

Shared Autonomy for a Wearable Supernumerary Robotic Limb

A complete account of the `kortex_ws` system:
what was built, what was measured, and what is still open

Platform	Two Kinova Gen3 7-DOF arms, backpack frame
Master	Instrumented mannequin arm, Teensy 4.1
Middleware	ROS 2 Jazzy, Fast DDS (SHM), WSL2
Repository	<code>kortex_ws</code> , 392 commits, 2026-07-14 to 2026-08-25
Scale	16 packages, 164k lines of Python, 114 test modules
This edition	every program, every failure, every change
Report date	26 August 2026

The complete edition. Part one is the narrative account; Chapter 17 is the defect register — 265 numbered faults with their mechanisms; Chapter 18 is the full change log, all 392 commits; the appendices list every program in the repository with its own description. It supersedes no existing document and updates none.

Abstract

This report documents a wearable supernumerary robotic limb system built over six weeks: two Kinova Gen3 seven-degree-of-freedom manipulators mounted on a backpack frame, worn by one person and driven by a second person through an instrumented mannequin master arm, a virtual-reality motion-capture path, or a typed or spoken natural-language goal. Around those three input routes sits a shared-autonomy layer, a perception stack that measures the working environment rather than being told about it, and a safety architecture whose central object is the body of the person wearing the machine.

The engineering content is substantial — sixteen ROS 2 packages, about 164 000 lines of Python, 114 test modules, 233 analysis and verification scripts, 71 design and findings documents, all catalogued program by program in the appendices — but the more transferable content is methodological. The project’s governing rule is that *a surprising measurement is evidence about the instrument until the instrument has been cleared*, and that rule has been correct seventeen separate times. Chapters 12 and 13 catalogue the instruments that produced confident wrong answers: a clearance model that returned the same number for an arm inside a mannequin and an arm in free space; a status topic that republished a frozen snapshot with fresh timestamps; a test gate that could invent a failing test out of a string literal; a render check that filed a scene containing nothing as passing because the wearer occupied 18 % of the frame.

Chapter 17 is the complete register of that work: **265 numbered faults**, in the order they were found, each with its symptom, its mechanism and its resolution. Chapter 18 is the corresponding change log — all 392 commits, oldest first, with what each one changed and why.

The system’s present capability is stated without hedging. It can measure a table and enumerate what stands on it to sub-millimetre accuracy; it can plan a grasp and refuse an unsafe one by name; it can execute a four-cube colour-matched pick-and-place on both arms from a typed English sentence; and it maps a volume with both arms and plans against the map. It cannot yet place a hand inside the 30 mm grasp capture gate from an open-loop plan — the measured worst-case positioning error is 33.4 mm and one term of the budget remains unmeasured — and its wearer guard cannot react to a person who moves quickly, because in five of six modelled limb motions the limb travels further than the entire 150 mm clearance floor before the guard hears about it. No human data has been collected; nothing has been worn.

Contents

Abstract	1
1 Introduction and scope	9
1.1 The system in one paragraph	9
1.2 What this report is	9
1.3 The scale of the artefact	10
1.4 How the work was done	10
1.5 What is deliberately absent	11
2 The physical platform	12
2.1 The arms and the mount	12
2.2 The master arm	13
2.3 The gripper, and a constant that was a curve	14
2.4 The compute environment, and its five consequences	14
3 Software architecture	16
3.1 Eight first-party packages, one-way dependencies	16
3.2 Node inventory	16
3.3 The five operating modes	17
3.4 The follower, and the absence of a planner	17
3.5 One source per constant	18
4 Kinematics, workspace, and the wearer	19
4.1 What limits the workspace	19
4.2 The pinned wrist, and what it costs	19
4.3 The wearer, and the 150 mm floor	20
4.4 Can the work be in the centre?	20
4.5 Neither arm crosses the centreline	21
4.6 The home pose	22
5 Teleoperation	23
5.1 The command path	23
5.2 Motion generation: from a clamp to Ruckig	23
5.3 The simulation-to-real gap, measured	24

5.4	The bridge deadband	24
6	The virtual-reality path, which is really motion capture	25
6.1	The route decision	25
6.2	It is not virtual reality; it is motion capture	25
6.3	Three safety gaps that desk operation turns from rare into routine	26
6.4	The observer e-stop	26
6.5	The mapper	26
6.6	One button starts VR	27
6.7	Passthrough	27
7	Perception	28
7.1	Detection is not the bottleneck	28
7.2	What is built	28
7.3	Measure the environment, then plan against it	29
7.4	Tracking the wearer	30
7.5	The scene camera, and what has never been attached	31
8	Autonomy, language, and planning	32
8.1	Why a grammar and not a language model	32
8.2	The task from a typed sentence	32
8.3	The window is part of the parser	32
8.4	Voice	33
8.5	The mislabel control	33
8.6	The planner that was built	33
9	The experiment programme	35
9.1	Four tasks, five modes	35
9.2	What each task is blocked on	36
9.3	The recorded clip set	36
9.4	T2's tray, and unfalsifiable criteria	37
9.5	Three instruments that reported defects that did not exist	37
10	Safety engineering	38
10.1	The layered defences	38
10.2	Fault injection	38
10.3	The latch pattern	39
10.4	Anonymity, enforced in code	39
10.5	And a demonstration is not evidence	39

11 The window	40
11.1 The rule	40
11.2 What the window is	40
11.3 The audit, and what it found	40
11.4 And look at it	41
12 Verification methodology	42
12.1 The instrument failure catalogue	42
12.2 Worked examples	42
12.3 Two clients, one planning scene	43
12.4 The test suite	44
13 The real-hardware campaign	45
13.1 The first real-arm session, 2026-08-20	45
13.2 The cube pick, 2026-08-25	45
14 How good is it, honestly	48
14.1 The short answer	48
14.2 The positioning error budget	49
14.3 The reaction budget	49
14.4 The failure mode of losing tracking	50
15 What is open	51
15.1 Blocked on the laboratory	51
15.2 Blocked on ethics	51
15.3 Blocked on measurement	51
15.4 Built but not yet fed, or not yet wired	52
16 Timeline and conclusion	53
16.1 How the project actually went	53
16.2 What was built	53
16.3 What was learned	54
16.4 Where it stands	54
17 Every failure	55
17.1 2026-07-31 — the first build	55
17.2 2026-08-05 — the mount, and the hardening pass	57
17.3 2026-08-06 — the capture, and what it invalidated	60
17.4 2026-08-07 — the task scenes, and the geometry that stopped a programme . .	62
17.5 2026-08-08 to 08-13 — the two-person reframing and the GUI generations . . .	63

17.6	2026-08-15 — the workspace measured against the wearer	64
17.7	2026-08-17 — the wrong wrist, and the run that did not start from home . . .	71
17.8	2026-08-18 — language, the grasp constant, and the centre priced	73
17.9	2026-08-19 to 08-20 — the VR path, and the first real arm	75
17.10	2026-08-22 to 08-23 — the window, the motion generator, and the accuracy pass	78
17.11	2026-08-25 — the cube pick on real hardware	83
17.12	Still open	84
18	Every change	86
	2026-07-14	86
	2026-08-08	86
	2026-08-09	89
	2026-08-10	98
	2026-08-11	106
	2026-08-12	115
	2026-08-13	120
	2026-08-15	125
	2026-08-16	127
	2026-08-17	129
	2026-08-18	133
	2026-08-19	135
	2026-08-20	136
	2026-08-21	137
	2026-08-22	137
	2026-08-23	140
	2026-08-24	142
	2026-08-25	145
A	Every program: the packages	146
A.1	srl_teleop	146
A.2	srl_perception	151
A.3	srl_autonomy	153
A.4	srl_experiments	154
A.5	srl_vr_teleop	156
A.6	srl_vr_autonomy	157
A.7	srl_experiments/experiments — the task layer and the archive	157
A.8	Launch files	161

B Every program: the scripts	163
B.1 Measurement — each produces a number and writes a baseline	163
B.2 Verification — each produces a pass/fail verdict	166
B.3 Audit — checks the repository against its own declarations	168
B.4 Sweeps — explore a parameter space and record the whole surface	169
B.5 Searches over configuration space	170
B.6 Solvers — poses, layouts, transits	170
B.7 Recording and capture	171
B.8 Rendering	171
B.9 Probes — small targeted experiments	171
B.10 Session checks	172
B.11 Calibration	172
B.12 Diagnosis	173
B.13 Fault injection	173
B.14 Pick execution	173
B.15 Operator-facing tools and windows	174
B.16 Scanning	174
B.17 Staging	175
B.18 Analysis	175
B.19 CAD	175
B.20 Finders	175
B.21 Generators	176
B.22 Runners	176
B.23 VR tooling	176
B.24 Motion helpers	177
B.25 Location	177
B.26 Builders	177
B.27 Gates	177
B.28 Appliers	177
B.29 Choosers	178
B.30 Drivers	178
B.31 Execution	178
B.32 Readers	178
B.33 Scoring	178
B.34 Mapping	178
B.35 Merging	179
B.36 Modes	179
B.37 Automation	179

B.38 Observer	179
B.39 Hand-eye	179
B.40 Guarded motion	180
B.41 Completion	180
B.42 Follower control	180
B.43 Description	180
B.44 Capture	180
B.45 Sampling	180
B.46 Settling	181
B.47 Surveys	181
B.48 Re-look	181
B.49 Retraction	181
B.50 Revalidation	181
B.51 Recording layout	182
B.52 Safe motion	182
B.53 Servoing	182
B.54 Simulated sessions	182
B.55 Status	182
B.56 Accuracy	183
B.57 Session faults	183
B.58 Clips	183
B.59 Environment	183
B.60 Archived runs	183
B.61 Generators	183
B.62 Gripper	184
B.63 Instruction	184
B.64 LeRobot	184
B.65 Pilot runs	184
B.66 Planning	184
B.67 Posing	185
B.68 Proofs	185
B.69 Virtual hardware	185
B.70 Shell entry points	185
C Every program: the tests	188
C.1 srl_autonomy — 10 modules	188
C.2 srl_experiments — 35 modules	188
C.3 srl_perception — 17 modules	191

C.4	<code>srl_teleop</code> — 43 modules	192
C.5	<code>srl_vr_teleop</code> — 10 modules	195
D	Recorded baseline artefacts	196
E	The hard constraints, in full	198
F	Glossary	200

CHAPTER 1

Introduction and scope

1.1 The system in one paragraph

Two Kinova Gen3 seven-degree-of-freedom arms are mounted on a backpack frame worn on a person's back. A second person drives them. The driving input can be an instrumented mannequin arm — a physical linkage with a potentiometer on each joint and an inertial measurement unit at the wrist, read by a Teensy 4.1 over serial — or a pair of Meta Quest controllers held as six-degree-of-freedom motion capture, or a sentence typed or spoken in English. Between the input and the metal sits a shared-autonomy layer that can supply wrist orientation, infer a goal, arbitrate a handover, or take the task over entirely. The whole stack is ROS 2 Jazzy on Ubuntu under WSL2, with Fast DDS restricted to shared memory transport.

Two people, and this is structural. The person wearing the arms and the person driving them are different people. Both are participants; both consent separately; the wearer has their own outcome measures. The wearer is not part of the apparatus, and the wearer's body is in the robot's collision model. Every safety argument in this report follows from that one sentence.

1.2 What this report is

It is a complete technical account of the work to 26 August 2026, written as a standalone document, in four parts:

- **Chapters 1–16, the narrative.** What the system is, how each subsystem works, what was measured, and what it is honestly capable of.
- **Chapter 17, every failure.** The defect register: 265 numbered faults in the order they were found, with the mechanism named in each.
- **Chapter 18, every change.** All 392 commits, oldest first, each with its own account of what it changed.
- **Appendices A–C, every program.** Every module, script and test in the repository, with its size and its own description, followed by the baseline artefacts, the hard constraints and a glossary.

It does not update, replace or summarise the existing thesis draft in `thesis_report/`; it is a separate synthesis, assembled by reading the source tree, the test suite, the recorded baselines and the 7099-line diagnostic history in `docs/system/findings.md`.

Three conventions are used throughout.

- **Every number is a measurement or it says it is not.** Where a quantity has never been measured, this report writes UNMEASURED rather than an estimate. A budget with a guessed term hides the term you should go and measure.

- **Defects are reported with their mechanism.** “The check was wrong” is not useful; “the check compared a constant that includes the immobile mount stub against a floor that constant can never violate” is.
- **Claims are attributed to the artefact that establishes them.** Where a claim rests on a script, a test or a recorded baseline, that file is named.

1.3 The scale of the artefact

Measure	Value
Commits	392 (2026-07-14 to 2026-08-25)
ROS 2 packages built	16
First-party packages (<code>srl_*</code>)	8
Python in <code>src/</code>	373 files, 83 910 lines
Python in <code>scripts/</code>	233 files, 80 732 lines
Shell entry points in <code>scripts/</code>	28
Test modules	114
Design and findings documents	71
Recorded baseline artefacts	195 files in <code>recordings/baselines/</code>
Verification clip cells	5 modes $\times$ 5 tasks, 25 planned, 31 clip directories
Diagnostic history	7 099 lines, <code>docs/system/findings.md</code>

Table 1.1: The repository, measured on 2026-08-26.

The commit distribution is informative about how the work was done: 22 commits on 8 August, 60 on the 9th, 52 on the 10th, 60 on the 11th, then a tapering run through the second half of the month as the work shifted from building to measuring and then to hardware. The single commit on 2026-07-14 is the initial import; the substantive project is a four-week sprint in August.

1.4 How the work was done

The project has one methodological rule, stated in the repository’s own standing instructions and enforced by the test suite:

Before reporting a bad measurement, validate the measuring tool against ground truth. A surprising failure is evidence about the instrument until the instrument has been cleared. This has been the fault seventeen times.

Four corollaries have accumulated around it, each from a specific incident:

- **Every analysis script gets a known-answer test.** Synthetic data is acceptable only where the ground truth is *constructed* (arithmetic, geometry) and never where it is *rendered*.
- **A check that cannot fail on a deliberately broken input is not a check.** Several checks in this project passed for weeks and were later shown to be unfalsifiable; each now has a fault-injection argument (`--inject-float-mm`, `--self-test`) that makes it fail on demand.

- **A result contradicting an earlier measurement is an instrument check, not a finding,** until the two are reconciled.
- **Repeat before believing.** TRAC-IK restarts randomly, so one call is one coin flip. Reachability is certified at $N = 10$ over a path densified to 20 mm, and usable poses stay 20 mm inside the last pose that passed 10/10.

Chapter 12 gives the catalogue of instrument failure modes this produced, which is arguably the most reusable output of the project.

1.5 What is deliberately absent

Three whole categories of work were considered and declined, each for a recorded reason rather than by omission.

- **No end-to-end vision-language-action policy.** There is no corpus; collecting one requires human data that ethics has not cleared; the available GPU is an RTX A500 with 4 GB of VRAM shared with the detector; and a learned policy would learn *around* an unmeasured camera extrinsic instead of exposing it.
- **No local large language model for command parsing.** A grammar was chosen instead, because a grammar can explain a refusal and can be swept over 75 phrasings with a scored outcome per phrasing.
- **No cyclic (real-time) Kortex control path.** It is unusable over WSL2 — every cyclic write is a network round trip — and the project records an explicit instruction not to spend another day on it.

CHAPTER 2

The physical platform

2.1 The arms and the mount

Two Kinova Gen3 seven-degree-of-freedom manipulators, each with a Robotiq two-finger adaptive gripper and a wrist-mounted colour-and-depth camera module, are carried on a frame worn on the back. The mount geometry was one of the longest-running engineering problems in the project and is worth stating as it was resolved rather than as it was first written.

2.1.1 The mount orientation defect

The arms initially rendered in a posture that did not match the physical rig. Two hypotheses were possible: the home joint angles were wrong, or the mount rotation was wrong. The project resolved this by evidence and recorded the resolution as a hard constraint, because the wrong answer is dangerous:

Home joint angles are ground truth and must never be changed to make a pose look right. The simulation-to-real bridge replays simulated joint angles onto the physical arm; any difference it does not catch is commanded as a jump. The defect was the mount rotation — specifically an angle placed in the wrong `rpy` slot — and it was fixed there.

The mount was then re-derived rather than tuned. A sweep of 108 candidate mount geometries was recorded and, deliberately, *not* applied; the finding from it was that no rotation of the mount fixes the binding constraints, because three of them are not rotational in origin. One criterion in the original brief was shown to be unsatisfiable by proof rather than by failure to achieve it.

2.1.2 The proximal collision

A genuine mechanical finding, not a modelling nuisance: with a shoulder-mounted pair of arms, the segments that threaten the wearer are the proximal ones — the upper arm and the tube between shoulder and elbow — and these are exactly the pairs a conventional Semantic Robot Description Format (SRDF) file excludes from collision checking, because on a desk-mounted robot they can never collide with anything. Forty-four proximal pairs are excluded in this project's SRDF. The consequence is stated as a hard constraint and repeated throughout this report:

avoid_collisions is not the wearer check. A pose that MoveIt reports as valid can have the arm's tube inside the person. Wearer clearance must be measured geometrically. An SRDF exclusion silences an alarm; it does not move the metal.

2.1.3 The mount caps every clearance figure

`base_link` sits 0.1610 m from the wearer's torso and no joint moves it. The 150 mm clearance floor therefore has 11 mm of headroom before the arm has done anything at all, and a clearance reading of exactly 0.1610 is a measurement of the mount, not of the arm. This single fact invalidated several earlier clearance numbers, and the project keeps a dedicated ledger (`docs/system/clearance_gap_ledger.md`) recording which historical workspace figures survived the discovery and which did not.

2.2 The master arm

The master is an instrumented mannequin arm: a potentiometer on each of seven joints per side, plus a wrist inertial measurement unit, read by a Teensy 4.1 and streamed over serial. It is not a haptic device and provides no force feedback.

2.2.1 Channel health, and why it is measured every session

The master's channels are individually unreliable. The newest recorded baseline (`recordings/baselines/channels_20260806.json`, 20 440 frames) gives the following verdicts.

Channel	Verdict	Updates	Dropout %
l_j1	INTERMITTENT (2.2 % dropouts)	168	2.2
l_j2	INCOHERENT (5 % of updates jump >60°)	172	0.0
l_j3	INCOHERENT (8 %)	261	0.0
l_j4	INCOHERENT (6 %)	97	22.4
l_j5	INCOHERENT (12 %)	82	40.4
l_j6	INCOHERENT (8 %)	36	70.4
l_j7	ALIVE (range 26°)	27	0.0
r_j1	ALIVE (range 195°)	248	0.0
r_j2	ALIVE (range 261°)	265	0.0
r_j3	INCOHERENT (14 %)	37	79.7
r_j4	ALIVE (range 291°)	242	0.2
r_j5	DEAD (range 0.0°, 0 updates)	0	100.0
r_j6	ALIVE (range 250°)	309	0.1
r_j7	DEAD (range 0.0°, 0 updates)	0	100.0

Table 2.1: Master channel health, 2026-08-06 baseline. Seven of fourteen channels are unusable; the cause is a soldering problem, with `l_j2` and `l_j4` named by regression.

INCOHERENT is a verdict the project had to invent. A channel can be alive, can have low dropout, and can still be useless because its updates jump more than 60° between samples — a signal that is present, fresh and wrong. Before this verdict existed, such channels were counted as working.

First action of every lab session: `bash scripts/check_channels.sh`, about three minutes. Which channels are frozen is decided from the *newest* `channels_*.json`. A baseline older than the wiring makes the software freeze channels that now work, and nothing downstream will disagree with it. Target: twelve or more of fourteen coherent.

2.2.2 Sensing ladder and degraded mode

Because channels fail, the master sensing path is a ladder rather than a single algorithm. Position is computed in a spherical mode by default, with a measured axis map `(("+x", "+y", "+z"))` that is deliberately not “fixed” — the sign convention of the direction unit vector is documented as correct and counter-intuitive. A complementary filter fuses the wrist IMU. A gyro-aided azimuth path exists, is measured, and is not enabled by default. Every channel is individually disableable through `channel_manager`, and a `degraded_mode` exists with a measured cost expressed as radial extent rather than convex hull, because convex hull flatters a degraded arm.

A `virtual_tensy` script provides a synthetic master so that the whole teleoperation path can be exercised without the physical linkage, and the serial port is held exclusively so that two readers on one Teensy is impossible rather than merely detectable — an earlier incident in which two `master_pose_node` instances split the serial stream invalidated a full day of measurements and is now hard constraint number 3.

2.3 The gripper, and a constant that was a curve

The Robotiq two-finger gripper’s fingers swing on a four-bar linkage. The consequence is that the distance from the wrist frame to the midpoint between the finger pads is *not* a constant.

Gripper state	Wrist-to-pad-midpoint (m)
Wide open	0.09833
Closed on a 40 mm cube	0.10976
Closed on a 20 mm cube	0.11179

Table 2.2: Measured by forward kinematics along the mimic chain, `scripts/measure_pad_mid_ee.py --by-width`.

This single measurement resolved an argument the repository had with itself for five days. One earlier finding said the grasp constant was “13.47 mm too long”; another said T1’s grasp was “built 11.43 mm short”. Both numbers are points on the same curve, and the two claims were comparing different gripper states. It also explains why the evidence appeared to flip between runs: a verification script read the finger tips at whatever opening the simulator happened to be left at.

The operational consequence is direct. Solving a grasp with the hand open and then closing it drives the pads 11.43 mm further along the tool axis — into the table. Grasp poses are now solved at the closed-on-object width, and `pad_mid_ee_for(width_mm)` is the table that supplies the number.

2.4 The compute environment, and its five consequences

The whole system runs under WSL2 on a laptop with an RTX A500 (4 GB VRAM). That environment is not incidental; it determines five separate architectural decisions.

Constraint	Consequence
UDP discovery is dead on this host	<code>FASTDDS_BUILTIN_TRANSPORTS=SHM</code> is mandatory in every terminal. Stale <code>/dev/shm/fastrtps_*</code> segments must be cleared with the stack stopped.
Every cyclic write is a network round trip	The Kortex real-time cyclic path is unusable; the high-level API bridge (<code>kortex_highlevel_bridge</code>) is the only supported route to the arm.
<code>/dev/snd</code> contains only <code>timer</code>	No microphone can be opened. Every voice number in the project was produced from synthesised audio, and is a parser score, never a transcription score.
No <code>adb</code>	The VR path had to work through the headset's own browser over Wi-Fi, which turned out to be an advantage: it works on a borrowed headset with no sideloading.
4 GB of shared VRAM	CUDA motion generation (cuRobo) and GPU simulation (Isaac Lab) are excluded by hard requirement, not by preference. Torch runs CPU-only by choice.

Table 2.3: WSL2 and the laptop GPU as design constraints.

The arm permits exactly one Kortex session. The bridge must be stopped with `SIGINT`; `SIGKILL` leaks the session and the next run cannot connect. The check is to `grep` for `kortex session closed cleanly`.

A late and expensive discovery, on 2026-08-25: Fast DDS shared memory on ROS domain 0 became polluted such that no *new* process could join it, while every already-running node continued to work normally. A minimal publisher and subscriber measured 0 of 3 messages delivered on domain 0 and 3 of 3 on domain 7. The real-arm tooling now runs on `ROS_DOMAIN_ID=7`. In the same session, an unset `RMW_IMPLEMENTATION` was found to make `rcply` discover nothing at all while the ROS graph was plainly present in other tools — a failure that looks exactly like a code bug and is not one.

CHAPTER 3

Software architecture

3.1 Eight first-party packages, one-way dependencies

Package	Role
<code>srl_teleop</code>	Teleoperation only: master sensing, IK follower, clutch, scaling, motion generation, simulation-to-real bridge, e-stop, the GUI.
<code>srl_perception</code>	Cameras, AprilTag, colour and shape detection, open-vocabulary detection, depth grasping, table scene analysis, segmentation lifting, the world model, wearer tracking, calibration sweep, joint-space planner.
<code>srl_autonomy</code>	Grasp generation, intent inference, handover arbitration, named places, referring expressions, voice intent.
<code>srl_experiments</code>	Tasks, conditions, logging, analysis, session management, narration, the work-surface owner.
<code>srl_description</code>	<code>srl_dual.urdf.xacro</code> — arms, mount, wearer.
<code>srl_moveit_config</code>	TRAC-IK, controllers, SRDF.
<code>srl_vr_teleop</code>	Quest bridge, pose mapper, VR safety, VR gripper, wearer view.
<code>srl_vr_autonomy</code>	VR handover arbiter, VR intent source.

Table 3.1: The first-party packages. Vendored third-party packages (`ros2_kortex`, `ros2_kortex_vision`, `ros2_robotiq_gripper`, `serial`) bring the total build to 16.

The dependency arrow runs one way. `srl_teleop` imports nothing else in the repository. It is the baseline condition of every experiment; if it imported `srl_autonomy`, every comparison against the baseline would be circular.

3.2 Node inventory

`srl_teleop` alone exposes roughly forty console entry points. They group into six functions:

- **Master sensing** — `master_pose_node`, `master_imu_node`, `pot_bridge`, `fake_pot`, `channel_manager`, `capability_node`, `pointing_direction_node`.
- **Following** — `ik_follower_node`, `leader_follower_node`, `srl_teleop_node`, `fsr_gripper_node`.
- **Safety** — `estop_node`, `mount_guard_node`, `blocking_aggregator`, `recovery_manager`, `participant_safety_node`, `real_arm_gate`, `arm_link_monitor`, `selftest_node`, `preflight`.
- **Real hardware** — `real_homing_node`, `sim_to_real_bridge`, `kortex_highlevel_bridge`, `mock_real_stack`.
- **Visualisation and operation** — `gui`, `teleop_gui`, `console`, `launcher`, `rviz_master`, `live_monitor`, `dashboard`.

- **Instrumentation** — `fault_injector`, `full_state_recorder`, `teleop_recorder`, `block_monitor`, `divergence`.

`rviz_master` deserves a note, because it encodes a philosophy. It draws three states, not two: `COMMANDED`, `ACTUAL`, and `PLANNED-BUT-REFUSED`, the last of which cannot be drawn at all without a reason string attached. The wearer is drawn inflated by the clearance floor, built from the body the guard is actually enforcing (not a separate copy), with the nearest segment coloured and labelled by body part. It is a pure marker builder with 32 checks and needs no display to test.

3.3 The five operating modes

Key	Name	Input	Assistance
01	<code>01_master_teleop</code>	Mannequin master arm	none
02	<code>02_vr_teleop</code>	Quest controller, full 6-DOF	none
03	<code>03_shared_autonomy</code>	Master input	wrist orientation supplied
04	<code>04_vr_shared</code>	VR input	wrist orientation supplied
06	<code>06_full_autonomy</code>	Spoken or typed goal	the system does the rest

Table 3.2: The modes. There is no 05; the numbering is historical and is preserved so that recorded data keeps its labels.

The same waypoints are *commanded* under every mode — the task runner builds them with no mode argument at all — but the robot does not *perform* identically. Measured on 2026-08-15 with no operator present, task T0’s left-arm path length was 1.407m under mode 01 and 2.034m under mode 03. This is finding 3 in the project’s history, and settling it required refuting two alternative explanations; the effect is real, and its magnitude is not yet known, because the recorded clip set cannot separate mode from the upstream node set that each mode brings with it.

3.4 The follower, and the absence of a planner

Until 2026-08-22 nobody had written down the following sentence, and it is the single most clarifying fact about the control path:

This project had no motion planner. `ik_follower_node` solves one inverse-kinematics query per commanded pose through MoveIt’s `/compute_ik` (TRAC-IK, `solve_type: Distance`, 5ms timeout) and clamps the joint step per cycle. That is pose streaming with a rate limiter. There is no OMPL in the command path, no `move_group.plan()`, no `compute_cartesian_path`. MoveIt is used for exactly two things: the IK service and the planning scene.

For teleoperation that is the correct architecture. The operator’s hand is the input, there is no goal to plan to, and the real problem is smooth, safe following. For autonomy it is not correct, and the cost was measured: a pick whose pregrasp and grasp poses are both individually clear, streamed as a straight line in joint space, breaches the wearer clearance floor at 0.1462m at step 29 of 64. The path checker catches it — and catching it means the run stops. Chapter 8 describes the joint-space planner built in response.

3.5 One source per constant

A recurring class of defect in this project is a constant with more than one storage site, where a change reaches some consumers and not others. Each instance was fixed by collapsing to a single owner and pinning the collapse with a test.

Quantity	What went wrong	Test
Home joint angles	Stored in five places, not two: <code>config/home_positions_*.txt</code> (the source, 5 live consumers), the URDF's two spawn blocks, and private hardcoded copies inside <code>pot_bridge.py</code> and <code>srl_teleop_node.py</code> .	<code>test_home_has_one_source.py</code>
Work-surface height	Two constants 150 mm apart: <code>BENCH_TOP</code> 1.10 positioned every object and <code>TABLE_TOP</code> 0.950 was the only geometry. Nothing compared them.	<code>test_one_work_surface_height.py</code>
Pad offset	The declared grasp offset was 13.45 mm longer than the measured one. It was invisible because it <i>cancelled</i> : the scene drawing used the same constant, so the picture was always right whatever the value. What did not cancel is where the fingers went.	<code>test_pad_offset_has_one_source.py</code>
Wearer posture	The posture variable reached the URDF but not <code>mount_guard_node</code> , which built its wearer at import time. A posture that reaches one and not the other measures the old wearer under a new name.	<code>test_wearer_posture_has_one_source.py</code>
Continuous joint indices	<code>CONTINUOUS_IDX</code> had six independent definitions.	<code>test_motion_generator.py</code>

Table 3.3: Constants that had more than one owner, and the tests that now prevent it.

CHAPTER 4

Kinematics, workspace, and the wearer

This chapter contains the project’s hardest-won results. The short version is that the arms move far less usefully than a specification sheet suggests, that the reason is almost never a joint limit, and that the binding constraint changes identity depending on which direction you ask about.

4.1 What limits the workspace

Direction	What binds
Forward and outboard	the pinned wrist
Inboard (towards the body centreline)	the wearer
Down	the table
Any direction	<i>not</i> a joint limit — no direction is joint-bound

Table 4.1: The binding constraint per direction, with the mechanism named.

4.2 The pinned wrist, and what it costs

Until 2026-08-22, every commanded waypoint carried a fixed wrist orientation — the “workspace anchor”, a stored constant of approximately 30.7° approach direction. This was never sized. When it was, the result was startling.

Orientation policy	At the work point (12 walks)	At home (28 walks)
Pinned (before 2026-08-22)	0.065 m	0.460 m, 6 wearer-bound
Cone 15° (the default now)	0.304 m	0.492 m, 6 wearer-bound
Cone 45°	0.325 m	0.530 m
Position only	0.362 m	0.549 m

Table 4.2: Mean reach over the direction walk, 150 mm wearer floor enforced throughout, computed through the follower’s own candidate list. `scripts/measure_orientation_cost.py`.

At the home pose the wrist cost 89 mm. **At the point where the work actually happens it cost a factor of seven**, and eleven of twelve direction walks were wearer-bound. The mechanism is not the wearer: a fixed six-degree-of-freedom pose on a seven-degree-of-freedom arm spends the entire redundancy, and the redundancy is precisely the joint that moves the elbow out of the person while the hand stays where it was put.

Freeing only the wrist *roll* is worth exactly +0 mm, and is refused by name. `orientation_policy` defaults to `exact` in the library, so nothing changed until a mode opted in; the follower has run a 15° cone since 2026-08-22, which retrospectively invalidated six verification scripts that were still asking for an exact orientation (Chapter 12).

4.3 The wearer, and the 150 mm floor

The wearer is a geometric body in the collision model — torso, head, hips and, since 2026-08-15, arms. The clearance floor is 150 mm from any moving part of the arm chain to any part of that body.

4.3.1 The wearer’s posture is a variable

SRL_WEARER_ARMS offers five settings through a single source, `wearer_posture.py`, which both writes the URDF’s arm block and builds the guard’s own body. Clearance from the arm at home, by posture:

Posture	Clearance at home	Against the assumed model
out (arms held out to the sides)	0.0933 m	−224 mm — breaches the floor
behind (clasped behind the back)	0.2779 m	−39 mm
down (the assumed model)	0.3170 m	—
folded	0.3211 m	+4 mm
none	0.3211 m	+4 mm

Table 4.3: Identical on both arms. This is why hard constraint 11 forbids instructing a wearer to clasp their arms behind their back or hold them out to the sides: with a back-mounted rig, both put the wearer’s own limbs inside the floor at the home pose.

4.3.2 The wearer’s size is a variable too

Posture became a variable on 2026-08-15 and size did not, which meant that every figure produced afterwards silently described a 300 mm upper arm and a 360 mm chest. SRL_WEARER_SIZE and `config/wearer_sizes/` close that, through the same single source, with the `mannequin` setting reproducing the shipped body byte for byte.

What a real body costs, measured whole-arm through IK at $z = 1.120$:

- Arms down the sides: the left column *improves* by 25 mm ($0.375 \rightarrow 0.350$).
- Arms folded: 25 mm *worse* ($0.325 \rightarrow 0.350$), because the chest is 70 mm wider.

What binds changes identity with the body. For the mannequin it is a limb; for a real person it is the torso, every time. The operational consequence: folding the arms is worth 50 mm to the mannequin and nothing at all to this person.

4.4 Can the work be in the centre?

The most operationally important question about a supernumerary limb is whether it can work in front of the wearer’s own body, where the wearer’s own hands are. The project asked this four separate ways and the answer is consistently no.

Question	Answer	Evidence
Innermost safe column at $z = 1.120$	left 0.325 m, right 0.450 m, $N = 10$ over the full path	<code>measure_centre_vs_height.py</code>
With the wearer's arms deleted entirely	still shut: $\min x $ 0.300 (L) / 0.375 (R). Every column from 0.125 to 0.300 binds on the <i>torso</i> . Arms clear buys 25 mm (L), 75 mm (R).	<code>centre_vs_wearer_posture.json</code>
On the surface, searched properly	3 360 cells, table height and distance both swept, objects resting: no configuration puts $x \leq 0.10$ on the surface. Closest confirmed at $N = 10$: $x = 0.350$, left arm, table top 1.00, near edge 0.530.	<code>search_centre_on_surface.py</code>
Top-down on a surface	0 of 840 cells , any table height 0.70–1.10, either arm. Not a trade-off against centre-on-surface — centre-on-surface is achievable at 350–450 mm off centre; top-down-on-surface is not achievable at all.	<code>centre_on_surface.json</code>

Table 4.4: Four independent searches for a workspace in front of the body.

Every survivor of the surface search needs the object at the very front edge with zero overhang, because the pinned wrist arrives from below.

4.5 Neither arm crosses the centreline

Measured at every cross-side placement slot and every cross-side object: 0 of 10 IK solutions, both arms. This is not marginal — nothing solves. It is a hard architectural fact that decides what the two-arm task can be: an object can only be delivered to the target on its own side, so the randomised stage of the pick-and-place task draws the *side* and lets the colour follow, and the task builder refuses a cube whose colour names the other side's pad.

The same fact killed a planned inter-arm handover task outright: the two arms' reachable sets are disjoint, so there is no point at which one can hand to the other. That is recorded as a finding, not as a failure to implement.

4.6 The home pose

4.6.1 The change, and the two things it was

On 2026-08-15 the simulated home pose became the “presentation pose”: wrists level, hands in front of the chest. The change was made only after the entire task set was re-measured, and it was carefully separated into two changes of which only one is dangerous.

- **Moving home** cost nothing measurable. With the workspace anchor unchanged, every task performs exactly as before.
- **Re-deriving the anchor to match a level home** costs task T2 its right arm, so it is refused by name in the hard constraints.

Simulated and real home disagree on purpose. The physical arms are still at the legacy Kortex home, where the wrists point up by $+85^\circ$ and $+79^\circ$. The bridge *refuses to enable* on the roughly 1.94rad difference and says why, rather than commanding it as a jump. The new pose must be captured on the arms before any real session.

4.6.2 The pose in pixels

A separate investigation asked whether the rendered home pose was wrong, and found that the render was right and the numbers were about something else. Measured from the transform tree at boot: elbows 0.0793 m (left) and 0.2715 m (right) below their shoulders; the left elbow 188 mm outboard of its own hand; hands 1.10 m apart against a 0.36 m torso; forearm mirror residual 0.2826 m. The wrists are level to -0.01° and always were. It is not a cached configuration: the live joint state matches the source file to 7×10^{-5} rad.

Can it be fixed? Partly. The innermost hand column that works for both arms is $|x| = 0.450$ against a torso half-width of 0.18 — so “hands at torso width” is, once again, the torso. The best achievable mirror residual is 0.161–0.164 m and does not improve with more search branches (4 branches versus 31 changes it by 1 mm), so the asymmetry is structural: the two mounts’ base axes mirror exactly but differ by 168° of roll, and two identical arms on mirrored mounts cannot mirror. This was measured and deliberately not applied.

4.6.3 A run must start from home

From the second run of a session onwards, it did not. Nothing returned the arms to home when a run ended and nothing checked at the start, so run 2 began 1.2338 rad (left) and 0.6248 rad (right) away from home, measured at the first commanded waypoint. Boot and run 1 both read 0.0000. The task runner now stages before the task and *refuses* rather than running from an unknown pose; `--allow-unhomed` overrides, loudly. The recorded clip sweep always staged, so the recorded clips were never affected.

CHAPTER 5

Teleoperation

5.1 The command path

Master pose is read, mapped through the spherical position mode and the measured axis map, scaled, gated by a clutch, and turned into a commanded end-effector pose. The IK follower solves one query per pose and hands joint targets to the motion generator. Orientation is held at the workspace anchor by default; mode 03 and 04 replace it with an autonomy-supplied wrist.

The clutch is a genuine engineering component rather than a button: a boot reference that was originally the master frame origin (wrong, and fixed), a live button map with a lab check, a second press that cancels a pending engagement, and an indexing behaviour measured over ten cycles against the virtual board. The first such run showed four cycles of 0.00 mm, which was a real bug and not a measurement artefact — a distinction the project makes explicitly because the reverse is far more common here.

5.2 Motion generation: from a clamp to Ruckig

Until 2026-08-23 the follower’s motion generation was a per-joint step clamp. Each joint was clamped independently, which has three consequences that were measured rather than argued:

Quantity	Per-joint clamp	Ruckig
Hand excursion off the commanded line, left arm	51.3 mm	0.0 mm
Hand excursion off the commanded line, right arm	39.2 mm	0.0 mm
Peak commanded joint speed vs <code>joint_limits.yaml</code>	12.53×	1.00×
Joint arrival	spread over two cycles	all seven on one cycle

Table 5.1: Measured by `scripts/measure_teleop_motion.py` against a 30 mm grasp gate. 17.5 rad/s was being commanded against a declared limit of 1.3963 rad/s; the limits file had never been read.

This is not a tuning result. Swept against its own step size, the clamp’s excursion *settles* at 68 mm — there is no step size that fixes it, because the defect is the independence of the per-joint clamps, not their magnitude.

Ruckig was selected on published characteristics (time-optimal, jerk-limited, type V, 19.8 μ s mean for seven degrees of freedom, no dependencies) and adopted as the default in every mode. Velocity limits are the robot’s own; acceleration and jerk are **assumed and labelled as such**, because `joint_limits.yaml` declares `has_acceleration_limits: false` for all fourteen joints. `motion_generator:=legacy` reproduces any recording made before 2026-08-23.

Five defects in the author’s own implementation were caught by checks during this work, and all five are instructive: re-planning every cycle destroyed phase synchronisation; what forced the re-plan was floating-point noise; the fallback path claimed to be jerk-limited by reporting

zeros; the off-path metric had a 0.05 mm floor and reported that floor as behaviour; and the arrival-cycle test used an absolute tolerance on a per-joint quantity.

5.3 The simulation-to-real gap, measured

Every joint parks 0.305° short of its target, on the side it came from. 211 of 252 recorded per-joint errors fall at ± 0.25 – 0.30° with the sign a coin flip — which makes it a *bound*, not a spread.

At the end effector that is 7.2 mm RMS on a 100 mm move, on both arms. Pushing the recorded joint errors through the Jacobian reproduces the Cartesian error to 90 % ($r = 0.923$), which establishes that it is a constant terminal offset rather than a gain — two models that are indistinguishable at 100 mm and differ by a factor of five at 20 mm. A single parameter per arm removes 74 % and 76 % of it, held out on an axis the parameter was never fitted on, where a full Cartesian 3×3 correction scores 93.5 mm. The compensation is “overshoot each joint by ε in the direction of travel”: no matrix, no basis, no step length.

The data for this had existed in `recordings/baselines/arm_directional_calibration.json` since it was recorded, and **nothing read it**.

A correction of the project’s own earlier reporting belongs here. A “speed sweep” was reported as showing the error to be speed-independent across a $5\times$ range. It was not a sweep: `vmax_rad_s` was read once at bridge start-up, and 7 of 36 runs moved faster than the limit they were nominally commanded with. The three speeds are three replicates of one condition, and how the error varies with speed has never been measured. A control test now asserts that the sweep still looks inert, so that a genuine sweep will make it fail and force the stale sentences to be rewritten.

5.4 The bridge deadband

The high-level bridge had a 1.0° deadband inside which the proportional term is off by design — $3.3\times$ the 0.305° park error it was permitting. It is now 0.10° , with an opt-in `terminal_overshoot` that applies the compensation at the setpoint and refuses to run in the “on” state with a zero overshoot value. Neither has been tried on hardware; both are live parameters and four runs would settle which wins.

CHAPTER 6

The virtual-reality path, which is really motion capture

6.1 The route decision

The second input route uses a Meta Quest headset and its two controllers. The route was decided on facts rather than preference, because the obvious route was blocked: there is no `adb` on this machine, so a sideloaded native application was never available.

What was built instead is a WebXR client served by `quest_bridge_node` directly to the headset's own browser over the laboratory Wi-Fi. The node serves *both* the client page and the secure WebSocket from one TLS origin, which means one certificate exception covers both. The certificate carries the machine's IP address in `subjectAltName`, and WSL2's mirrored networking makes this box's LAN address identical to Windows'.

■ The advantage of being forced onto the browser route is that it works on a *borrowed* headset: no sideloading, no developer account, no settings changed.

Running this path for the first time on 2026-08-19 found two defects that no amount of reading would have found:

- **The bridge node had never started at all.** Not once. A member assignment `self.clients` shadowed a read-only `Node` property, and the node raised on construction.
- **Its axis map was for the standard ROS frame, not this repository's.** A hand held three metres forward commanded three metres to the wearer's *right*.

6.2 It is not virtual reality; it is motion capture

The shipped operating arrangement, clarified on 2026-08-19, is:

■ The operator sits **across the room, facing the wearer**, holds the controllers as six-degree-of-freedom motion capture, and watches the robot directly. **Nobody wears the headset** — it stands on a shelf as the tracking reference.

This has a geometric consequence that had been promised in a docstring for months and never implemented. Facing someone and copying their motion is a *reflection* — determinant -1 — which mirrors every orientation while positions still look approximately right. The alignment is therefore a *yaw*, never a mirror, and no value of the parameter can produce one. The yaw is measured rather than guessed by `scripts/calibrate_operator_yaw.py`, which refuses a calibration motion under 100 mm or more than 35° off horizontal.

One ambiguity this uncovered and did not settle: the repository disagrees with itself about whether world $+x$ is the wearer's right, and the arms are physically mounted the other way round. That is recorded as open.

6.3 Three safety gaps that desk operation turns from rare into routine

6.3.1 The tracking reference can be knocked

In desk operation the headset *is* the tracking reference, and a nudge announces nothing: poses stay valid, the rate stays 90 Hz, controllers stay tracked, and every pose after the knock is expressed in a rotated frame. `vr_safety_node` now latches the first head-mounted-display pose and freezes past 20 mm or 2°. The freeze deliberately survives the ordinary unfreeze path, because poses flow the entire time the frame is wrong. `/vr/rebase_reference` is a deliberate act and tells the operator to re-calibrate.

6.3.2 Controller tracking loss froze nothing

`/vr/tracking_ok` was derived from the arrival time of the last frame, stamped *before* the per-controller validity check. It therefore caught a dropout, a sleeping headset and a backgrounded application — but not the case it is named after. Measured with the real Quest: covering a controller gave **zero** loss transitions while the stream ran on at 90 Hz. There is now a per-hand `/vr/controller_valid_<hand>`, and the mapper freezes that arm and refuses to engage it.

6.3.3 Nothing consumed the freeze

The safety node computed freezes that no node subscribed to. This is the project’s canonical “feature present but does nothing” defect: a value was being *stored* correctly and no consumer *read* it.

6.4 The observer e-stop

An interlock required `/vr/observer_estop_present` before real motion. Nothing ever published it, which made the interlock a permanent refusal — exactly the kind of gate somebody routes around. It is now a heartbeat: `scripts/observer_estop.py` publishes at 5 Hz, re-confirms every 30 s, and `vr_safety_node` treats two seconds of silence as *withdrawal*. It had previously been a latch, so a publisher that stopped left the system believing somebody was still standing there holding the button.

6.5 The mapper

`vr_pose_mapper` holds the arms while it runs, so the staging move cannot execute concurrently; the recording sweep stops it for staging and re-isolates it afterwards. Its more damaging defect — it did not reset between runs, which cost mode 02 every grasping task — is fixed in the node itself: `reset()`, a `/vr/reset` trigger service and a `/vr/reset_request` topic clear references, anchors, the filter and the thumbstick scale, and the mode isolation calls it on *every* run and refuses the mode if it will not answer. The accumulating term was the rate limiter, which is now published as `lag_m` so it can be seen.

A related visibility defect: the mapper published its state only inside the engaged branch, which made “disengaged”, “reset”, “frozen” and “the process died” one single observation. A 2 Hz idle heartbeat now carries the same keys with `engaged: false`.

Mode 02 still cannot grasp. All three of its grasping tasks fail; the pads miss by 88 mm rising to 206 mm, against a 30 mm gate, and the miss *accumulates*. The other four modes close at 0.0000 m.

6.6 One button starts VR

START VR TELEOP at the top of the GUI's RUN tab runs twelve steps in an order that is load-bearing — the simulation before the mapper, the certificate before the bridge, the port before the bridge — each with a plain-words failure message and the fix presented as a button. It is idempotent: pressing it on a running system says “already done”. It cannot reach a real arm, by construction and by test. `scripts/verify_vr_one_button.py` runs it for real with nothing typed: 22 checks, 0 failed, 10 of 10 machine-side steps, stopping honestly at the absent headset.

Three findings came out of building it, and all three are about self-inflicted faults:

- **“READY TO OPERATE” with no headset in the building.** The final step checked that the pose topic was *listed*, and the bridge creates its publishers at start-up — so the topic exists the moment the bridge does. The operator would have squeezed the grip, watched nothing happen, and had a window still saying ready. It counts message *arrivals* over a window now.
- **Clearing stale shared memory wedges the ROS daemon**, and the bring-up then blamed the simulation — false, specific and confident, which is the worst combination, because it sends you to the wrong log. Measured: after a clear, `ros2 topic list` returned 2 topics against a fully running stack.
- **Three repairs recreated their own fault.** Clearing shared memory restarts the daemon, which orphans the daemon's own segments — four repairs in a row. And the sweep destroyed a *starting* simulation, because a process coming up has segments it has not mapped yet, so they look unowned. Staleness is now asked of the kernel through `/proc/*/maps` with a 20 s age floor.

6.7 Passthrough

Passthrough (`immersive-ar` in the Quest's own browser) is built and is an *addition*. It adds nothing to the arm's latency, and that claim is structural rather than measured: there is one `requestSession` call site and one frame loop serving both modes, pinned by a test. It reports `environmentBlendMode`, because a granted AR session that composites opaque shows the operator a black room while the page believes it showed them the laboratory. In desk operation nobody wears the headset, so passthrough is worth nothing there — which is stated rather than quietly omitted.

CHAPTER 7

Perception

7.1 Detection is not the bottleneck

This is the most important sentence in the perception chapter and it is deliberately placed first.

On 2026-08-21 both real arms were driven repeatedly and the perception stack found the target cube **every time** — FastSAM at score 1.0, 1 166 depth points, zero false positives from the teal robots that had fooled a colour-only filter three times. **Nothing was picked up.** The reasons were calibration, kinematics, and eight defects in the code that commands the arms. A better detector fixes nothing that is currently broken.

7.2 What is built

Component	What it does
<code>prompt_detector.py</code>	Open-vocabulary detection from a noun phrase: YOLO-World (~60 M parameters) for boxes, FastSAM-s for masks, HSV plus depth as a no-model fallback. The backend is reported on every result , so a detection never silently implies the good one was running.
<code>rgbd_grasp.py</code>	Geometric grasp: principal axes of the depth cloud, close across the short axis, approach along the free one. No model, no GPU, and every refusal names a cause — no depth, wider than the 85 mm jaw, too few points.
<code>grasp_pipeline.py</code>	Reach and safety check: seeded IK, wearer floor, $\pm\pi$ seam margin.
<code>table_scene.py</code>	Support-plane RANSAC constrained near a given up-vector, so it fits a <i>table</i> and not the back wall; ties break to the lowest plane, because a cluttered table's object tops can out-vote the surface.
<code>segment_lift.py</code>	FastSAM cuts every region out of each frame; each mask is lifted through that frame's own depth; masks are split by height layer and nested masks suppressed.
<code>world_model.py</code>	Many views into one map: surface measured, objects fused by overlap, disconnected lumps split apart, fragments rejoined.
<code>joint_planner.py</code>	RRT-Connect in joint space with a <i>VoxelWorld</i> built from the map.
<code>wearer_tracker_node.py</code>	MediaPipe Pose, markerless, metric landmarks lifted by PnP.

Table 7.1: The perception stack.

Two design decisions are recorded with their reasons. The grasp is geometric and deliberately not learned: a parallel jaw on a rigid object is a solved problem, a learned model would need GPU budget that does not exist here, and it could not explain a refusal. And range is used as a discriminator where colour cannot be: the teal robots in the laboratory have a shadowed hue of 76 and the target green cube has 74 — two apart, inside noise — but the robots stand 2.8 m away while the rig works under 1.6 m, so range separates them cleanly.

7.3 Measure the environment, then plan against it

The operator’s own words prompted this work: *“there is so much space to calibrate, why can’t you create an algorithm to properly figure out the environment”*. The criticism was exactly right.

Almost everything in this repository that touched an object was **told where it is**. The pick-and-place task built its path from four coordinates in a source file; the table was at 1.250 m because a constant said so. All of it was verified at $N = 10$ over densified paths, and **none of it survives a different table, a different object, or the same object moved 50 mm**. The number in the file becomes a lie that the arm acts on confidently.

Three pieces that could have sensed instead already existed and had never been composed: the table analyser was used only to *print a description*; the planner’s `VoxelWorld` had been constructed nowhere outside its own self-test, so no path this project ever planned knew that the table was there; and `work_surface.set_measured()` existed precisely so the surface height could come from depth, and nothing called it.

7.3.1 The sweep: a printer probe, not a pan-and-scan

Two further pieces of operator feedback drove the design: *“the gripper and the whole joints move everywhere without the gripper being stable”* and *“imagine how a 3D printer works... our robot is in 3 space, it should map all the sides with all the lengths”*.

The first was the important one. The wrist had been *re-aimed at every cell*: the tool axis pointed at each grid point, so the orientation changed between neighbours 90 mm apart. The arm swung continuously and no two views were taken from the same attitude — which means a residual wrist error was a *different* error in every frame and could never cancel or be measured.

A printer’s probe does not re-aim. It holds one attitude and translates. So does this sweep now: one wrist direction per pass, with the camera grid being the work grid rigidly translated back along that fixed ray, so consecutive cells differ by a pure translation. Several passes at different yaws supply the sides, because a side is invisible from a single elevation. The sweep is $2 \text{ layers} \times 3 \text{ yaws} \times 24 \text{ cells} \times 2 \text{ arms} = 144 \text{ cells}$ and 13.4 m of travel, in a straight serpentine (1.140 m of travel against 1.702 m for the obvious alternative — measured, not asserted).

Stationary is not the same as arrived. A joint can sit 0.02 rad from its commanded value and still be moving through it, and a frame captured at that moment is smeared across two poses while every downstream statistic reports fine. `is_still` watches for 400 ms and requires no joint to move more than 0.0015 rad — a tenth of the arrival tolerance.

7.3.2 What the map contains

Quantity	Measured
Surface plane inliers	694 455
Surface height	1.25026 m (declared 1.250; +0.2 mm)
Surface tilt	0.036°
Objects recovered	6, every one exact in x, y to a millimetre
Object dimensions	every dimension within 3 mm
Cells swept	109; 74 reachable, 35 outside the envelope
Coverage of reachable cells	98.65 %
Stillness refusals	0
Sweep duration	13 min 15 s with the cache warm

Table 7.2: recordings/baselines/world_map.json. Objects are cut from the *picture* by segment_lift, not clustered from the point pile — pinned by test_objects_come_from_the_picture.py.

An earlier reported surface error of -5.2 mm was itself an instrument defect: `surface_z` was reporting the fitted plane’s intercept *at the origin*, which is a point inside the wearer.

7.3.3 Two lifetimes, one command

Bundling reachability and scene mapping into one command hid that they are facts about different things:

- `reachable_cells.json` is a fact about the **robot**: which camera poses the arm can reach. It is a one-time measurement, keyed on every bound, layer, facing, elevation and step, so moving the mount, changing home or editing the URDF invalidates it automatically.
- `world_map.json` is a fact about the **scene**. Somebody moves a cube and it is wrong immediately.

There is no incremental re-map: `build()` fuses every view from scratch, which is fine for “new cell, new table” and much too slow for “I put a different object down”. Until that exists, `pick_from_map` reports the map’s age on every run and **refuses to move off a map older than 30 minutes**. Planning from an old map is harmless and only reported; driving the arm at coordinates measured in a previous session is the failure that costs something, and a stale file looks exactly as authoritative as a fresh one.

7.4 Tracking the wearer

The clearance floor is enforced against a body. Until 2026-08-20 that body was always a mannequin. `wearer_tracker_node` replaces it, per part, with a measured one.

Quantity	Measured
MediaPipe Pose inference	28–34 ms per frame
Frame to estimate	61.6 ms median
End to end	62–562 ms (the spread is the <code>scene_hz</code> rate limit)
Objects held in the planning scene	twelve <code>wearer_*</code> primitives

Table 7.3: `scripts/measure_wearer_tracking_rate.py`.

The camera may only make the wearer BIGGER. That is the whole safety case in one rule. `fuse()` keeps whichever of the tracked and mannequin primitive is *closer* to the robot, per body part, with no mode switch anywhere. A body injected 0.10, 0.30, 1.00 or 5.00 m further away changes nothing.

Six faults were injected through the live node — dark frame, blank frame, two people, empty frame, occluded arms, plus a control — and the node refused 7 of 7 times, with each refusal required to *name* the injected fault. That naming requirement caught a false pass in which the two-person gate had never actually run.

The substitution is per part, so a measured torso can stand beside a mannequin upper arm in one scene. And the floor follows it: `mount_guard_node` used to build its wearer at import time, which meant the body MoveIt planned against and the body the 150 mm floor was enforced against were two different bodies, with the floor holding the mannequin. It subscribes now, through the same `fuse()`, so the floor cannot be loosened — silent, stale, gated-out and further-away all return the mannequin unchanged, and `/wearer/enforced` publishes which body is in force.

7.5 The scene camera, and what has never been attached

`scene_camera_node` exists, refuses rather than republishing when it has no data, and publishes a zero intrinsic matrix until calibrated. Both calibration tools have self-tests that pass exactly (focal length 0.43 %, principal point 4.5 px; extrinsic 0.0000 m and 0.000°). An attachment script finds the USB bus ID and prints the two `usbpd` commands.

No camera has ever been attached to it. The device is the laptop webcam; `usbpd bind` needs Administrator and is refused without it. Everything in this section is verified against mocks, and the repository keeps “as built” and “inferred” in separate documents for exactly this reason.

CHAPTER 8

Autonomy, language, and planning

8.1 Why a grammar and not a language model

Mode 06 takes a goal in English and does the rest. The parser is a grammar, and the decision not to use a local language model is recorded with its reasons: a grammar can explain a refusal, can be swept exhaustively over a phrase set with a scored outcome per phrase, and fits inside a 4 GB VRAM budget already occupied by the detector. The stronger version of the same argument is recorded against end-to-end vision-language-action policies, with an instruction not to reverse it later without reading the reasoning.

8.2 The task from a typed sentence

Parser	Correct	Asked	Refused	Misunderstood
Current grammar	34	14	27	0
Shipped grammar at commit b782a9b	21	14	37	3

Table 8.1: 75 phrasings, run through both parsers on the same cases; the old grammar was checked out of git and run in its own interpreter. `scripts/sweep_t1_instructions.py`.

Ten of the fourteen asks carry a scored reply, and all ten resolve correctly in twelve replies (two need a second turn). Replies are scored on their own line and are *never* counted as correct — an ask is an outcome in its own right, and the singular is load-bearing: the design position is that ambiguity should ask, and that the ask must be answerable in the same box.

The phrase set covers selectors, compound instructions, bare picks, politeness and misspellings. Two findings from building it are worth keeping:

- **The one misunderstanding was a compound instruction that executed half of itself.** That is the worst possible failure mode for a command parser, and it is why the current count of misunderstandings is reported as the headline number rather than the count of correct parses.
- **Politeness became a second action.** “Please” was parsed as a verb-like token in one construction.

A third defect sat underneath both: the grounding layer never read the declared colour, so a colour named in the sentence was not compared against the colour the camera saw.

8.3 The window is part of the parser

The GUI’s `Instruct` tab is not a text box that shells out. It shows, in order: what the parser understood *before* anything moves; what the camera detected, with the classifier’s own HSV

margin per object; the announced intention; and a confirm click. Live state comes from the runner’s own progress lines, and questions are answerable in the same box. The check that matters is pinned by `scripts/verify_prompt_panel.py`:

| Confirming hands the runner an argument vector carrying this task, mode 06, the detections it was planned from, and the sentence **verbatim**.

8.4 Voice

Voice reaches the same parser over `/voice_transcript`. The honesty constraint here is unusually strict and is repeated in three documents:

| `/dev/snd` on this machine contains only **timer**, so no microphone can be opened. Every voice number in this repository was produced from Piper-*synthesised* audio — word error rate 25.2%, zero exact transcripts, one voice, no room, no noise. **A voice score in this repository is a parser score, never a transcription score.** Speech recognition accuracy against a real speaker is unmeasured.

8.5 The mislabel control

An autonomy system that plans from a declared object property rather than an observed one will do the wrong thing confidently, so a control exists to prove which one is in force. A cube is declared green and rendered blue. The declared path sends it to the green pad at $x = -0.290$ on the *right* arm; the camera’s path sends it to the blue pad at $x = +0.290$ on the *left* — 0.580 m apart and a different arm entirely.

That control had been refusing to run since the task rebuild, because its own single-arm look cannot see a row that straddles the centreline. It now takes `--vision`: the staged two-arm detections that the recorded clip was itself planned from.

8.6 The planner that was built

`joint_planner.py` is RRT-Connect in joint space, with every design choice made for this specific robot:

- **Joint space, not Cartesian.** The binding constraint is the distance from the whole *arm* to the wearer, not from the hand. A Cartesian planner would solve IK at every node anyway.
- **The wearer floor is the collision check**, through the same moving-chain clearance the guard enforces — not the SRDF, which is blind to the pairs that matter.
- **Every edge densified at 0.05 rad**, matching the path checker. An RRT that checks only its nodes reproduces the streaming path’s defect one level up.
- **Straight line tried first.** Most moves are fine, and planning a clear move turns 60 ms into seconds.
- **Shortcut, then resample to the follower’s step.** Handing the follower waypoints further apart than its maximum step means it invents the intermediate motion with an unchecked straight interpolation — the exact defect the planner exists to remove, reintroduced at the last step.

The alternatives were assessed against the same 4 GB ceiling and recorded with sources: cuRobo is the strongest thing in the space and needs roughly 2.5 GB at batch 1024 on a 48 GB card, so it is the right answer on a bigger machine; Isaac Lab’s minimum specification is 8 GB and current documentation lists 16 GB; MoveIt Servo is the right *shape* for teleoperation and is worth evaluating; MoveIt’s own octomap is the right idea with an implementation known for voxels that never clear, which is why this project builds a per-frame `VoxelWorld` and deliberately does not carry occupancy between frames.

`pick_from_map` is the `VoxelWorld`’s first consumer outside its own self-test: pregrasp, grasp, lift, carry, place, retreat, with every coordinate taken from the measured map and fifteen narration phases published on `/robot_say` and shown in the window.

CHAPTER 9

The experiment programme

9.1 Four tasks, five modes

Key	Task	What it is
T0	Target reaching	Six labelled, coloured spheres; the arm moves between distinct positions.
T1	Colour-matched pick and place	Four cubes resting on a table, two coloured pads straddling the centreline at ± 0.290 m; each cube must end on the pad of its own colour, run by the arm that can reach that pad. Stage 2 randomises the <i>side</i> with a stored seed.
T2	Bimanual coordinated carry	A rigid tray held by both grippers with a ball on it; tilt and separation logged continuously through the carry.
T3	Circuit box and multimeter	Manipulation of two objects of very different size.

Table 9.1: The task set. Task keys `m0 m1 m1s2 m2 m3` are displayed as T0–T3; the archived `t*/e*` sets are refused by name so that old data cannot be silently mixed with new.

Each task carries numbered acceptance criteria — twelve for T1 alone — and each criterion is checked by an instrument rather than by inspection. T1’s criteria include ones that only exist because they were once violated: cubes must *rest* on the table rather than float; four separate close-and-open cycles rather than one; each cube moves only while gripped; the pads must sit at the innermost column that is both reachable and clear of the wearer, with the distance from the centreline *stated* rather than hidden.

9.2 What each task is blocked on

Task	Status
T0	Runs. Breaches the wearer floor by roughly 142 mm (left) and 137 mm (right), with 4 IK failures per arm.
T1	The healthiest task. Stage 1 walked three times sequentially and once independently: 0 IK failures, 0 breaches, worst clearance 0.1920 (L) / 0.2148 (R), pad miss 0.00–0.01 mm. Stage 2 is the same geometry with the side drawn; eight seeds walked, all clean. Only T1 stage 1 keeps the wearer floor throughout.
T2	Breaches by 150 mm / 147 mm, and its left arm fails one waypoint. Its right grip presents 113.9 mm of tray to an 85 mm hand, so that grip cannot close.
T3	Breaches by 88 mm on the left arm. The circuit box presents 195 mm of box to an 85 mm hand — 170 mm even top-down — and therefore cannot be grasped under any approach.
T4 (inter-arm handover)	Abandoned as a finding. The two arms' reachable sets are disjoint; the premise of the task is false.

Table 9.2: Task status. The grasp-geometry column is the one that decides feasibility: the hand opens 85 mm, and the object either fits across the closing axis or it does not.

9.3 The recorded clip set

The primary evidence artefact of the project is a set of rendered clips: every task under every mode, filmed from eight angles, each opening on the presentation pose, each carrying a title card.

Quantity (re-recorded 2026-08-23)	Value
Cells	17, at 8 angles
Grasp success	100 %
Positioning error	0.000 mm
T1 placement error	319.7 mm → 56.3 mm
Gripper motion	8/12 → 12/12
Real gaps	none

Table 9.3: `scripts/record_everything.sh`. The previous set is archived rather than deleted, because it remains the evidence for an earlier finding.

Two things the re-record found that nothing else could:

- **T2's grippers had never closed.** The elastic tray had been blamed and fixed; the fresh clips still read 0.000 rad for all 620–1219 samples on both arms in all five modes. The runner holds a grip until the pads *reach* the object — which is unsatisfiable for T2, which declares no grip object at all. It is applied on the spot now when there is nothing to arrive at: 0.0000 rad → 0.5176 rad.
- **The scene measured stage 2 at the wrong approach.** A pad-offset guard read `if self.task == "t1"`, and stage 2 — rebuilt on stage 1's geometry and commanding stage 1's approach — fell through to the anchor. That is 99.1 mm out on the left arm and

83.3 mm on the right, against a 30 mm capture gate. The runner commanded all four picks and both grippers closed; the scene recorded two grasps and filed “2 object(s) never moved”. A broken measurement that read as a flaky task.

9.4 T2’s tray, and unfalsifiable criteria

T2’s tray was drawn as the *live* gripper separation plus 60 mm, so it stretched from 0.487 m to 1.211 m against a 0.560 m specification. Two of the task’s four acceptance criteria — “tray held by both grippers” and “ball visible on the tray” — therefore could not fail, because the tray deformed to meet whichever hands existed. It is now drawn at the specification length and oriented along the gripper line, and the ball falls off past 6.8° of tilt and stays off.

■ This is the archetype of a whole class of defect in this project: a criterion that passes because the thing being measured was built to fit the measurement.

9.5 Three instruments that reported defects that did not exist

- The accuracy table mapped objects to gripper pads with ± 2 , a single-arm assumption, and therefore scored two cubes against the pad 0.58 m away.
- The status table called T1 under four modes a real gap, when T1 runs under mode 06 only.
- The gripper-motion verifier demanded a release from a carry task that never releases.

All three now read the task’s own declaration. And a bare `except` written into the third fix reproduced the same class of fault immediately, which is recorded because it is the kind of thing that is easy to omit from a write-up.

CHAPTER 10

Safety engineering

10.1 The layered defences

Layer	What it does
<code>estop_node</code>	Parks the arms first, then talks to the driver.
Dead-man	Keyed on the <i>source</i> timestamp, not on arrival.
<code>block_monitor</code> / <code>blocking_aggregator</code>	Central blocking-condition registry with first-out annunciation and automatic expiry.
<code>mount_guard_node</code>	Enforces the 150 mm wearer floor against the fused body.
<code>participant_safety_node</code>	Session limits for participant runs, separate from developer limits.
<code>real_arm_gate</code> / <code>preflight</code>	Arming, stray-process detection, continuous-joint winding check.
<code>recovery_manager</code>	Recovery paths (b)–(e), each injected and tested.

Table 10.1: The safety layers.

Never wait for a service inside the e-stop trip path. Doing so once blocked the e-stop for 4.0 seconds. Park first, then talk to the driver. Detection was never late — only the halt was, which is a distinction that took a trace to establish.

10.2 Fault injection

The project’s confidence in its safety layer comes from injection rather than from reading. The injection suite went 8/10, then 10/10, then **14/14**, and six full experiments were run with faults injected mid-trial and all six passed.

Four harness bugs were found during that work, **all of which made a failed injection look like a pass**. That is the transferable lesson: a fault-injection harness is itself an instrument, and it must be cleared before its verdicts are believed.

Two structural fixes came out of it:

- **The dead-man’s real hole was the source timestamp.** It could not see the failure that actually happens — a publisher that keeps publishing stale content with fresh stamps.
- **An expired blocker is not permission to move.** Automatic expiry of a blocking condition had been treated as clearance; it is now an explicit state that must be resolved.

10.3 The latch pattern

Several components had a latch that could not be reset by the disappearance of the thing it was latching — the observer e-stop believed somebody was present after the publisher stopped; the follower’s e-stop block was invisible exactly when it mattered. The systemic fix was to convert presence latches into heartbeats with an explicit silence timeout, and this pattern now appears in three separate nodes.

10.4 Anonymity, enforced in code

`write_manifest()` raises on `name`, `email`, `dob`, `address` and `phone`. This is a code-level constraint rather than a policy document, because a policy document does not stop a keystroke.

10.5 And a demonstration is not evidence

Demonstration clips carry that caveat in their caption and produce no trial data, no condition and no metric. This is hard constraint 13, and it exists to stop a persuasive video from being cited as a result.

CHAPTER 11

The window

11.1 The rule

On 2026-08-22, at the operator's request, the project adopted a rule that changed how every subsequent feature was delivered:

The window is the interface. Every capability reaches it, and every change is checked in it. A feature reachable only from a terminal does not exist for the person running the session.

Four clauses follow from it. If you add or change a capability, it gets a control in the GUI *in the same change*. A press must leave a trace, and a control that changes what the next run does must say so — which is why every toggle, drop-down and spin box logs its new value. After pressing everything, the window must still work: the audit re-checks the layout, the tabs, the e-stop and the instruction box at the *end* of its sweep, because a control that works on a fresh window and stops working after somebody explored the panel is a real defect and the audit is the only thing positioned to catch it. And you must run the audit before claiming anything works.

11.2 What the window is

One command — `bash scripts/start_gui.sh` — sources everything, refuses a second window, and opens the GUI. RViz, showing the *commanded* state, is embedded where a window manager exists (detected from the X root window, not assumed), beside a native panel drawn from the `real_*` frames showing the *actual* state, with a divergence readout under both. The left column is grouped SET UP / RUN / STATUS. A connection panel names eight faults in plain words and offers the fix as a button.

11.3 The audit, and what it found

Audit generation	Checks
First passing run (2026-08-22)	91, 0 failed
After RViz embedding and the connection panel	163, 0 failed
Current	257

Table 11.1: `scripts/verify_gui_buttons.py`. Since 2026-08-22 it checks not only that a button can be pressed but what the press *did*: the e-stop must reach `/estop`, service buttons with no stack must say so, and the experiment panel's own defaults must produce a command the dispatcher accepts.

Getting to the first passing run found four defects, and the way they hid is the point:

- The audit had **hung on a modal consent dialog** ever since that dialog was added, so it never reached the presses that would have revealed the rest.
- Gui had no `log` method, so five session buttons raised an exception inside a Qt slot — which PyQt swallows — and did nothing, silently.
- `proscan` was never imported, so the stray `robot_state_publisher` preflight check — hard constraint 3's own guard — had never run.
- The Launch panel rendered three of its four groups, so one whole group of buttons did not exist.

11.4 And look at it

Three defects lived for weeks behind a passing audit: RViz embedding an *invisible* window; a control column capped below its own content; and every mode button below the fold. **None of them is visible from a return code.** The rule is now to launch it and screenshot the window — and to grab the window by ID, because a grab of the X root records black under WSLg.

CHAPTER 12

Verification methodology

This chapter is the one most likely to be useful outside this project.

12.1 The instrument failure catalogue

Every row below was a real incident here. The right-hand column is the mechanism, which is what makes the row reusable.

Symptom	Mechanism
A statistic is 0.000 everywhere	The recorder oversamples a slower sensor, so adjacent rows are identical.
Zero variance	A dead channel, or a cached value being republished.
Data is fresh but never changes	Stale content republished with a <i>new</i> timestamp. Key on the source stamp.
A learned model scores near zero	The synthetic renderer is out of distribution.
A metric swings between runs	It is a minimum over directions; use medians.
Everything matches	Substring or prefix matching (<code>t0</code> inside <code>t001; t1s2.startswith(t1)</code>).
Results depend on run order	Scene or state persisted; <code>remove_furniture()</code> clears only <i>its own</i> ids.
Every pose returns one value	Forward kinematics is evaluating the <i>current</i> state and ignoring the solution it was given.
A feature is “present” but does nothing	The check verified that a field is <i>stored</i> , not that a consumer <i>reads</i> it.
A gap that is not a gap	A by-design absence rendered identically to a defect.
A clip verifier fails a good clip	It matched the <i>requested</i> RGB, not the <i>rendered</i> colour — RViz shades everything.
A test breaks with no behaviour change	The test sliced the source “before the first <code>def</code> ”.
A success count is identical either way	A tolerance so loose that it binds nothing; measure the achieved value instead.

Table 12.1: The instrument failure modes seen in this project. The repository’s own instruction is to check this list first.

12.2 Worked examples

Four are worth stating in full, because the mechanism in each is subtle.

12.2.1 A render check that could not fail

Removing a bounding box left a label reading its corners, so the render tick raised an exception on frame 1 and the scene came up with no table, no cubes, no pads and no workspace marking. The check filed it as OK at 18 % ink, *because the wearer is 18 % of the frame* — and a real scene reads 46–50 %. The renderer now subscribes to `/task_objects` and refuses to shoot without the task’s own namespaces present.

12.2.2 The test gate could invent a failure

`check_tests.py` matched `^FAILED (\S+)` over the whole pytest output — and the docstring checker *echoes* the source lines it objects to. So the string `FAILED = "failed"` appearing in a source file became a failing test named `=` on a suite where everything passed.

■ A gate that cries wolf is a gate that somebody bypasses. It is now pinned by `test_the_test_gate_cannot_invent_a_failure.py`.

12.2.3 The frame extractor never extracted a grasp

It mapped event wall-clocks through the *scene node*’s start time. Under mode 06 that is about 300s before the video capture begins, so every grasp event landed at +380s inside a 48s clip and was skipped in silence. It printed “10 frames” and exited 0 — openings and final frames only. The sweep now writes a capture clock into `clip_meta.json`, and the extractor refuses a clip it cannot map and fails when no grasp frame comes out.

12.2.4 Six verifiers asking the wrong question

The single most expensive instrument failure in the project’s history:

- `verify_msc_tasks.py` had raised `AttributeError on import` since T1 became two-armed — it had not run at all.
- About twenty scripts solved IK at the live *home* wrist, which is 42.9° from the anchor the task runner actually commands. One task’s own pick reads 10/10 at the anchor and 0/10 at home.
- They asked for an *exact* orientation while the follower has run a 15° cone since 2026-08-22. Correcting that took T1 stage 1 from 32 IK failures to 0, and the choreography routine’s total from 171 to 68.
- `normalise()` returned radians while its two sibling functions took degrees.
- `verify_t1 --stage2 N` overwrote the stage 1 baseline file.

12.3 Two clients, one planning scene

A verifier run that overlapped a still-running sweep read 18 failures of 171 where two clean runs read 0 — the sweep was applying and removing its own collision slab in the same planning scene. This is hard constraint 3 one level down: never run a sweep and a verifier against one `move_group`.

12.4 The test suite

Measure (run 2026-08-26)	Value
Test modules	114
Tests passed	1 322
Skipped	1
Failed	3
allowed (test_flake8, test_pep257)	2
not on the allowlist	1
Wall-clock	234.99 s

Table 12.2: Live result of `python3 scripts/check_tests.py`. The two allowed failures are pre-existing style checks (the package uses double quotes); the gate knows the allowlist and refuses anything else.

12.4.1 A live finding from running the gate for this report

The gate refused, and it was right to. `test_home_has_one_source.py::test_urdf_and_config_agree` fails on the current working tree:

Left arm	j1	j2	j3	j4	j5	j6	j7
URDF (working tree)	1.1619	0.9974	2.2173	-2.4289	-2.6699	-2.1000	-0.1308
config/home_positions_left.txt	1.1568	0.9826	2.2323	-1.6462	-2.6840	-1.1242	-0.1194

Table 12.3: Worst disagreement 0.9758 rad on joint 6, against a tolerance of 0.001 rad.

The cause is an uncommitted edit from the 2026-08-25 hardware session: the URDF’s spawn blocks were changed to a manually set pick pose (left) and a pose captured from the real arm (right), and the configuration file — which is what every node actually loads, including the bridge’s home gate — was not changed with them.

This is precisely the drift that `test_home_has_one_source.py` was written to catch, working as designed, on the day it mattered. It is reported here rather than fixed, because deciding *which* of the two poses is intended to be home is a decision about the hardware, not about the code.

12.4.2 The test names are the documentation

The suite is named in sentences rather than in symbols, and reading the file list is a fair summary of what the project believes about itself:

```
test_a_grip_with_no_object_is_not_left_pending.py,
test_block_expiry_is_not_permission.py, test_clearance_guard_cannot_pass_on_no_data.py,
test_closed_on_nothing_is_not_a_grasp.py, test_objects_come_from_the_picture.py,
test_objects_on_table_can_fail.py, test_readiness_never_green_on_unknown.py,
test_the_grasp_pipeline_looks_before_it_says_no.py, test_the_planner_goes_round_not_through.py,
test_the_robot_maps_before_it_moves.py, test_the_test_gate_cannot_invent_a_failure.py,
test_wearer_presence_is_explicit.py.
```

CHAPTER 13

The real-hardware campaign

Nothing in this repository had ever run against a real arm until 2026-08-20. Six days of hardware work followed, and they produced more findings per hour than any comparable period of simulation work — almost all of them about tooling rather than about robotics.

13.1 The first real-arm session, 2026-08-20

The sequence that worked is recorded as an ordered procedure: tear down everything, launch with the master disabled, prove discovery, bring up *one* arm, then the VR chain, then the observer last, then the headset. Alongside it is a trap table in which every row cost time:

- The launch script swept the ROS daemon’s own shared memory.
- `timeout N ros2 ...` sends SIGTERM, which kills DDS participants mid-handshake.
- A deaf observer — the e-stop gate did not gate the thing it is named after.
- **Two arms cannot share the Kortex high-level path over WSL.** Measured at 8–12 Hz with 519 ms spikes for two arms, against 21 Hz for one.
- The wearer clearance check was measuring almost nothing.

13.2 The cube pick, 2026-08-25

The first sustained attempt to drive a real arm from its own gripper camera. The grasp geometry was never the hard part; every hour went to instruments that reported confident wrong answers.

13.2.1 The finding that governs everything else

The gripper camera’s extrinsic calibration is wrong by about 8.5° , and it rotates with the wrist. The table cannot move, so the table normal measured from any arm pose must be the same vector. Measured from four poses: 9.74° , 15.42° , 16.26° and 18.14° off world vertical — a worst pairwise disagreement of 8.45° , which is 34 mm of position error at 0.23 m and 59 mm at 0.40 m. No open-loop grasp planned through that transform can land inside the 30 mm capture gate, and none did.

The depth camera itself is excellent and was never the problem: the table plane fits at 94.8% inliers with 0.46 mm RMS, and a real 40 mm cube measures $39.2 \times 38.9 \times 40.9$ mm. Only the transform out of the camera is bad. The pick therefore servos closed-loop in the camera’s own frame, where the data is good, and nulls the pad-to-cube error by iteration — because a closed loop converges through a wrong extrinsic and an open loop cannot.

13.2.2 Seven instruments that produced a false pass

Instrument	What it claimed	Ground truth
ClearanceModel via link origins	367.0 mm of clearance	The arm was physically against the mannequin — and it returns 367.0 for a clear pose too. A constant dressed as a measurement.
/real_status_*	Pose matched the saved file to 0.144°	Published by a leftover homing node republishing a <i>frozen</i> snapshot with fresh stamps after its input died.
in_contact / force_n	25–41 N, “in contact”	No free-space baseline exists, so it cannot separate contact from gravity and payload.
A fixed 2.5 s settle	“Did not arrive, 7.2° out”	The bridge closes error at $k_p = 0.5$ (about a 2 s time constant); it was still converging.
Joint error by subtraction	“STALLED at 360.076°”	Joints 3, 5 and 7 are <i>continuous</i> : -179.05° and $+181.02^\circ$ are the same pose. The run had arrived.
self.q read in a publish loop	“Moved 0.032° — arm not responding”	<code>spin_once(timeout_sec=0.0)</code> starves the subscription; telemetry showed the arm had moved about 7°.
Table-plane-only check	path Every leg cleared by more than 100 mm	It checked the hand against the <i>table</i> . The 94° transit swung the arm into the wearer.

Table 13.1: Real-hardware instrument failures, 2026-08-25.

13.2.3 Defects in the vendored camera stack

- The vendored `kinova_vision` launch file publishes a static transform putting `camera_link` to `depth` at $(-0.0195, -0.005, 0)$ against the robot description’s $(-0.0275, -0.0096, 0)$ — 8.0 mm and 4.6 mm apart, and two writers on one static edge.
- Both arms publish the *same unprefixd camera_link*, so left and right collide in one global transform tree. A `launch_tf` argument (default true, behaviour unchanged) now allows the frames to be remapped onto the description’s.
- `loadCameraInfo()` declares its parameters unconditionally and runs again on reconnect, so all four camera nodes abort with `ParameterAlreadyDeclaredException` the moment a stream recovers.

13.2.4 What the pick pipeline now does

Two commands: `bash scripts/bringup_arm.sh left` (one Kortex session, the camera on the description’s frames, idempotent), then `python3 scripts/srl_pick_cube.py`, which finds a view-point that actually *sees* the cube, measures the table plane and the cube in the camera’s own frame, servos the pads onto the cube, closes, lifts and verifies.

Three geometric facts are built into it. The wrist camera cannot see its own grasp — the last 80 mm push the cube out of the bottom of the frame, which is geometry and not failure, so the last good fix is carried in world coordinates and the remainder completed from it. All joint arithmetic goes through `ang_wrap`. And motion is judged only through a freshness-checked joint read.

The outstanding safety gap, stated plainly: the wearer clearance model samples link *origins* only, so it misses the arm’s body between joints. Until it measures link *geometry*, a “pass” from it means nothing — and hard constraint 11 is therefore currently unenforced in code.

CHAPTER 14

How good is it, honestly

Every number in this chapter is regenerated by `python3 scripts/measure_control_budget.py`, with the output stored in `recordings/baselines/control_budget.json`.

14.1 The short answer

Question	Verdict
Can it see a table and say what is on it?	Yes — any height 0.40–1.40 m to 0.1 mm; object centres to 0.9 mm
Can it plan a safe grasp?	Yes , and it refuses unsafe ones by name, including a sweep whose two endpoints are clear and whose middle passes through the wearer
Can the arms move freely?	Mostly, now — 0.065 to 0.304 m mean reach at the work point since the wrist stopped being pinned
Will the hand land where it was told?	Not inside the grasp gate — worst case 33.4 mm against a 30 mm gate, and one term is unmeasured
Does the guard know where the wearer is?	Not in time — in 5 of 6 motion cases the limb travels further than the entire floor before the guard hears
Is the assumed posture safe if tracking drops?	No — arms held out to the sides give 93 mm at home, a 57 mm breach, while the fallback assumes 317 mm

Table 14.1: The system, assessed 2026-08-22.

14.2 The positioning error budget

Term	mm	Kind
Declared vs measured pad offset (T0, T2, T3)	13.45	systematic
Terminal joint error, uncompensated	7.24	systematic
Shell bias from one view, <i>before</i> the plane	10.5	systematic
the same, <i>with</i> the support plane	0.0	—
joint error, <i>if</i> the overshoot is switched on	1.88	—
Depth noise at 0.4–0.8 m	2.0	random
IK solve residual	0.2	random
camera_link vs the physical module	UNMEASURED	systematic
RSS of the measured terms	18.64	
Worst case (they add)	33.39	
The grasp capture gate	30.0	

Table 14.2: The control budget. It improved from RSS 18.64 and worst case 33.39 after the pad-offset derivation and the shell-bias correction; before those it was worse still.

Read that as: **the system is over budget on paper, and two of the three largest terms already have fixes sitting switched off.** With the support plane in the path (0.0 mm) and the terminal overshoot enabled (1.88 mm), the worst case falls to roughly 17 mm — comfortably inside — *provided* the unmeasured term is small.

The unmeasured term is the one to go and measure. The wrist camera’s pose is forward kinematics, which is exact given the robot description, and nothing has ever checked the description against the physical module. A 10 mm error there is invisible and lands in every grasp. The 2026-08-25 hardware session sized the *rotational* part of it at 8.45°, which is worse than a 10 mm translation, so this is now the cheapest high-value calibration remaining in the project.

14.3 The reaction budget

Tracking latency is 62–562 ms end to end. Limb speeds below are literature ranges and are labelled as assumptions rather than measurements.

Limb motion	Latency	Travel	Vs the 150 mm floor
A deliberate reach	62 ms	50–93 mm	62 % of it
A deliberate reach	562 ms	450–843 mm	exceeds it
A quick reach or fidget	62 ms	93–155 mm	exceeds it
A quick reach or fidget	562 ms	843–1405 mm	exceeds it
A startle or flinch	62 ms	155–248 mm	exceeds it
A startle or flinch	562 ms	1405–2248 mm	exceeds it

Table 14.3: Five of six cases move further than the whole clearance floor before the guard knows.

The floor is a *distance*, not a reaction time, and nothing in this system reacts to a wearer who moves quickly. That is the single most important control problem in the project, and it is not a software bug — it is the consequence of a 150 mm margin, a roughly half-second perception chain, and a person who can move a metre in that time.

Three things would help, in order of honesty, and none is done:

1. **Inflate the floor by the reaction budget.** The guard should keep $\text{floor} + \text{latency} \times \text{assumed speed}$, not a fixed 150 mm. That is a one-parameter change and it makes the margin mean something.
2. **Cut the latency.** 562 ms is the rate limit, not the tracker — the tracker itself is 28–34 ms per frame. Most of the spread is recoverable.
3. **Slow the arm near the person.** Speed is already a live parameter and clearance is already computed. Nothing currently couples them.

14.4 The failure mode of losing tracking

`fuse()` is correct about its own rule — the camera may only make the wearer bigger — and it falls back to the mannequin when the estimate is stale, silent or gated out. **That fallback is the gap.** If the wearer has raised their arms and tracking drops, the guard reverts to the assumed `down` posture and believes there is 317 mm of clearance where there is 93 mm.

The system's current answer is an instruction to the wearer, recorded as hard constraint 11. That is a real mitigation and it is not a control system.

Markerless tracking here is not a nicety; it is load bearing — and its failure mode is to quietly restore an assumption that is 224 mm optimistic.

CHAPTER 15

What is open

15.1 Blocked on the laboratory

- Seven of fourteen master channels are incoherent — a soldering problem, with `l_j2` and `l_j4` named by regression.
- The two arms' reachable sets are disjoint, so any handover task needs the right arm re-parked in hardware.
- The right arm's home joint values have never been read from hardware.
- Two real Kortex sessions have never been opened simultaneously (and the measured rate says they should not be).
- Detection rate at working distance is unmeasured.
- Speech recognition against a real speaker is unmeasured, and cannot be measured on this machine.
- There is no `adb`, and this machine is not on the laboratory network.

15.2 Blocked on ethics

No human data has been collected at all. Operator and wearer are different people and consent separately. Worn operation means more than 17 kg on somebody's back with no gravity compensation, moving under partly autonomous control, within reach of a head and torso, driven by a person who did not choose the wearer's motion. The ethics document describes the study as it is actually built, *including the parts that are not ready*, on the stated principle that an ethics application which overstates readiness is the wrong kind of document.

15.3 Blocked on measurement

Ranked by how much they cost if left alone:

1. **The gripper camera's extrinsic calibration.** Sized at 8.45° of wrist-dependent rotation; it is the largest term in the positioning budget and it is the reason no open-loop grasp has landed.
2. **The wearer clearance model.** It samples link origins, not link geometry, and returns the same number for an arm inside a person and an arm in free space. Hard constraint 11 is not enforced in code.
3. **The reaction budget.** The floor does not include the latency term.
4. **How the terminal joint error varies with speed.** Never measured; the run that was reported as a speed sweep was three replicates of one condition.

5. **The size of the between-modes difference.** The effect is real and its magnitude is unknown, because the recorded set cannot separate mode from the node set each mode brings.
6. **T1’s exact grasp**, which is refused by T1’s own table: it solves 10/10 with no furniture and needs a 5.0° tilt with the table present. The follower’s cone supplies the tilt, at a cost of 9.58 mm measured against 8.58 mm predicted — inside the gate, and not zero.

15.4 Built but not yet fed, or not yet wired

- `/viz/envelope` and `/viz/refused_in` are built and tested and not yet published to.
- Predictive avoidance has a real null-space implementation — Jacobian by finite differences, closed-loop task-space correction with an end-effector residual of 0.00001 m — and is worth 0.0 to 5.6 mm. Every one of 28 refusals names the end-effector link, because the *hand* is what is inside the person and the null space holds the hand fixed by definition. It is not wired into the follower, and now for a better reason than before.
- An incremental re-map. The world model rebuilds from scratch, which makes “I moved one object” a 13-minute operation.
- The LeRobot export exists and is worth having; the policies are blocked three independent ways (no corpus, 4 GB VRAM, and a policy would learn around the unmeasured extrinsic rather than fix it). Twenty-three episodes and 20 440 frames of real teleoperation are exported, with the eight channels the newest baseline calls incoherent or dead dropped and *named* in the dataset’s own metadata.

Timeline and conclusion

16.1 How the project actually went

Period	What happened
2026-07-14	Initial import.
Late July	Teleoperation, collision safety with the wearer in the model, gripper root-cause work, e-stop, first real-robot bring-up documents. The finding that the arms cannot reach a table as originally mounted.
2026-08-05	Restructure into six packages with one-way dependencies. The mount orientation defect found and fixed — twice, the second time by derivation. The proximal collision identified as a mechanical finding. Hardening by fault injection: 8/10, then 10/10, then 14/14.
2026-08-06 to 08-08	Dual arm; the fifth collision, invisible in simulation. Trajectory capture, and the discovery that the recorder oversampled the sensor and therefore faked every per-row statistic. The two-person reframing: operator and wearer become separate participants.
2026-08-09 to 08-13	The heaviest building period — 60, 52 and 60 commits on successive days. GUIs, VR on the vendor protocol, full autonomy and the six modes, the holographic arm panel.
2026-08-15	The workspace measured against the wearer for the first time. The home pose becomes the presentation pose. The centre searched properly and refused. Predictive avoidance built, measured, and found to be worth almost nothing — twice.
2026-08-16 to 08-18	The task set re-recorded. T2's elastic tray. T1 from a typed sentence. The grasp constant that had been 13.47 mm wrong since it was written.
2026-08-19 to 08-20	The VR path over Wi-Fi, and the discovery that its bridge had never once started. Desk operation reframed as motion capture. The first real-arm session.
2026-08-22 to 08-23	The GUI rule. The motion planning research. Ruckig. The sweep, the segmenter and the world model — measure the environment, then plan against it. The accuracy pass, and six verifiers found to be asking the wrong question.
2026-08-24 to 08-25	The window audit and its remaining defects. The cube pick on real hardware, and seven instruments that lied.

Table 16.1: The project in eleven phases.

16.2 What was built

A complete shared-autonomy stack for a two-armed wearable robot: three input routes (a physical master linkage, six-degree-of-freedom motion capture, and natural language), five operating modes, a perception pipeline that measures its environment rather than being told about it, a jerk-limited motion generator, a joint-space planner whose collision check is the wearer rather than the SRDF, a safety architecture built on heartbeats rather than latches, a single operating window through which every capability is reachable, and a verification apparatus of 114 test modules, 233 analysis scripts and 195 recorded baselines.

16.3 What was learned

The engineering is replaceable; the method is not. Three lessons generalise beyond this robot.

- 1. A surprising measurement is evidence about the instrument.** Seventeen times in six weeks, the surprising result was the instrument and not the system. The cost of checking the instrument first is minutes; the cost of not doing so was, on several occasions, a day.
- 2. A check that cannot fail is worse than no check,** because it converts an unknown into a false belief. Several of this project's checks passed for weeks while measuring nothing: a clearance floor compared against a constant that included the immobile mount; a tray that stretched to meet whichever hands existed; a render check satisfied by the wearer's own pixels. Every check here now has an argument that makes it fail on demand.
- 3. Say what is unmeasured.** The single most useful line in the control budget is the row that reads UNMEASURED, because it is the row that tells the next session what to go and do. A budget with a guessed term in it would have looked better and would have hidden the 8.45° extrinsic error that was costing every grasp.

16.4 Where it stands

The system sees well, refuses well, and explains itself well. It does not yet place a hand accurately enough to close a grasp from an open-loop plan, and it cannot react to a person who moves quickly. Both of those are named, sized, and have a next action. Nothing has been worn, and no human data exists; that is a deliberate ordering, not an oversight.

CHAPTER 17

Every failure

This chapter is the project’s defect register: every fault found between 2026-07-14 and 2026-08-26, in the order it was found, with its mechanism and its resolution. It is assembled from the 7 099-line diagnostic history, the commit log and the test suite.

Three kinds of entry appear, and they are deliberately not separated, because in this project they were rarely distinguishable at the moment of discovery:

- **System defects** — the robot, the safety layer or the task did the wrong thing.
- **Instrument defects** — the measurement was wrong, and the system was not. *These outnumber the first kind.*
- **Findings** — nothing is broken, and the platform cannot do the thing that was asked.

Entries are numbered **D1** onward and referred to by number elsewhere in this report.

17.1 2026-07-31 — the first build

Teleoperation, collision safety, the gripper, the e-stop, and the first attempt at a real-robot bring-up.

D1 Commanded pose is not a proxy for the end effector under motion

The only recording with a moving master (right arm, 2 097 samples) measured the software mapping at gain +1.034 and the *actual* end-effector gain at +0.100: tracking RMS 0.0396 m, max 0.2956 m. Every earlier sub-millimetre tracking figure had been taken with the master at rest and proved nothing about tracking. *Mechanism:* that recording ran with **anchored** orientation, where IK was failing over 90 % of the time, so the follower had almost nothing to publish. *Consequence:* no experiment may use commanded pose as a proxy for end-effector pose; all position measures come from tf2.

D2 The wearer was invisible to MoveIt

`avoid_collisions=True` was set on every IK request, and `human_backpack.xacro` carried 27 `<visual>` elements and exactly *one* `<collision>`; no human link appeared in the SRDF at all. MoveIt cannot avoid what it cannot see, so the flag had been doing nothing for the person wearing the machine. *Fix:* 13 collision elements added, paired with 36 SRDF exclusions for the permanently-adjacent mount links, with everything distal of `half_arm_1` left enabled.

D3 The right gripper exported zero command interfaces

`right_robotiq_85_left_knuckle_joint/position` read [unavailable] [unclaimed]. The URDF was correct. *Mechanism:* the vendor macro declared `<gpio name="reactivate_gripper">` *unprefixed*, so on a dual-arm robot both hardware components registered the same GPIO interface names and the second to load exported nothing. An earlier vendor patch had prefixed the hardware *name* and missed the GPIO.

D4 Three independent clutch glitches

Button bounce produced double toggles; a *moving* reference could be latched; and pre-disengage filter state bled into the first frames after re-engagement. *Fix:* 50 ms debounce, a quasi-static gate (2 mm over 100 ms), a reference averaged over five frames, and an explicit EMA reset on engage.

- D5 Both Robotiq drivers defaulted to the same serial port**
Both real gripper drivers defaulted to `COM_port=/dev/ttyUSB0` and would have fought over one device the first time real hardware was attached. Found by a full scan of `<gpio>`, `<joint>` and `<sensor>` names across all four `ros2_control` components.
- D6 The task volume in the brief is not reachable**
The volume named in the project brief measures 5.6% IK success with collisions on and 11.1% with them off. The same $0.5 \times 0.3 \times 0.2$ m box moved to the arm's own side at chest height measures 77.8–100%. *Finding, not defect:* a back-mounted arm at $z = 1.25$ cannot work a table at $z = 0.75$ – 0.95 across the body centreline.
- D7 The right arm's home sat 5° from the $\pm\pi$ seam**
Seam margin 0.087 rad on `joint_5`, where a wrap executes as a 358° rotation, with 29.1° of headroom against a 30° target. *Fix:* home replaced by the left pose mirrored about $x = 0$ — mirroring the *pose* and solving IK, not negating joints, because both arms use the same non-mirrored vendor macro. Seam margin $\rightarrow 1.268$ rad.
- D8 No table height reaches 80% IK, and the binding constraint is not reach**
Best 77.8% at chest height; 16.7% at table height. Of 18 table-height samples: JOINT/IK 9, orientation 1, collision 1, reach 1. The Gen3 spans the distance easily (0.74 m required against 0.902 m of reach) and cannot *fold* into the required configuration inside its joint limits. No mount orientation fixes that, because orientation does not change how far the arm has to fold.
- D9 Tilting the mount down makes it worse**
Swept without touching the URDF (rotating the base by δ is exactly equivalent to rotating targets by $-\delta$). Aiming the base downward points the arm into the wearer's body, so it must reach further round. The one angle that improves both table and chest reach ($+0.3$ rad) was recorded and deliberately not applied, because applying it invalidates `P_HOME`, the anchor and every clearance figure.
- D10 Pointwise IK scoring chose the worst home**
The right home had been selected because it scored 100% *pointwise*. Under continuous reachability — each step seeded from the previous solution, as the follower does — it is the worse of the two arms: 15 of 26 directions usable, isotropy 0.32. *Mechanism:* pointwise IK asks “could the arm get there from anywhere”; teleoperation asks “can it get there from where it already is, without a branch switch”.
- D11 MoveIt's IK plugin ignores the constraints field**
An `OrientationConstraint` of $(0.2, 0.2, \pi)$ returned *exactly* the same IK success as an exact pose — left 79.3% both ways, right 68.9% both ways, over 270 calls. Identical to the digit is not a coincidence: this `/compute_ik` setup is not constraint-aware. The tilt orientation mode was therefore implemented and not shipped.
- D12 real_robot mode had never successfully run**
`ik_follower_node` read `self.max_vel` nine lines before assigning it, raising `AttributeError` on every `real_robot:=true` start. *And the fix exposed a second defect:* once the crash was repaired, the later line would have *overwritten* the 0.10 rad/s safety cap the block had just applied. A clamp discarded one line later is worse than no clamp, because the warning still prints.
- D13 New launch files were silently not installed**
`setup.py` listed launch files by hand, so `mock_real.launch.py` and `real_arms.launch.py` did not install and did not error. Replaced with a glob.
- D14 Kortex driver deactivation is one-way**
Deactivating the hardware component tears down the API router; `on_activate` then fails permanently with `KBasicException: Router is not active`. *Consequence:* the driver-level e-stop cannot be undone in-process — recovery is a relaunch, and the gate is built around that.

D15 **/real/joint_states is frozen while the component is inactive**

313 samples over 5 s gave one distinct value per joint, peak-to-peak 0.000e+00, identical to a reading taken 40 minutes earlier. The broadcaster keeps publishing at 100 Hz and the content is a cache. Any hold-versus-limp test done through that topic reports “holding” regardless of what the arm does. Same zero-variance signature that identified the dead j7 potentiometer.

17.2 2026-08-05 — the mount, and the hardening pass

Two mount fixes, the proximal collision, fault injection from 8/10 to 14/14, and the two safety defects that had hidden for months.

D16 **The mount angle was in the wrong rpy slot**

At the legacy home the real arms point forward; in simulation the same angles put the left hand 0.49 m behind the wearer and the right hand 0.24 m above a 1.75 m person’s head. *Mechanism:* the mount was `rpy="0 ±0.6 0"`, and in this file’s convention x is lateral and y is fore/aft — so a rotation about y is a *lateral splay*, not a forward pitch. The old mount splayed both arms 34.4° outward with zero forward tilt. *The tell:* the hand-written sign flip on the right arm. A pure forward tilt is a roll, and a roll needs no flip under mirroring.

D17 **The arm’s shape was ruled out before the mount was blamed**

Every link’s pose relative to its own `base_link` was compared against the stock vendor description at the same angles: maximum deviation 0.000000000 m and 0.0000024°. Three further checks ruled out a joint-zero convention error, including reading the vendor driver’s own source (degrees to radians plus a wrap, nothing else). *Recorded because the alternative hypothesis — per-joint offsets — would have been a large and wrong change.*

D18 **Link naming is viewer-perspective, not anatomical**

`left_leg`, `left_mount` and `human_left_upper_arm` all sit at *positive* x , i.e. on the wearer’s anatomical *right*. Kinematically irrelevant; decisive when matching a simulated arm to a physical arm in the room.

D19 **The mirror-symmetry criterion is unsatisfiable, and it is a proof**

With mirror-symmetric mounts, $|\text{right } P_{HOME} - \text{mirror}(\text{left } P_{HOME})|$ *does not depend on the mount rotation at all* — the rotation cancels algebraically. It is 1.3837 m for every mirror-symmetric mount, old or new. *Cause:* the home joint angles park the two arms asymmetrically. Fixing it needs a re-parked right arm in hardware, not a different mount.

D20 **The proximal collision: 48 of 48 IK poses refused, including the pose the arm was standing in**

The first candidate mount was applied and `/compute_ik` returned `NO_IK_SOLUTION` for every pose on both arms. `/check_state_validity` named it at once: `left_shoulder_link` against `human_left_upper_arm`, depth 0.0240. *Mechanism:* the offline search scored clearance from *distal link origins*, which cannot see a proximal link and cannot see a contact nowhere near a link origin. *This is a mechanical finding:* the mount origin is 0.1342 m from the wearer’s upper-arm axis — 38 mm of surface clearance — against a base tube 46 mm in radius and 171 mm long, moved by no joint.

D21 **The first mount fix routed the arm through the wearer**

It searched only *endpoint* criteria and had no path or intermediate-link constraint, so it found rotations satisfying the endpoint by putting `base_link` and `shoulder_link` **75 mm inside the torso**, with ten link pairs in collision. *Nothing objected*, because the offending pairs had been SRDF-excluded and the offline check excluded the same pairs by the same argument. **An exclusion silences the alarm; it does not move the metal.**

D22 **The old mount position made 150 mm of clearance impossible**

At $(\pm 0.15, -0.12)$ the base tube starts 0.010 m from the torso box, so over *every* rotation the ceiling for `base_link` was +0.0097 m — infeasible by a factor of 15 before any rotation was chosen. The mount was moved onto the back of the pack.

D23 Clearance and reach are in direct conflict

Moving the mount back to gain clearance pushes the task volume toward the edge of the arm's 0.902 m reach: mount $y = -0.12$ gives 0.026 m of clearance at 67 % of reach; $y = -0.32$ gives 0.211 m at 88 %. Two of the seven mount criteria (clearance ≥ 150 mm, IK ≥ 80 %) cannot both be satisfied at this scale, and the resolution is mechanical: a longer-reach arm, a task volume closer to the body, or a mount raised above the shoulder.

D24 The wearer model has static self-collisions

Head against torso 0.040 m, hips against legs 0.050 m — two fixed links overlapping by construction, nothing to do with the arms, and previously reported as an arm problem.

D25 The mount guard's first version cried wolf

Bounding spheres fired at 0.113 m on a geometry with 0.166 m of real clearance. *Recorded because the failure mode matters*: a guard that cries wolf gets switched off, which is worse than no guard.

D26 Four mechanisms could stop all motion silently

The blocking was usually correct; the defect was that blocking was *indistinguishable from working*. *Fix*: every mechanism that can stop motion registers with a `BlockMonitor` that is loud and by name on the first block, escalates past 3 s, and *raises at registration* if the blocker cannot state how it clears — a blocker that cannot say how it clears is a latch.

D27 `ik_inflight` latched active forever

The instrumentation's own bug: `clear()` sat on a line only reached when `pending` was already `False`, which never happens on a healthy system. It held for 65 s reporting "waiting 0.00 s".

D28 `/estop_reset` is a service, not a topic

The first injection run published a `Bool` at it, nothing listened, the e-stop stayed latched, and **six subsequent faults reported NOT HANDLED for that one reason**. Tests must reset to a known-good state between faults.

D29 Channel auto-disable defaulted to off

`channel_manager`'s auto-disable threshold was 0 frames, so a channel could drop out for an entire trial with nothing marking it dead. Set to 25 frames — 0.5 s at 50 Hz, the measured length of a real j4 burst — with a required-channel dropout now publishing a trial ABORT.

D30 The dead-man's enable was latched at construction

`estop_node` read `deadman_enabled` once, so setting it at runtime did nothing — and `participant_safety_node`'s attempt to force the dead-man on for a participant session would have failed silently.

D31 `ros2 param set` hangs on this machine

It goes through the daemon. Anything that shells out to it fails silently and reports success it has not earned. All parameter changes go through a parameter client, are read back, and are echoed `old -> new`.

D32 Duplicate node instances corrupt the blocking view

Three stale followers published under the same unit key and overwrote each other's state.

D33 Six files used the wall clock to measure intervals

The same bug class that had produced a -2321 ms latency reading, fixed once in the bridge and left everywhere else — including `sim_to_real_bridge` (the command path) and `real_homing_node` (which drives the real arm). All converted to `time.monotonic()`; wall-clock *timestamps* deliberately kept.

D34 The dead-man could not see the failure that actually happens

`master_pose_node` does not go silent when the master degrades: on a validation failure it substitutes `last_good` and keeps publishing at 50 Hz — and it stamped that substituted frame `now`. A dead-man keyed on message *arrival* therefore could not fire at all. **Only a clean unplug produced silence, which is why every cable-pull test passed and the mechanism looked correct for months.** *Fix, at both ends:* the source stamps the time of the *data* it was built from; the dead-man keys on the header stamp, and warns if a stamp is empty. Measured after: trips at 0.94–0.99 s on a frozen-but-publishing master, 0.97 s on a silent one, no false positive over 243 fresh frames.

D35 The e-stop halt was 4.0 seconds late

`trip()` called the real-driver halt first, which made two `wait_for_service(timeout_sec=2.0)` calls on a service that does not exist on any simulation or mock stack — inside the e-stop’s own 10 Hz timer on a single-threaded executor. Measured: decision at 0.97 s, trip logged at 4.98 s. **Detection was never late; the halt was, and the log line that would have revealed it was itself behind the block.** *Fix:* park first, publish `/estop_state` first, and never wait for a service there.

D36 An arm that had never published exempted itself from its own dead-man

if `t` is `None`: `continue`. Only the left master usually publishes, so the right arm’s dead-man was permanently disarmed.

D37 A reset was measured against staleness that predated it

So a latch caused by a brief gap re-tripped on the next tick and could not be cleared. Age is now measured from the later of “last fresh data” and “armed/reset”.

D38 The follower’s e-stop block was invisible exactly when it mattered

The `estop` blocker was set and cleared only inside `on_pose`, and the commonest reason for the e-stop to latch is the dead-man — which fires *because* the master went silent, so `on_pose` stops being called at the same instant. From the aggregated view the follower simply went quiet, which is indistinguishable from a crashed node.

D39 The latch pattern, and why grading instances was the wrong fix

Every blocker is written as `if <bad>: block(); return` followed by `clear()`, so the clear sits on the fall-through path of the guard it reports on — fine while the guard is mutated elsewhere, and a latch the moment the guard also stops the callback running. **It happened five times, the fifth inside the code written to catch the first four.** *Structural fix:* auto-expiry of any blocker not re-asserted for 2.0 s.

D40 An expired blocker read as an all-clear

Expiry set `active=False`, and every *consumer* read that as clear — `preflight`’s “no active blockers” check would have passed on a crashed guard and let a participant session start. Expiry is now a third state: `blocked` stays `True`, because “nobody is asserting it” is not evidence the condition went away. *Two placement rules are load-bearing:* the follower’s refusal sits at the publish rather than at callback entry (refusing at entry would prevent the re-assertion that clears the unknown), and `state_unknown` is excluded from its own trigger or it re-arms itself forever.

D41 Four fault-injection harness bugs, each producing a table of clean runs while testing nothing

(1) A fixed delay is not “mid-trial” — trials run 0.4–2.5 s with gaps, so one offset landed inside a trial in some runs and in a gap in others. (2) The publisher must already be discovered: building the injector node after the trigger costs about 1 s of DDS matching, longer than a trial. (3) `/trial_state` persisted across experiments, so the previous run’s final trial index still satisfied “trial 3 is running” and the fault fired into the *next* experiment’s startup. (4) `/participant/abort` is a one-shot event, so a fault raised between two trials was seen by neither. *And the assertion must require the injected cause*, or an unrelated standing fault invalidates a trial and scores as a pass for an injection that never landed.

D42 A wholly-invalid session analysed cleanly

Before `validity.py` existed, a session in which every trial was invalidated analysed to “0 valid trials” followed by empty means and NaN intervals — indistinguishable from a null finding by anyone reading the output. One shared definition of “analysable” now exits non-zero when no trials survive or when more than half were excluded.

D43 One experiment had no run or analyse scripts at all

E6 shipped with a config, a protocol and scenarios, and nothing to run. Found by attempting to inject a fault into all six.

D44 A fifth unprefix resource, invisible to every URDF check

`tcp/twist.*` and `reset_fault/*` are hardcoded string literals in the driver C++, not in any xacro. *And the gripper one is worse:* an earlier fix prefixed the GPIO *declaration* in the xacro, so afterwards the URDF declared `left_reactivate_gripper` while the driver still exported `reactivate_gripper` — they disagreed. It went unnoticed because that driver is COLCON_IGNORED and every gripper test to date ran against mock_components, which reads the URDF and never runs that code.

D45 arm=:right would have opened a session against the left arm

`real_arms_highlevel.launch.py` passed `left_robot_ip` unconditionally, so selecting the right arm would have connected to the left one and homed it.

D46 The two-arm start waited for one homing completion, not two

The first arm finishing ended the wait and enabled the bridge while the second was still moving.

D47 The homing “stall” was a wrong exit test

Completion required every joint inside the 1° deadband while the acceptance criterion is 0.05 rad (2.86°). A joint resting at 2.5° is homed by the project’s own definition and could never satisfy the exit test, so homing ran on until the stall timer fired. **The deadband’s job is to suppress the correction, not to decide when the arm is home.**

D48 Three more homing diagnoses that did not exist

Hysteresis was missing, so a joint would stop just inside the band, drift out, get kicked and overshoot back in — the observed 2.2–3.0° hunting. OSCILLATING is now reported separately from STALLED, from error sign changes, because “stalled” sends you looking for a seized joint on an arm that is moving. And BELOW VELOCITY FLOOR is a third diagnosis: at 2.5° of error the law commands 0.0218 rad/s, and if the arm cannot execute that the error can never close.

D49 The FSR gripper node was in no launch file

It had only ever been started by hand, so a launched stack silently had no grippers at all.

D50 Three bugs in the trajectory-capture analyser, all found by known-answer testing

(1) `C_left_up` matched the direction `left` before `up` — the arm names collide with the direction names, so every up/down segment was filed under left. (2) Correlation included dropout zeros, so a dropout-ridden channel correlated with anything that moved: it was measuring the fault, not crosstalk. (3) The linearity R^2 was meaningless (0.000 for every healthy channel) because it regressed against *sample index*, measuring the shape of the operator’s sweep rather than the sensor.

17.3 2026-08-06 — the capture, and what it invalidated

The recorder, the channel verdicts, and a 35-minute capture that measured almost nothing.

- D51 The recorder oversampled the sensor, and that faked every per-row statistic**
 Rows at 50.3 Hz against potentiometers updating at 14.7–17 Hz, so **82–88 % of adjacent rows are bit-identical**. Median $|\Delta|$ over rows was 0.000 on all fourteen channels — that is the recorder, not the sensor; over distinct updates the real noise floor is 0.2–5.4°. **The “spread exactly 0.0 over 40 samples” signature used for weeks to identify a dead channel is this artefact**, not evidence.
- D52 Two priors that had been wrong for weeks**
 Left j7 is the *cleanest* channel on the left arm — 0 % dropouts, 0 % implausible jumps, merely stiff at 26° of range — and had been recorded as railed and dead. And half the left arm is INCOHERENT, which is worse than had been documented.
- D53 INCOHERENT: the verdict that did not exist**
 A failing potentiometer does not go quiet. It returns values spanning a wide range that are *not a trajectory*: consecutive updates unrelated, jumping tens of degrees in 60 ms. Range alone calls that healthy, which is how a broken channel keeps being trusted. The rule is now > 5 % of updates jumping > 60°.
- D54 Range must be circular, or a 360° sensor reports 314 363°**
 Unwrap-and-subtract is invalid for a rotary sensor: across a dropout gap the unwrap cannot know how far the joint moved and the offset runs away. Range is now the smallest arc containing every sample, bounded by 360 by construction. Samples outside $[0, 360]$ — 0.2–0.8 % serial parse glitches, in bursts up to 27 — are rejected first.
- D55 The 35-minute capture was clutched out and mostly invalid**
 41 of 42 directional segments were recorded with the clutch *disengaged* on the arm under test, simulated end-effector span 0.000 m. *Mechanism*: GATE_INDEX mapped each arm to its *own* clutch button, so every gating press disengaged the clutch being measured — **and the comment directly above it stated the correct rule while the code did the opposite**. The real arms also never moved (the CSV has no `real_*` columns at all), and the e-stop was latched for all of one block because the both-button trigger fired on any two button *edges* within 0.4 s and gating taps supplied them.
- D56 The curses GUI crashed on launch, and its test passed on the broken code**
 It failed with no tty and under `TERM=dumb`, and the cleanup’s own call failed too and replaced the traceback. **The old test drove the program through script -qec, which allocates a pty, and set TERM=xterm** — it constructed exactly the environment where the bug cannot occur, then checked only for panel captions, never the exit code or a traceback.
- D57 GUI control keys blocked the draw loop**
 For seconds at a time when a target node was absent, so the console appeared hung exactly when it was being used to find out why something was hung.
- D58 The reachability measurement ran from the wrong pose**
 Both arms were parked 155–166° from home, left over from the capture, so “reachability from home” was never measured; the right arm reported 0.000 m in all 26 directions. *Fix*: the tool now *refuses to sample* unless every joint is within 0.05 rad of home, and prints the check. That assertion is what was missing — without it the tool happily measured the wrong question and reported it under the right name.
- D59 Isotropy swinging between runs was the statistic, not the measurement**
 Two consecutive runs from an identical pose gave isotropy 0.28 then 0.12. At $N = 10$ the interquartile range is **0.000 m on every direction, both arms**. *Mechanism*: isotropy computed as the minimum over directions of a *single* sweep is the statistic most sensitive to a rare tail failure. On medians it is stable.

D60 The IK “sticking” was compounding, and it was in the measuring tool
Per-call false-negative rate on a known-solvable target is 0.0–0.4 %, sporadic, never the same direction twice. A sweep is a chain of up to 90 calls and any one failure ends the direction early: at $p = 0.002$ that is about 13 % of directions, matching the 9-of-26 early terminations observed. `ik_follower_node` never suffers it because it re-seeds and retries — so **the sweep was more brittle than the system it characterises**. Given the follower’s retry, early terminations went 9 to 0 and 3 to 0, and the medians barely moved.

D61 Raising the IK solver timeout does nothing
Requested timeouts of 5, 20, 50 and 100 ms all returned a median solve time of 6.5–7.0 ms. The request timeout is ignored on this path; `kinematics.yaml` governs.

17.4 2026-08-07 — the task scenes, and the geometry that stopped a programme

Two arms with no shared workspace, a mount sweep recorded and not applied, and the standing rule written down.

D62 The two arms share no workspace at all
Every task target failed — 0 of 19 — with collisions off and with yaw sampled, so it is reach and not scene collision. The decisive measurement is a $7 \times 3 \times 3$ grid over the frontal volume: **0 of 63 cells reachable by both arms**, 16 left-only (all at $x \geq +0.4$), 18 right-only (all at $x \leq -0.4$), and 29 by neither — **including the entire centreline**. *This explains the operator’s original symptom directly:* “it does not reach toward the centre” is not a control fault; the centreline is reachable by neither arm.

D63 A correction to that framing, recorded because it was acted on
The 0-of-63 result asks whether both arms can reach *the same point*, which is what an inter-arm handover needs and nothing else does. A coordinated carry needs two *distinct* points 300 mm apart, which is strictly weaker. Re-measured properly, three of five bimanual tasks are feasible and two are blocked.

D64 The coordinated carry’s feasible band is at chest height, not table height
 z 1.10–1.30 rather than the protocol’s 0.885 m. The tray needs a stand, not a table. And the transport is a *lift*, not a carry: grips and vertical lift solve 20 of 20, carrying toward the wearer solves **0 of 20**, because the feasible band is only 50 mm deep in y .

D65 The disjointness is the wearer *and* the kinematics, in that order
With collisions off, 3 of 63 shared points exist kinematically — and *the wearer removes all three*.

D66 The mount sweep is an upper bound, and its best candidates move the arms toward the person
108 candidates evaluated by the target-transform trick, which does *not* move the wearer with the mount, so the sweep is purely kinematic. The best buildable candidate improves shared points from 3 to 22 of 63 — and it moves the mounts inboard, forward and lower, i.e. towards the wearer. *Recorded and deliberately not applied.*

D67 Detection scored 0–4 %, and the renderer was out of distribution
Before reporting it, the model was checked against a real photograph: seven detections at confidence 0.89–0.91, against **zero** on the synthetic scene at a 0.01 threshold. **The model is fine; flat coloured rectangles on a flat grey background are nothing like the photographs it was trained on.** This is the mirrored-AprilTag renderer bug in a new costume: a synthetic harness that measures itself. Detection rate at working distance remains UNMEASURED, and the 95 % gate is neither passed nor failed.

D68 A library device-placement bug that looks like a model failure
Calling `set_classes()` after the model is on CUDA raises Expected all tensors to be on the same device. Set classes before the first GPU predict.

D69 Forbidden targets could still be offered to the operator

The keep-out filter ran *after* resolution, so the ambiguous branch was free to offer a position the robot must never reach — and asking the operator to choose an option that will then be refused invites them to say “the far one” and trust it. The exclusion moved into the world model’s own match, so a forbidden object is never returned to any consumer, with a separate count so the refusal can still name the real reason.

D70 The wake-word check passed while nothing was running

Its previous version reported a pass against a dead executive. It now sends a probe command first and *refuses to report at all* if nothing answers.

D71 A pose that passes one IK call is not a reachable pose

Every scenario in the project had been verified at $k = 2$, which lets a 60 % pose through 36 % of the time — often enough to be written into a protocol, rare enough to look like bad luck on the day. Measured: a pose at (0.15, 0.35, 1.36) is 72–82 % feasible per call and passes a 5-of-5 check 19–38 % of the time, while 20 mm lower it is 100 % (40 of 40). *Both halves of the fix are load-bearing*: $N = 10$, and over the *whole path* densified to 20 mm, because the arm flies the gaps — one task declared four waypoints spanning 200 mm and left 150 mm unexamined between each pair.

D72 And the audit itself was wrong, three times

Its first run reported every waypoint of one task unreachable: it assigned the “left” arm to the $-x$ end, and `left_*` links sit at *positive* x in this model. Then it re-rolled a static hold pose once per waypoint, which turns a 60-of-60 pose into a random failure somewhere along the path. Then a bare dump call silently deleted the scenarios another script had merged in. *Fifth, sixth and seventh instances of the standing rule.*

D73 A target trajectory could leave the sphere that had been verified

The pursuit task’s Lissajous exceeds its own amplitude by $\sqrt{1 + 0.35^2} = 1.06$, so the marker could leave the shell that had been verified reachable — the “scenario that fails IK on the day and wastes a participant” failure, caught by a known-answer test before a participant saw it.

D74 The carry path template carried toward the wearer

Reported as “0 of 19 placements”; the placements were fine and the path template moved in the one direction that leaves the feasible band.

D75 A metric that recovers a known answer in a unit test can still be wrong end to end

The pilot injected interference coefficients of 90 and 60 and the fit returned 358 and 289. **The claim was wrong, not the metric**: the generator injects speed sensitivity in the error amplitude *and* in a tracking lag whose positional error also grows with speed, so the fit recovers *total* speed sensitivity, which is what the quantity is defined as. A calibration block whose error is linear by construction now recovers 90.0 and 60.0 exactly, end to end through the real pipeline.

17.5 2026-08-08 to 08-13 — the two-person reframing and the GUI generations

Who is a participant, what a clearance floor means, and a document that pointed at the wrong window for two days.

D76 IK-valid is not clearance-valid

Recording every task surfaced what no amount of IK verification would: the carry paths bring the arms within 48–62 mm of the wearer’s torso — contact-free, IK-valid, and below the 120 mm floor the real-robot mode enforces. **MoveIt’s `avoid_collisions` is binary contact; the real-robot floor is a margin.** A scenario can pass one and be refused by the other.

D77 Fixing it outboard propagates through three tasks

Forward in y does not work — clearance improves and only 1 of 6 poses stays reachable. Outboard in x does: a half-span of 0.25 m gives 0.134 m at 6 of 6 reachable. *But* a 0.25 m half-span is a 500 mm tray, which breaks the sling task’s retention (it needs $L \geq 507$ mm), rescales the tilt threshold (60 mm over 500 mm is 6.8° , not 11.3°) and moves a container opening from 300 to 500 mm. Recorded, not applied.

D78 Four instrument bugs in the verification recorder

(1) The clearance model was *invented* — hardcoded world-frame boxes reporting 0.064 m where the shipped guard reports far more; two clearance metrics are worse than one. (2) The tracking metric included the approach from home, where the command is deliberately far ahead of the arm: 224 mm on a run that tracks to 2.1 mm once moving. (3) Paths were too short and too fast — a three-frame clip of an arm asked to cross half a metre in 0.1 s. (4) A per-frame 3-D axes put the full sweep at three hours.

D79 The primary GUI was named wrongly for two days, and a reader followed it

The project’s own instructions named `console` as primary from 2026-08-08 to 2026-08-10 while every piece of GUI work went into the Qt window. **A reader following that file ran a GUI that received nothing, reported the new panels as missing, and was right.**

D80 The window offered three generations of task set at once

41 buttons across `t1`–`t9`, the superseded E-series and the current set, so **a button labelled “T3 rigid carry” ran a 300 mm tray against a 500 mm current specification.** The old sets are archived with a README naming what superseded them, and the runner refuses them by name rather than silently.

17.6 2026-08-15 — the workspace measured against the wearer

The longest single day in the history: the SRDF gap, the home change, the centre, predictive avoidance, and the clips looked at frame by frame.

D81 The SRDF gap reaches the task coordinates

With the wearer measured geometrically rather than through `avoid_collisions`: **70 of 143 waypoints of the shipped layout inside the 150 mm floor, worst 58.7 mm, at zero IK failures**; the idle arm’s park pose at 30.6 mm for the whole clip; and the left arm’s reachable survey cells 72 of 265 inside the floor, worst -2.7 mm. **Negative means the metal is inside the person**, and those cells were inside the workspace marking a participant is told to work in.

D82 A survey tested every right-arm cell at the left arm’s wrist

`survey_work_surface.py` passed no arm argument, so it defaulted to the left arm’s wrist-to-pad offset — 48.3 mm from where the right hand closes. **The decision to move a task to the right arm was taken on that survey.**

D83 Slicing an IK response always returns the left arm

`res.solution.joint_state` is the whole robot, 26 joints, left arm first, so `positions[:7]` gives the left arm whatever arm was asked about. The right arm’s clearance therefore read *one* value, 0.1610 m, across 80 cells while the left varied from -0.003 to 0.161. **Caught by the “zero variance” row of the instrument table before the number was used for anything.**

D84 A verifier carried its own copy of a task’s layout

Written down once and never touched, it verified those stale coordinates through three layout changes and reported zero failures each time.

D85 A recorded seed that nothing read

The runner wrote `--seed` into the trial manifest and never read it: the layout came from the builder with no arguments, so the randomised stage ran the same four objects in every trial with a different seed number recorded beside them. **A recorded seed that nothing reads is worse than no seed — it is a claim of randomisation in the data file.**

D86 Spoken commands still resolved against the unsafe region

Two of three consumers were switched to the clearance-checked cell set when it was measured; `named_places.py` — the module that turns “move to the left side” into a pose for the autonomy executive — was not. **It was safe, and safe by luck:** the centroid of the unsafe set happens to clear the floor. A re-survey that moved the centroid inboard would have returned a pose inside the wearer, and the only check that pose ever faced was the one that cannot see it. It now *refuses* a region file without clearance data rather than falling back.

D87 A table quoted the safe boundary against the unsafe count

The region row printed the clear set’s x -range beside the IK set’s cell count, so the 175 mm inboard strip that is the entire subject of the measurement did not appear at all. **Two numbers from the same measurement printed side by side, looking consistent because one was copied from the other’s source.**

D88 The recording sweep had only one witness for a moving arm

It decided a clip was good from the runner’s exit code, which carries its own travel gate — both measuring end-effector travel, in different processes, off different tf2 listeners. So the runner can see motion the scene node never received, and clips reporting TRAVEL L 0.00 R 0.00 with four objects carried 0.000 m were recorded OK. A second gate now reads the scene node’s own travel, extracted as a function so it can be handed a broken input.

D89 An inward scan assumed the reachable set is an interval

It stopped at the first column that failed, and the reachable set is not an interval in x : one row fails at 0.55 and succeeds at 0.25–0.50, so the scan reported “nothing at this y ” for a row with eleven working columns in it.

D90 The tray was elastic, and two acceptance criteria could not fail

It was drawn as the *live* gripper separation plus 60 mm, recomputed every tick: 0.487–1.211 m against a 0.560 m specification, a $2.5\times$ stretch. **“Tray held by both grippers” cannot fail — the tray is the segment between the grippers.** “Ball visible on the tray” cannot fail either. And the declared failure mode never happens: under full autonomy the tray is past the 6.8° ball-drop angle for **89 % of the carry** and the ball never moves. *The task is not at fault:* the schedule commands exactly 0.500 m at all 21 waypoint pairs, so the stretch is a *tracking error being rendered as elasticity instead of as error*.

D91 And the previous fix here was the opposite error

A rigid tray pinned to *one* arm’s pads, so the right arm held air. The fix replaced a wrong-but-rigid tray with a right-looking elastic one.

D92 A misreading of my own, corrected in the same pass

The front view appeared to show the tray crossing the wearer’s head. It does not — at the highest waypoint pair the tray clears the head sphere by 248 mm, and the overlap is projection. **It was checked because a marker is not a collision object**, so `avoid_collisions` would not have objected if it *had* been true.

D93 The workspace marking ran 25 mm past the table’s near edge

43 of 439 cells — the whole nearest row — drawn over air on the wearer’s side of the table edge, **and the randomised stage put objects there**, one coming to rest 24 mm beyond the front edge, because the sampler draws from cells and nothing in that pool knows where the furniture is.

- D94 Achieved motion is not identical across modes, with no operator present**
 The task specification stated that identical commanded waypoints mean identical robot performance, so every measured difference comes from the operator. **The first clause is true and enforced in code; the second does not follow.** Scripted, no operator: one task's left path is 1.407 m under mode 01 and 2.034 m under mode 03 — 44 % — against a 3 % same-mode spread. *Consequence:* a mode difference in a trial cannot be attributed to the operator until the no-operator difference has been subtracted, and it never had been.
- D95 A measurement hold of 0.29 seconds**
 One task's card says both arms hold still while a reading is taken. Under mode 01 the window in which both objects are held at once is 0.29 s and the box travels 21 mm — it is never held up. The clip is not wrong about anything it claims; it simply does not show the thing the card describes.
- D96 The VR mapper holds the arms, and it cost seven clips their opening pose**
 Seven of 25 clips opened on the wrong pose — all five of one mode and two of another, the only two modes that run the mapper. **Isolated by experiment with a control either side:** mapper absent, staging arrives at 0.0000 rad; mapper started exactly as the isolation starts it, staging *does not arrive*, 2.076 rad out; mapper killed again, arrives at 0.0000. The reported joint error was identical at the start and end of the deadline — **that is a held arm, not a slow one** — and the follower's own pause does not silence the publisher that wins.
- D97 A wrong hypothesis, recorded because it was acted on**
 The first explanation was that the staging trajectory had a fixed 2.5 s duration whatever the distance. **That defect is real, and it is not what was happening** — with duration scaling in, the arm was given 8.8 s and a 12.8 s deadline and finished exactly where it started. **A fix that removes a real defect can still leave the symptom, and reporting the symptom as fixed because a plausible cause was addressed is how this project's standing rule gets broken from the inside.**
- D98 Mode 02 cannot grasp, and the miss accumulates**
 Three of 25 cells failed and all three are that mode's three grasping tasks. Against a 30 mm capture gate the pads missed by 88.1, 115.4, 156.2 and 205.9 mm while the other three modes closed on all four objects at 0.0000 m. **The increments — 27.3, 40.8, 49.7 mm — are the diagnostic:** a constant offset repeats one number; a growing miss means the arm falls further behind on every object.
- D99 And it is the mapper's lifetime**
 Re-recorded with one change — a fresh mapper per clip instead of one shared across the mode's five — and all five passed, with all four objects closing at 0.0000 m. **The ordering says the same thing:** the mode's first clip, on a mapper seconds old, passed, and every clip after it failed. The sweep is sound now; *the data path still starts the mapper once per session*, so a real trial block under that mode would walk into exactly this.
- D100 The complaint about the opening pose named two poses, and one of them was already right**
 “Wrists up and elbows splayed” is the home pose for the wrists and the presentation pose for the splay. The presentation pose's wrists have been level since it was built — `elevation_deg: 0.0` for both arms, recorded before the session began.
- D101 A claim withdrawn on measurement: the limb never rode over the shoulders**
 An earlier session recorded that “the limb rises above the shoulder before descending”. The elbow term could not see it, because it measured one joint rather than the limb, so an apex term was added for the whole capsule chain against the wearer's shoulder line. **The new term found nothing:** the apex is identical before and after, and was already 118 mm under the shoulder line. **It was an impression taken off a render**, and what reads as “over the shoulder” in the picture is the arm being nearer the camera than the wearer is.

- D102 The hand span is 3.1 times the torso width, and it is the torso that says so**
 1.100 m of hand span against a 0.360 m torso. Measured geometrically, a hand at $|x| = 0.18$ puts the arm 16 mm *inside* the torso, and $|x| = 0.45$ is the nearest either hand comes with the 150 mm floor intact.
- D103 Top-down is strictly worse for the pick-and-place task**
 It keeps all four picks and loses *every one* of the four places on wearer clearance — down to 12 mm against a 150 mm floor.
- D104 A presentation test that could not fail on an ungraspable object**
 It asked only whether the pads were *centred* on the object, and duly reported 195 mm of circuit box lying across an 85 mm hand as PRESENTED. The aperture is part of the verdict now, and a 200 mm control object proves it can fail.
- D105 Three of four tasks breach the wearer floor**
 On the waypoint list the recorder actually sends: T0 breaches by 141 and 132 mm with 4 IK failures per arm; T2 by 145 and 60 mm; T3's left arm by 88 mm. Only T1 has ever had the geometric wearer pass.
- D106 The table was never white in the picture, and nothing could have noticed**
 The specification requires a white table; the requested colour was (0.94, 0.94, 0.95) and the top renders at RGB (118, 118, 118) — mid grey — while the arm's own meshes in the same frame reach (208, 207, 210). **The audit passed it every time because it read the requested colour and never looked at a pixel.** *Mechanism, measured:* a marker's material takes ambient = $0.5 \times$ colour and the only light is a headlight on the camera, so an up-facing face gets the ambient term alone — and the renderer does *not* clamp, so 1.5 and 2.0 continue the same line to 190 and 253. The top is now requested at 1.88 and renders at about 240.
- D107 Two risers stood on the table holding nothing**
 Carried in every clip of four tasks, they belonged to a bench only three other tasks have, stopped 60 mm short of the objects above them, and invited exactly the reading the scene must not invite — that the objects rest on something.
- D108 A capture ordering fault that looked like a rendering defect**
 The first render of the white table came back with the table present and the marking, the objects and the pads all missing. The obvious reading was that an out-of-range colour had broken the marker array. **It had not:** 220 markers were being published at 10 Hz throughout, and the scene node had been started *before* RViz existed. “The marking is missing” and “the renderer was not up yet” produce the same picture.
- D109 Moving home is two changes, and only one is dangerous**
 Home is the origin of where the arm rests *and* of the pinned approach anchor, and **they are separable in the code, which nothing had noticed** — the anchor is read once at node construction and nothing recomputes it from the live home. Measured over every task at $N = 10$: with the anchor kept, every task performs exactly as before. **With the anchor re-derived to level, one task loses its right arm entirely.**
- D110 An instrument defect caught by a control, in that same measurement**
 The first version took the clearance of whichever solve happened to run *last* per waypoint — one sample of a null space the solver re-seeds randomly, and the seed is precisely what the experiment changes. Corrected to the worst over all N . **The known-answer control then flagged a disagreement, which is the correction working,** because worst-over- N can only be lower. The control was made one-sided rather than the measurement tuned to pass it.
- D111 A staging move that did not arrive, and a script that refused to report**
 On one run the middle condition was refused outright: the 1.94 rad staging move did not arrive inside the deadline, so the script reported *nothing* rather than reporting numbers for a pose the arms were not in. **That refusal is the reason the other numbers can be believed.**

D112 The home pose had five storage sites, not two

The “update both places” warning undercounts, and the two sites nobody names are the dangerous ones: `pot_bridge.py` and `srl_teleop_node.py` each carried their own hardcoded copy. **Both are installed executables that no launch file or script references**, so nothing would have caught the drift — and after the home change each was one command away from driving an arm to the superseded pose.

D113 A reference file that had gone stale in the way that matters

`real_home_reference.txt` carried per-joint deltas computed against a simulated home superseded *twice* since, so it read like a homing estimate and was arithmetic against a pose that no longer exists. The deltas were removed rather than updated, and the header now says plainly that no code reads it.

D114 The guard against hardcoded copies took three attempts to become a check

Version 1 anchored on `home... = [` and missed the dict form the real code used — **it passed a deliberately broken input, which makes it not a check**. Version 2 matched any seven-float list with “home” within 200 characters and false-positived on an offset *from* home. Version 3 parses the file and asks whether a literal seven-element pose is assigned to something named `home` — a question about the syntax tree rather than about characters.

D115 A double hyphen in an XML comment took out the entire robot description

-- cannot appear inside an XML comment. A comment added beside the home change used it the way every Python file here does, and xacro refused the description outright with “not well-formed”, taking out the arms, the wearer, MoveIt and everything reading `/robot_description` — naming a line and column rather than the mistake.

D116 What the home change actually cost, on the applied tree

Re-verified at $N = 10$ over the full path: the randomised stage’s seed 0 right arm **regressed to 25 of 98 waypoints inside the floor**, and one task’s left arm **now fails a waypoint**. **The mechanism is the IK seed** — the solver seeds from the live joint state, so the resting posture selects which null-space branch it lands in — and the right arm’s `joint_7` is additionally stored *wrapped*, the same physical pose 2π away as a seed.

D117 The teleoperation workspace centre is now stale by 0.2 m and deliberately unapplied

It is documented as `offset = P_HOME` and is stale by 0.188 m (left) and 0.222 m (right) after the home change. The master node seeds three quantities from it, so the first commanded teleoperation frame would move the arm that far. The replacement values are recorded and not applied, because changing it changes what live teleoperation does — which is the study’s baseline condition.

D118 The centre, on the surface: 0 of 3 360 cells

Table height and distance both swept with objects *resting*: 22 of 3 360 cells survived the screen, and **every survivor has zero overhang** — the object must sit at the very front edge. *Mechanism*: the pinned wrist’s tool axis is 30.7° above horizontal, so the hand arrives from below and over a slab that trailing volume is inside the slab. Top-down on a surface is **0 of 840** at any table height on either arm. (*All 22 survivors were withdrawn on 2026-08-17 — see “Every reachability sweep in the repository solved at the wrong wrist” below.*)

D119 Predictive avoidance, first build: the null space is worth 4.4 mm

A seed-fan sampler was built, driven straight at the wearer’s head and torso, and measured: best clearance gain **+0.0044 m at the head and +0.0001 m at the torso**. **The fan is not the problem** — its own control found and scored 90 and 82 *distinct* joint solutions. The null space is being searched and does not contain anything better.

D120 Predictive avoidance, second build: the null space is real now, and the reason it cannot help is the finding

A genuine Jacobian projection was built — finite-difference Jacobian, $N = I - J^+J$ verified idempotent and symmetric, a clearance gradient that returns nothing if any probe is missing, and a **task-space correction each step, because a finite step along a tangent leaves the manifold**: 5.5 mm of hand drift over eight steps, reduced to under 0.1 mm by one Newton step. It is worth the same as the sampler it replaced: 0.0 to 5.6 mm. **Every one of 28 refusals names the same link: the end effector.** The *hand* is what is inside the person, and the null space is by definition the family of postures that hold the hand's pose fixed. *Refusing is the right answer, and it is right for the right reason.*

D121 A bug of my own that the numbers caught

The first integration accepted the projection's result whenever it beat the fan's clearance and marked the step COMPLETED — which bypassed the floor. It reported 10 of 15 torso steps completed *while the projection's best gain for that target was 0.0000 m*: a completion rate produced by the bookkeeping rather than by the robot. The corrected number is 4 of 15. The same summary printed “lockups N ” where N was the count of *refused* steps.

D122 The runtime wearer had no arms

`clearance.py` — what the follower, the homing node and the bridge enforce — modelled a torso, a head and a pair of hips. The offline survey has always included the wearer's arms, and on the inboard side those arms are exactly what binds the right arm. **The planner's wearer and the follower's wearer were different people, and the follower's had no arms.**

D123 The verification renderer drew the wearer's arms 150 mm low

Its specification applied a -0.15 m offset assuming the link frame sits at the shoulder; the description puts the collision at the link origin with no offset. The drawn upper arm sat 150 mm below the one the score used, and the hands were absent entirely.

D124 Two postures could not report, because the control moved with the thing under test

The sweep required an outboard cell to clear the floor and required clearance to fall as x goes to zero. Under two of the five wearer postures the wearer's own arm moves toward that cell, so both controls failed *on exactly the postures the sweep exists to measure*. **A control whose ground truth moves with the thing under test is an outcome in disguise.**

D125 Two stacks ran at once and a measurement talked to the wrong one

A malformed shell line left an orphaned runner, so a second `move_group` came up beside the queue's. One run then read a robot description holding a different wearer posture while its own process reported the posture it had asked for. **The URDF control caught it**, which is the whole reason it exists.

D126 The published innermost columns were stale, and nothing had re-run the sweep

Two instruments built for different questions agreed with each other and disagreed with the stored baseline. **The left arm had gained 100 mm inboard and the right had lost 50 mm** at the home change, and neither was noticed because nothing re-ran the sweep afterwards.

D127 The rendering harness wrote into a directory that did not exist

Its scratch path was hardcoded to one session's temporary directory, long deleted, so every render check, cached config and frame grab wrote into nothing.

D128 A blank white frame was reported as captured

The first still came back as a path to an empty viewport and the code called it a shot. The check now measures the fraction of viewport pixels differing from the modal colour and refuses below 2%; the known-blank frame scores 0.0003, **and the window chrome has to be cropped first, because menus and a status bar alone score 0.132.**

D129 “Mirror residual 0.0000 m” came from sixteen failed lookups

Perfect symmetry and an empty loop are the same picture. The reader spins and retries now, and the missing-frame control is what caught it.

D130 The shared environment script never exported the one mandatory variable

`scripts/env.sh` did not export `FASTDDS_BUILTIN_TRANSPORTS=SHM` — the variable the project’s own hard constraint calls “NOT optional”. It was set only in the shells the session launcher opens, so a stack started that way and any process started by hand could not see each other. Measured: with it unset, `ros2 service list` hung past 30s and a measurement reported “no /compute_ik” while `move_group` was up and answering. **A transport mismatch reads exactly like a dead stack**, and this cost two full measurement runs.

D131 The randomised stage had no colour rule and nothing drawn to place onto

Stage 2 placed objects onto coordinates drawn from each arm’s surveyed cells — arbitrary points, no colour — and the scene’s pad block ran for stage 1 only, so there was nothing on screen to place onto. **The half of the task that exists to demonstrate divided attention had no colour matching in it at all.**

D132 And the colour must follow the whole draw, not the per-arm index

Keyed on the per-arm index, a 3/1 split gives three of one colour and one of the other, which is a different task from the one the specification declares.

D133 The mode difference is real, and its size is not knowable from this set

Two alternative explanations were tested and refuted: it is not recording length (runs of identical duration differ by 27 % and 34 %) and it is not run-to-run noise (**four modes share a net displacement to four decimal places** — that is what running the same thing looks like, not what independent noise looks like). **But both metrics are properties of the observation window:** one sums over recorder ticks, and the other is measured from the first pose the scene node *happened to see*, not the pose the task started from.

D134 A bad draw was a stale pool

The randomised stage’s seed 0 put 26 waypoints inside the floor with zero IK failures while seeds 1 and 2 were clean, which made it look like one unlucky draw. **It was the pool:** the region file had been surveyed before the home change and its right-arm cells reach 50 mm further inboard than the arm can now work. Eleven stale cells were dropped — **the class fix rather than the one-draw fix**, because re-drawing seed 0 would have left the other stale cells in the pool for the next seed to find.

D135 A verifier refused to report, and the refusal was correct

Its fourth control — a known-good pick from the legacy scene — read UNREACHABLE while the other three passed. **It will not print a count when a control says the instrument is wrong.** The legacy coordinates had not been re-derived since the home change, and the refusal is that verifier telling the truth about itself.

D136 Three verifiers had been failing for the life of the clip set, and the missing piece was the producer

Their own source carried the right diagnosis — *nothing in the clip path writes the grip trace at all, 0 files on disk across 25 recorded cells* — and they refused to report a pass on zero inputs, which is correct and was ignored because it looked like tooling noise. *Three more faults sat behind it:* their file glob knew only the legacy four-level path, they *crashed* on a three-level path, and the grasp-task list held retired names in lower case while the path spells the task in capitals — which is how “checking gripper motion in 5 runs” and “grasp clips: 0” appeared together.

D137 And then it spoke: the carry task’s grippers are open for all 992 samples

Fully open on both arms, while the task’s own schedule commands them closed on *every* waypoint with a comment saying they stay closed. The other three tasks cycle correctly through the same command path. **So “tray held by both grippers” is false in the pixels, and has been for as long as the task has existed** — invisible because the tray was drawn between the two grip points whether or not anything was gripping. *Rigid or elastic, a board drawn between two open hands still looks held.*

D138 One task wrote an empty trace because its clock started in another task’s branch

`self.t0` was set inside the carry branch, so a task with no carry never started its clock and appended nothing. **An empty file and a stationary gripper are precisely the two things these verifiers exist to tell apart.**

D139 A grasp criterion read a file the current recorder does not write

It read the *legacy* recorder’s capture file, so the dictionary was empty and the criterion was False for every clip whatever the fingers did.

D140 A grasp criterion asks the pinned wrist for something it cannot do

It requires the gripper’s principal axis to rotate more than 8° during the clip; measured, $0.3\text{--}2.1^\circ$. **That is not a defect, it is the platform** — the anchor is written into every waypoint, so the wrist does not rotate during a pick and place. *Not silently weakened:* the criterion is right for an arm that can rotate its wrist during a grasp and wrong for this one, and which of those is true is a platform decision rather than a verifier tuning parameter.

D141 A verifier crashed exactly when it had something to report

“e.g. %s” % worst[0] where `worst[0]` is a 3-tuple. That line only runs *when there is a failure to report*, so the verifier threw an exception whenever it had something to say and printed a clean table the rest of the time.

D142 And then it spoke: the choreography envelope was taken from a scene with no furniture

45 of 382 and 49 of 554 waypoints unreachable. The box was $|x| \in [0.26, 0.50]$, taken from free-space anchors with no wearer floor, against per-arm working columns of 0.325 and 0.450. **Moved to one symmetric band rather than two**, because these routines are built on unison, canon and mirroring and an asymmetric envelope makes the arms trace visibly different shapes. Re-verified 0 of 1298.

17.7 2026-08-17 — the wrong wrist, and the run that did not start from home

The single most expensive instrument fault in the project, and a home check that measured the wrong instant.

D143 The observe pose arrives inside a tolerance, and vision deprojects from the pose it did not reach

Detection error inside the recording sweep was 28–31 mm against 1.3–3.0 mm standalone, and **every object was displaced in the same direction** — a constant camera-pose error, not accumulating lag. **The object whose 31.0 mm detection error and 30.9 mm recorded miss are the same number is the proof.** *Mechanism:* the staging helper verifies arrival with a 0.02 rad per-joint tolerance, and inside the sweep the follower is streaming position commands to the *same controller*, so the arm settles at the edge of that tolerance — and 0.02 rad on the proximal joints of a 0.7 m arm is centimetres at the camera. **The one-publisher-per-controller rule arriving in the vision path, where it is quieter, because nothing fails:** it returns four objects, in the right order, with the right colours, and every coordinate wrong by about the width of the capture window.

D144 And the two obvious fixes are both wrong

Widening the 30 mm gate would grasp from a pose that is still 31 mm wrong and hide the fault in exactly the clips meant to be evidence for it. Tightening the tolerance alone turns a wrong answer into a timeout, which is the third row of the same table.

D145 A run does not start from home, from the second run of a session onward

Measured at the instant the first waypoint is published: boot 0.0000, run 1 0.0000, **between runs 1.2338 rad (left) and 0.6248 (right)**, run 2 the same. **Nothing returned the arms to home when a run ended and nothing looked at where they were when the next one started.** The recording sweep stages before every clip, which is exactly why this survived. *And it is not only the working arm:* a left-arm task left the right arm 0.6248 rad out, because the runner parks the idle arm and never brings it back.

D146 A readiness gate that could only ever say UNKNOWN

`session_manager` imported a module that does not exist, so every joint state callback raised `ImportError`, a bare `except` swallowed it, and the readiness panel reported “home reference not loaded” for the life of the node. **It survived because it never went green on a wrong answer** — and UNKNOWN is a legitimate reading for a gate with no data *yet*, so nothing distinguishes “no data yet” from “no data ever” from the outside. **A gate that can only ever say UNKNOWN is worth a test of its own.**

D147 And the home check still measured the wrong instant

The pre-run check passed and a clip recorded two hours later shows arms visibly off home. Between the check and the first published waypoint the scene node comes up, the manifest is written, and under full autonomy the arm is driven to an observe pose. Measured at the first commanded waypoint with everything upstream reporting success: **left 0.7955 rad from home, right 0.6249**. The clip’s own record carried `opened_on: presentation`, which records *whether staging succeeded* — it says nothing about where the arms were when the task first commanded anything.

D148 Every reachability sweep in the repository solved at the wrong wrist

The shared rig read the *live* end-effector orientation off `tf2` at construction — the home wrist — while the runner writes the workspace anchor into every waypoint of every task under every mode. Measured: **32.26° apart on the arm the pick-and-place task runs on**. *And the documented work-around did not work:* one script passed “its own anchor” to the solver and the value it passed *was* the home wrist; another had no orientation argument at all, so the instrument cited as the evidence for the flagship task could not have asked for the right orientation even in principle. **What it did not overturn:** the shipped layout re-verifies at the anchor, 0 failures, twice. **What it did:** 22 surviving centre-on-surface cells became **0 of 3 360**.

D149 A one-axis sweep of a two-axis constraint reads the best cell off the wrong axis

“Raising the surface to 1.020 costs 0 of 54 pick waypoints” was measured on the six *pick* paths; the path the task sends is 171 waypoints and also contains the placements, the transits and the standoffs. Acting on it put the full-path verifier at **36 of 171**. Swept properly in height *and* near edge together, the best free pair is 0.980/0.100.

D150 Two surface constants 150 mm apart, and nothing compared them

One literal positioned every object, pad and marking tile; the other was the only surface with geometry. **It survived a code read because it was written down** — a comment stating “it is 170 mm below the work plane and nothing rests on it”. *And a first attempt collapsed them into one owner, which is wrong in the other direction:* that makes the objects rest on the table by definition and the render disagrees. They are two facts, and the gap between them is a third.

- D151 A check that reported BLOCKED and could not report anything else**
 “It reports BLOCKED today” is not evidence that it can report anything else. An injection argument now displaces the scene so one instrument answers three ways: 0 mm gives BLOCKED (exit 3), +5 mm gives DRIFT (exit 1), and −120 mm gives **PASS** (exit 0). *And two ordering bugs in that control were caught by running it:* the plane was shifted *after* the objects were checked, so both non-zero cases reported the wrong verdict.
- D152 A ternary with the same answer on both sides**
`cubes = T1_CUBES if task == "t1" else T1_CUBES`. Asking the check about stage 2 measured stage *one’s* objects and stage one’s single-arm pad pair; stage 2’s own layout had never been looked at. **In a file written the same day as another fix for the same class of fault.**
- D153 Support geometry, re-priced at the anchor: five times worse**
 Every number in the support note had been solved at the home wrist. With the lips really in the planning scene and the question asked at the anchor: 109 of 171 IK failures against the old table’s 22.
- D154 The workspace marking was 116 mm below the work it bounds**
 The previous session had moved it down to the table, on the argument that “paint goes on the thing it is painted on”. **That is a good argument about paint and the wrong argument about this marking** — it is the boundary a participant is told to keep the work inside, and drawing it under the work put the whole task outside its own boundary.
- D155 A number in the report that could not be reproduced, said so rather than rounded to**
 The report was “left 0.945 against a pad at 0.987, a 42 mm gap”. The code has one marking height and one pad height; they measure 0.9815 and 1.0980, so the gap is 116.5 mm and identical for both arms. **The only 42 in the geometry is a pad offset component.** The defect was real regardless.
- D156 The marking was 192 filled tiles with a bounding box that asserted a third of a region nobody measured**
 It is an outline now — an edge wherever a cell adjoins a non-cell, so concave stays concave — 20 bars instead of 196 markers, painted on the surface and clipped to it, with 40 cells dropped for having no surface under them and the count printed rather than absorbed.
- D157 Removing the bounding box left a label reading its corners, and the render check filed the empty scene as OK**
`tick()` raised on its first frame, so the scene came up with **no table, no objects, no pads and no marking** — **filed OK at 18 % ink, because the wearer is 18 % of the frame.** With the scene publishing, the same views read 46–50 %. **An ink fraction answers “is anything drawn”, not “is the scene drawn”.** The renderer now subscribes to the task’s object topic and refuses to shoot without the task’s own namespaces.
- D158 A seed with 16 IK failures that always had them**
 Identical at the raised surface and at the old one, so it is the anchor fix and not the surface. The recording sweep runs a different seed, so no recorded clip is affected — and the published claim of “0 IK failures on stage 1 and all three stage-2 seeds” had been measured at the home wrist.

17.8 2026-08-18 — language, the grasp constant, and the centre priced

A grammar that dropped a clause in silence, a constant that had been 13 mm wrong since it was written, and five levers priced against the centre of the table.

D159 The shipped grammar could not say the task’s own sentence, and one failure was in the dangerous direction

“Pick up the blue cube and put it on the blue pad” parsed as a grab with **the place clause silently dropped**. Executing it grabs an object and stops, and the operator has heard an acknowledgement. The other two phrasings were refused outright.

D160 The target must be read from the head, not the sentence

The extractor takes the *first* colour it finds, which on “put it on the blue pad” is the *destination’s* colour — so the arm would go looking for a blue object nobody named.

D161 And the two-targets check had to be scoped to the head as well

It counts distinct colours and nouns and refuses on more than one, so “pick up the blue cube and put it on the green pad” — one object, one place — was refused as an ordinary instruction. Surface words are deliberately kept out of the noun vocabulary for the same reason.

D162 Dropping verbs from the vocabulary to stop them being repaired broke ten tests at once

A correctly spelled **grab** is one edit from **gray**, so with verbs no longer counted as words it was *repaired into a colour*. **“Never repair a verb” and “never touch a verb” are different rules and both are needed**: one set to leave alone, and a strict subset that may be repaired *into*.

D163 Politeness became a second action

“Could you please just put the blue ones on the blue pad for me *when* you get a chance” was refused as more than one action. **when** is one edit from **then**, the repair took it, and **then** is a sequence word. **A word that decides whether a sentence is refused is as load-bearing as the verb**, so negations, sequence words, conjunctions and relational words are no longer repair targets.

D164 A compound instruction executed half of itself

“Put the blue ones on the blue pad and the green ones on the green pad” — two complete instructions sharing a verb — **planned the first clause and dropped the second in silence: two objects moved of four, reported as a success**. Three of 75 phrasings did it, all compounds. *Mechanism*: the destination split cuts at the *first* destination cue, so the head was “put the blue ones”, and the two-targets check counts targets in the head — one target, passed. **Every part behaved as designed**.

D165 The before-column came back identical the first time it was run

Which is indistinguishable from “the change did nothing”. The temporary tree holding the old grammar was appended to the path *after* the real one, so the import found the current file. The runner now asserts that the module it imported came from the tree it wrote.

D166 The grasp constant had been 13.47 mm too long since it was written

It was *derived* — the magnitude of a world-frame vector recorded at the anchor — and a test asserted the two agreed to 0.1 mm, which they did, **because one was arithmetic on the other. A test that derives its expectation from the thing it is checking cannot fail**. Forward kinematics on the two finger-tip links says 0.09833 m, with the two arms agreeing to 0.000 mm. **The symptom had been readable the whole time**: a pad miss of 13.48 mm on all four objects of both arms, *identically*, which is the signature of a constant and not of a path.

D167 And correcting it broke the approach, which is the useful part

At the shipped heading the gripper base fouls the table by 5.2 mm and **0 of 40** grasps solve. Swept at $N = 10$ per cell, 40 of 40 solve from heading -45 to -57.5° and 0 of 40 outside it. The heading is now -50 : the middle of that window, and the only value that also solves at a different elevation, so it has margin in both directions rather than merely working.

D168 One run in three put a pad slot inside the wearer’s own upper arm

The same waypoint read 0.1611, 0.1787 and 0.1483m against a 0.150 floor over three walks — **with zero IK failures every time**. That is null-space branch scatter landing on a column with no margin to absorb it. The pads moved to a band where the closest thing to the person is the mount, so a bad branch cannot make it worse.

D169 Neither arm crosses the centreline

0 of 10 IK solutions at every cross-side pad slot and every cross-side object, both arms. **Not marginal — nothing solves**. So an object can only be delivered to the target on its own side; the randomised stage draws the *side* and the colour follows, and the builder refuses an object whose colour names the other side’s pad rather than emitting a path whose every waypoint fails IK for a reason nobody would connect to colour.

D170 The specification audit had been dead since the task rebuild

It called a builder that had moved, so **the pre-recording pass — the thing that exists to catch “reported done and later found absent” — died with AttributeError the first time it was run afterwards**, and its checks were still written for the single-armed version of the task.

D171 Two objects stayed on the table and the run said exit 0

The clip failed only because the object that was missed happened to be `items[0]`, **which is the only item the sweep’s placement check ever read** — for a one-object task that is the task; for four objects it is a quarter of it, and the other three could not fail.

D172 And the two failures were not the same defect

One object was *reached and lost to timing* — the pads were 11.0mm from it with the hand still open, and the fingers did not read closed until the arm had moved 48.8mm on. The other was *never reached*, stopping 42.3mm short, 58% of the way down its own descent.

D173 A schedule whose result depends on what else is running on the machine

The same run, launched by hand with nothing recording, picked all four. The dwell at the pick is fourteen waypoints, tuned without the recording load; under eight video captures and RViz the follower lags past it. *Mechanism*: the grip *command* had been gated on arrival — twice — but **the schedule itself never was**. Ticks per waypoint are fixed; the arm’s lag is not. *Two quantities were being confused*: arrival (the pads are not on the object yet) and the grip (the change was commanded and the fingers have not done it).

17.9 2026-08-19 to 08-20 — the VR path, and the first real arm

A bridge that had never started, a frame that was for a different world, and a day of discovery partitions we caused ourselves.

D174 The WebXR bridge had never been started. Not once.

`self.clients = set()` — and `clients` is a read-only property on the ROS node base class, so assigning it raises during construction. The node had been carried in the tree, in the launch file and in two documents as “the fallback for a headset without USB access” since it was written, **and the failure is on the first statement of its constructor: a single attempt to run it would have found it**.

D175 And its axis map was for the standard ROS frame, not this project’s

Measured through the real node, real socket, real topic: a hand moved 3m forward and 1m right commanded **3m to the wearer’s right and 1m backward**. *And the two transports genuinely need different conversions*: WebXR is right-handed, the vendor route is left-handed, and copying one into the other inverts every rotation. One test exists purely to fail if somebody “unifies” them.

D176 A certificate exception is granted per origin, and a WebSocket cannot prompt for one

Serving the page on one port and the socket on another means the operator accepts one warning, gets a page that looks completely healthy, and a socket that fails with no reason shown anywhere. **From inside a headset that is indistinguishable from a bridge that is down.** Both now come from one port and one TLS context.

D177 Prove the network before the certificate

Laboratory Wi-Fi commonly has client isolation — both clients reach the internet, neither reaches the other — and from inside a headset that looks exactly like a certificate problem, so the natural reaction is to go and fight the certificate. A deliberately plain-HTTP reachability page settles it first, with a WebSocket echo beside it because some networks pass HTTP through a proxy that eats the upgrade.

D178 The mapper’s accumulating term is the rate limiter, and it is a lag

The filter is rate-limited, so whenever the command moves faster than the limit it falls behind and cannot catch up while the motion continues. **That is why the four misses grew by 27, 41 and 50 mm instead of repeating one number.** It is published as `lag_m` now, with a warning past the 30 mm capture gate.

D179 Tracking loss did not drop the clutch

Staying engaged across a dropout means resuming from a filter latched before the loss against a controller that has since moved — and the clutch’s “zero jump by construction” guarantee is void for exactly the motion nobody saw.

D180 An analyser that called a working gripper broken, caught by its own self-test

It correlated the trigger against the commanded angle over the whole trace and returned $r = 0.033$ on a known-good *latching* trace. **A latching gripper deliberately holds the firmest value seen while the trigger falls away, so a whole-trace correlation is near zero for a perfectly working one.** Correlated over the pre-latch region it reads 0.989 on the same trace.

D181 The alignment rotation was promised in a docstring for months and never existed

The code added the controller displacement *raw*, so the controller frame *was* the world frame by assumption — true only for an operator standing behind the wearer facing the same way, which is not how the rig is driven. **And the intuitive fix is a reflection:** “they’re facing me, so mirror it” has determinant -1 , which looks right applied to position and mirrors every *orientation*. It is exposed as one angle precisely so that no value of the parameter can produce one.

D182 An ambiguity uncovered and not settled

The documents say world $+x$ is the wearer’s right. The arms sit the other way round — `left_end_effector_link` at $x = +0.117$ and `right_end_effector_link` at $x = -0.160$. Either $+x$ is the wearer’s left, or the arm named `right_` is physically on the wearer’s left. **Those need different fixes and only one is an axis-map change. Still open.**

D183 Controller tracking loss froze nothing

The tracking flag is derived from the last received frame, stamped on *every* arrival — *before* the per-controller validity check. So it caught a network dropout, a sleeping headset and a backgrounded application, and **not the case it is named after.** Measured with the real headset: covering a controller for 5 s produced **zero** tracking-loss transitions while the stream ran on at 90 Hz. Head-worn this is nearly harmless because the hands are in the headset’s cameras; with the headset on a shelf, occlusion is routine.

D184 Nothing consumed the freeze

The safety node computed freezes, published them, and no node listened. So “the observer e-stop was withdrawn” and “the tracking reference moved” were both states the system could be in *while still commanding the arm*. **It caught its own author immediately:** the first chain test came back “clutch did not engage” because the observer e-stop publisher had been killed in a restart.

D185 The tracking reference is a physical object, and a nudge is silent

Every other failure in this system announces itself. A knocked headset leaves the poses valid, the rate at 90Hz and the controllers tracked, and silently rotates the frame every subsequent pose is expressed in. **The rotation threshold is the one that matters:** a 10° twist barely moves the headset and puts every command 10° wrong. *And the freeze must survive the ordinary unfreeze path*, because poses flow the whole time it is wrong.

D186 The mapper was reporting follower lag as a clutch jump

At engagement the command is set to the arm’s own pose, so the jump the *arm* makes is zero by construction. The published quantity was how far the *follower* had fallen behind its last target: **a real run reported 150 mm of “re-engage jump” across an engage on which the arm moved zero**. Published under that name it would have gone into the write-up as evidence the clutch is bad.

D187 Three successive measurement harnesses recorded exactly zero

Same cause each time: a transform listener constructed on the node joins its default mutually-exclusive callback group, so a lookup inside a timer callback blocks *every other callback on that node* — including the subscriptions. Given its own executor it recorded 2 146 samples immediately.

D188 And then a result that was 0.000 everywhere, which is not a result

Every clutch quantity read 0.000 mm. The data said why: the commands were moving correctly and **the end effector never moved at all — one unique row in 2 146 samples**. The follower log had it: 47 of 64 IK solves succeeded, so the synthetic drive had pushed the target into a region where IK fails. **That makes the clutch numbers degenerate, not good**, and the re-engage jump is still not measured.

D189 The launch swept the ROS daemon’s own shared memory

The clear removed the segments unconditionally, and the daemon is a participant. **It does not die when they vanish — it goes deaf**. Node listing then returns nothing against a fully healthy stack, and the bring-up reports a discovery problem: *true, specific, confident, and pointing at the wrong thing, because the sweep two seconds earlier caused it*. **Four full teardown-and-restart cycles were spent before the sweep was suspected**, because the message was so specific that it read as evidence about the network.

D190 And the sweep’s own live-process guard named three nodes by hand

So the VR stack, the real-arm bridge and the observer e-stop were all invisible to it: a sweep “with the stack stopped” could still be a sweep under a live observer e-stop. **A guard that names processes by an explicit list will be wrong the moment a new process exists. This one was wrong for the observer e-stop.**

D191 timeout N ros2 ... sends SIGTERM

A hard-killed DDS participant holding shared memory poisons discovery for everything that joins afterwards. There were ten such call sites — including `ros2 topic hz`, which never exits on its own and is therefore *always* killed.

D192 The observer interlock did not gate the thing it is named after

The safety node tested the observer flag directly in the freeze path with no parameter; the parameter gated only the enable service. Setting it false therefore disabled the real-arm request and left the clutch held shut anyway. **In the laboratory this presented as a broken clutch:** grip pressed, gripper moved, arm dead, and the only explanation in a log on a machine the operator could not see while wearing the headset.

D193 Two arms cannot share the high-level path over WSL

One arm sustains 21.0 Hz with 12–33 ms of send latency; two arms give 8.1–12.2 Hz with latency **spiking to 519 ms**. That is past the bridge’s 0.5 s watchdog, so it zeroes the speed and **the joints stop while still being commanded** — and the homing node then reports BELOW VELOCITY FLOOR, a true statement about a symptom two layers down that sends you to the homing gains instead of the network. *Both arms froze simultaneously, which is the giveaway.*

D194 The real-arm clearance check was measuring almost nothing

It iterated three wearer parts against a model holding **twelve** — never the neck, the thighs, or the wearer’s own arms, which are the geometry that binds the right arm inboard. **And it swept only the distal links**, starting at the forearm: the shoulder and both half-arm tubes — the segments a shoulder mount actually swings across a person’s head and chest — were never measured. *The hand can be a metre clear while the upper tube is inside somebody’s neck, and the check reports the hand.* **Not re-measured:** every clearance figure from a real-arm run before this date was taken with the proximal half of the arm invisible.

D195 And the one that is not a code bug

Every real-arm run so far has used the setting that *deletes the wearer from the model entirely*, printing “clear inf m” on every homing tick. Correct for a bench-mounted rig with an empty harness, and wrong the instant anybody stands in the arms’ volume. It was left set after the situation changed. **No real-arm run has ever happened with a wearer in the model.**

17.10 2026-08-22 to 08-23 — the window, the motion generator, and the accuracy pass

Four defects behind a passing audit, five defects in my own new code, and six verifiers asking a question the robot does not ask.

D196 The wearer floor in the grasp pipeline could never fire

The scorer returns two clearances. The one it read is the *whole* chain, including the immobile mount stub — and that stub is nearer the wearer than anything the joints can move, so it reads the constant **0.2202 m in every pose the arm can reach**. It was compared against a 0.15 m floor. **Every grasp this pipeline has ever approved was checked against a test that could not fail.** It reads the moving-chain clearance now, and a test aims the hand at the middle of the wearer’s torso and requires a refusal that names the wearer.

D197 Refusals asserted instead of measuring

“This point is outside the arm’s 902 mm reach” is a conclusion about the arm drawn from a search. It reports the distance the seeded search actually reached now. *Stated as a hardening rather than a fix:* the test could *not* make the old seeding fail.

D198 The GUI audit had hung on a modal dialog since that dialog was added

So it never reached the presses that would have revealed the rest. Behind it: the window class had no logging method, so **five session buttons raised inside a Qt slot — which PyQt swallows — and did nothing, silently**; the process-scan module was never imported, so the stray-publisher preflight — *hard constraint 3’s own guard* — had never run; and one panel rendered three of its four groups, so a whole group of buttons did not exist.

D199 Three defects that lived for weeks behind a passing audit

RViz embedding an *invisible* window; a control column capped below its own content; and every mode button below the fold. **None of them is visible from a return code.** *And a trap in the looking:* a screen grab of the X root records black under WSLg — the window must be grabbed by its ID.

D200 The window was six pixels too wide, and the compositor answered by not drawing it

The window sized itself with a hardcoded resize to 1920 pixels on a 1920-pixel screen, so the right edge landed at 1926. Qt renders such a window correctly and X reports it viewable; **WSLg declines to composite it and gives you a taskbar icon that will not open.** *The capture tool had been naming the fault the whole time* — it refused the window with BadMatch, which is what X says about a window not wholly on-screen — **and it was read as noise for most of a session.** The startup line offered exactly one explanation, “it is behind another window”, and it was the wrong one.

D201 The gripper camera panels were subscribed to topics nothing publishes
Found by the same look.

D202 The per-joint step clamp was the entire motion generation, and it has three separable faults

Not synchronised: a joint needing 0.10 rad arrives in one cycle and one needing 0.50 takes two, so the joints sit at different fractions of their own travel and the hand leaves the line the IK solution implies — **51.3 mm off-path on a rig whose grasp gate is 30 mm.** **Not continuous:** from rest the first cycle commands a whole step, whose implied acceleration is $1343\times$ the jerk-limited one. **It never read the joint limits:** 17.5 rad/s against a declared $1.3963 - 12.53\times$ — and the limits differ per joint, which a single step size cannot express. *And it is not a tuning value:* swept against its own step size the excursion does not tend to zero, it *settles* at 68 mm.

D203 And the metric in the diagnosis conflates two things

“Where the hand is at cycle k against where it would be if every joint were at fraction k/N ” charges a jerk-limited generator for having a velocity profile — it reads 69.8 mm on the new generator and the number means nothing. The reported figure is now off-path distance to the *curve*, which is parameterisation-free.

D204 Re-planning every cycle destroys phase synchronisation

A re-plan cannot reproduce the phase-synchronised profile it is halfway through: the joints’ fractions of travel spread and re-converged, leaving the commanded path 0.0264 rad off the straight line against 8.9×10^{-17} when riding one trajectory. **Every other check passed either way;** the straight-line check is the only thing that saw it.

D205 And what forced the re-plan was floating-point noise

The target was recomputed each cycle as `pos + wrap(target - pos)`, which is not bit-identical to `target`. The goal moved by about 10^{-17} every cycle and the library compares by value, so every cycle looked like a new target.

D206 The fallback claimed to be jerk-limited by reporting zeros

It returned an acceleration of all zeros because it does not model one, and the continuity check reads that field — **so the fallback passed by fabricating the quantity being checked.** It reports its real implied acceleration now ($62.5\times$ the jerk-limited step) and a test requires it to be over $10\times$: this backend must never be able to claim smoothness.

D207 The off-path metric had a resolution floor and reported it as behaviour

A 4001-point reference curve over a 0.4 m path is a 0.1 mm grid, so the nearest-sample distance bottomed out at half a grid step and a generator that is on the line to 2×10^{-16} rad measured 0.0509 mm off it. **The instrument’s resolution read as the robot’s behaviour.** Refined by bisection, the same path reads 9.4×10^{-9} mm.

D208 An arrival test with an absolute tolerance on a per-joint quantity

Under a 10^{-6} rad gate a phase-synchronised move reported arrivals spread over two cycles: every joint was at the same *fraction* throughout, and the ones moving 0.02 rad were inside the gate a cycle before the one moving 0.50. The travels span $25\times$, so an absolute gate measures the instrument’s scale.

D209 CONTINUOUS_IDX had six definitions

Seven modules each declared which joints are continuous. **They agreed, which is luck rather than design** — it is a fact about the URDF, and seven copies of it is the same shape as two home poses.

D210 The pad midpoint is not a constant, it is a function of the opening

The gripper’s fingers swing on a four-bar: wrist-to-pad is 0.09833 m wide open, 0.10976 on a 40 mm object, 0.11179 on a 20 mm one. **Both numbers this repository argued about are points on that curve** — the 2026-08-18 note calls one “13.47 mm too long” against the other and treats the difference as an error in one of them. *Neither was wrong; they are two gripper states, and the quantity was compared as though it did not have one.*

D211 And it explains why the evidence flipped

The verifier read the finger tips out of `/compute_fk`, which uses the simulation’s *current* gripper joints — whatever the last thing to touch the hand left them at. On one date that was an open hand and the pad miss read 0.00 mm, which is the measurement quoted as the evidence for the constant. Five days later, **same geometry, same code, it read 13.52 mm**, because the hand was nearly shut. **A measurement whose answer depends on leftover state.**

D212 And the instrument could not have done it before

The forward-kinematics helper set every gripper joint to zero and ignored the gripper’s mimic tags entirely, so the finger tips — the only part of the robot that touches anything — could not be placed anywhere except one arbitrary opening.

D213 The declared pad offset was 13.45 mm out, and it was invisible because it cancelled

The scene *draws* each object through the same offset, so the picture was right whatever the constant was. **What did not cancel is where the fingers went.** Now derived from the measurement: 13.45 mm $\rightarrow$ 0.05 mm, with 6 024 IK calls and 0 failures on re-verification.

D214 The shell bias was fixed in the scene reader and not in the pick path

One module had corrected it for two days; the function that `find_object` and the recorder actually plan grasps with never had it — 10.5 mm in the budget, and two readers of the same scene disagreeing. One corrected-centre function now, used by both, which *says* the centre is the shell’s when there is no support height rather than reporting a biased number as a measurement, and *refuses* an object whose top is below the surface it is said to rest on.

D215 A verifier had raised on import since the task became two-armed

It read a constant deleted seven days earlier. **It did not verify the task wrongly; it did not run at all, and nothing said so.** *The second time that file has held its own stale copy of the layout.*

D216 About twenty scripts solve at the home wrist and call it the anchor

Measured against the shipped home: **42.94° apart on the left arm.** The fault was fixed in one module five days earlier *and nowhere else*. What it costs, measured: one task’s own pick solves **10 of 10 at the anchor and 0 of 10 at the home wrist** — which is why a verifier’s “known-good pick” control was failing on a task that runs.

D217 The verifiers ask for EXACT while the follower has run a 15° cone since the day before

So a waypoint the robot solves by tilting two degrees read as unreachable: 32 IK failures per arm $\rightarrow$ 0; a choreography routine 64 of 382 $\rightarrow$ 0; the total across three routines 171 $\rightarrow$ 68. *The commanded orientation is candidate 0, so nothing that solved before stops solving.*

D218 One function returns radians while its two siblings take degrees

So a description printed “within a 0 deg cone” for 0.2618 rad — and worse, another verifier passed the radian value on to the candidate generator, making its first “cone” run a **0.26°** cone that rescued almost nothing. **The value was right in one place and wrong in the other, and the sentence was wrong in both.**

D219 A stage-2 run silently overwrote the stage-1 baseline

Same output file for both stages, so it replaced the record a test reads — and did, turning three green tests red with a result from a different stage.

D220 Moving the grasp changed the task side and not the scene side

The clip came back “1 object never moved” on a run whose runner exited 0, whose four objects were all detected at the right coordinates, and whose gripper closed. **Nothing downstream could tell, because the scene is also what scores the grasp** — it measures the pads against the object *it* drew, so a consistent-but-wrong pair scores perfectly and moving one side scores a miss. *That disagreement has now produced that sentence three times: 49 mm, 111.8 mm and 11.43 mm.*

D221 The carry task’s grippers had never closed, and the tray was not why

The elastic tray was blamed and fixed, and the fresh clips still read **0.000 rad for every one of 620–1219 samples on both arms in all five modes**. *The cause is a gate: the runner holds a commanded grip until the pads reach the object* — right for a pick and **unsatisfiable for a task that declares no grip object**, which this one does, because its own specification says the grippers start closed on the tray and stay closed. The change stayed pending for the whole run, every run.

D222 Three instruments reporting defects that did not exist

(1) The accuracy table mapped objects to pads with **i % 2** — a single-arm alternating row — and scored two objects against the pad **0.58 m away**: placement read 319.7 mm with 264 mm of variance on clips whose own scene log says “placed 56 mm from target”. (2) The status table called a task under four modes a REAL GAP when it declares it runs under one mode only — *a by-design absence rendered identically to a missing clip*. (3) The gripper-motion verifier required a RELEASE from a carry task whose whole definition is that it never releases, behind a hardcoded task-name exemption.

D223 And a fallback written into that third fix did exactly the same thing

The helper wrapped its import in a bare `except` returning the conservative answer. **The import failed, so every task came back “releases” — including the one the helper was written for** — while the same function returned the right answer whenever it was called from anywhere else. *A fallback that turns a wiring failure into a plausible default, written into a fix for that same defect.*

D224 The wrist was re-aimed at every cell of the mapping sweep

The tool axis pointed *at* each grid point, so the orientation changed between neighbours 90 mm apart: the arm swung continuously and no two views were taken from the same attitude, **which means a residual wrist error was a different error in every frame and could never cancel or be measured**.

D225 Stationary is not the same as arrived

A joint can sit 0.02 rad from its commanded value and still be moving through it, and a frame captured then is smeared across two poses *while every statistic downstream reports fine*. The stillness test now watches 400 ms and requires no joint to move more than 0.0015 rad — a tenth of the arrival tolerance.

D226 The surface height was reporting the plane’s intercept at the origin

Which is a point inside the wearer, and it produced a -5.2 mm surface error that was reported as a measurement. The true figure is $+0.2$ mm.

D227 A docstring quoting numbers that had not been measured

The first draft of the sweep claimed “3.12 m against 1.68 m, 46 % less”. The measured figures are 1.702 and 1.140, 33 %. **Written before the check was run, which is the thing this register exists to stop**; corrected in place with the mistake left visible.

D228 Speech built by deleting words from writing

The narration regex-stripped the coordinate out of a finished sentence and left the preposition behind, so the robot said “*Left arm: reaching at.*” The spoken form is not the written form with bits removed.

D229 The stillness/self-test check grepped its own source and found the words it was searching for

The first version of the “this stage imports no task module” check searched its own text. It parses the syntax tree now.

D230 The test gate could invent a failing test

It matched `^FAILED (\S+)` over the whole test output, and the docstring checker *echoes* the source lines it objects to — so `FAILED = "failed"` in a source file became a failing test named `=` on a suite where everything passed. **A gate that cries wolf is a gate somebody bypasses.**

D231 The frame extractor never extracted a grasp

It mapped event wall-clocks through the *scene node*’s start time, which under full autonomy is about 300s before the video capture begins — so every grasp landed at +380s inside a 48s clip and was skipped in silence. **It printed “10 frames” and exited 0:** openings and final frames only.

D232 The scene measured the second stage at the wrong approach

A pad-offset guard read `if self.task == "t1"`, and stage 2 — rebuilt on stage 1’s geometry and commanding stage 1’s approach — fell through to the anchor: **99.1 mm out on the left arm and 83.3 mm on the right, against a 30 mm capture gate.** The runner commanded all four picks and both grippers closed; the scene recorded two grasps and filed “2 object(s) never moved”. *A broken measurement that read as a flaky task.*

D233 A one-button bring-up that said READY with no headset in the building

Its final step checked that the pose topic was *listed*, and the bridge creates its publishers at start-up — so the topic exists the moment the bridge does. **The operator would have squeezed the grip, watched nothing happen, and had a window still saying ready.** It counts message *arrivals* over a window now. *This is the “feature present but does nothing” row landing in the step whose whole job is to say ready.*

D234 Clearing stale shared memory wedges the daemon, and the bring-up blamed the simulation

Measured: after a clear, topic listing returned 2 topics against a fully running stack — and the diagnosis produced was false, specific and confident, which is the worst combination, because it sends you to the wrong log.

D235 Three repairs recreated their own fault

Found only by pressing the button repeatedly from a dirty machine. Clearing shared memory restarts the daemon, **and that orphans the daemon’s own segments — four repairs in a row. And the sweep destroyed a starting simulation,** because a process coming up has segments it has not mapped yet, so they look unowned. Staleness is asked of the kernel now, with a 20-second age floor.

D236 The observer e-stop interlock was unsatisfiable, and silence read as presence

Nothing ever published the presence topic, so the interlock was a permanent refusal — exactly the kind of gate somebody routes around. And it was a *latch*, so a publisher that stopped left the system believing somebody was still standing there holding the button. It is a 5 Hz heartbeat now, with two seconds of silence treated as withdrawal.

D237 The mapper was silent while disengaged

Its state was published only inside the engaged branch, which made “disengaged”, “reset”, “frozen” and “the process died” **one single observation.**

D238 The clearance floor and the planning scene held two different bodies

The wearer tracker reached MoveIt’s planning scene and *not* the guard, which builds its wearer at *import* time — so the body MoveIt planned against and the body the 150 mm floor was enforced against were two different bodies, and the floor had the mannequin.

D239 A wearer-fallback check that had never actually run

The two-person gate produced a false pass until each refusal was required to *name* the injected fault.

17.11 2026-08-25 — the cube pick on real hardware

Seven instruments that produced confident wrong answers in a single session, and two transport traps.

D240 The gripper camera's extrinsic is wrong by 8.5° , and it rotates with the wrist

The table cannot move, so the table normal measured from any arm pose must be the same vector. Measured from four poses: 9.74° , 15.42° , 16.26° and 18.14° off world vertical — **a worst pairwise disagreement of 8.45°** , which is 34 mm of position error at 0.23 m and 59 mm at 0.40 m. **No open-loop grasp planned through that transform can land inside the 30 mm capture gate, and none did.** *The depth camera itself is excellent and was never the problem:* 94.8% plane inliers at 0.46 mm RMS, and a real 40 mm cube measured at $39.2 \times 38.9 \times 40.9$ mm.

D241 The clearance model returns a constant dressed as a measurement

Asked about the pose in which the arm was *physically against the mannequin*, it returned **367.0 mm** — the same value it returns for a visibly clear pose. It samples link *origins* only, so it misses the arm's body between joints. **Hard constraint 11 is currently unenforced in code**, and a pass from this model means nothing.

D242 A status topic republishing a frozen snapshot with fresh timestamps

A pose read from it matched a saved file to 0.144° while the arm was somewhere else. It is published by a leftover homing node, not by the bridge, and when its input died it kept publishing the last snapshot with fresh stamps. **Key on distinct source stamps.**

D243 A contact signal with no free-space baseline

25–41 N reported as “in contact”. Without a baseline it cannot separate contact from gravity and payload.

D244 A fixed settle time against a proportional controller

“Did not arrive, 7.2° out.” The bridge closes error at $k_p = 0.5$, about a 2 s time constant, and a fixed 2.5 s settle leaves a third of the error standing. **Arrival is waited for, not timed.**

D245 Joint error by subtraction on continuous joints

“STALLED at 360.076° .” Joints 3, 5 and 7 are continuous: -179.05° and $+181.02^\circ$ are the same pose. **The run had arrived.**

D246 A publish loop starving its own subscription

Spinning with a zero timeout while publishing at 20 Hz let the cached joint vector go seconds stale, so a motion check read 0.032° on a move that was really running and aborted it — **telemetry showed the arm had moved about 7° .**

D247 A path check that checked the hand against the table

Every leg cleared by more than 100 mm. **The 94° transit swung the arm into the wearer.**

D248 Domain 0's shared memory was polluted so that no *new* process could join

Minimal publisher and subscriber: **0 of 3 messages on domain 0, 3 of 3 on domain 7.** Existing nodes keep working, which is exactly what makes it hard to see. The real-arm tooling moved to domain 7.

D249 An unset middleware variable makes rclpy discover nothing

While the ROS graph is plainly there in other tools.

D250 Two writers on one static transform edge

The vendored camera launch puts `camera_link` to `depth` at $(-0.0195, -0.005, 0)$ against the robot description's $(-0.0275, -0.0096, 0)$ — 8.0 mm and 4.6 mm apart.

D251 Both arms publish the same unprefix camera frame

So left and right collide in one global transform tree.

D252 A vendor camera node aborts the moment a stream recovers

`loadCameraInfo()` declares its parameters unconditionally and runs again on reconnect, so all four camera nodes abort with `ParameterAlreadyDeclaredException`.

D253 And the wrist camera cannot see its own grasp

The last 80 mm push the object out of the bottom of the frame. **That is geometry, not failure:** the last good fix is carried in world coordinates and the remainder completed from it.

17.12 Still open

Faults that are named, sized, and not fixed.

D254 The wearer clearance model, still

Link origins, not link geometry. The most important open item in the project: the 150 mm floor is not enforced against a real arm pose in code.

D255 Mode 02 still cannot grasp

88 to 206 mm against a 30 mm gate, and the miss accumulates. The mapper resets in the node now; the recorded set has not been re-taken under that mode.

D256 The exact grasp pose is refused by the task's own table

It solves 10 of 10 with no furniture and needs a 5.0° tilt with the table in. The follower's cone supplies it, and the tilt costs $2|pad|\sin(\theta/2) = 8.58$ mm predicted, **9.58 mm measured** — inside the gate, and not zero.

D257 Two choreography routines have 68 unreachable waypoints

At the anchor with the 15° cone, down from 171 at the home wrist. Deliberately not re-recorded: **filming a routine the verifier will not certify is how a clip of the robot stalling gets filed as a demonstration.**

D258 One carry clip closes on nothing

0.793 rad, past the free-air bound, with the gripper silhouette unchanged between its open and shut frames. The other four close on the tray.

D259 The repository disagrees with itself about which way is right

The documents say world $+x$ is the wearer's right; the arms sit the other way round. Two different fixes, and only one is an axis-map change.

D260 The terminal joint error stays uncompensated

7.24 mm, against a modelled 1.88 mm with the overshoot enabled. That and the deadband change are two corrections for one error and **neither has been tried on hardware.**

D261 How the terminal error varies with speed has never been measured

The run reported as a speed sweep was three replicates of one condition, because the velocity limit was read once at bridge start-up — and 7 of 36 runs moved *faster* than the limit they were nominally commanded with.

D262 The size of the between-modes difference is unknown

The effect is real and refuted against two alternatives. The recorded set cannot give it a trustworthy magnitude, because both metrics are properties of the observation window and the set has one run per cell.

D263 The camera-to-module transform is the largest unmeasured term in the budget

There is no CAD of the gripper in this repository — only the master arm’s — so it needs the hardware. The 2026-08-25 session sized its rotational part at 8.45° .

D264 The guard cannot react to a person who moves quickly

In five of six modelled limb motions the limb travels further than the entire 150 mm floor before the guard hears about it. **Not a software bug** — a consequence of a fixed margin, a half-second perception chain, and a person who can move a metre in that time.

D265 And the home pose has drifted between two of its storage sites again

Measured while writing this report, 2026-08-26: the working tree’s URDF spawn blocks were updated during the 2026-08-25 hardware session and `config/home_positions_left.txt` — *the file every node actually loads* — was not. Worst disagreement **0.9758 rad on joint 6** against a 0.001 rad tolerance. **The single-source test caught it, which is what it was written for.**

CHAPTER 18

Every change

All 392 commits in order, oldest first, each with its own account of what it changed and why. Messages in this repository are written as findings rather than as labels, so this chapter doubles as the project diary. Bodies are condensed to their opening argument; the full text is in `git log`.

2026-07-14 — 1 commit

df8cc54 **Initial commit**

2026-08-08 — 22 commits

2b4f133 **Restructure into six packages; vendor serial; add autonomy, VR and GUI**

Six packages with a one-way dependency arrow. `srl_teleop` depends on nothing in-repo, because it is the baseline condition of every experiment and a dependency on `srl_autonomy` would make the comparison circular. FILES MOVED, NOT DELETED The restructure relocated six modules and two launch files.

844e19e **README: fix the patch command; record a verified clean-clone build**

The documented invocation was wrong and a fresh clone could not be rebuilt from it. `git apply -directory=src` doubles the path, because the patches already begin with `src/` – every hunk failed with `"src/src/ros2_kortex/...: No such file or directory"`. Correct form is `patch -p1 -forward -fuzz=3` from the repo root. The fuzz is needed: 0003 applies at `"fuzz 3 (offset -10 lines)"` because `ros2_robotiq_gripper` main has moved since the patch was cut.

bcd034 **NEXT_SESSION: add a CHECKPOINT so a fresh session starts oriented**

Records what works with its evidence, what is blocked and on what, the lab order, and the standing caveat that every protocol number is IK feasibility in simulation – nothing has been driven by a human through the master arm, and the DIRECT condition of every task depends on channels that are currently incoherent.

4b019dc **T2 scenarios: four verified, and the documented geometry was wrong**

The bimanual runner refused t2 by name because `verify_scenarios.py` never generated any. `scripts/verify_t2_scenarios.py` now generates four graded scenarios and verifies the COMPLETE fill path – pick, lift, transit, above-opening, release, retreat – at every waypoint, while the holding arm is simultaneously solvable at the hold pose. Endpoint-only checking would have passed paths that carry the block through the wearer, which is how the T3 respec failed the first time. 4 of 4 verified, path 6/6 each.

- 398f8b7 **Audit every protocol coordinate at N=10 over the whole path; instrument T2**
 AUDIT — T2's coordinates passed a single IK call and failed a repeated one, so every figure in every protocol was re-verified. scripts/audit_scenario_reachability.py grades each pose SOLID (N/N) / MARGINAL (1..N-1) / FAIL (0) over the whole DENSIFIED path rather than its endpoints. Result: 17/17 scenarios and 17/17 protocol-table coordinates SOLID, 0 MARGINAL. Getting there changed a lot. WHICH FIGURES CHANGED
- cad57ac **Two-person reframing: literature, hypotheses, T8/T9, measures, session order**
 The arms are worn by one person and driven by a DIFFERENT person. That is not an emphasis change – it changes who is a participant, what is measured, and which literature the work sits in. THE GAP, CORRECTED TWICE ————— The brief's reading was "Fusion established the operator/wearer architecture but did not study shared autonomy within it." Correct, and not the whole story. Two corrections that must survive into any write-up:
- efa280c **T8/T9 analysers, and a known-answer test that hardened T9's manipulation check**
 T8's analyser reports ANTICIPATION first and its SIGN before its magnitude: negative means the wearer began moving before being asked, which is a predictive model of another person's intent built by someone with no control over the limbs and no proprioception of them. It also reports repositions per trial alongside latency, because a dyad can cut latency by the wearer shuffling constantly, which is not learning and is worse for the wearer. Both analysers refuse to invent numbers from empty columns.
- 0d471b7 **Task-band clearance probe: which link, and how far the band must move**
 Recording every task showed every chest-band scenario at 48-67 mm of clearance – IK-valid, contact-free, and below the 120 mm real_robot floor. That is a property of the BAND, not of any one scenario, so it gets its own measurement rather than an inference from 87 clips.
- 8c692e7 **T8/T9 protocols**
 T8 is well-posed by construction: the operator cannot reach (verified IK failure at nominal stance) and the wearer cannot manipulate (no control input at all), so a trial only succeeds through coordination. Neither is told whose move it is – who initiates is a dependent variable and instructing it would destroy the measure. The operator holds still while the wearer repositions, because an arm moving toward a person who is themselves moving is the configuration with the least margin and the wearer cannot see it.
- 1e9001a **setup.py: the bimanual analysers were never installed**
 EXP_SCRIPTS globbed experiments/*/run_*.py, one level deep. The bimanual set nests one deeper – experiments/bimanual/t3_coordinated_carry/analyse_t3.py – so the glob matched run_bimanual.py and NONE of the seven analysers. Every 'ros2 run srl_experiments analyse_t3.py' has failed since the restructure and looked like an installation problem rather than a packaging bug.
- 036e4a6 **87 verification clips, and the clearance finding resolved by measurement**
 Every task x scenario x autonomy condition driven in sim and recorded: 87 clips, 7 tasks. Zero pass through the wearer. Zero show a stationary robot. Task-phase tracking 0.0-4.7 mm RMS across all of them. recordings/verification/INDEX.md lists every clip with WHAT IT SHOULD SHOW, because an index of filenames tells a reviewer nothing – they still have to work out what correct looks like before they can tell whether they see it.

- a47dd45 **RViz screen captures: 87 clips, objects attached and carried, 0 static**
 REAL SCREEN RECORDINGS, not TF plots. All 87 task x scenario x condition runs now carry rviz.mp4 alongside the TF clip, with the task objects visible as markers at their real dimensions and posed from the LIVE gripper pose every frame. x11grab ON WSLg's :0 RECORDS BLACK, and this is now in CLAUDE.md so it cannot be lost again. A full-screen grab with RViz plainly visible measures mean pixel value 0.0 – XWayland windows are composited by Wayland and their pixels never reach the X root window x11grab reads.
- 2747eea **thesis: Imperial MSc template scaffolded, evidence audited, first full draft**
 PHASE 0-1. Template extracted to thesis_report/_template (kept pristine) and read. thesis_report/EVIDENCE.md maps every claim the thesis might make to the file that backs it, and lists nine gaps. Three findings from the evidence pass that shape the whole document:
- a36590c **Four-view clips with the gripper actually operating: 87/87, fingers cycle 45/45**
 THE GRIPPER NEVER OPENED, and that was a real bug, not a rendering artefact. mock_components boots the Robotiq driven knuckle at 0.793 rad – fully closed on nothing – and nothing in the recording path ever commanded it. So every previous clip showed a shut hand while the "grasp" existed only in the marker layer. The object-attachment check did not catch it and could not: the marker genuinely was following the gripper. It was following a gripper that never opened.
- 3a4c160 **Remux two oversized gripper captures (62 MB -> ~1 MB)**
 Their x11grab was never finalised, so the container carried the whole recording rather than the trial. GitHub warns above 50 MB and the content is identical after a stream copy.
- 52727a7 **thesis Phase 3: artefacts ingested, criteria set E'1-E'9, Chapter 1 drafted**
 INGESTED the external design record (45 files, 47 MB) into thesis_report/_source/artefacts/ with PROVENANCE.md: kinematics.cpp (the C++ master_pose_node.py cites by name), J1-J7 printed links, PotArm.step, 8 BOM PDFs, backpack and stand prints with sliced gcode, the WH148 PH1 pot CAD, the MuJoCo log dated 7 Jun 2026, and the 12 master trajectories of 27 Jul 2026. Originals untouched.
- 66ac1ac **Real grasp sequence wired to the existing generator; thesis draft committed**
 THE GRASP WAS FAKE. The arm moved its end effector to a point, the marker attached, and it carried on: no approach vector, no wrist alignment, no pre-grasp standoff, and fingers that never moved. The marker layer was substituting for a grasp rather than following one.
- abd0c74 **Five-task consolidation, real grasp, figures, and the thesis in full draft**
 PART 1 – FIVE TASKS. Nine families consolidated to five. The criterion for the bimanual three was strict: one arm alone must be unable to complete the task GIVEN UNLIMITED TIME. Because the reachable sets are disjoint that can only be met two ways, and both are used: a rigid object spanning the dead band (Tasks 3 and 4) and two targets requiring simultaneous satisfaction (Task 5).
- 502a473 **Five-task verification reproduced from home; single-owner gripper**
 N=10 full-path verification re-run with both arms verified at 0.0000 rad from home: 0 failures, and byte-identical to the previously committed run. That is the first reachability figure in this project to reproduce across runs rather than be re-asserted; it reproduces because the 500 mm respec moved every coupled waypoint off the feasibility boundary. Two real defects in the grasp, both found in the RECORDED TRACE rather than by inspection: 1. Two owners wrote the gripper on the same cycle.

da9950b **Rewrite the thesis: reframe as graceful degradation, add two chapters**
FRAMING. Channel loss is presented as an operating condition rather than as a list of specific faults. Any body-worn master routing fourteen analog channels through rotating joints will lose channels, so the architecture is described in those terms: per-channel health verdicts computed over distinct sensor updates, validation scoped to the channels the active mode consumes, run-time channel disabling with published capability loss, and a reduced-degree-of-freedom fallback.

92d9834 **Write the abstract and acknowledgements**
Abstract: opens on why the problem is specific to this class of system (the base is a person), states the two design premises, and carries the numbers that matter – 63.8 mm cost at the last degradation rung, 6 of 14 channels healthy, 54.5 mm of commanded motion injected by the failing ones against a 272 mm master reach, 137.7 deg measured where 90 deg was intended, 14/14 faults handled, 0 of 63 shared workspace cells with a 0.25 m dead band, and the buildable mount fix at 22 of 63.

ae68df1 **Part 1: measure whether the grasp is real. Three instrument bugs found first**
Every figure below comes from a bare sim stack with mock hardware, and each carries how it was obtained. INSTRUMENT, VALIDATED BEFORE USE. Reading the two finger-tip frames out of TF gives the gap between LINK ORIGINS, not between pad faces. Taken raw it says the gripper never closes below 50.7 mm and that grip_for()'s linear model is 52 mm wrong. Both are artefacts.

2026-08-09 — 60 commits

e22b59c **Part 2: the capability ladder, selected live from whatever channels survive**
Not a workaround for specific broken joints. Any body-worn master routing 14 analog channels through rotating joints loses channels, so the position mapping is SELECTED every cycle from whatever passes health, and the selection is published with what it cost. srl_teleop/capability.py is pure (no ROS handles, no I/O) so the ladder is testable against known channel sets without a stack.

a870374 **Part 3: scene fingerprinting, so calibration runs only when the bed changed**
srl_perception/scene_fingerprint.py is the matcher and is PURE – no ROS handles, no camera, no I/O beyond an explicit path – because that is the part testable against CONSTRUCTED ground truth. Two points 30 mm apart really are 30 mm apart, which is the one kind of synthetic data the standing rule permits, and it is why these figures can be trusted where a rendered-image test of the same logic could not.

9b7862b **Part 4: verify all four mode paths. The VR path was broken; fixed**
METHOD. Every hop confirmed from the LIVE ROS graph: a hop counts as CONNECTED only when the topic has both a publisher and a subscriber. Reading an import and concluding the path exists is the mistake this script exists to avoid, and it is how the defect below survived. THE VR COMMAND PATH DID NOT REACH THE ARMS. quest_vendor_bridge – the transport the project adopted – published /vr_pose_<side> and /vr_gripper_<side>. Nothing in the workspace subscribes to either.

- c4332b3 **Part 5: free-form language. Opens the noun, not the verb**
 THE TENSION WITH THE RECORDED DECISION, RESOLVED RATHER THAN IGNORED. This project chose a deterministic grammar over a language model, and the reason holds: a grammar that does not recognise an utterance returns unparsed and the robot asks again, whereas a model asked for structured output can return a different, plausible verb that nothing downstream can distinguish from a correct parse. Free-form input does not change that. It changes WHERE the openness lives. VERBS stay closed.
- 485fad5 **Part 6: Qt operations GUI with RViz embedded, Helvetica, everything indicated**
 RVIZ EMBEDDING: INVESTIGATED, AND ROUTE 1 VERIFIED FROM PIXELS.
 1. X11 reparenting. QX11EmbedContainer is gone in Qt5; the equivalent is QWindow.fromWinId() wrapped by QWidget.createWindowContainer(). WORKS. Proven under Xvfb by screenshot: the host label renders above and the complete RViz UI renders inside the container at 31 fps.
- 5b5abe4 **Part 7: one-command Quest connection, supervised, plus a drawn control diagram**
 scripts/vr_connect.sh finds adb, waits for the headset, opens the reverse tunnel, starts the bridge, and then SUPERVISES the tunnel for as long as it runs. WHY SUPERVISED RATHER THAN OPENED ONCE. adb reverse dies with the USB connection. Opened once at start-up, a nudged cable leaves the bridge running and the headset silently unable to reach it, which from the operator's side is indistinguishable from a frozen application.
- 0264277 **Part 8: six operator aids evaluated, one built, one honestly unfinished**
 EVALUATION of all six against what this rig has actually measured, in docs/system/08_operator_aids.md. Four rejected for reasons specific to this hardware rather than general ones:
- a5ae397 **Part 9: audit every reported number; re-recording not done**
 AUDIT, in docs/system/09_results_audit.md. For each figure in Parts 1-8: how it was measured, whether the instrument was validated against an independently known answer, and a verdict. VALIDATED 14 SOUND 15 UNVERIFIED 4 STRUCTURAL 1 The reason this exists: across this project the measuring tool has been the fault EIGHT times, three of them in this run of work. The base rate is high enough that an unaudited number is not evidence.
- a63cebb **Resolve the boundary-node contradiction: it was the harness, twice**
 VERIFIED: the warning arrives with 88 mm of travel still remaining, after 334 mm of travel, identical in 3 of 4 repeats. THE CONTRADICTION WAS MINE, NOT THE NODE'S, IN TWO SEPARATE PLACES. First, the drive test commanded an IDENTITY orientation. Measured: identity is unreachable at 0 of 6 points along that line while the home orientation reaches 6 of 6. "Wall at 0 mm" was the correct answer to the question actually being asked.
- 48dcdfa **Task 2 loses DIRECT and VR as a finding; cut the boundary node; adb; methods**
 1. TASK 2 HAS NO DIRECT OR VR CONDITION, and the objects were NOT made easier. Teleoperated grasping is not achievable with this master: a top-down grasp needs 169.7 deg (left) / 164.6 deg (right) from the anchor orientation_mode "fixed" pins the wrist to, and nothing measures the wrist to command it. VR inherits the same limit because it drives the same follower through the same pinned path.

- 2d3f9dc **Item 1 part one: five-task sweep built, verifier validated, 4 clips recorded**
 THE SWEEP HAS NOT BEEN RUN. 4 of 38 clips exist. Stopping at the item boundary with the infrastructure finished and the verifier cleared, rather than leaving a half-recorded sweep. BUILT * scripts/final5_runs.py drives the recorder from the CURRENT five-task spec: 14 runs, 38 clips, 266 files at 7 angles, under f1..f5 so nothing collides with the 87 stale nine-task clips. * record_rviz.py -final5 -resume.
- 9cd2728 **Item 1 complete: all 38 clips re-recorded, 39/39 pass a validated verifier**
 Five-task geometry (500 mm tray, 540 mm sling) with the single-owner gripper fix. 7 angles per clip, 266 files, 132 MB, largest file 5.3 MB. INDEX.md carries the Task 2 caveat at the top and the caveat is burned into every f2 overlay in red. A REAL STALE-GEOMETRY BUG, which is what the re-record existed to find. record_rviz.SLING_L was still the retired 350 mm while the spec is 540 mm.
- 54ec2bc **Track every angle for the five-task clips**
 The brief asked for one file per angle. The ignore rule that kept only the quad tile was written for the retired nine-task sweep, whose rushes were interrupted mid-encode and could not be stream-copied back to a sane size. The five-task clips are recompressed at crf 30: 266 files, 132 MB, largest 5.3 MB, which is small enough to track. A single angle cannot answer every question.
- e5d81a4 **Item 2: wrist camera relay. Logic measured; GUI widget NOT visually confirmed**
 srl_teleop/camera_relay.py holds the staleness rule, shared by the GUI widget and the VR publisher so the two cannot drift apart. MEASURED against a synthetic 15 Hz camera on both arms: measured rate 14.2 Hz, reported from ARRIVALS not from configuration transport latency median 2.7 / 3.1 ms, p95 7.6 ms (header stamp to receipt) after the source stops: live -> NO SIGNAL, painting disabled IT SUBSCRIBES, IT NEVER OPENS A DEVICE.
- d4187c9 **Item 3: the instrument-validation chapter, the thesis's methods contribution**
 CHECKED WHAT EXISTED BEFORE WRITING. The master arm chapter (2799 words), the MuJoCo-to-ROS development path (1631), and all four control-mode flowcharts were already written in the earlier rewrite. Duplicating them would have been the easy way to look productive. The genuinely missing piece was the methods contribution, now Chapter 6: "When the instrument is the fault".
- 1394a05 **Item 4: the lab list, ordered so each step decides whether the next is worth it**
 docs/NEXT_SESSION.md now opens with the hardware session. Everything above the line needs the lab; nothing below it does. 0. Exactly ONE stack, and clear stale /dev/shm segments with the stack stopped. Those segments silently kill newly launched nodes – empty logs, no processes, no error – which cost hours on 08-09. Kill by explicit PID: pkill -f <pattern> matches the shell running it and killed the working shell three times in one session. 1. Repair LEFT j2 and j4, and nothing else.
- 1288444 **Confirm the GUI camera panel renders, from pixels, in both states**
 Both states screenshotted and committed to docs/img/: live both feeds painting, labelled LEFT / RIGHT, captions "live 13.8 Hz 320x240" in green source stopped both panels black, "NO SIGNAL" in red, and "last frame 5.9 s ago" That second shot is the one that matters. It is the visible-degradation claim made good: the panel does not dim or freeze the last frame, it stops painting and says why, because a frozen picture of a workspace cannot be told from a live picture of a workspace that is not moving.

- 9d72c6d **f4 clip binaries from the post-sling-fix pass**
Six left-behind re-encodes of the S1_short_lift/direct angles. Same clips, committed so the working tree is clean before consolidating branches rather than discarded by a checkout.
- 75d3800 **Final f4 clip binaries, left changing by orphaned capture processes**
The sweep left ffmpeg x11grab processes running against the f4 clips, so those files kept changing under git and blocked a branch switch. Killing them made the tree stable. Same orphan class as Part 4, where 183 stale /dev/shm segments from an earlier sweep silently killed newly launched nodes.
- 15cbb64 **Job A: restructure recordings by control mode, and five silent breakages**
recordings/verification/<mode>/<task>/<scenario>/<condition>/ Control mode and autonomy condition are independent axes and the old tree conflated them: "direct" under the mannequin and "direct" under VR shared one folder name with nothing in the path to tell them apart. Six numbered mode directories (01_master_teleop .. 06_full_autonomy) so they sort by capability rather than alphabetically, which would put full_autonomy between direct and shared. THE FINDING THAT SHAPED THE MIGRATION.
- c29e9be **Job B: workspace audit. Report only, nothing fixed**
scripts/audit_workspace.py, repeatable, 232 python files. Every check exists because reading did not catch that bug in this project. CLEAN. The five recurring bug classes are essentially gone: wall-clock intervals 0, block()-without-clear 0, second-stack starts 0, unresolved entry points 0, missing referenced paths 0, and rejected task args 0 – that last being the class that once made five GUI buttons do nothing.
- adaae8 **Fix the four vacuous checks; wire the post-hoc stage holm was written for**
THE FOUR VACUOUS CHECKS: 0 remaining. verify_final5 now RAISES if densify returns no waypoints instead of reporting "TOTAL FAILURES: 0" on a path it never checked – that check sits under the result this project leans on hardest. Same guard on the place path, on verify_scenarios's centres dict (all() over an empty dict is True), and on verify_autonomy's expected-words list (an empty expectation passes anything the robot says, including nothing). HOLM: THE GAP WAS NOT A MISSING CALL. holm() had nothing to correct.
- 64fe28a **Job C: /mnt/c works, first CAD measurement, no library needed**
C1: /mnt/c RECOVERED ON ITS OWN. It reads fine and so does the project directory; the mount entry is the same 9p one that was returning EIO earlier, so that was transient. No Windows-side command needed. Prior work confirmed present: mujoco_menagerie, MUJOCO_LOG.TXT, singlekinova.py, srl_teleop.py, testjoints.py, basic_control/, 3dprint/, BOM/. C2, PARTLY. Recommendation: NO CAD LIBRARY for this part of the question.
- 59908d0 **C2 finished: the CAD validates the FK link lengths; the mount angle does not**
LIBRARY. pythonocc-core is NOT on PyPI – it ships through conda and this workspace has none. cadquery bundles OCP, a binding to the same OpenCascade kernel, and installs with pip: identical kernel, different packaging. For overall bounding extents no kernel is needed at all, since STEP is a text format, and measure_cad_step.py does that with a regex.

a7e5204 **C3: prior work recovered and read; two corrections to the development chapter**

The development chapter was written from reconstruction. The actual programs have now been recovered and archived at `thesis_report/_source/prior_work/`, and file dates give the calendar directly: 7 June to 27 July 2026. The chapter cites the source instead of supposing. WHAT THE PRIOR WORK ACTUALLY WAS. Five programs import MuJoCo and NONE contains an inverse-kinematics call of any kind – searching the whole set for "inverse" or "jacobian" returns nothing.

12fa17f **C4 part one: the first real RViz figure, and the participant-Results skeleton**

THE REPORT HAD NO RVIZ VISUALS AT ALL. It now has one, captured from the running system on Xvfb rather than drawn, cropped to the 3-D view and annotated: left arm at +x, right arm at -x, gripper, backpack mount, and the wearer marked as being IN the collision model. It sits in the workspace chapter, where it SHOWS the disjoint-workspace result instead of only asserting it: both hands finish on the same side of the body, 1.46 m apart.

6789d2f **C5: state that the CAD is not the kinematic source, with each claim checked**

A new section in the master arm chapter, so nobody later mistakes the mechanical drawings for the model the software uses. Three statements, each verified rather than asserted: 1. THE CAD CARRIES NO ARTICULATED JOINTS. Both STEP files declare AP214 (AUTOMOTIVE_DESIGN), a schema that IS capable of carrying kinematic structure, so the absence is meaningful rather than a format limitation.

3c5df49 **C4 part two: CAD renders as vector; and a correction to my own C4 claim**

CORRECTION FIRST. The previous C4 commit said the RViz capture recipe was "one command per view". It is not. `shoot.sh` accepts yaw, pitch and distance arguments and IGNORES all of them – the camera pose comes from the `.rviz` config, so every invocation renders the same view. Two "different" captures came out byte-identical, which is how it was caught. The fix is a per-view `.rviz` config with the camera pose written into it, which `record_rviz.py:ensure_display()` already does and `shoot.sh` should reuse.

77f3394 **Job A: audit clean, but the pot repair does not take effect yet**

THE HEADLINE, and it changes what Job B can claim. `degraded_mode.decide()` reads a STORED BASELINE FILE, not live hardware. `channels_20260806.json` still says 6/14 coherent, so `master_pose_node` engages degraded mode at startup and freezes eight now-working channels, silently. stale baseline 6/14 coherent -> degraded engages: True repaired arm 14/14 coherent -> degraded engages: False Both directions verified.

cafaa5c **Job B.1: wrist orientation capability after the pot repair**

Master can now express 180 deg of tip reorientation against the 169.7/164.6 needed for a top-down grasp, so the repair restores the input side. The robot is now the binding constraint: over a 27-pose grid, fixed 74.1/85.2%, tilt 73.5/81.2%. Tilt costs 0.6-4.0 pp against fixed but neither clears the 90% bar, so tilt is not shipped.

- 7f17263 **Job B.2: capability delta from the pot repair, and what is lab-blocked**
 Left arm's commandable set goes from a surface (radial extent 0.000 m, radius not commandable at all) back to a shell (0.139 m). Right arm unchanged, its frozen channels being rolls that carry no radial information. Job B's azimuth, position-ladder and smoothing questions are fits to recorded frames and every recording predates the repair, so they are named as lab-blocked with the exact capture that unblocks them rather than re-run on pre-repair data.
- e12efe0 **Job C: fix grasp poses driving the fingers through the object**
 candidates() returned the bare centroid while the fingertips sit 0.1118 m along the tool +z and the grasp quaternion aims it down, so every object in the catalogue was penetrated by 51.8-105.8 mm. grasp_offset() now raises the tool by half-height plus pad. Root cause of the missed check: nothing in the autonomy path publishes the perceived object as a CollisionObject, so avoid_collisions=True ran against a world without it and passed 100%.
- ca68cc9 **Job D: mode command paths verified from the live graph**
 Modes 4, 5 and 6 cannot command the arm. handover_arbiter publishes /autonomy/assist_pose_<arm> and nothing anywhere subscribes to it; and autonomy_executive's complete publisher list is speech, stage, decision and estop, with no pose, trajectory or action client. Every stage before the last is real; the last hop was never connected.
- ba22676 **Job E: publish the calibration sweep to RViz as live markers**
 scene_markers.py renders each fingerprint verdict as a coloured box plus a label carrying name, verdict, confidence and displacement, on /scene/markers. SAME is deliberately desaturated so the verdicts needing attention carry the colour; a move is drawn as an arrow from its stored pose; a dropped object is held for drop_hold_s at its stored pose before being released, because an instant removal is indistinguishable from an object that was never there.
- a91f8bc **Job F: precision/speed dial with an enforced safety invariant**
 One dial in [0,1] moves scale, smoothing, velocity cap and step limit together; scale is geometric so the low end gets the resolution. SAFETY_PARAMS names the nine things it must never touch and assert_safety_unconditional raises on any of them, checked across all 101 dial positions so a future key added to RANGE fails the test rather than shipping. rebase_anchor keeps the command continuous to 1e-12 through a dial change, including far from the engage point where the uncorrected jump is worst.
- 6bab268 **Blocker 1: connect the autonomy command path through the teleop safety stack**
 ik_follower_node now subscribes to /autonomy/assist_pose_<arm>, which had zero subscribers anywhere – the reason modes 4, 5 and 6 never drove the arm. It enters at request_ik, the same door teleop uses, so collision-aware IK, the clearance floor, the guard, the flip reject and the e-stop are the same code with no per-mode exemption and no private route to the controller. autonomy_executive gained command()/release() and a 20 Hz pump; its previous complete publisher list was speech, stage, decision and estop.
- c6c31fe **docs: standing work brief as the session resume point**
 docs/WORK_BRIEF.md carries the multi-part brief verbatim with a STATUS line per part, so a fresh session resumes from "Read CLAUDE.md, NEXT_SESSION.md and docs/WORK_BRIEF.md" with nothing re-pasted. Parts 1-4 are the user's text verbatim. Parts 5-8 (Tasks A/B/C and the dual-view GUI) are recorded as TEXT NOT SUPPLIED rather than reconstructed: only the one-line forward reference naming them exists, so any specification written here would be invented and then graded against itself.

- 6736ee7 **Part 1: autonomy moves the arm – root cause was a missing import**
 ANSWER: YES. 1092 trajectories at a reachable target and 0.0700 m / 0.1204 m of real tf2 EE displacement over two legs, residual 0.0000 m on both. THE BUG. ik_follower_node.py had no `import time` while the autonomy path calls `time.monotonic()` at lines 713 and 733. A missing name is evaluated only when the path runs, so the node imported, launched, logged "Tracking enabled", and died on the first `/autonomy/assist_pose_<arm>` message with `NameError`. The followers launch without respawn, so it stayed dead.
- cf9e4cd **docs: WORK_BRIEF tracks the 7-part brief; Part 1 marked DONE**
 Supersedes the earlier 4-part version. Parts 1-7 verbatim with STATUS lines and commit hashes, START HERE pointing at Part 2. Part 1 recorded DONE at 6736ee7 with the Part 1 findings that feed directly into Part 2's diagnosis (bug class 5 instance found, SHM-at-startup confirmed necessary including the `sem.fastrtps_* glob` miss, follower respawn left as a deliberate open decision).
- 9e0dce4 **Part 2: full diagnosis – one bug wore three costumes, two instruments lied**
 DIAGNOSIS. Part 1's missing import killed the follower, and a dead node publishes nothing, so three separately-filed faults were one corpse seen from three angles: "autonomy does not move the arm", `/ik_status_<arm>` publishes an empty data array" (Part 2D.3), and "probe_dial_safety recorded zero clearance samples". Measured after the fix, `/ik_status` carries 11 fields with clearance at index 5 exactly as documented. D3 is closed by the Part 1 fix. A. Five bug classes. A1 one instance (fixed).
- b6e5a8b **docs: Part 2 marked DONE, START HERE points at Part 3**
 Also records the live-stack state and the parked left arm, so the next session re-homes before any measurement that assumes the home pose.
- c0ff406 **Part 3.1: language sweep – 7 MISUNDERSTOOD found, all seven closed**
 40 phrasings, 13 categories, written against the parser's STRUCTURE rather than alongside it. The prior self-test (`measure_freeform_language.py`) reported 0 MISUNDERSTOOD over 30 phrases, but every one of those was written with the grammar it tests – bug class 5, a test constructing the environment where the bug cannot occur.
- 759dfe6 **Part 3.2: the gripper no longer inherits a leftover as its open reference**
 The 2F-85 HOLDS POSITION when its commanding process goes away. So a node killed mid-grasp left the fingers part-closed and the next start found them there with nothing able to say whether that was a grip or a leftover. Two opposite mistakes follow and no default avoids both: open unconditionally -> drops whatever the hand was holding never open -> the travel range is wrong for the whole session THE MARKER IS WRITTEN WHEN THE GRIP IS TAKEN, NOT WHEN THE PROCESS EXITS. This is the decision the whole thing rests on.
- c51ee0f **Part 3.3: the pot repair could not take effect, and the warning had never fired**
 TWO DEFECTS, AND NEITHER WAS "THE WARNING IS TOO QUIET". 1. THE RUNTIME READ A DIFFERENT FILE FROM THE ONE THE LAB WRITES. `check_channels.sh` saves the new reference as `channels_<today>.json` and diffs against `ls -t channels_*.json | head -1`. `degraded_mode.default_baseline_path()` returned the literal string `"channels_20260806.json"`.

- d5bbec8 **Part 3.4: five tasks -> three, verified N=10 over the full path, 116 min**
TASKS A, B AND C, in src/srl_experiments/experiments/abc/. A POSITIONING uncoupled; attention contrast; Fitts; warm-up B COORDINATED CARRY physical coupling, rigid AND compliant C DUAL PURSUIT simultaneity across the disjoint sets; b_cross The binding constraint is the WEARER, not the science: >17 kg, 12 min continuous and 48 min total pack-on, and a two-hour session. Two modes x five tasks was costed at 167 minutes and does not fit.
- 6b9d78a **docs: Part 3 marked DONE, START HERE points at Part 4**
Findings write-up in NEXT_SESSION.md. Records the three items that turned out worse than the brief described: the gripper's startup verdict was re-decided mid-motion, the degraded-mode warning had never fired at all, and the pot repair could not reach the runtime because the lab and the code disagreed on which baseline file is current.
- 95adcf4 **Part 4: commanded vs actual, with the divergence named – and two panels proven not to work here**
WHAT SHIPPED. scripts/srl_gui.py rebuilt around the question the brief asks: what is the gap between what was commanded and what the arms actually did?
- dd5ae82 **docs: Part 4 marked DONE, START HERE points at Part 5**
Records the two findings that changed the design: X11 reparenting embeds but does not clip without a window manager, so two side-by-side RViz panels are not available on this host; and /real/tf – which the brief names – does not exist, the real arm being prefixed into the shared /tf.
- 93fcd5c **Part 5 preliminaries: capture frame time properly, and correct /real/tf**
FRAME TIME IS NOW MEASURED, NOT READ OFF A SCREENSHOT. Part 4 could only report frame time by reading the status bar in a captured image, because the harness got an EMPTY string back from the GUI's stdout. Chasing that turned up three faults, two of them in the measuring tool and one of them serious: 1. `pgrep -f "Xvfb :99"` MATCHED ITS OWN SHELL COMMAND LINE, so `ensure_xvfb()` believed the server was up and never started it. Every GUI run then died with "could not connect to display :99".
- 3597e0a **Part 5: the clip verifier now refuses to report until it catches a broken clip**
THE NEGATIVE CONTROL. Before any verdict is printed, the verifier is shown seven CONSTRUCTED clips whose answers are known and must get every one right. It refuses to report at all otherwise, because "0 clips failed" and "nothing was actually checked" print identically – and this file has produced the second while reporting the first.
- 93ce00c **Part 5 (partial): tasks A/B/C are runnable through a mode's own command path**
WHAT NOW WORKS, AND IT IS THE THING THAT WAS BLOCKING THE WHOLE PART. Job A found that NO clip in this repository had ever been recorded through a control mode: every one came from `record_rviz.py`, which calls `/compute_ik` directly and never publishes `/master_arm_pose_*`, so no follower, clutch, anchor, orientation lock or safety guard was in the path. The mode axis of the recordings tree was empty because nothing could fill it. `experiments/abc/run_abc.py` fills it.
- 39f8869 **docs: Part 5 partly done, resume point is the capture sweep**
Records what is proven (the A/B/C runner drives the arms through a mode's own command path; the verifier refuses to report until it catches a broken clip) and what is not (the 7-angle sweep, mode 01 suppression, mode 02 untested).

- bc3b890 **Part 5: both blockers resolved – mode 01 diagnosed, VR driven end to end**
 BLOCKER 1 – autonomy_has_control is REFUTED. It was never suppression. The decisive evidence was /ik_status, whose layout is [success, fail, direct, slewed, rejected, clearance, blocks, redundancy]: success fail redundancy retries before 0 8 48 after a mode-01 run 0 15 (+7) 90 (+42) after a mode-04 run 8 (+8) 15 90 Mode 01 made SEVEN IK calls and FAILED ALL SEVEN, burning 42 redundancy retries (7 x redundancy_samples). Every BlockMonitor blocker stayed clear throughout.
- b071909 **docs: all four modes drive the arms; the sweep needs mode isolation**
 Records the measured travel per mode and the precondition the sweep must honour: after a VR run, modes 01/04/06 reported 0.0000 m, and killing vr_pose_mapper restored them to 0.2000 m. The mapper stays engaged and keeps publishing on /master_arm_pose_<arm> – one source at a time, at the process level.
- 131f2af **Part 5: the 7-angle sweep, recorded through the GUI, with isolation enforced**
 TWELVE CLIPS, 3 tasks x 4 modes, 8 files each (7 angles + the quad tile), every one started by pressing the GUI's own button (Gui.on_launch on the real manifest). These are the FIRST clips in this repository recorded through a control mode: Job A found every earlier clip came from a recorder calling /compute_ik directly, with no follower, clutch, anchor or safety guard in the path, which is why the mode axis of the tree was empty. THE ISOLATION RULE IS IN THE CODE AND IT FIRED ON THE FIRST RUN.
- a843c67 **docs: Part 5 DONE – 12 clips through the GUI, isolation enforced**
 Records the two findings that shaped it: master:=false is required for a scripted-operator stack (master_pose_node is otherwise a second publisher and the isolation check refuses to record), and the A/B/C clips carry no task objects so the object-in-frame criterion is not claimed for them.
- b4da854 **chore: ignore RViz per-view log files written beside the clips**
 They are transient stderr from the capture displays, not evidence, and they were showing up as modified on every sweep.
- e2dc845 **Clips: 06_full_autonomy recorded – 3 tasks, 7 angles + quad, captions burnt in**
 FIRST MODE IN THE ORDER, and pushed on its own so a dead session still leaves it on the remote. Full autonomy needs no operator input, which is why it goes first. A pick and place LEFT descends 0.10 m, closes, lifts 0.14 m, carries to the bin, releases; RIGHT holds still B hold and place LEFT holds the work at x=+0.25; RIGHT brings the part down 0.15 m and places it C multimeter LEFT presents and holds steady; RIGHT probes down 0.11 m, holds contact, retracts THE CLIP TASKS ARE BUILT FROM VERIFIED COORDINATES ONLY. [...]
- 6799649 **Clips: 01_master_teleop recorded – 3 tasks, 7 angles + quad**
 Scripted master input, entering at /master_arm_pose_<arm> – the same topic master_pose_node publishes and the same follower callback live teleop uses, so the pose dead-band, collision-aware IK, redundancy re-seed, clearance floor, step guard and every blocker are all in the path. The stack runs with master:=false, which is what makes the isolation check pass: master_pose_node is otherwise a second publisher on that topic and the sweep correctly refuses to record.

515264b **Clips: 03_shared_autonomy recorded – 3 tasks, 7 angles + quad**
Autonomy commanding through /autonomy/assist_pose_<arm>, the arbiter's topic, entering request_ik at the same door teleop uses. Isolation verified before every clip: 0 idle publishers, as predicted. 3/3, captions burnt in.

2026-08-10 — 52 commits

b4913dc **Clips: 02_vr_teleop recorded – 3 tasks, 7 angles + quad**
Scripted Quest input on /vr/controller_pose_<side>, through vr_pose_mapper, which is the only publisher on /master_arm_pose_* – and the isolation check predicts exactly that (1 idle publisher, not 0 as for the other modes) and verified it before every clip. The VR path is genuinely relative: the mapper anchors on the clutch engage and commands motion since.

fa1ec28 **Clips: 04_vr_shared recorded – all five modes now complete**
VR transport UP and autonomy owning the pose: vr_pose_mapper runs, but the command enters at /autonomy/assist_pose_<arm>, so the isolation check predicts 0 idle publishers on the follower's input (not 1 as for pure VR) and verified it before every clip. That the two VR modes predict DIFFERENT counts, and both were met, is the check doing real work rather than rubber-stamping. 3/3. Fifteen clips across five modes, 8 files each.

b7fda18 **Clips: the language sweep on video – 40 phrasings, every outcome captioned**
2m21s, one card per utterance, grouped by category, colour-coded by outcome. Every card is the SHIPPED grammar's real answer: voice_intent.parse then resolve_target against a constructed scene, the same path scripts/sweep_language_vision.py runs. The robot's spoken reply on each card is read off that call, not written by hand. CORRECT 10 ASKED 6 REFUSED 24 MISUNDERSTOOD 0 30 of 40 utterances end with the arm untouched, and the video says so: ASKED and REFUSED are SUCCESSES.

f1c6fdf **INDEX.md: generated from disk, the progress file and the verifier**
24 clips indexed, 24 verified. The five most important at the top, with how to open them from Windows (the \\wsl\$ path, or explorer.exe from a WSL shell). GENERATED, NEVER HAND-WRITTEN. This file was once overwritten by a script that dropped 38 clips and discarded a caveat with them, so every row is read from the clips themselves, the sweep's progress file and the verifier's own output – it cannot claim a clip that is not there or a pass that was not measured.

7de5beb **Graphs: language outcomes and per-mode EE travel, from the sweep's own data**
Both read their numbers from measurements rather than restating them: language_outcomes from recordings/baselines/language_vision_sweep.json, and mode_travel from the tf2 path integrals the recording sweep produced. MISUNDERSTOOD is plotted even at ZERO and annotated as the dangerous category. A chart that omits an empty bar invites the reader to forget it was measured – and zero is the entire result, so it has to read as a measured zero and not as an absence.

68732d6 **docs: clips recorded – 15 across five modes, plus the language sweep**
Records what is on the remote and names the one outstanding item (GUI tutorial clips) with the pieces that already exist for it.

- daa3df5 **Clips: objects in the scene, grasped for real – 06_full_autonomy re-recorded**
 The first sweep drove each mode's command path with NO OBJECTS PUBLISHED, so fifteen clips evidenced that a mode commands the robot and nothing about anything being picked up – and task C had no multimeter in it at all. The pixel check could only ever ask "did the arm move". NOW: table, teal container, orange block, green circuit box, tray, and a multimeter dummy body, at the verified task coordinates.
- b2e282f **Clips: 01_master_teleop re-recorded with objects grasped**
 A block GRASPED->RELEASED carried 0.412 m; B part GRASPED->RELEASED carried 0.125 m; C multimeter GRASPED and held. Same grasp evidence as full autonomy, through the master command path.
- f4bef7e **Clips: 03_shared_autonomy re-recorded with objects grasped**
 A 0.412 m, B 0.125 m, both GRASPED->RELEASED; C GRASPED and held.
- b0a3d94 **Clips: 02_vr_teleop re-recorded with objects grasped**
 A carried 0.206 m, B 0.062 m – almost exactly HALF the other modes' 0.412 and 0.125. That is vr_pose_mapper's x0.5 scale showing up in the object's own travel, which is a stronger confirmation the VR path is real than the arm motion alone: the block is being carried by the gripper, so it inherits the scale. C GRASPED and held, carried 0.000 m – the present-and-hold task.
- 3feecfc **Clips: 04_vr_shared re-recorded – all 15 now have objects grasped**
 A 0.412 m, B 0.125 m GRASPED->RELEASED; C GRASPED and held. Full distance, not VR's half, because autonomy owns the pose in this mode – the scale difference tracks which path is commanding, exactly as it should.
- 4f0ef88 **GUI tutorials: six workflows, narrated, clicks are real**
 launch_a_mode run_a_task read_the_divergence precision_speed_dial
 estop_trigger_clear switching_modes Every step calls the widget's own click() or the handler it is wired to, on the real GUI against a live ROS graph – the same route verify_gui_buttons.py uses.
- 31f1d6a **docs: 15 of 15 clips show the task completing, measured not judged**
 Records the carry distances per mode and the VR half-scale confirmation, the six GUI tutorials, and the one stale leftover directory from the superseded study-taskset sweep.
- 90e7920 **Three one-liners for things that were believed working**
 1. recovery_manager IS NOW LAUNCHED. It owns fault paths (b)-(e) – Teensy reconnect, Kortex session loss, camera/zero-detection abort, network dropout – each verified by fault injection and NEVER STARTED BY ANYTHING. Installed, documented, inert. A recovery layer that is not running is indistinguishable from one that is, right up until a fault. Verified in the graph: 'recovery_manager up.
- b67739f **GUI resolved to one: console features ported, task set trimmed to A/B/C**
 srl_gui.py is now the strict superset and the only launcher a lab session needs. Three things happened, and the third was a real defect. PORTED FROM srl_console (the Dear PyGui tool it supersedes): Charts rolling strips for end-effector height, clearance to the wearer against its floor, and sim->real divergence. The divergence strip renders "no data – not a flat line" when nothing is publishing, because a flat trace at zero is exactly what a missing real arm must never look like.

- 7f48aff **Clip frame rate: measured, not raised – 56% of every clip was dead lead-in**
 "5-13 fps against 12 requested" turned out to be three separate things, and raising the capture rate would have fixed none of them. WHAT THE CONTAINER HEADER PROVES: nothing. x11grab always writes its requested rate – when the source has not repainted it grabs the same pixels again – so every clip on disk reported 12 fps whatever RViz managed. Counting frames that DIFFER from their predecessor is the only honest measure, and across nine clips it gave 0.6-1.3 fps.
- 1c5a01c **Stale numbers re-measured, and three regressions the archive move caused**
 ONE PLACE WITH LIVE NUMBERS. CLAUDE.md now carries a CURRENT COUNTS block stating what was measured today, with the command that measures each. The historical log keeps its dated figures untouched – "36 unit tests pass" in a 2026-08-05 section was true on 2026-08-05 and rewriting it would destroy the record. What was wrong was having no current figure to read instead.
- 251a5ed **Consolidate: one owner for the grasp thresholds, three verifiers archived**
 GRASP CONSTANTS: six definitions -> one. srl_teleop.gripper_state now owns grip_for(), holding(), and every threshold; record_rviz, clip_tasks, vr_gripper_node and verify_gripper_shutdown import them. TWO HAD ALREADY DRIFTED, and neither would have crashed: verify_gripper_shutdown FREE_AIR_RAD = 0.80 gripper_state FREE_AIR_RAD = 0.74 the same name for two different quantities – one is the knuckle's mechanical stop, the other the bound that separates "closed on nothing" from "closed on something".
- d787d97 **Seven gripper clips are entirely black, and the verifier could not see them**
 FOUND BY LOOKING AT THE FRAMES, which is the only reason it was found. verify_rviz_clips.py globbed rviz_front.mp4 and opened nothing else, so "24 of 24 clips pass" meant 24 FRONT VIEWS passed. Measuring all eight angles directly: 01_master_teleop/C/S1_present_probe gripper mean 1.0/255, 0 changing frames 02_vr_teleop/C/S1_present_probe gripper black 03_shared_autonomy/A/S1_single_arm gripper black 03_shared_autonomy/B/S1_bimanual gripper black 04_vr_shared/B/S1_bimanual gripper black 04_vr_shared/C/S1_present_pr [...]
- 2ba2ce8 **Black gripper views: root-caused to pkill leaving a live process that renders nothing**
 DIAGNOSED, NOT GUESSED. Reproduced by alternating the active arm four times and grabbing the display each time: step 0 arm=left brightness 99.00 left_rviz=True right_rviz=False step 1 arm=right brightness 98.92 left_rviz=True right_rviz=True step 2 arm=left brightness 0.04 left_rviz=True right_rviz=True step 3 arm=right brightness 0.04 left_rviz=True right_rviz=True The gripper camera is bolted to <arm>_end_effector_link, so it needs a different config per arm, and ensure_display kept ONE display for it: on an arm [...]

- 6bec842 **Task A is a carry now; framing on the workspace; the verifier can fail**
 TASK A GEOMETRY. A_BIN 0.30 -> 0.62, so the transport is 0.300 m lateral against 0.130 m vertical. It was 0.132 m of which 0.130 was vertical: the block hovered directly above the bin and the "place" lowered it 130 mm. scripts/sweep_task_a_bin.py walks the bin outward, verifying the WHOLE densified path at N=10 per candidate rather than checking one chosen value: $x = 0.35 \dots 0.68$ PASS (lateral 0.030 .. 0.360 m) $x = 0.71$ fail <- the IK boundary, 3863 calls IK IS NOT THE BINDING CONSTRAINT – the bench is.
- 0b29ed3 **Re-record mode 01: task A is a carry, framing tightened, grasps inside the clip**
 01/A grasp at +5.6 s of 15.7 s 01/B grasp at +4.4 s of 12.4 s 01/C grasp at +5.0 s of 13.2 s WHAT I SEE, having looked at contact sheets of all three: A the orange block is gripped at the right of the bench, lifted, carried ACROSS to the teal bin and is visibly INSIDE it in the final frame. This is the change that mattered – the previous version lowered the block 130 mm into a bin directly beneath it.
- a609c04 **Re-record all 15: every clip places its object at 0 mm from target**
 mode A B C 01_master_teleop 0 mm placed held 02_vr_teleop 0 mm placed held 03_shared_autonomy 0 mm placed held 04_vr_shared 0 mm placed held 06_full_autonomy 0 mm placed held every grasp lands inside its clip: +0.8 to +26.6 s of 12-34 s THREE DEFECTS FOUND BY RECORDING AND LOOKING, in the order they surfaced. 1. THE OBJECT WAS PINNED TO THE WRIST, NOT THE FINGERS.
- d2b3918 **FROZEN check used a statistic that dwell dilutes; use the max instead**
 Two clips that visibly move were failed as FROZEN at delta 0.011-0.012 against a 0.02 floor. The statistic was the mean absolute frame-to-frame difference over EVERY frame and EVERY pixel, so a small arm moving in a mostly-static frame scores near zero once the clip carries a dwell – and task A now holds 4.5 s at the bin on purpose, so the check was penalising the fix for the previous defect. The question the check asks is "did anything ever happen".
- 521f510 **CLAUDE.md current counts: the 15 clips and their placement accuracy**
- 33cf23c **Clip verification green: 54 of 54, all eight angles per clip**
 15 clips indexed, 15 verified 54 of 54 clips pass every automatic check (includes the older 00_unclassified set) Every angle of every clip is now checked for a picture and for motion – the pass that previously read "24 of 24" while seven gripper views were black for 34 s each, because the tool globbed rviz_front.mp4 and opened nothing else.
- 850a625 **Part 1 (INCOMPLETE): the bench was decoration and the objects floated**
 REPORTING THIS AS UNFINISHED. The scene machinery is built and verified; the task geometry that has to sit on it does NOT yet verify, so nothing is re-recorded. WHAT WAS WRONG, both confirmed by measurement: 1. THE BENCH WAS DECORATION. clip_scene published a MarkerArray to /task_objects and NOTHING to /planning_scene. Asked directly, move_group returned "collision objects: NONE" while the bench looked solid on screen.

- d9fc66c **Part 1 DONE: the sweep was self-cancelling; 40 failures -> 0 with the bench in**
 THE HARNESS BUG, FIRST, BECAUSE THE GEOMETRY WAS NEVER THE PROBLEM. 40 failures at every edge from 0.10 to 0.26 m. The cause: `on_bench()` placed objects at `BENCH_NEAR_Y + depth/2`, so moving the edge moved every object with it and the wrist-to-edge geometry – the only thing that decides whether the gripper hits the bench – was INVARIANT. The parameter had been cancelled out of the quantity being scored. A flat score was the harness saying so.
- 7b67a3f **Empty-world audit: the wearer was fine, the WORLD was not; guard added**
 THE WEARER IS LOADED, AND I WAS WRONG TO SAY OTHERWISE FIRST. My initial check looked for `human_torso` and `human_head`, found neither, and I raised the alarm. The links are named `torso`, `head`, `hips`, `left_leg`, `right_leg` plus the six human arm links – 11 in the runtime URDF, all present in TF. A wrong name reads exactly like a missing link, which is the same class of error as everything else in this audit.
- 083dc32 **The right arm CAN support B and C: standoff was the lever, not the workspace**
 YOU ASKED WHETHER THE COLLAPSED BAND MEANS THE WORKSPACE CANNOT CARRY THE TASKS. It does not, and here is why the two figures are not in conflict. The band that collapsed 15/19 -> 1/19 was probed at a FIXED $z = 1.15$ and $x = +/- 0.32$. The tasks do not live there: they live at the bench edge, $y = 0.09..0.20$, $z = 1.06..1.20$. Same rig, different place. "It passes" and "the envelope collapsed" were both true and neither answered the question that matters – HOW CLOSE TO THE EDGE IS EVERY POSE.
- 26adc92 **Part 3: scene calibration, and the tolerance is +/-10 mm not +/-20 mm**
 WHAT `scene_fingerprint_node` DOES TODAY: consumes `/perception/objects` (it never opens a camera – two processes on one camera is the serial-port fault in a new costume), accumulates during a sweep window, compares against `config/scene_fingerprint.json` at 15 mm / 15 deg with a 250 mm gate, and re-registers ONLY what moved.
- ccfb27b **Part 4 (font sub-item): Helvetica resolves, and it was never being requested**
 "Helvetica is not resolving" was not a fontconfig problem. Checked at both layers before changing anything: `fc-match Helvetica -> LiberationSans-Regular.ttf` `QFontInfo(QFont("Helvetica")) -> "Liberation Sans"` Both already resolved. The actual cause: NOTHING EVER ASKED FOR HELVETICA. `srl_hud.py` requested "DejaVu Sans" and "DejaVu Sans Mono" throughout, so the alias was never exercised and the GUI simply rendered in DejaVu.
- 147e704 **CAD analysis: the alternation is confirmed and the interior chain agrees to 1 mm**
`/mnt/c` IS HEALTHY. The I/O errors in `CLAUDE.md` are gone – a WSL restart fixed the dead 9p mount at some point. No remount needed, nothing for you to run from Windows. READER: `cadquery-ocp`. pip wheels, no root, no conda, and it is the full OpenCASCADE kernel rather than a parser – bounding boxes, volumes and surface classification all need the kernel. `pythonocc-core` is conda-first with unreliable pip wheels; headless FreeCAD needs apt and ~1 GB for the same kernel.
- 820351c **pos_tol 15 -> 10 mm, and the lab placement requirement recorded**
 (a) THE TOLERANCE WAS LOOSER THAN THE THING IT PROTECTS. The grasp's capture half-window is 17.5 mm on the widest object, so at 15 mm an object could be declared UNCHANGED by the fingerprint and then not be picked up.

- f16b15f **Pure-CAD master arm URDF: builds, parses, and looks like the printed arm**
 PURE CAD, NOT HYBRID, as decided. Nothing in master_arm_cad.urdf.xacro comes from the measured lengths – mixing them in would throw away the only thing this model is for, which is being an independent check. srl_dual.urdf.xacro is UNTOUCHED.
- 970d212 **Perception without fiducials: the accuracy is in the FIT, not the detector**
 THE QUESTION WAS ASKED WRONG BEFORE. "Can vision hit sigma ≤ 2 mm" treats the detector as the thing that localises. It is not. The pipeline is detect -> crop the depth -> fit, and the box only has to point at the right REGION. So the two halves separate, and one of them is measurable here rigorously. MEASURABLE HERE, by the project's own rule – synthetic is legitimate where the ground truth is CONSTRUCTED, not RENDERED.
- 11e5f21 **Lab procedure for the detector measurement, with what NOT to re-measure**
 The lab session is the only route that uses our objects in our lighting, so the procedure says plainly what is already settled and what is not. ALREADY KNOWN, do not re-measure: the pose accuracy is not the detector's job. The known-dimension fit gives 1.2 / 1.8 / 8.0 mm and tolerates a sloppy box; the naive centroid is wrong by 13-45 mm; range barely matters. WHAT THE LAB MUST MEASURE: the DETECTION RATE and the box quality.
- 346f5b6 **LM-O evaluation harness, ready to run when the download lands**
 measure_detector_on_real_rgbd.py runs the SAME estimator as the constructed measurement – segment to the nearest surface, then fit known dimensions – but against REAL RGB-D with real sensor noise and ground-truth 6D poses, and with the detector given nothing but an open-vocabulary prompt. It reports detection rate, median box IoU against ground truth, and the known-fit pose error with the fraction inside 10 mm.
- 62eb61f **Multimeter respec'd 50x90x130 -> 30x70x45; LM-O measured; architecture written**
 1. THE MULTIMETER, FIXED. It bound the object set three ways at once, and the three constraints pull on DIFFERENT dimensions – so only changing both works: candidate pose err capture sigma ≤ 2 50 x 90 x 130 (old) 8.0 mm 17.5 mm no 35 x 90 x 130 7.9 mm 25.0 mm no <- narrow only: pose barely moves 50 x 90 x 60 2.2 mm 17.5 mm no <- shallow only: capture unchanged 30 x 70 x 45 1.7 mm 27.5 mm YES <- all three WIDTH sets the capture window $((85-w)/2)$; DEPTH conditions the fit, because a deep object is seen from one face [...]
- 3a950cc **PART 5: the empty RViz panel was a QoS durability mismatch, order-dependent**
 ROOT CAUSE, and it explains why it looked intermittent rather than broken. robot_state_publisher LATCHES /robot_description – one message at startup, TRANSIENT_LOCAL, never republished. Confirmed from the live graph: publisher durability = 1 (TRANSIENT_LOCAL). verification_capture.rviz asked for it with Durability Policy: Volatile A volatile subscriber that arrives AFTER the latched message was sent never receives it, so the RobotModel has no description and the panel renders an empty scene.

c0d99e0 **(a) CAD panel root-caused: a config key in the wrong place is SILENTLY IGNORED**

SAME CLASS AS THE TRANSIENT-LOCAL BUG, exactly as you said it would be. ROOT CAUSE: the hand-written config put `Fixed Frame` directly under `Visualization Manager` instead of under `Global Options`:. RViz does not error on a misplaced key – it silently uses the `DEFAULT` – so the fixed frame fell back to one that does not exist, link transforms could not be resolved, and the panel rendered a single stub. `Marker Scale: 0.05` being ignored was the corroborating clue I had already seen and not followed.

4df7bbf **PART 6: READY TO RUN, the participant session, and failure handling**

NEVER GREEN ON UNKNOWN — enforced STRUCTURALLY, not by remembering. `srl_experiments/readiness.py` has no boolean anywhere, because a boolean cannot represent "nobody asked". Three states: OK / FAIL / UNKNOWN. Readiness is built from `REQUIRED` with every entry already UNKNOWN, and checks `OVERWRITE`. A check that crashes, is skipped, or was never wired writes nothing – so it CANNOT write OK. Adding a name to `REQUIRED` and forgetting to implement it makes the system NOT READY, which is the correct direction to fail in.

136674e **Tab-selection gap closed — and it was TWO silent failures compounding WHY THE PANEL COULD NOT BE CAPTURED FROM THE RUNNING GUI: 1. My `-tab` argument was never added. The patch matched `ap.add_argument("-no-rviz", action="store_true")` but the real line is multi-line with a `help=` on the next, so the replace silently did nothing. 2. `parse_known_args` then SWALLOWED `-tab ready` without a word. It is there so ROS can have its `-ros-args`, and it accepted an unknown flag silently.**

382b280 **Part 4: the arm panels became diagrams, and one of them found the seam risk**

Both master arms and both robot arms are now kinematic diagrams rather than tables. The master chain is drawn at `TRUE` bend angles with a uniform scale-to-fit, so the picture IS the posture; roll joints carry a rotating index mark, bend joints are hinges, and under each node sits the live angle and the `TIME SINCE THE LAST DISTINCT VALUE` – the second number being the one this rig actually needs, because its characteristic failure is data that arrives on time and never changes.

3ca6b4b **Archive the superseded clips: they are evidence, and the note says of what**

`recordings/verification/ -> archive/recordings/verification_20260810/`, moved under the standing never-delete rule with `GEOMETRY_NOTE.md` stating exactly what geometry they were recorded against and why they were replaced: `planning_scene EMPTY` – `clip_scene` published a `MarkerArray` to `/task_objects` and nothing at all to `/planning_scene` the bench `DECORATION` – `avoid_collisions` could not see it, so the arm swept through a surface that looked solid on screen the objects `FLOATING` – 0.107 to 0.357 m of clear air under eve [...]

- c2d0b46 **Re-record mode 06_full_autonomy — and two grasp bugs found by the first take**
 3 of 3 clips, 8 angles each, against the verified geometry. Against the stack's own record over the three runs: 54 of 54 IK windows at 100%, mount guard PASS at 0.1610 m worst clearance (floor 0.15), zero clearance-floor blocks. The one block was a 0.38 s flip_reject on the right arm that self-cleared – the guard doing its documented job, not an IK failure. TWO REAL BUGS, BOTH FOUND BY THE FIRST TAKE FAILING RATHER THAN BY READING. 1. gripper_state.holding(knuckle, width_mm) HAD ONLY A LOWER BOUND.
- cf14987 **Re-record mode 01_master_teleop — 3 of 3, the grasp fixes hold on a second path**
 A grasp at +5.2 s of 19.1 s, placed 0 mm from target B grasp at +4.4 s of 14.8 s C grasp at +3.8 s of 13.7 s The two grasp fixes from the previous commit were found on the autonomy path; this is the first evidence they hold on a different one. No spurious t=0 grasp, no grasp outside the window, on any of the three tasks.
- dd4c13e **Re-record mode 03_shared_autonomy — 3 of 3, and the clips are 13.4 fps not 5.4**
 A grasp at +4.6 s of 17.5 s, placed 0 mm from target B grasp at +20.7 s of 30.1 s C grasp at +4.3 s of 13.6 s B's 30.1 s against ~14 s on the other paths is the arbiter blending toward the target rather than a stall – checked by looking, not assumed: both arms are engaged in every sampled frame, the left holds its pose throughout, and the right travels continuously to the part and on to the green box. DELIVERED FRAME RATE RE-MEASURED. 403 frames over a 30.1 s window = 13.4 fps against 15 requested.
- 2fe0899 **Mode 02_vr_teleop: A recorded, B and C did NOT — reported, not papered over**
 A grasp at +15.8 s of 29.2 s, placed 0 mm from target OK B run exited 0, NO GRASP RECORDED at all FAIL C run exited 1, NO GRASP RECORDED at all FAIL NOT RE-RUN AND HOPED. What I can say from looking at B's frames: the right arm DOES travel to the part and ends up beside it; the part never moves; the burnt-in caption correctly turns red and reads "DID NOT COMPLETE – run exited 0: NO GRASP RECORDED at all". So the transport moves the arm and the failure is at the grasp, not at the command path.
- 0d774dd **Record the re-record's state and the one open VR question in NEXT_SESSION**
 Three modes complete (9 clips), VR partial (1 of 3), 04_vr_shared not run. Names the measurement that settles the VR grasp question – pad-to-object distance at closest approach – and says explicitly not to tune the 30 mm gate or the mapper before taking it.
- 48ec186 **VR grasp failure SETTLED by measurement: neither hypothesis was right**
 The pair of numbers clip_scene now logs answers it outright: VR A (works) min_pad_obj 0.000 m, knuckle 0.4136 at that instant VR B (failed) min_pad_obj 0.000 m, knuckle 0.0 at that instant min_pad_obj_closed 0.339 m So the VR path does NOT land short – the pads reach the part EXACTLY – and the gate is NOT too tight. When the fingers closed they were 339 mm away, and when they were at the part they were fully open. The close was MISTIMED, and neither of the two hypotheses I offered was correct.

e642506 **The physics gate FAILS, and it fails in both directions**

The gate: close the shipped Robotiq 2F-85 on a 40 mm cube in Gazebo, hold it without jitter, drop it when released. Rig is the real Gen3 + 2F-85 hung upside down from a fixed post, every joint at zero, so the approach is top-down with no IK anywhere and the lift is a fold that preserves tool orientation exactly. No ros2_control, no MoveIt, no ROS. Result at the SHIPPED fingertip friction ($\mu=100000$): lifts 0.256 m, survives a 90 deg arm rotation at 0.24-0.48 mm RMS – and will not let go.

2026-08-11 — 60 commits

264529a **Section 3: T3 replaces Task B; ALOHA written up; the randomisation region is EMPTY**

THREE PARTS, and the third is the one with teeth. 1. T3 REPLACES TASK B. The slot keeps the key "B" – renaming it would break the dispatcher, the GUI buttons and run_experiment.sh, which is the exact "button labelled T3 runs something else" failure the archive README exists to prevent. The CONTENT is T3 and only T3: the compliant (sling) half is gone, 18 trial slots become 9. WHICH T3, because there are two and only one is usable.

c16737c **Archive README: the two T3s, and which one is the regression**

Verified after the scope revert that the last commit touched ONLY the B slot: TASK_A and TASK_C are byte-identical to their pre-change text, the Task C header and OPTIONAL_PICK_PLACE are present, and the archived file contains no A or C content. Nothing needs restoring.

6591a83 **Gates 1-3 measured. Gate 1's premise is wrong, Gate 3 is impossible.**

GATE 1 – the timeline is not 2.5x over, and the cap does not need raising. The brief assumes a real-arm cap of max_vel_rad_s 0.05. The shipped value is 0.15 (teleop.launch.py real_max_vel_rad_s); 0.05 is the HOMING velocity and real_max_step_rad, which are different parameters.

87e61fb **Gate report and experiment architecture (A-J), gates answered first**

docs/research/12_experiment_gates_and_architecture.md. Gates are measured, the architecture is drawn around what they returned rather than around the brief's assumptions. Counterbalancing checked rather than assumed: williams_square(5) returns 10 sequences (odd n needs 2n), and check_balance says n=15 gives position imbalance 0 but SEQUENCE imbalance 2. n=10 and n=20 are perfect. Recruit 20 or state the residual – do not use 15 and call it counterbalanced.

69ee55b **Build 1/5: the task layer, five adapters, and the mode-independence PROOF**

Build order item 1, done before any task code exists, because if the property does not hold then nothing built on top is comparable. task_actions.py – seven verbs (REACH_TARGET, PICK_OBJECT, PLACE_OBJECT, GRASP, RELEASE, TRANSFER, RETURN) over an Adapter with exactly four methods. It names no mode. Phases are part of the trace signature so an adapter cannot quietly skip one – teleporting to the object would produce a trace missing DESCEND and the proof would catch it.

7a11b89 **Build 2/5: T0 target reaching – seeded, region-bounded, and it REFUSES**

Extends experiments/abc/task0.py rather than adding a parallel spec. The fixed A/B/C/D set stays as the Fitts CALIBRATION; the randomised L1-L3 / R1-R3 set is the study instrument. Both share the same measured BAND so neither can drift from the geometry the other was verified against. sample_targets(arm, seed) 3 spheres per arm, reproducible from the seed, drawn ONLY from the band verified with the bench in the planning scene, with an optional live-IK validator applied to every candidate.

dcea886 **T1 and T2 re-measured against the corrected spec, before building either**
T1 – THE CUBE COSTS NOTHING AGAINST A CYLINDER, and the switch was unnecessary. A cube admits four grasp yaws, a cylinder all of them; measured on identical poses in each arm's own graspable strip: fixed cube(4 yaw) cylinder(8) top-down left 12/12 100% 12/12 12/12 12/12 right 5/12 42% 11/12 92% 11/12 92% 12/12 100% 11/12 either way. Gate 2's 5/12-vs-11/12 was FIXED versus YAW-FREE, not cube versus cylinder.

4f53151 **T1 layout does NOT fit at the current bench edge – measured, holding before changes**

Option 3 (left arm only) is recorded, and three of its four reasons hold. The fourth – "four cubes and both planes fit in the left arm's region" – is FALSE at the current bench edge, and the brief said to come back rather than change anything, so nothing has been changed. scripts/verify_t1_layout.py searches rather than testing one layout: it enumerates every cell that is BOTH supported (near face at or behind the bench edge, i.e.

d44dbb8 **The general constraint: support must come from BEHIND; T0 is the control**

Two things to carry forward, recorded rather than remembered. 1. WHY A SHELF CANNOT HELP (docs/system/12_randomisation_region.md). The pinned tool axis is measured at (-0.153, +0.846, +0.511): 120.8 deg from straight down and 30.7 deg ABOVE horizontal. The hand enters from the near side and from BELOW, and the fingers close underneath the object and lift into it.

c7336dd **The rail does NOT restore MASTER_TELEOP. Modelled, swept, and controlled.**

A cantilevered lip flush with the bench top, thin in z so the space BENEATH it stays free for the fingers – the only support geometry that can satisfy the 30.7-degree near-side-from-below approach. Swept at three edge positions, bench in scene, full densified pick path (standoff, descend, lift). rail edge FIXED cells TOPDOWN cells 6-item layout none (0.245) 0 18 no (18564 combos searched) 0.230 0 19 no (27132) 0.210 0 20 no (38760) 0.190 0 22 no (74613) FIXED IS ZERO AT EVERY RAIL POSITION.

76631ff **Parts 1 and 2: T1 settled on option 4 with a verified layout; regression fixed**

PART 1 – T1 SETTLED. Options tested in the order the brief set. Option 2 (smaller standoff) cannot even be tested: at the current bench edge FIXED has zero supported cells on the GRASP POSE alone, so there is nothing for a smaller standoff to rescue, and with a rail at y=0.19 the grasp pose is still zero – the rail occupies exactly the volume the fingers enter, as the general constraint predicts. –sweep-standoff REFUSED rather than reporting a ladder with no rungs.

- 23c3a25 **Blockers A and B settled; T2 re-spec'd; T3 built; all four MSc tasks verify 0**
 BLOCKER A – the free-space check was the broken one, and it was hiding more than T2. Settled three independent ways, bench in scene: ARITHMETIC two declared T2 grip poses – S2's start at (+/-0.25, 0.35, 1.10) – lie INSIDE the bench slab (z 1.06-1.10, y 0.245-0.63). No solver involved, so it cannot be a solver artefact. PER WAYPOINT N=10, both arm assignments: S1 4/4 fail, S2 9/11, S3 8/10 = 21, exactly the number the regression block reported. CLEAR BAND feasible from z=1.28 at y=0.35 and z=1.30 at y=0.38.
- e874da1 **Resume point: parts 3-4 done, part 5 (recording) is the next boundary**
- b038d9c **Part 5 start: archive the superseded clips, record the two distinctions**
 ARCHIVED the 15 A/B/C clips to archive/recordings/verification_20260811/ with a GEOMETRY_NOTE naming BOTH reasons they are superseded, not just the obvious one: 1. Every Task B clip was recorded against the declared band z 1.10-1.30, which is now known to be inside the bench slab (z 1.06-1.10). Two of its declared grip poses are geometrically INSIDE the slab – arithmetic, no solver – and per-waypoint at N=10 with the bench in scene the three paths fail 4/4, 9/11 and 8/10.
- 1180cec **Part 5 wiring: clip paths for all four MSc tasks, BUILT but NOT YET VERIFIED**
 msc_clip_tasks.py – the recording paths for T0/T1/T2/T3, in the same registry shape as clip_tasks.TASKS so record_abc_sweep.py can drive them without a second sweep. Extends, does not duplicate. Every coordinate comes from an already-verified source and nothing new is invented: T0 from task0.sample_trial at a FIXED seed (the randomised L1-L3 / R1-R3 set is the study instrument – TARGETS_LEFT is the A/B/C/D Fitts calibration set, and a clip must show what a participant sees), T1 from the option-4 layout, T2 from t [...]
- 52cd314 **Resume point: clip paths unverified, no clip recorded**
- ccefd1d **Clip paths VERIFIED through IK – and the first run of this was vacuous**
 THE CLIP PATHS NOW PASS: 103 distinct waypoints at N=10, both arms, bench in scene, 0 failures. 5264 IK calls. t0 L 9 distinct 0 fail R 8 distinct 0 fail t1 L 61 distinct 0 fail R 1 distinct 0 fail t2 L 4 distinct 0 fail R 4 distinct 0 fail t3 L 8 distinct 0 fail R 8 distinct 0 fail Controls all correct, including the must-fail one: a pose inside the bench slab comes back unreachable. THE FIRST RUN OF THIS WAS A VACUOUS PASS AND I NEARLY REPORTED IT.
- 5c5465a **clip_scene: item tables for all four MSc tasks, and the guard T0 needs**
 Sweep wiring, continued. clip_scene.py now has an item table for t0/t1/t2/t3: t0 NO OBJECTS – and tick() now RETURNS EARLY on an empty item dict. Without that guard next(iter(self.items.values())) raises StopIteration; with a placeholder object instead, the one task whose whole point is that there is nothing to grasp would show something to grasp. T0 runs in every mode precisely because it has no object.
- ef7a90d **Sweep wiring complete – and the MSc task names COLLIDE with an archived refusal**
 Both wiring items done, plus one thing the resume point had not anticipated. THE COLLISION. run_experiment.sh refuses t1|t2|...|t9 BY NAME – they are the archived 300/310 mm bimanual set, and the refusal is the guard that stops a run producing a plausible number against the wrong tilt criterion. The MSc brief also calls its tasks T0-T3.

8f71470 **Mode 06: 32 clips recorded, ALL FOUR TASKS FAILED. Not a completed mode.**

The sweep ran end to end for the first time on the MSc set and REFUSED all four tasks: FAIL run exited 1; no scene_events.json – the scene never ran 32 mp4s exist (4 tasks x 8 angles) and 0 scene_events.json. The clips render; nothing has confirmed an object was ever in them. They are NOT a recorded mode and are not citable. Reported as a failure rather than as 32 files. Three bugs found by running it, each of which hid the next:

7d2a5ce **Clean up: the sweep leaks one Xvfb per angle, and the locks are self-worsening**

Found after the failed mode-06 run: 8 orphaned Xvfb servers – one per angle – and 8 stale /tmp/.X9*-lock files, left because the sweep failed before its teardown ran. Killed by explicit PID, locks removed by explicit path. Recorded in NEXT_SESSION because it is not tidiness.

5a6a597 **Xvfb teardown on EVERY exit, proven by killing mid-angle; 32 bad clips deleted**

ITEM 1 – THE LEAK IS FIXED AND THE FIX IS EXERCISED ON THE FAILURE PATH. record_rviz.teardown_displays() kills every Xvfb and RViz BY PID – pkill -f has killed the shell running it three times in this project – and removes /tmp/.X<n>-lock and the socket for each of the 8 views. record_rviz.stale_displays() the precondition, returning what is live or still locked so the sweep can NAME it.

7df7c81 **Xvfb teardown proven on the failure path; three recording bugs found by running**

THE LEAK IS FIXED AND THE FIX IS EXERCISED WHERE IT MATTERS. teardown on try/finally AND on SIGINT/SIGTERM/SIGHUP – a finally does not cover the signal, which is how this sweep actually ends when a session runs out. Proven by SIGTERMing a run mid-angle: "killed 16 process(es), removed 8 lock(s)", then XVFB=0 LOCKS=0. A precondition now refuses on any live Xvfb or stale lock, naming each. THE VERIFIER'S BLIND SPOTS were already closed; this session VERIFIED that rather than assuming it.

0e8e39c **Recording blocked: no joint_state_broadcaster. Chain diagnosed to one contradiction.**

VERIFIER (item 1) – all three named blind spots green, re-run BEFORE any recording as asked: 7/7 controls correct including black_capture and hud_text_only, plus the per-angle control in both directions – "black gripper is CAUGHT" and "all-good clip is not flagged". It refuses to report until those pass. RECORDING (item 2) – still blocked, but the chain is now established rather than suspected: probe: TIMEOUT joint_state_msgs=0 ik=True followers=1 1.

2be27d6 **The respawn storm fixed at SOURCE; my master:=false diagnosis was wrong**

ITEM 1 – WHICH LINK WAS LYING: NONE OF THEM. Run directly, run_teleop.sh gate:=false master:=false yields 0 master_pose_node lines, so "\$@" forwarding and the launch file's IfCondition are both honest, and sim_session passes the flag correctly. MY PREVIOUS REPORT WAS WRONG.

- 0a80b7f **Mode 06 recorded: 2 of 4 pass the sweep, and LOOKING shows none is correct yet**
 FIRST CLIPS THAT ARE NOT WORTHLESS. The stack comes up, the arms move, the scene runs, captions burn in, and scene_events.json is written. T0 and T2 passed every automatic check; T1 and T3 failed with "NO GRASP RECORDED at all". THEN I EXTRACTED FRAMES AND LOOKED, WHICH IS THE POINT OF ITEM 4, AND BOTH "PASSING" CLIPS ARE ALSO WRONG: T0 Both arms reach out, caption right, arms clear of the wearer – and there are NO SPHERES AND NO BENCH in the frame.
- 24e013a **Four mode-06 defects fixed; T1/T3 grasp diagnosed from pad position**
 T1 and T3 failed for DIFFERENT reasons, and the existing pad diagnostics said so without any guessing at schedule indices: T1 cube_0 min_pad_obj_m 0.0957 knuckle 0.4235 closed_m 0.0957 T3 box min_pad_obj_m 0.0978 knuckle 0.0 closed_m None T1: the fingers CLOSED (knuckle 0.42) 95.7 mm from the cube, against a 30 mm gate. 0.0957 is |PAD_OFFSET| to the millimetre.
- dd89e1d **Mode 06 re-record: 4/4 FAIL, and the failure MOVED – resume point written**
 The four fixes are in and the scene is right: scene_events.json records fixtures, T0 lists its six spheres, T2 lists its ball, and the subject check's control fails as required. The re-record still failed, and reporting it as progress would be false. WHAT ACTUALLY HAPPENED: all four gated the capture on TIMEOUT, which means graph.moved() never fired – the arm did not move – and the runner exited 1 for its own reason, 'no arm moved at least 0.010 m'.
- f92297e **kill_stack() was blind: ps comm truncates at 15 chars, so it killed almost nothing**
 THIS IS WHY THE 06 RE-RECORD FAILED 4/4, AND IT IS NOT THE SCENE. stack_pids() compared ps -o comm against a set of full node names. comm truncates at 15 characters, so three of the five entries could never match: robot_state_publisher (21) -> comm reports robot_state_pub ros2_control_node (17) -> ros2_control_no ik_follower_node (16) -> ik_follower_nod Only move_group, rviz2 and spawner were ever killed.
- 2d1b5be **Resume point: the 06 blocker is DISCOVERY, not the scene**
 Replaces yesterday's resume point, which drew the wrong conclusion from the right data. It blamed the scene changes for four failed recordings. They were not the cause and the pad distances it quoted were measured on an arm with no TF; that paragraph is withdrawn. The sweep was recording a graph that had 66 leaked stack processes, 10 leaked launch parents, no move_group, no /tf publisher and 153 stale shm segments – because kill_stack() had never been able to see any of it (ps comm truncates at 15 chars).
- 6a77a0d **Narrow the 06 blocker: the stack is fine, the SEPARATELY-STARTED process is isolated**
 Decisive evidence, from the stack's own log: fsr_gripper_node reports 'OPEN REFERENCE CONFIRMED from /joint_states at 0.0000 rad' for both arms. So /joint_states is published and received. The broadcaster is not dead and the topic is not dead. What fails is any process started SEPARATELY from the launch: the readiness probe (joint_state_msgs=0 while ik=True, three times), a fresh 'ros2 node list -no-daemon', and 'ros2 topic hz /joint_states'. Nodes launched together see each other; a new process joins nothing.

- bbbb9cb **The environment theory is WRONG – tested, reverted, recorded**
 Sourcing scripts/env.sh in wait_ready() was the one-line fix my own resume point predicted. It was tried and it made the probe WORSE: without env.sh joint_state_msgs=0 ik=True with env.sh joint_state_msgs=0 ik=False With the variables pinned the probe stopped seeing even /compute_ik. So the missing ROS_DOMAIN_ID / ROS_AUTOMATIC_DISCOVERY_RANGE / RMW_IMPLEMENTATION are not what isolates a separately-started process, however well the theory fitted CLAUDE.md's own account of env.sh.
- 3518ed2 **Exclude three more candidates by measurement; the signature points elsewhere**
 (a) The machine's DDS is FINE. Sanity floor with NO stack: ros2 topic pub in one bare shell, ros2 topic hz in another -> 5.002 Hz. Separately-started processes CAN discover each other here, so 'wsl -shutdown' is not indicated and the fault is not the box. (b) Discovery RANGE is not it. teleop.launch.py pins SUBNET via setdefault and this machine has eth0/eth1 DOWN with only eth2 up, so SUBNET multicast with no healthy interface was a good theory.
- 0608c22 **ROOT CAUSE: Fast DDS shared memory. Discovery was fine; DATA never flowed**
 The probe was instrumented to print LISTS instead of counts, per the resume point, and settled it in a single run: probe sees 1 nodes: ['sim_ready_probe'] probe sees 81 topics /joint_states publishers=1 subscribers=7 PRESENT in topic list Discovery was never broken. The probe found all 81 topics and counted the publishers and subscribers correctly – it knew exactly who was there. What never arrived was DATA: zero samples on a topic with one publisher and seven subscribers already matched.
- 81dcceb **The 06 blocker was NOT the transport: it was clearing /dev/shm under the launch**
 Five sessions of transport and environment theories all measured the same three zeros, and the reason they could not tell each other apart is that both things they were varying were fine. WHAT IS ACTUALLY WRONG. sim_session cleared every /dev/shm/fastrtps_* segment and launched the stack in the same instant.
- 2808bbe **Per-task scenes: T0 loses the bench, T1 gains its planes, and "objects rest on a surface" is measured to be impossible for this wrist**
 FOUR SCENE DEFECTS, and the third one turned into a platform finding. (a) T0 HAD FURNITURE IT DOES NOT USE. It is reaching only – no objects, no grasp, nothing to rest on – and it was being handed the bench, the bin and a circuit box. With them came the bench's limits: T0's target band was derived as the largest rectangle BOTH arms can work with the bench in scene, 200 x 80 mm, which is why its three targets landed 76 mm apart.
- e775862 **T0's three targets become three DIRECTIONS, re-derived in T0's own empty scene**
 The targets were 76 mm apart and the whole task was 0.219 m of commanded travel. They came from task0.BAND, which is the largest rectangle BOTH ARMS can work WITH THE BENCH IN THE SCENE – 200 x 80 mm – and T0 has no bench. The band was correct for the question it was derived for and wrong for this one, and nothing said so because the same scene was applied to every task.

- 825408f **Resume point: the geometry is clean, mode 06 is recorded by the NEXT session**
 What changed about the state of the project, in one line: mode 06 is no longer BLOCKED, it is simply not RECORDED. Every earlier resume point named a blocker – the scene, then discovery, then the transport – and all three are settled and measured.
- cca29fc **Archive the pre-fix mode-06 clips with a geometry note before re-recording**
 Four clips recorded against a scene that has since changed in four ways, from a run that gated every capture on timeout with the arm at home. They are evidence about the failure, not about the tasks, and the note says which.
- 73d8f46 **Mode 06 recorded, and SIX defects found by looking at the frames**
 All four tasks pass the sweep AND inspection. Mode 06 is citable. task angles EE travel L/R (m) grasp/release sweep T0 8 1.716 / 1.477 - / - OK, motion-gated T1 8 2.001 / 0.239 4 / 4 OK, placed 3 mm T2 8 0.711 / 0.604 - / - OK, motion-gated T3 8 0.249 / 0.361 2 / 2 OK, placed 0 mm Every clip is 7 angles plus the quad, captions burnt in (mode, task, scenario, expected, caveat, result), Xvfb :91-:98, capture gated on MOTION in all four – not one gated on timeout.
- 8d2e050 **Mode 01 recorded, and T1's LAST cube exposed a lag defect that had to be fixed in the TASK, not the mode**
 Mode 01 master teleop: four tasks, eight angles each, all motion-gated. task EE travel L/R (m) grasp/release sweep T0 1.569 / 1.283 - / - OK T1 2.099 / 0.000 4 / 4 OK, placed 0 mm T2 0.711 / 0.604 - / - OK T3 0.909 / 1.016 2 / 2 OK, placed 0 mm THE DEFECT, and it is the reason to record one mode at a time. T1's first mode-01 pass grasped THREE of four cubes and still passed the sweep, because the placement check reads items[0] and cube_0 was fine.
- 5b2cb67 **Mode 03 recorded: four tasks, all four grasps closing at 0.0 mm**
 task EE travel L/R (m) grasp/release sweep T0 1.541 / 1.294 - / - OK T1 2.165 / 0.515 4 / 4 OK, placed 16 mm T2 0.711 / 0.605 - / - OK T3 0.911 / 1.023 2 / 2 OK, placed 0 mm Every capture gated on MOTION, eight angles each, captions burnt in. THE LENGTHENED PICK HOLD HOLDS UP ACROSS MODES, which is what it was for.
- addf3b1 **Mode 02 VR teleop recorded, and the VR lag shows up exactly where it should**
 task EE travel L/R (m) grasp/release sweep T0 1.707 / 1.453 - / - OK T1 2.118 / 0.480 4 / 4 OK, placed 22 mm T2 0.718 / 0.575 - / - OK T3 0.904 / 0.998 2 / 2 OK, placed 0 mm THE ISOLATION RULE WAS ASSERTED, NOT ASSUMED, and it named the right thing: isolated: no unowned publisher on the follower input (the MAPPER is the only publisher; the runner drives it upstream on /vr/controller_pose_*) VR IS MEASURABLY THE LAGGIEST PATH, and it falls out of the numbers rather than being asserted: mode grasp at clip T1 placed c [...]
- 1dcac33 **Mode 04 recorded – ALL FIVE MODES DONE, one identical task definition**
 04_vr_shared task EE travel L/R (m) grasp/rel sweep T0 1.760 / 1.478 - / - OK T1 2.160 / 0.446 4 / 4 OK, placed 9 mm T2 0.710 / 0.604 - / - OK T3 0.909 / 1.024 2 / 2 OK, placed 0 mm THE WHOLE SET, 20 clips x 8 angles = 160 files, every capture gated on MOTION and not one on timeout: mode T0 travel L/R T1 g/r T1 placed worst closed 06_full_autonomy 1.716 / 1.477 4/4 14 mm 0.0000 m 01_master_teleop 1.569 / 1.283 4/4 0 mm 0.0000 m 03_shared_autonomy 1.541 / 1.294 4/4 16 mm 0.0000 m 02_vr_teleop 1.707 / 1.453 4/4 22 mm [...]

- aea7379 **Resume point: all five modes recorded, and the seven defects only LOOKING found**
 The recording is done. What the file now leads with is the part that matters for whoever picks this up: four of the seven defects passed the sweep, and every one of them was an arm moving with objects moving alongside it – the shape that looks correct in every metric.
- 29146f1 **Freeze the verified mode-06 clips: archived copy, README, and a tag**
 The four mode-06 clips are the first in this project to pass BOTH gates – the automatic sweep and frame-by-frame inspection – and four of the seven defects found on the way there had passed the sweep, so a clip set that survives both is not cheap to reproduce. Copied to archive/recordings/mode06_GOOD_20260811/, 32 mp4 plus every scene_events.json and the sweep's progress file, 19 MB.
- 061f77b **Real-robot readiness: a paper audit, and the safety path is weaker than the arm path**
 docs/system/14_real_robot_readiness.md. No hardware was contacted. Every verdict is read off the code and the machine's current state, and where the code and CLAUDE.md disagree the code wins and the disagreement is named. THE HEADLINE: the ARM path is in better shape than the SAFETY path.
- a81797d **Task 1: a real two-level table, per-cube slots, measured workspace markings – and the front centre is NOT REACHABLE, which changes 2(b)**
 VERIFIED N=10 OVER THE FULL PATH AFTER EVERY MOVE: 6304 IK calls, TOTAL FAILURES 0, all four controls correct, every task against its own scene. (b) THE FRONT CENTRE IS NOT BUILDABLE, and this is the measurement rather than an opinion.
- ac0de7d **Note in the frozen archive that the scene changed after those clips**
 The clips are untouched and still valid for what they show. The README now says what moved afterwards – the two-level table, the planes 15 mm outboard, the per-cube slots and the workspace markings – so a reader does not assume the current scene matches, and marks T1 in that set as superseded geometry.
- ea0c961 **Resume point: items 0-2 done, 3-5 not started, and the region item 3 needs**
 The file now opens with the six-part brief as a state table and the measured reachable region item 3 samples from, with the two limits that travel with it – the survey box truncates it and cells were tested at the grasp pose only. It states 2(b) as a finding rather than an omission: zero reachable cells anywhere with $|x| \leq 0.10$, nearest reachable x is 0.30, so planes centred in the front centre cannot exist on this rig.
- be28e16 **Fix the two silent real-robot failures, and pin both against regression**
 Both had the same shape: a safety mechanism that runs, reports success, and is about something other than the real arm. Neither showed up in any check, because every check was satisfied – the clearance number was a number and the halt logged a line. (a) THE CLEARANCE MEASURED THE WRONG ARM AND SAID NOTHING. `measure_clearance()` looked up `f"{arm}_{link}"` – unprefix, the SIM tree – while the real arm publishes under `real_*` (both real launches run `robot_state_publisher` with `frame_prefix="real_"`).

- f3e8a5e The unprefix-name class, swept: five more services and one live bug**
 This class has now appeared six times and every instance was found separately – reactivate_gripper as a GPIO, the Robotiq COM port, tcp/twist.* and reset_fault/* as C++ literals, reactivate_gripper again in the driver, and last commit’s emergency halt. Finding them one at a time IS the defect, so this is the sweep rather than another instance. THE RULE APPLIED, because "prefix everything" is not it.
- f1d3122 Make the test suite unavoidable: the session runner refuses to RECORD while it fails**
 Two tests broke last session and went unnoticed for a whole session because the suite was not run. Of the three ways to make that impossible, I chose the third, and the reasoning is the point:
- 0b843dd T1 Stage 2: both arms at once, positions from the surveyed region – and the region had to be re-surveyed to be usable**
 A THIRD KIND OF BIMANUAL, and the write-up must not merge it with the other two. The project already separates: T2 PHYSICAL COUPLING – one rigid body at two grips 500 mm apart. Neither arm’s pose is free given the other’s: a height difference at one grip IS a tilt at the other. Remove an arm and the task is IMPOSSIBLE. Tilt and separation are JOINT metrics no single arm can produce. T3 BIMANUAL BY ROLE – two objects, two places, two jobs. Neither arm constrains the other.
- e4bce0c Item 4: the sequence, run end to end – and THE PIPELINE PRODUCES NO DATASET**
 The question was "so I know the pipeline produces a complete dataset before a participant sits down". Run end to end in sim, the answer is NO, and that is the deliverable. CAPTURED: 0 sample rows across 0 files run_abc.py is the ONLY entry point the MSc tasks have – run_experiment.sh –taskset msc dispatches straight to it – and it contains ZERO references to TrialLogger.
- 8cc305b Resume point: the data path is missing, and item 5 is not started**
 The file now opens with the two things that decide what happens next: the MSc set has a CLIP path and no DATA path – measured, 0 sample rows across 0 files – and the demonstration mode is not started, with a note that its perception half rests on a detection rate that is still unmeasured. Everything done this session is a one-line summary underneath, including the two silent real-robot failures and the five further unprefix services the sweep found.
- abc00ae The MSc tasks now HAVE a data path, and every mode moves**
 Item 4’s finding was that the whole study’s output was missing: run_abc.py was the only entry point the MSc tasks had and it never constructed a TrialLogger. Measured then: 0 sample rows across 0 files. Measured now, all 19 cells of the sequence, every task in every mode it runs in: 19 of 19 cells OK, 0 FAIL 1520 sample rows across 19 files 42 of 66 sample columns populated THE LOGGER IS OPENED BY run_abc ITSELF, not by a wrapper.
- 9fa4352 Choreography verified; unmask the failure the teardown was eating**
 THE TEARDOWN WAS DESTROYING THE EVIDENCE. run_one returns grabs=None on every failure path, and stop_grabs(None) raised AttributeError – before the sweep logged WHY the run failed. So a real, nameable failure surfaced as a stack trace in the cleanup code, pointing at the wrong file. Fixed at both ends: stop_grabs tolerates an empty map, and the sweep prints the reason at the moment it decides, not after teardown.

6d2c3cd **Mock RGB-D at the camera boundary; T1 stage 2 recorded**

MOCK THE BOUNDARY, THEN SAY WHERE IT IS. Same pattern as mock_real.launch.py and the desktop Quest mock: stand a plausible producer at the edge so everything inside it runs, and be explicit about what that cannot show. Publishes on the topics the real driver owns – /<arm>_camera/color/image_raw (rgb8), /color/camera_info, /depth/image_raw (16UC1 MILLIMETRES). Depth is mm on purpose: vlm_object_locator carries a mm-or-metres branch and the mm side has never run.

af5a297 **Status table, read from disk – and two false positives in it**

Every MSc task x every mode, with four columns counted SEPARATELY because they fail separately: built, verified N=10 over the full path, clip on disk, trial rows on disk. "It works" has meant each of these at different times in this repo. task B master vr_tel shared vr_shr full_auto T0 target reaching 9 CD CD CD CD CD T1 pick and place 10 CD CD CD CD CD T1 s2 both arms 10 .D .D .. .D CD T2 coordinated carry 9 CD C. C. C. CD T3 circuit + meter 10 CD C. CD C.

2026-08-12 — 30 commits

8212e58 **Fix the stale spec count the suite caught; resume point**

test_only_the_two_CURRENT_task_sets_are_offered asserted 20 MSc specs (4 tasks x 5 modes) against 25. The CODE was right: T1 has two STAGES and they are separate dispatcher keys with separate session cells, because stage 2 is the simultaneity condition and a participant must do stage 1 first or they learn the task and the simultaneity at once. A stale constant in the test written to catch drift is precisely the drift it exists to catch – which is an argument for the test, not against it.

159637f **T2/T3 VR rows: the path is fine, the PLAN does not ask. Table now says so**

CAUSE, MEASURED BEFORE CHANGING ANYTHING. The missing T2/T3 VR data cells are not the VR data path failing again. msc_session.PLAN runs T2 in the two ANCHORS ONLY and T3 in the anchors plus shared autonomy – the deliberately half-crossed design the module already documents and costs out. The logged cells match the plan exactly: planned cells 19 cells with rows 19 PLANNED BUT MISSING DATA : NONE logged though NOT planned: NONE And the path itself was re-proven rather than argued.

cd3fac5 **T1 stage 2 filmed in all five modes – no real gaps remain**

clip dirs 25 logged cells 19 of 19 planned REAL GAPS – missing clip : NONE REAL GAPS – planned, no data : NONE Inspected, not just counted: the 02_vr_teleop clip shows the right caption, two yellow cubes on the left arm's side and two cyan on the right, both arms moving at once and neither entering the other's cells.

13eb160 **SCAN POSE: the wrist cameras cannot see the work surface from home**
THIS IS THE FIRST THING THAT WOULD GO WRONG ON CAMERA DAY, and it is now answered before anyone stands in a lab. Measured in sim: at home the bench projects to pixel (954, 539) and the table to (1115, 765) for the right camera – IN FRONT of it and outside a 640-wide image. The left camera is parked looking up and back.

812cafe **Choreography FILMED – three routines, eight angles, caveat in every caption**

Built and verified last session (2420 IK calls, 0 failures); now recorded. 06_full_autonomy/d1/D1_wave OK 8 files, quad 06_full_autonomy/d2/D2_mirror OK 8 files, quad 06_full_autonomy/d3/D3_sweep OK 8 files, quad -> recordings/demo_20260812/ THE CAVEAT IS IN THE PICTURE, not only in the module. Every frame carries: "A DEMONSTRATION IS NOT EVIDENCE. This routine produces no trial data, no condition, no metric and no comparison, and nothing in it may be cited as a result.

4d97734 **CONFIRMED: the band is at the body, and the BENCH is why. Measured, not argued**

CAUSE CONFIRMED AND EXTENDED. The reported cause is right and understates it. The surveyed band is y 0.05..0.20; the bench surface spans y 0.245..0.630, so it overlaps the band by EXACTLY 0.000 m. The bench is entirely unreachable. And all four T1 cubes are over NO surface at all – not over the bench (they sit 15-75 mm in front of its near edge) and 0.170 m above the table. Nothing ever checked object-against-surface, so this was never reported.

b8d5b89 **Mount measured (it is NOT the fix); dance rebuilt with beat and phrasing**
=== 1. THE NUMBER I OWED YOU. (a) IS NOT THE FIX === Furniture transformed per arm along with the target – the honest version of the mount experiment – probing 20 mm above the table top: baseline L 0.050 R 0.050 tilt -30 deg L 0.125 R 0.150 tilt -45 deg L 0.200 R 0.225 fwd 0.15 + tilt -30 L 0.125 R 0.150 fwd 0.30 L 0.050 R 0.050 The best candidate buys 0.150 m and STILL leaves the work at the waist.

0f790af **ONE TABLE: bench deleted, work moved out front – NOT YET RECORDABLE**

APPLIED * the bench is gone from every MSc scene. It cost 0.250 m of forward reach (no furniture 0.425, table only 0.425, bench only 0.175) and its surface overlapped the reachable band by EXACTLY ZERO, so nothing could rest on it. The A/B/C set keeps it – those clips were verified against it.

dce8a7d **The dance timing DOES survive to the arm – measured, three stages**

CLAIM CHECKED BEFORE ACTING ON IT, and it does not hold. The recorder does not teleport through waypoints: routine recorded clip flow 18.1 s 49.8 s pulse 17.4 s 50.9 s play 25.0 s 68.1 s The clips are LONGER than the routines, not shorter – real time plus the approach from home and the settle. run_abc publishes one waypoint per spin(0.05), which is wall-clock 20 Hz, so duration is honoured by construction; there is no per-waypoint duration for the recorder to ignore because the runner IS the clock.

f84c22b **LIFT MEASURED: it is not the limitation. The LEFT ARM cannot grasp at all**

Asked at every real T1 grasp point, N=10 per 5 mm step, lift = the last height that passed N/N (a MARGINAL height does not count): PINNED, scene left -, -, -, - NONE REACHABLE PINNED, scene right 300, 300, 300, 300 >= 300 mm PINNED, no furniture left -, 300, 300, 300 cube_0 unreachable PINNED, no furniture right 300, 300, 300, 300 >= 300 mm FREE wrist, scene left -, -, -, - NONE REACHABLE FREE wrist, scene right 300, 300, 300, 300 >= 300 mm 300 mm is the SWEEP CAP, not a limit – the right arm never ran ou [...]

- ae7c140 **Placement answered from scene_events, not frames: 20/20 on their OWN plane**
TOTAL own 20 WRONG 0 off 0 never released 0 Four cubes x five modes, every one on the plane of its own colour. The question no longer depends on frame sampling: a cube is 40 mm, a plane is 140 x 100 mm and the capture runs at 5-11 fps, so the moment of release can fall between frames entirely – colour-pixel counting is the wrong instrument for a question about POSITION. THREE VERDICTS, NOT TWO.
- 27a97ed **Could NOT reproduce the Section 1 premise – reporting before building on it**
Everything in items 1b, 1c and 3 rests on "TRAC-IK honours OrientationConstraint and top-down is 0/128 with the wearer". Item 3 would write that into the thesis as a design finding, so it was probed first. It does not reproduce here.
- b861660 **FK CHECK: the constraint IS honoured. Tolerance artefact confirmed**
pinned and top-down are 169.7 deg apart (tool axis 120.8 deg) request achieved exact pinned, no constraint 0.00 deg from pinned pose PINNED + constraint TOP-DOWN 0.05 REFUSED 4/4 exact top-down, no constraint 0.00 deg from top-down pose TOP-DOWN + constraint TOP-DOWN 0.05 0.00 deg from top-down ROW 2 IS THE WHOLE ANSWER, and it is why the test asks for a pose and a constraint that DISAGREE.
- 91e6dd5 **TOP-DOWN TESTED AS A REAL APPROACH: it is complementary, not better**
N=10, full path, wearer AND furniture, constrained yaw-free IK at 0.05 rad – the mechanism modes 3/4/6 would actually use: task pinned near-side top-down constrained T1 0/4 reachable 0/4 reachable T2 2/2, lift >= 300 mm 0/2 reachable T3 0/2 reachable 1/2, lift >= 300 mm THE TWO APPROACHES ARE COMPLEMENTARY, NOT ORDERED. Each rescues a task the other cannot do.
- fe6db97 **T1 layout: the PEDESTALS are fatal in every option. Removed**
Three options, N=10, wearer and furniture, before choosing anything: option cubes planes left arm, table only 2/4 1/2 left arm, stands ON 0/4 0/2 RIGHT arm, table only 4/4 2/2 <- the only clean pass right arm, stands ON 0/4 0/2 split, table only 3/4 2/2 THE PEDESTALS ARE FATAL EVERYWHERE – 0/4 in every layout that has them.
- f2da765 **The workspace marking is STALE – the mismatch is 170 mm, not 15 mm**
Measured against WORKSPACE['left'] = x 0.300..0.700, y 0.050..0.200: object x outside by y outside by cube_0 60 mm 10 mm cube_1,2,3 0 10 mm plane_0 70 mm 170 mm plane_1 0 170 mm WIDENING x TO 0.365 WOULD NOT FIX IT, so it is not applied. The dominant error is in y and it is 170 mm on both planes. WORKSPACE was surveyed from the work plane WITH THE BENCH IN SCENE, where the reachable band really was y 0.05..0.20.
- e18717d **Two design properties written up: mode-independence and the 40 mm floor**
MODE-INDEPENDENCE IS STRUCTURAL, NOT EMPIRICAL, and that is the stronger claim. In run_abc.py the commanded trajectory is `wp = spec["build"]()` – built from the TASK spec with NO mode argument. `mode` selects only the topic the poses go out on, the isolation check and the logging labels. Every mode therefore commands identical waypoints to the same follower under the same limits, so 100% / 100% / zero variance is the NECESSARY consequence rather than a surprising measurement.

- f82b814 **MEASURED: the wrist DOES point up at home. +30.8 deg left, +22.1 deg right**
 arm tool axis elevation angle to table normal left (-0.153, +0.846, +0.511) +30.8 deg 120.8 deg right (+0.259, +0.890, +0.376) +22.1 deg 112.1 deg left fingertip sits +40 mm ABOVE the EE origin right fingertip sits +49 mm ABOVE the EE origin SO THE OBSERVATION IS CORRECT AND IT IS NOT A FRAME BUG. The gripper approach axis points UP AND FORWARD at the home pose on both arms, 112-121 degrees away from the table normal, and the fingertips sit ABOVE the wrist rather than below it.
- a3af107 **Pad measurement is INVALID – FK returned the home pose. Not reporting it**
 The per-grasp pad measurement ran to completion and its output must not be used. Every row reports the SAME EE and pad height regardless of the grasp requested: t1 left (0.320, 0.190, 1.120) EE z 1.146 PAD z 1.186 t2 left (0.250, 0.350, 1.320) EE z 1.146 PAD z 1.186 t3 left (0.357, 0.095, 1.065) EE z 1.146 PAD z 1.186 Three grasps 255 mm apart in z returning one number to the millimetre.
- 8e360bb **PADS MEASURED PROPERLY: not in the table – 40 mm PAST the object, upward**
 Instrument first, because the last one lied. /compute_fk returned the home pose whatever solution it was given, so it is not used. The EE->pad offset is CONSTANT in the end-effector frame, so pad_world = p_grasp + R(q) @ offset is exact and needs no solver. Validated by recomputing the home pad position from the offset: error 0.00e+00 m (left) and 5.55e-17 m (right). Negative control: a grasp 300 mm below the table is flagged at -260.5 mm, so the check CAN fail.
- bf9cdc0 **Part 2b: the work surface height has ONE owner, and 20 mm is now NAMED**
 FIXED AT SOURCE, not with a check bolted on top. srl_experiments/work_surface.py holds the height; clip_scene READS it rather than defining it. The fallback literal survives only inside an except branch that WARNS, because a silent fallback to a hardcoded height is precisely how a measured surface can exist and reach nothing. THE SPLIT THAT MATTERS is DECLARED versus MEASURED. Declared is what the sim builds and what every task coordinate was verified against; it is a constant and should be.
- 04c8f30 **PART 3: twelve new faults injected. THREE FAIL SILENTLY**
 fault_injector.py handles 14 and predates the camera work, the work-surface owner and the object-pose gap. These twelve had never been injected. fault what named recovers object_rotated_30deg SILENT NO no object_missing SILENT NO no wearer_moves SILENT NO no camera_garbage PARTIAL yes unknown gui_killed PARTIAL yes re-launch, session resumable table_20mm_high DETECTED yes yes table_20mm_low DETECTED yes yes table_never_measured DETECTED yes n/a object_moved_after_scan DETECTED yes re-scan (srv_resweep) camera_absent [...]
- 0e78162 **A1 first half: attachment tests ORIENTATION, so a rotated object now FAILS**
 WHAT WAS SILENT. Attachment tested DISTANCE alone – near = d <= GRASP_NEAR_M – so an object at an angle was grasped as though square and the sim reported a clean grasp. On a 40 mm cube a 30 deg error is about 20 mm of misalignment across the face; in reality the fingers catch a corner or shove the cube away. Nothing anywhere compared the gripper’s closing axis to the object’s, so simulation could not show it and it would have appeared on real hardware as an inexplicable miss. NOW MEASURED.

0a2c93c **Part F: the accuracy table is now a REPRODUCIBLE SCRIPT, not a quoted figure**

Flagged three times that the 100% / 0.000 mm / zero-variance numbers lived only in session reports and could not be re-derived. scripts/accuracy_table.py derives them from scene_events.json, which every clip already writes, so the table is a consequence of committed data rather than a recollection. No new trials are needed to reproduce it. AND THE NUMBERS DO NOT MATCH WHAT WAS QUOTED.

15af670 **Split CLAUDE.md: 98k tokens -> 11 kB. Nothing deleted, verified line by line**

CLAUDE.md was 4789 lines and 98.2k tokens, a TENTH of a session's context window consumed before any work started, every session, whether or not any of it was relevant. It is now 209 lines and about 3k tokens. docs/system/findings.md 2935 lines + the old header as an appendix docs/system/hardware.md 689 lines docs/system/wsl.md 375 lines docs/system/architecture.md 632 lines NOTHING WAS DELETED AND THAT IS CHECKED, NOT ASSERTED.

5f50355 **PART C: three dance buttons dispatched to the WRONG taskset and exited 1**

FOUND BY RUNNING, which is the only way this class is ever found. [1] dispatch d1 FAIL exit 1 unknown msc task key 'D1' [1] dispatch d2 FAIL exit 1 [1] dispatch d3 FAIL exit 1 I added d1|d2|d3 to the MSc case arm, which hardcodes -taskset msc, so run_experiment.sh d1 exited 1. THE GUI HAPPENED TO SURVIVE IT: its spec appends -taskset demo and argparse takes the last one, so the button worked while the command it claims to run did not.

4977a43 **Part D blocked: the two gates the brief records as closed are OPEN**

Checked in the repo rather than assumed: T1_CUBES still at POSITIVE x – T1 still runs on the left arm at 0/4 WORKSPACE still y (0.05, 0.20) – the 170 mm stale value pad offset no 0.098 / tool_axis anywhere in the task or action code Recording against that films the left arm failing to reach four cubes, inside a marking that excludes both targets, with the pads landing 39.5 mm past each object. That is a fourth superseded set and it costs an hour to produce. NOT RECORDING.

271eb34 **Sweep names what it is about to film, and refuses the wrong taskset**

CORRECTION TO THE PREMISE, checked rather than accepted: the switch already exists and already defaults to the MSc set –

396f418 **T1 mirrored to the right arm: 49 failures -> 1. Not yet zero, so NOT recording**

MEASURED AT EVERY STEP, N=10 full path, wearer and furniture in scene: starting point (left arm, pedestals) 49 failures mirrored to the RIGHT arm 11 cube row moved into the surveyed band 2 planes pulled 20 mm nearer 1 T1 now drives the RIGHT arm: 4/4 cubes and 2/2 planes at the grasp points, 146 waypoints, 8 grip changes. T1_ARM is honoured by the path and the grip schedule instead of "left" being hardcoded in three places, so the side is one edit rather than four.

c4c4040 **THE LAYOUT GATE IS CLOSED: 0 failures across all five tasks, N=10 full path**

t0 50 distinct waypoints, 0 failures t1 80 distinct waypoints, 0 failures t1s2 50 distinct waypoints, 0 failures t2 22 distinct waypoints, 0 failures t3 16 distinct waypoints, 0 failures Wearer and furniture in the scene, N=10 per waypoint over the densified path.

9db677f Archive note: everything before the T1 mirror is superseded

Records what changed and why the old clips should not be used: T1 moved to the right arm from 0/4 on the left, the bench and the pedestals are gone, and the cube row moved into the surveyed band. The layout now verifies at zero IK failures, N=10 full path, wearer and furniture in scene. mode06_GOOD is untouched.

88e5fe6 Information card plays FIRST, instead of an overlay across the footage

The caption used to sit on top of the arms for the whole clip, competing with the thing it describes. A viewer reads it while trying to watch the motion. A card that plays first is read once and then gets out of the way. Six seconds of full-frame card, then the footage, on EVERY angle rather than only the front one. Card states the task, how it is being driven, what happens and what to watch for.

df7d3d5 T1's scene still watched the LEFT arm after the task moved right

'NO GRASP RECORDED at all' on a run where the arm plainly picked cubes up and placed one 112 mm from target. The task moved to the right arm; clip_scene still had arm='left' hardcoded on every T1 cube, so the right gripper closed on the object while the scene watched the left one. Now follows MCT.T1_ARM, so the side is one edit rather than two files that can disagree. Scenario renamed S1_left_arm -> S1_right_arm, which was the visible tell and which I left stale when I mirrored the task.

fb7e425 Orientation check REPORTS instead of gating; it was refusing every grasp

I added it last session to make a rotated object visible in sim. It made attachment conditional on the gripper being within 20 deg of square, and the pinned wrist sits well off square, so a live sweep produced four clips reading 'NO GRASP RECORDED at all' on runs where the arm visibly picked cubes up. The check now measures and warns without deciding whether the object attaches. max_yaw_err_deg still lands in scene_events.json and the warning still fires, which is what made the failure visible.

d2a0ece Camera was framing where the work USED to be

T1 moved to the right arm, so its cubes sit at x -0.32..-0.50. The capture camera still looked at x=0, which put the whole task at the right edge of frame and partly outside it, with the empty left-arm workspace marking occupying the middle of the picture. A viewer sees a marked box with nothing in it and the work happening off the edge. Focal point x 0 -> -0.10, the true centre of every task's span (-0.54..0.34), and distance 2.25 -> 2.55 so the whole span fits. Found by extracting a frame and looking at it.

81eae18 Part E: recording is clean, the CAMERA is aimed at the wrong side

Mode 01 recorded 5 of 5 with 0 failures, twice. T1 shows 4 grasps with the pads on the cube at 0.000 m, and the card plays first and reads well. Both bugs from the previous sweep stayed fixed. What is wrong is the framing, and I found it by extracting frames and looking rather than by any check: the work sits at the right edge of frame and partly outside it, while the EMPTY left-arm workspace marking holds the middle of the picture. A viewer sees a marked box with nothing in it.

- 8deb5ca **Framing: eliminated two wrong controls by experiment, found the third**
 The front view's aim comes from NEITHER of the things I edited. Established by running, not by reading: `verification_capture.rviz` aims the standalone RViz only. Edited twice, the frame never moved. `__FOCUS` in `record_rviz.py` aims the other angles. Set to 0.15 and to -0.10; the front view was IDENTICAL both times. the FRONT view FOLLOWS THE TASK `record_rviz.py` line 121 says so, and the config cache holds `front_t1.rviz` next to `front.rviz`. That is the one to fix.
- 71a39bc **Framing fixed: TASK_FOCUS aims the front view, and T1's entry was stale**
 TASK_FOCUS in `record_rviz.py` aims the FRONT view per task. T1's entry was (0.34, 0.22, 1.19) which was the OLD left-arm cube row. T1 moved to the right arm at x -0.32..-0.50, so the task sat at the right edge of frame while the empty left-arm marking held the middle of the picture. Now (-0.30, 0.22, 1.19) at distance 1.45. Checked by looking: the wearer is fully in shot, the right arm reaches out over the work, the blue and green cubes sit on their matching mats, and the cyan right-arm marking encloses the work.
- 0777214 **Mode 01 recorded against the fixed layout and framing**
 5 of 5, 0 failures. Cards play first, the work is in frame with its marking around it, and T1 shows 4 grasps with the pads on the cube.
- 4f3386c **Sweep: 19 of 25 cells recorded, and two patterns worth more than the count**
 01_master_teleop 5/5 T1 4/4 T1S2 4/4 T3 2/2 02_vr_teleop 2/5 T1 0/4 T1S2 0/4 T3 0/2 03_shared_autonomy 4/5 T1 0/4 T1S2 4/4 T3 2/2 04_vr_shared 5/5 T1 4/4 T1S2 4/4 T3 2/2 06_full_autonomy 3/5 T1 0/4 T1S2 4/4 T3 2/2 dance 1/3 02_vr_teleop loses EVERY grasp while its arms travel 1.0 to 2.6 m, so the pose command lands and the gripper command does not. It is not that VR is broken: 04_vr_shared drives the same controllers and gets everything. The fault is specific to the 02 path.
- dfdc1e4 **TASK_SPEC is the source of truth, and the pre-recording pass found nine gaps**
 docs/TASK_SPEC.md holds the brief verbatim as acceptance criteria, and scripts/audit_task_spec.py asks every MUST BE TRUE item of the CODE before a single clip is filmed. 12 self-test probes, each breaking one input on purpose, because a check that cannot fail is not a check. 40 PRESENT, 0 MISSING, 1 BLOCKED. Nine items were missing when it started:
- 5d4d133 **Archive the three superseded clip sets with a note naming all nine changes**
 recordings/verification, `verification_20260813` and `verification_20260811_t1rework` move to `archive/recordings/superseded_20260813_prespec/`. `mode06_GOOD` is untouched. GEOMETRY_NOTE.md names what changed under them, item by item: the re-surveyed marking, cells instead of a bounding box, one marking for a one-arm task, the presentation pose, T2's carry series, T3's card, the wrong arm in T1's expect text, and the leaked loop variable that measured left-arm yaw against the right gripper.
- 9a465f9 **Found what makes T1 flaky: ee_for() uses the LEFT arm's pad offset**
 A single T1 clip under 01_master_teleop, the mode that used to score 4/4, records NO GRASP AT ALL with the pads stopping 31.4 mm from three of four cubes against a 30 mm gate, while the arm travels 2.72 m. THREE CUBES REPORTING THE IDENTICAL 0.0314 IS THE CLUE. They sit in a row 60 mm apart, so an identical closest approach to all three is the perpendicular distance from a path running parallel to the row: a CONSTANT offset, not the accumulating lag the earlier note described.

- 1db80dc **One pad offset per arm, shared by the task layer and the scene**
 PAD_OFFSET was the LEFT arm's wrist-to-pad vector and was used for both arms. Measured off TF at home, which IS the pinned anchor and the only orientation a task path ever holds: left (-0.0171, +0.0945, +0.0572) identical to the old constant right (+0.0289, +0.0995, +0.0421) 48.3 mm away from it ee_for(obj, arm) now takes the arm and defaults to left, so the A/B/C set does not move. Every MSc task passes its own. AND clip_scene._pad_offset() reads the SAME table instead of sampling live TF.
- d387b23 **MOVE_TO reaches the resolver, and T3's four points go on the box**
 TWO GAPS, and both were "the piece exists and nothing connects to it". 1. THE MOVE_TO VERB. voice_intent has parsed "move to the front centre" since it was written and NAMED_PLACES said which places are reachable – and the executive fell through to "I don't know how to 'goto'", so the system refused a command it was designed to accept.
- cbc742e **Part 1: the pose in the screenshots is the PRESENTATION pose, and it is a T**
 IDENTIFIED FIRST, with numbers, because getting it wrong in either direction is expensive. scripts/describe_pose.py reports both poses through /compute_fk: HOME wrist axis +30.8 / +22.1 deg (UP) hand span 1.459 m PRESENTATION wrist axis +6.5 / +7.5 deg (LEVEL) hand span 1.633 m The discriminator is the wrist.
- 184633c **The clearance floor, not the search, is what forbids hands at torso width**
 CORRECTING MY OWN MEASUREMENT. measure_ready_pose_envelope.py reported both arms reaching $|x| = 0.14$ with a level wrist and I reported the brief as achievable. That sweep asked MoveIt's collision-aware IK, and the WEARER PAIRS ARE SRDF-EXCLUDED – the trap mount_guard_node exists for. Measured geometrically instead, with no exclusions: hand $|x| = 0.18$ clearance -0.016 m the arm is INSIDE the torso 0.20 -0.004 0.26 +0.002 .. +0.041 0.30 +0.019 .. +0.045 0.45 +0.151 ..
- cdcb99f **Looked at the render: the pose is still splayed, and the cost said it was fine**
 RENDERED IT AND LOOKED, which is the whole point of showing a pose before recording with it. scripts/render_pose.py stages a pose and grabs a still per view, reusing record_rviz's displays so the framing matches the clip. WHAT THE PIXELS SHOW, against a cost that scored it as satisfied:
- 586dcba **The staging move now pauses the follower, so the arm actually goes where told**
 THE CONTROL CAUGHT IT. A pose search reported "home tool axis +22.1 deg (TF -0.0)" and refused to save: FK computed home while TF showed the arm still standing in the previous staged pose. The arm had never been homed – stage_presentation_pose.py -home reported success and moved nothing. WHY. ik_follower_node streams position commands to the same arm controller this script publishes a trajectory to, and once the follower has a target it holds the arm there.
- 8e220e6 **A symmetric presentation pose, rendered and looked at. It is still not right.**
 All controls pass now that the arm is genuinely homed, and the pose is mirrored by construction: both hands at (+/-0.550, 0.360, 1.100), wrists level to 0.0 deg, wearer clearance 0.1610 m against a 0.150 floor, hand travel 0.19 and 0.24 m from home. WHAT THE RENDER SHOWS, and this is the part the numbers cannot say. It is no longer a T-pose and the two arms finally mirror each other, which is the fault the last render exposed. But:

- d34118f **The dance had no scene node at all, and its envelope was a tenth of the room**
TWO FAULTS, and the first explains why the dance clips were thin. 1. clip_scene REFUSED the dance by its own argument list. "d1" was not in -task's choices, so every dance clip was filmed with the scene node failing to start: no furniture, no markers, and no scene_events.json to say how far the arms travelled. The sweep launched it, argparse rejected it, and nothing downstream asked.
- 2f99716 **Dance envelope verified clean, framed per routine, and renders tidy up**
verify_dance_paths.py on the widened envelope: 1918 IK calls, 0 failures across all three routines, both arms, collision-aware with the wearer in scene, and all three controls correct. |x| 0.26..0.50, y 0.22..0.44, z 1.02..1.58 – so the inherited envelope holds on the paths that will actually run, which is what it needed to be checked for. TASK_FOCUS gains d1/d2/d3, centred on the wearer because both arms work both sides and pulled back to 2.10 m.
- aeb6279 **Watched the dance: the canon was invisible and the subject was too small**
WATCHED d1 rather than checked it, which is the only way this question can be answered. Frames at 8, 18 and 28 s: the arms tuck in, open out and unwind, so the motion is real and the widened envelope shows. Two faults in the picture: 1. THE CANON READ AS UNISON. Measured, mirror-corrected, the two arms differed by 68 mm on average – real in the numbers and a few pixels on screen at a 1-beat lag.
- b7f91f2 **Archive the dance and demo sets before re-recording, with what changed**
recordings/dance_20260812 and demo_20260812 move to archive/recordings/superseded_20260813_dance_prespec/. mode06_GOOD untouched. GEOMETRY_NOTE.md names five changes, and the first is the reason the rest were invisible: the dance clips were filmed with NO SCENE NODE, because clip_scene rejected "d1" in its own argument list. Then the envelope (0.18 x 0.14 x 0.18 m), the canon (68 mm, a few pixels), the framing (no TASK_FOCUS entry) and the presentation pose (lost to the follower).
- 2cdd912 **The THIRD consumer of PAD_OFFSET: the gate that decides when the hand closes**
T1 under 01_master_teleop recorded "NO GRASP RECORDED at all" with the pads 0.1 mm from the cube. The geometry was finally exact and the knuckle stayed at 0.0 for the whole clip. run_abc.py held PAD_OFF = CT.PAD_OFFSET – the LEFT arm's wrist-to-pad vector – and uses it to decide when the arm has ARRIVED and the pending grip may be issued. The task path and the clip scene were both moved to the per-arm table; this one was missed.
- c9d4e3c **Looked at the T1 framing: in frame, but not centred and nearly edge-on**
The framing was verified numerically and this is the first time it has been looked at. The card plays first and reads well, the work IS in frame and not cut off, and the wearer is fully in shot – so the TASK_FOCUS fix held. Two things the numbers could not say:
- 3f26c35 **Mode 01 records 5 of 5, and the grasp chain is fixed end to end**
With the third PAD_OFFSET consumer corrected, T1 records FOUR grasps and FOUR releases with all four cubes carried 0.16-0.30 m and the fingers closing 0.0-4.4 mm from the object, against a 30 mm window. T3 records two and two. Both were "NO GRASP RECORDED at all" one run earlier, with the pads 0.1 mm from the cube.

- cbede31 **Reap ORPHANED DDS segments so the presentation pose stops losing the port race**
 Five retries did not help, because retrying does not free a port. Measured mid-sweep: 156 fastrtps segments in /dev/shm, 76 held by a live process and 80 ORPHANED. Fast DDS leaks one per participant on this host and every clip runs the stager as a fresh process, so the range fills and the clip silently opens on home.
- 4ff4e58 **A failed restore disarms the arm, so restoring is now verified by read-back**
 MY OWN BUG, and the worst kind: one clip logged "paused (motion_enabled true -> false), RESTORE FAILED" and every later clip in that mode failed, because the follower it had silenced never spoke again. The single set_parameters call was believed on its return value alone. resume_followers() now retries and READS THE VALUE BACK, retrying again if it is still false, and the script exits non-zero if it cannot restore – even when the pose itself arrived.
- 48401ad **The SHM reaper was deleting the DISCOVERY PORTS. Exclude them.**
 /dev/shm holds two kinds of Fast DDS segment and they must be treated differently: fastrtps_port7400 a DISCOVERY PORT, the rendezvous every participant uses. Not mapped continuously by anybody. fastrtps_<hex> one per participant. These are what leak. The "orphaned" test – nothing in /proc/*/maps has it open – calls EVERY port segment orphaned, so the reaper deleted the graph's rendezvous points.
- 340a48d **Remove the SHM reaper. "Not currently mapped" is not "orphaned".**
 I added it to stop the presentation pose losing a DDS port race, and it broke the graph twice. Both failures measured, both worse than the problem: 1. IT DELETED THE DISCOVERY PORTS. fastrtps_port7400 is the rendezvous every participant uses and nothing maps it continuously, so the "orphaned" test called every port orphaned. After a clean restart that had just reported READY with 30 joint-state messages, no new process could join. 2. EXCLUDING THE PORTS WAS NOT ENOUGH.
- ec2da99 **Mode 01 recorded 5 of 5, every clip opening on the presentation pose**
 With the reaper gone and the graph healthy, 01_master_teleop completes all five MSc tasks at the corrected framing. Every clip logs 'presentation pose: OK' and the progress file records opened_on=presentation for each, so the claim is per clip rather than per run.
- 0d300f6 **Modes 01 and 04 recorded: 10 of 10 cells OK**
 01_master_teleop and 04_vr_shared both record all five MSc tasks at the corrected framing. 9 of 10 clips open on the presentation pose; the one that did not is recorded as opened_on=home rather than assumed.
- a1b6332 **Mode 06 records 5 of 5, and mode 03's failures are CAPTURE TIMING not grasps**
 06_full_autonomy previously scored 0/4 on T1 and now records all five tasks. With 01 and 04 also at 5/5, three of the four modes that showed the "T1 is flaky across modes" pattern are clean, which is what the left-arm PAD_OFFSET in run_abc's arrival gate predicted.
- fa7c397 **Sweep complete: 22 of 28 cells, and the two failure modes are DIFFERENT**
 01, 04 and 06 record 5 of 5 each; the dance records 3 of 3. 06 previously scored 0/4 on T1, so three of the four modes that showed last session's "T1 is flaky across modes" pattern are now clean – which is what the left-arm PAD_OFFSET in run_abc's arrival gate predicted. The six failures are two unrelated faults and must not be lumped together: 03_shared_autonomy – CAPTURE TIMING, not grasping.

6c934d9 **25 of 28 cells, every one opening on the presentation pose**

Mode 03 re-recorded 3 of 3 on a quiet stack, so its "GRASP OUTSIDE THE CLIP" failures were CONTENTION, not a synchronisation bug – the same three cells that failed at -16.4 s of a 4.4 s window now record cleanly with 4 grasps and 4 releases. Worth knowing before anybody rewrites the motion gate: the fault did not reproduce. Standing at 01, 03, 04, 06 at 5/5, the dance at 3/3, and all 25 successful clips recording opened_on=presentation.

341d0ce **Part 5: the T2 carry series found what an end-state check cannot**

The tilt/separation series was added because T2 could only be scored pass/fail at the end. On its first full sweep it earned that: mode tilt_rms tilt_max time above 6.8 deg 01_master_teleop 1.085 5.994 0.0 s 02_vr_teleop 1.643 3.522 0.0 s 03_shared_autonomy 1.146 5.723 0.0 s 04_vr_shared 10.482 29.475 8.28 s 06_full_autonomy 10.478 32.047 11.19 s T2's whole claim is that tilt past 6.8 degrees drops the ball.

2026-08-15 — 20 commits

d782743 **T1 to the left arm, and the workspace measured against the wearer for the first time**

The brief was four changes to T1 and one question about the workspace. The question turned out to be the session: every workspace number this project has ever quoted comes from /compute_ik with avoid_collisions, and the SRDF excludes torso/harness/backpack against each arm's base, shoulder and half_arm_1 – the pairs a shoulder-mounted arm actually threatens. So MoveIt returns valid for poses with the tube inside the person, and nothing had ever measured the distance.

b133a65 **Two witnesses for a moving arm, and the wearer reaches the spoken commands**

THE GATE. record_abc_sweep trusted run_abc's exit code as its only evidence that an arm moved. The two measure the same quantity in different processes off different tf2 listeners, so the runner can see motion the scene node never received – and that is what got filed on 2026-08-15: clips reporting TRAVEL L 0.00 R 0.00, four cubes carried 0.000 m, recorded OK. scene_travel_verdict() is a second, independent gate reading the scene node's own ee_travel_m.

e2e85b3 **Mode 01 recorded: 5 of 5, every clip on the presentation pose**

T0, T1, T1 stage 2, T2, T3 under 01_master_teleop, eight angles each plus the card. Both travel witnesses agree on every clip: t0 left 1.4071 right 1.4167 both arms reach t1 left 3.1056 right 0.5956 one-arm task, left t1s2 left 2.0120 right 1.9336 both arms, as stage 2 requires t2 left 1.0747 right 1.0071 bimanual carry t3 left 1.2575 right 1.2239 bimanual by role TWO BUGS FOUND BY RUNNING IT, both of which quietly damage a set. 1. THE LEDGER WAS OVERWRITTEN BY A SUBSET RUN.

35d032c **Modes 06 and 03 recorded, and the looking pass finds three things no check can**

10 clips, eight angles each, every one opening on the presentation pose. Both travel witnesses agree on all ten. THREE FINDINGS FROM THE PIXELS AND THE GEOMETRY, none of which any automatic check in this repository can produce, because each is a thing that is true BY CONSTRUCTION and therefore cannot fail. 1. T2's TRAY IS ELASTIC.

- 457c7d5 **02_vr_teleop: 5 of 5, and the cause was the mapper's LIFETIME**
 02_vr_teleop failed three of five cells on the first pass – T1, T1 stage 2 and T3, which are exactly its three tasks that record a grasp. It never touched a cube: the pads' closest approach was 88.1, 115.4, 156.2 and 205.9 mm against a 30 mm gate, while 01, 03 and 06 closed on all four at 0.0000 m. The final frame shows all four cubes still in their starting row. TWO CAUSES, BOTH ISOLATED, AND ONE OF THEM I GOT WRONG FIRST. THE MAPPER HOLDS THE ARMS.
- a3fd7b3 **All 25 cells recorded, and three verifiers were checking nothing**
 25 of 25 cells OK, every one opening on the presentation pose. 28 clip dirs, 19 of 19 planned data cells, no real gaps. verify_rviz_clips 28 of 28 with all seven of its own negative controls correct. 510 tests pass, 2 allowed. 04_vr_shared's two failures were the DEGRADED LONG-RUNNING STACK, not the mode.
- 6309856 **Resume note: the set is complete, and what to fix first**
 25 of 25 cells, every one on the presentation pose. 28 clip dirs, 19 of 19 planned data cells, no real gaps. 510 tests pass. The note leads with the clearance ledger, because the thing most likely to go wrong next is somebody citing a pre-fix workspace number in good faith.
- b762cad **Part 1: the centre is shut by the TORSO, not by the wearer's arms**
 The project's answer to "why can the work not be in the centre" has been the wearer's own forearm, measured against a mannequin whose arms hang rigidly at its sides. That is a modelling choice, so the posture is now a variable with one source and the sweep was re-run in four postures plus the limit case of no arms at all. The centre does not open in any of them.
- e48f5bf **Part 2: the render was right, and the numbers were about something else**
 Reported: both elbows 0.14 m below the shoulders, wrists level, symmetric to 1e-16. Rendered: elbows at shoulder height and wrists angled down. Measured from /tf, which is what RViz draws, with three stills from the same boot: elbow below its own shoulder 0.0793 m left, 0.2715 m right left elbow outboard of its hand +0.188 m hands 1.10 m apart, against a 0.36 m torso tool axis elevation -0.01 deg, so the wrists ARE level forearm mirror residual 0.2826 m, against 0.0000 m at the hands It is not a cached config.
- eea4cd0 **Part 3: the null space is real now, and it still cannot help**
 An earlier pass built a seed-fan sampler, measured it worth 4.4 mm, and said plainly that seed sampling is not redundancy resolution and the real answer is an explicit parameterisation. That was built. The conclusion did not change and it is now a much stronger statement.
- ca59be0 **Part 4: the tray was elastic and T1's two stages were two tasks**
 T2. clip_scene drew the tray between the grippers with length span + 0.06, where span is the LIVE distance between them, so the prop resized itself every frame to whatever the arms were doing. Over the shipped clips it ran 0.487 to 1.211 m against a 0.560 m spec, from 73 mm short to 651 mm long, held at both ends throughout. That is why T2-1 ("held by BOTH grippers") and T2-2 ("ball visible ON the tray") could not fail.
- 7673334 **Part 5: finding 3 is real, and its size is still not known**
 The claim under test was that scripted clips with no operator show different achieved motion by mode, made on a single run per cell against a 3% same-mode spread taken from the one cell recorded twice. A single repeat is not a measurement, so the two ways it could have been an artefact were tested directly against the recorded set. Not recording length.

- 165b238 **Part 6 (pre-record): T1 stage 2 seed 0 was a stale pool, not a bad draw**
 The N=10 re-verification before recording reproduced the known regression: stage 2, seed 0, RIGHT arm, 26 waypoints inside the 150 mm wearer floor, worst 0.1135 m, with zero IK failures. Seeds 1 and 2 were clean, which is what made it look like one unlucky draw. It was the pool. `work_surface_region.json` was surveyed before the 2026-08-15 home change and its right-arm `clear_cells` reach $|x| = 0.400$.
- 9efd6e1 **Part 6: mode 01 recorded, and the missing producer three verifiers wanted**
 Mode 01 recorded all five MSc cells on the teleop stack: every clip opens on the presentation pose, both arms travel (T1S2 1.635 / 1.881 m), the grasps land at +5.2 s and +6.5 s, T3 places 1 mm from target, and `verify_rviz_clips` passes 28 of 28 clips on every automatic check. Three other verifiers failed, and they were right to.
- a4cb334 **Part 6: mode 02 recorded, and T2 carries the tray with its hands open**
 Mode 02 recorded all five cells. With the grip trace now being produced, the three verifiers that had been refusing to report could finally run, and two more faults had to be cleared before they could: their glob knew only the legacy four-level path and CRASHED unpacking four components from the MSc's three, and GRASP_TASKS held lower-case retired names while the path spells the task in capitals, so no MSc clip was ever classified as a grasp clip.
- 678984b **Part 6: mode 03 recorded, and "0 of 10 genuine grasps" was two instrument faults**
 Mode 03 recorded all five cells. `verify_grasp_quality` ran on the MSc set for the first time and said 0 of 10 clips show a genuine grasp. The scene's own record says otherwise and is not ambiguous: T1 logs GRASPED and RELEASED for all four cubes, knuckle 0.383 to 0.395 against a needed 0.3812, `min_pad_obj_m` 0.0 on every cube, carried 0.23 to 0.40 m, all four finishing on their pads.
- f845a84 **Part 6: mode 04 recorded**
 All five cells OK on `04_vr_shared`, with the grip trace now produced so the three gripper verifiers have real input.
- d1dbe77 **Part 6: mode 06 recorded, all five modes now on the current geometry**
 All five cells OK on `06_full_autonomy`. Mode 01 is being re-recorded because its clips predate the grip-trace producer and the set has to be uniform.
- 08ba80b **Part 6: mode 01 re-recorded, so all 25 cells carry the grip trace**
 All five cells OK and five `grip_trace.json` on disk. The set is now uniform: every mode recorded on the current geometry with the producer in place.
- 25e1ee4 **Part 6 complete: 25 cells in five modes, plus the three routines**
 The dance could not be filmed before today and the check that knew it crashed: `verify_dance_paths` printed an example failure with "%s" % worst[0] against a tuple, so it raised `TypeError` exactly when it had a failure to report. Fixed, it then found d1 failing 45 of 382 waypoints and d3 49 of 554, at $x = -0.28$ on the right arm and $+0.272$ on the left. The cause is the same constraint as parts 1 and 2.

7752e5c **Part 7: looked at the clips, and four of the six things asked for cannot be shown**

Frames pulled from the re-recorded set and read against the MUST BE TRUE lists, pixels first with the measurement beside them. Stills in docs/img/home_render. symmetric home, elbows down NO 0.0793 (L) and 0.2715 (R) below their shoulders, forearm mirror residual 0.2826 objects ON the table NO they float; visible in T1, T1S2 and T3 pads and cubes in the CENTRE NO min $|x|$ 0.325 / 0.450, and 0.300 / 0.375 even with the wearer's arms deleted arms in FRONT of the person NO T1S2 shows both out at the table's ends the tra [...]

9d246dc **Home solved from constraints, T1 layout re-measured, vision drives the grasp**

THE HOME POSE WAS SOLVED, NOT CHOSEN. scripts/solve_home_pose.py states it as nine geometric constraints and searches the 14-DOF joint space for them, with FK validated against live TF to 0.06 mm. Achieved, measured from the render: elbows 0.2776/0.2794 m below their shoulders (was 0.0793/0.2715), camera 12.05 deg off vertical (was 56.4 mm BELOW the fingertips – the hand was upside down and nothing had ever constrained the roll), approach level to 1.5 deg and facing forward, whole-chain mirror residual 0.0272 m ag [...]

ffc50ba **Record T0 and T3 in mode 03 against the new home and layout**

Both cells clean, run exited 0. Mode 02 is deferred: it records a STATIONARY ARM on both cells (ee_travel 0.0000 either side) while its isolation reports correctly – vr_pose_mapper started, stopped for the staging move and restarted. Its clips are left at the committed versions rather than overwritten with failed runs.

ac3919b **Record T0 and T3 in mode 06 against the new home and layout**

Both cells clean. Mode 04 deferred with mode 02: both VR modes record a STATIONARY ARM (ee_travel 0.0000 either side) while every non-VR mode (01, 03, 06) records cleanly. The split is exactly the vr_pose_mapper path. Their clips are left at the committed versions.

639b2fc **Record T0 and T3 in mode 02, and fix the staging pose that was failing runs**

THE PRESENTATION POSE WAS STILL THE 2026-08-15 ONE. The sweep stages it before every clip, and after home was re-solved it sat 2.9465 rad (left) / 2.7723 (right) away – a 7.6-9.3 s move that regularly did not arrive. Cells then recorded a STATIONARY ARM (ee_travel 0.0000 both sides) and one clip logged "NOT STAGED – this clip opens on HOME", which breaks R-7 silently. The pose is now loaded from config/home_positions_{arm}.txt, so it cannot drift from home again.

1d001d2 **Record T0 and T3 in mode 04: all five modes now on the new home and layout**

T0 and T3 are complete across 01, 02, 03, 04 and 06 against the solved home, the re-validated region and the per-arm pad layout.

89c8dad **Record d1 and d2 in all five modes; d3 is REFUSED, not missing**

The restart-per-mode protocol is what makes these land: after any VR-mode cell the DDS graph is degraded and the NEXT run stages nothing and films a stationary arm, whatever mode it is. Clearing /dev/shm and relaunching before each mode gives 2 of 2 on 02, 03, 04 and 06. D3 IS NOT RECORDED AND MUST NOT BE. verify_dance_paths, N=3 collision-aware from the new home: d1 0 of 382 fail, d2 0 of 356 fail, d3 18 of 560 FAIL – e.g. right waypoint 124 at (-0.740, 0.435, 1.567).

e26389a **T1 is BLOCKED in the vision path, not the layout: 28-31 mm detections against a 30 mm gate**

THE GEOMETRY IS CLEAN AND THAT IS WHY THIS IS NOT A LAYOUT PROBLEM. verify `t1_paths` at the solved home, $N=10$, wearer and furniture, clearance geometric: 0 IK failures and 0 waypoints inside the 150 mm floor over 7522 IK calls, stage 1 and all three stage-2 seeds, both controls correct. `t1_paths.json` had been stale since 2026-08-15 – it predates the home solve – and is refreshed here. Stage 2 seed 0's right arm, 25 of 98 waypoints inside the floor at the old home, is CLEAN; that regression is closed.

3f1a509 **A run starts from HOME, and the second one did not**

The reported symptom was "RViz boots into the new pose but a task run starts from the old one". Measured on a fresh teleop stack, reading `/joint_states` against `config/home_positions_{arm}.txt` at the instant the FIRST waypoint is published on the mode's entry topic: at boot left 0.0000 right 0.0000 rad run 1, first commanded waypoint left 0.0000 right 0.0000 between runs (nothing else ran) left 1.2338 right 0.6248 run 2, first commanded waypoint left 1.2338 right 0.6248 So the pose is NOT lost between launch and the [...]

e68dfe7 **T1 from a typed sentence, grounded on what the camera saw**

WHAT THE SHIPPED GRAMMAR DID WITH T1'S OWN INSTRUCTION, measured before any of this was written: "pick up the blue cube and put it on the blue pad" -> grab / target "blue cube" / destination None. The destination was SILENTLY DROPPED, which is the dangerous category "put the green ones on the green mat" -> refused, "no known verb" "move that blue block to its colour" -> refused, "heard 'move to' but no place I know" One misunderstood, two refused, and the task the whole clip set is about could not be said.

b4d9813 **The T1 vision fault, reproduced exactly and fixed at both its causes**

REPRODUCED, and worse than the 2026-08-17 note recorded. Inside the recording sweep the four T1 cubes deprojected 31.7 / 32.7 / 34.0 / 35.7 mm from truth against a 30 mm capture gate, so NONE of the four was grasped – not just cube_3. The pad miss at closure in the same clip was 31.6 / 32.6 / 33.9 / 35.6 mm the same numbers to 0.1 mm. The grasp failure IS the detection error, one to one, which is the strongest evidence in the file that nothing else about T1 is wrong.

b1b664d **THE FOLLOWER PAUSE WAS DECORATIVE. That is the root of the T1 vision fault.**

`ik_follower_node` read `motion_enabled` ONCE at construction into `self.motion_enabled` and registered no parameter callback. `set_parameters` therefore changed the parameter server's copy and the follower carried on publishing – so every caller that "paused the follower" by lowering it paused NOTHING, and the node's own blocker recovery text (`ros2 param set motion_enabled true`) could not work either.

554924f **Restore the committed T1/01 clip: the last commit swept in the FAILED one**

The mode-01 recording that ran while the follower pause was still decorative overwrote the clip files on disk, and `git add -A` took them. That clip is the one whose four cubes closed 31.6-35.6 mm from the pads and which the sweep itself reported as NO GRASP RECORDED at all – it must not sit in the tree as evidence for anything. Restored from b4d9813, which is the 2026-08-16 set.

- 03a863b **The stillness gate WAITS for a still frame instead of refusing the first**
It refused the first bad frame, and inside the recording sweep that turned a transient into a lost clip – twice, both times reading EXACTLY 0.0170 m. A value that repeats to 0.1 mm across two differently loaded runs is a fixed lag, not an arm still travelling, and the refusal message reported only the distance so the two could not be told apart. It now prints both positions and the delta.
- 07df66b **T1 re-recorded under 01__master__teleop, and it grasps all four again**
The first T1 clip since the vision fault was found. Same code path, same sweep, same load as the clip that failed this morning. in-sweep detection error 1.30 / 0.50 / 1.30 / 2.30 mm was 31.7 / 32.7 / 34.0 / 35.7 mm min_pad_obj_closed_m 0.0000 on all four cubes was 0.0316 / 0.0326 / 0.0339 / 0.0356 against a 30 mm gate – 0 of 4 grasped carried 0.2074 / 0.3267 / 0.3005 / 0.2569 m was 0.0000 on all four placed 30 mm from target, was 101 mm left EE path 5.1561 m Every cube went to the pad of its own colour: cube_0 and [...]
- 9d99007 **T1 under 03__shared__autonomy: 4 of 4, every closure at 0.0000 m**
detection error 2.30 / 1.22 / 1.52 / 2.19 mm min_pad_obj_closed_m 0.0000 on all four carried 0.2118 / 0.2963 / 0.3020 / 0.2541 m placed 31 mm from target left EE path 5.1722 m Blue cubes released at x = 0.5952 and 0.5952 against a blue pad at 0.595; green at 0.8149 and 0.8223 against a green pad at 0.825.
- 63b5b37 **T1 under 04__vr__shared: 4 of 4, every closure at 0.0000 m**
detection error 1.48 / 1.55 / 2.08 / 3.09 mm min_pad_obj_closed_m 0.0000 on all four carried 0.2168 / 0.3282 / 0.2984 / 0.2452 m placed 30 mm from target left EE path 5.1551 m Blue released at x = 0.5951 and 0.5950 against a blue pad at 0.595; green at 0.8226 and 0.8211 against a green pad at 0.825. This mode also exercises the VR mapper being stopped for staging and restarted before the run, which is where 02 and 04 have failed before.
- 484b336 **T1 under 06__full__autonomy: 4 of 4, every closure at 0.0000 m**
detection error 1.94 / 1.36 / 0.92 / 1.94 mm min_pad_obj_closed_m 0.0000 on all four carried 0.2102 / 0.3240 / 0.3067 / 0.2568 m placed 30 mm from target left EE path 5.1843 m Blue released at x = 0.5951 and 0.5950 against a blue pad at 0.595; green at 0.8208 and 0.8227 against a green pad at 0.825.
- fa51d46 **T1 under 02__vr__teleop: 4 of 4, and that CONTRADICTS a documented state**
CLAUDE.md records 02_vr_teleop as 'cannot grasp – all three of its grasping tasks fail; the pads miss by 88 to 206 mm against a 30 mm gate, and the miss ACCUMULATES'. On T1, measured now: detection error 1.53 / 1.55 / 2.08 / 3.09 mm min_pad_obj_closed_m 0.0000 on all four carried 0.2130 / 0.3066 / 0.2662 / 0.2440 m placed 30 mm from target left EE path 5.1453 m Blue released at x = 0.5951 and 0.5958 against a blue pad at 0.595; green at 0.8176 and 0.8226 against a green pad at 0.825.
- 027d265 **The objects float because Rig was measuring at the wrong wrist, and T1-1 is BLOCKED**
THE INSTRUMENT FIRST. measure_what_binds.Rig – which verify_t1_paths, search_centre_on_surface, measure_objects_on_the_table and every other reachability sweep in this repo solves through – set self.quat to the LIVE end-effector orientation read off TF at construction, i.e. the HOME wrist. The tasks do not command that: run_abc.send() writes WORKSPACE_ORIENT into every waypoint of every mode.

- ca4bab1 **T1S2 seed 2 was always 16 failures, and the docs now say which wrist each number came from**
 The refreshed `t1_paths.json` baseline, at the anchor on the raised surface: stage 1 clean everywhere – the full 171-waypoint path, every per-object pick and both pad slots on both pads, 0 IK failures and 0 below the clearance floor at N=10 – and stage 2 seeds 0 and 1 clean on both arms. STAGE 2 SEED 2's LEFT ARM IS 16 OF 91, AND IT IS NOT THE SURFACE. Measured at the raised 0.980 and again at the old 0.950: identically 16.
- 658d861 **The arms are NOT at home at the first commanded waypoint, and now a clip fails for it**
 The operator looked at the last frame of the newest T1 clip and said the arms were at the old pose. They are, the clip is not stale, and nothing in the pipeline could see it. WHAT EVERY EXISTING CHECK SAID, recording T1 under `01_master_teleop: stage_presentation_pose.py` succeeded, so the clip records `opened_on: "presentation"`; the staged look reported `home` to 0.0000 rad; `require_home()` returned `homed` and the run exited 0; `verify_rviz_clips` passed 37 of 37.
- f413e21 **Two checks were pointing at the wrong thing: the home gate and the yellow cube**
 THE HOME GATE I ADDED LAST SESSION WAS MEASURING THE WRONG INSTANT, and it failed every clip including good ones. `run_abc` deliberately drives the arms to waypoint 0 and settles them there before it starts measuring – travel taken from wherever the previous run left the arm reported 0.0118 m instead of 0.4761 m, which is why that block exists. My probe sat at the first iteration of the send loop, i.e. AFTER that approach, called it "the first commanded waypoint" and read 0.7991 rad.
- e54049b **Mode 01: T1 and T1S2 re-recorded on the corrected geometry**
 Both cells, all eight angles, card first. Both open on the current home – AT HOME 0.0000 rad on both arms at the last instant before the approach, which is the reading that answers the question. T1 4 of 4 cubes closed at 0.0000 m, each placed on the pad of its own colour: blue slots at x 0.595, green slots at x 0.825. T1S2 4 of 4 closed at 0.0000 m, both arms, cubes blue/green by pad.
- 47ca64a **Mode 02: T1 and T1S2 recorded, and the detector stopped counting pads as cubes**
 T1 REFUSED TWICE BEFORE THIS, with "saw 8 cubes" and then "saw 9 cubes, the task expects 4", while the same code standalone on the same stack saw 4. The refusals were correct – they skipped rather than filming from a stale detection file – but the message was only a count, so the sweep's own dump was extended to list every accepted and rejected blob with its pixel size, the size a cube would be at that range, and its depth. The nine then split at a glance: 4 detections z 1.1118 ..
- 64e89f8 **Mode 03: T1 and T1S2 recorded on the corrected geometry**
 Both cells AT HOME 0.0000 rad on both arms at the pre-approach instant, 4 of 4 cubes closed at 0.0000 m, each on the pad of its own colour. Detection clean at four cubes with the work-plane and duplicate rules in place.
- 9cee332 **Mode 04: T1 and T1S2 recorded on the corrected geometry**
 Both cells AT HOME 0.0000 rad on both arms at the pre-approach instant, 4 of 4 cubes closed at 0.0000 m, each on the pad of its own colour.

- dcf421d **Mode 06: T1 and T1S2 recorded – all ten cells now on the corrected geometry**
Both cells AT HOME 0.0000 rad on both arms at the pre-approach instant, 4 of 4 cubes closed at 0.0000 m, each on the pad of its own colour. That completes T1 and T1S2 across all five modes, every cell re-recorded against the raised surface, the outline marking and the corrected detector. Every one reports AT HOME 0.0000 and 4 of 4 closures at 0.0000 m.
- 481fda2 **A clip counts for the task it CONTAINS, not the folder it sits in**
WHAT I WAS ASKED TO FIX, AND WHAT I FOUND. The report was that T1S2 showed as covered because T1's clips were being counted. It is not what is happening, and the numbers say so plainly: T1S2 has its own clips in all five modes, every one containing genuine stage-2 content (cube_left_0/1, cube_right_0/1), and `status_table` already keys on the exact directory name – the prefix bug (`"t1s2".startswith("t1")`) was fixed in that file earlier and its scar is in the comments.
- 34f9944 **The marking goes back to the plane it bounds, and the check can now fail on the real scene**
TWO THINGS WERE ASKED FOR, AND ONE OF THEM I CANNOT DO. Taking them in order. THE MARKING WAS 116.5 mm BELOW THE PADS, and that was my regression from the previous session. I moved it from the work plane down to the table on the argument that "paint goes on the thing it is painted on". That is wrong for the reason the frame shows immediately: the marking is the boundary a participant is told to keep the WORK inside, and drawing it under the work put the whole task outside its own boundary.
- e05e384 **T1S2 mode 01: re-recorded with the marking at the plane it bounds**
AT HOME 0.0000 rad on both arms, 4 of 4 closures at 0.0000 m. The markings now enclose the work instead of sitting 116.5 mm under it.
- 29f06fb **T1S2 mode 02: re-recorded with the marking at the plane it bounds**
AT HOME 0.0000 rad on both arms, 4 of 4 closures at 0.0000 m. The first attempt aborted: the virtual display would not render after three restarts and `record_rviz` REFUSED rather than filing a black view. Two orphaned `rviz2` processes from an earlier run were holding the display; the per-mode cleanup now kills viewers as well as displays.
- 5baa428 **T1S2 mode 03: re-recorded with the marking at the plane it bounds**
AT HOME 0.0000 rad on both arms, 4 of 4 closures at 0.0000 m.
- 7914958 **T1S2 mode 04: re-recorded with the marking at the plane it bounds**
AT HOME 0.0000 rad on both arms, 4 of 4 closures at 0.0000 m.
- efa878d **T1S2 mode 06: all five cells re-recorded with the marking at the plane it bounds**
AT HOME 0.0000 rad on both arms, 4 of 4 closures at 0.0000 m. That completes T1S2 across all five modes on the corrected marking height.
- b782a9b **Record the marking regression, the anchor re-price of supports, and T1S2 coverage**
findings.md carries the full account: the marking sat 116.5 mm below the pads because I moved it to the table last session on an argument about paint that is the wrong argument for a boundary; the on-table check was asking about T1S2 and measuring T1; and the support family, re-priced at the anchor, costs 109 of 171 waypoints where the home-wrist note claimed 22. Also recorded: the two ordering bugs in my own negative control, caught by running it rather than by reading it.

4fd9a76 **T1 verifies at the measured pad offset: symmetric pads, and the grasp lands on the cube**

The T1 rebuild and the mount change were in the working tree, uncommitted, with `test_t1_layout_is_verified` failing on all three of its assertions – the shipped constants and the walked record described different layouts. Walking the composed path found two real faults on the way. `grasp_frames.PAD_MID_EE` was 13.47 mm too long. It had been DERIVED, as the magnitude of `clip_tasks.PAD_OFFSET_BY_ARM` – a world-frame vector recorded at the anchor – rather than measured.

5f746df **T1 universally commandable: selectors, compounds, an answerable question, and stage 2 on stage 1's geometry**

THE ONE MISUNDERSTANDING, AND IT WAS LIVE. "put the blue ones on the blue pad and the green ones on the green pad" – two complete instructions sharing a verb – planned the FIRST clause and dropped the second in silence: two cubes moved of four, reported as a success. Three of the 75 phrasings did it, all compounds. `split_destination` cut at the first destination cue, so the head was "put the blue ones" and the two-targets check, which counts targets in the head, saw one and passed it.

3948edd **The pre-recording audit is alive again, and T1 passes all twelve of its criteria**

TASK_SPEC section 8 requires a code-side pass over every MUST BE TRUE item before any recording, and `scripts/audit_task_spec.py` is its mechanical form. It had been dead since the 2026-08-17 rebuild: it calls `msc_clip_tasks.t1()`, the builder moved to `t1_task` and the alias went with it, so the check that exists to catch "reported done and later found absent" died with `AttributeError` the first time it was run afterwards.

2851f71 **Drop three imports my own edits left unused**

7e5beb2 **The GUI button audit passes for the first time, and getting there found four defects**

`verify_gui_buttons` level 2 presses every button in the operations window and requires each press to produce an observable outcome. It had never completed: it hung on button 05 of 69 and was killed, every button after it unpressed, the run reporting nothing at all. What it was hung on is the modal CONSENT dialog on START SESSION, which is exactly what should block a machine.

3eddbea **The observe pose, re-solved for the geometry that exists, and a staging check that could not fail**

THE SOLVER REFUSED ON ITS OWN FK CONTROL, and it was right to. `srl_fk` compares itself against live TF recorded in `home_render.json`, and that recording is of the PRE-MOUNT-CHANGE robot: shoulder at 0.3473 against the 0.4977 the arm is at now, 0.1553 m apart, which is the 150 mm the mounts moved. The FK was correct and the reference was stale. Refreshed from live TF; FK now agrees to 0.000061 m.

6f77517 **Bring the state table and the spec up to what has been measured**

CLAUDE.md: the unit count, the marking (T1 and T1S2 draw their OWN cells, not the global survey, because not one surveyed cell is over this task's work), the observe pose, the GUI prompt box and the button audit. TASK_SPEC: R-5 now requires the typed instruction on the card verbatim, because under 06 the instruction IS the trial's input and a viewer cannot tell from the footage which sentence produced it. And a new P-D: every detection must be AT an object.

- 6bcc031 **sim_session lost every argument containing a space**
 " ".join(cmd) handed the shell a string, so -instruct "put every cube on the pad of its own colour" reached the sweep as -instruct put plus six unrecognised extras. The recording died with an argparse usage message AFTER the stack was up and the unit suite had run – a five minute setup spent to be told the command line was wrong. The shell is still needed (SRC sources the ROS setup and that has to happen in the same shell), so the arguments are quoted rather than the shell dropped.
- eabfb25 **The run started the schedule before the arms reached waypoint 0, and it cost the first grasp of each arm**
 MEASURED, on the first instruction-driven T1 recording. The clip ran, exited 0, travelled 4.69 m on the left and 1.43 m on the right – and TWO OF FOUR CUBES WERE NEVER PICKED UP: cube_0 left carried 0.0000 m hand closed 47.8 mm away cube_1 left carried 0.3125 m hand closed 0.0 mm away cube_2 right carried 0.0000 m hand closed 49.6 mm away cube_3 right carried 0.3204 m hand closed 0.0 mm away against a 30 mm capture gate. The pattern is the diagnosis: it is the FIRST cube of each arm, and only the first.
- 321217a **Stage 2 was commanded at the pinned anchor while its coordinates were solved at T1's approach**
 MEASURED, on the first stage 2 recording, and it read as a perfect approach and a dead gripper: every cube pads 1.4 to 4.2 mm away knuckle 0.00 throughout every cube carried 0.0000 m NO GRASP RECORDED AT ALL The pad distances look like the best numbers this task has ever produced. They are two descriptions sharing one error.
- fce9730 **One look per arm, because no single arm can see this layout's row at a usable range**
 MEASURED, and it is a property of the geometry rather than a bug. The rebuilt T1 spans 1.08 m of table with a pad pair in the middle. Asked for a pose from which ONE arm sees all twelve columns within 1.05 m, the solver returns NOTHING – and the poses that saw everything spanned 0.62 to 1.34 m. WHAT THAT SPAN COSTS. A 40 mm cube at 1.34 m is 18 px across and the same-coloured pad 60 mm behind it is 200.
- b422005 **The mat is excluded by WHERE IT IS, not by a six-millimetre threshold**
 The pads are the cubes' own colours – deliberately, so "each cube ended on the pad of its own colour" is judgeable from a frame – and on the rebuilt layout they are mats lying ON the table the cubes stand on. So a fragment of pad is the right colour, at very nearly the right height, and at 0.5 m it is the right SIZE too: the left camera returned five blue blobs where there are two cubes, the extra three being pad fragments 37 to 51 px against an expected cube of 38 to 45.
- 6951888 **Keep what a launched job said, and record where T1 stands**
 Gui.on_launch sent a launched job's stdout AND stderr to DEVNULL, so a job that REFUSED – naming its reason, as every refusal in this repository is written to do – produced a bare exit code and nothing else. Measured the hard way: run_abc exits 2 inside the recording sweep and runs perfectly outside it, and the one sentence saying which of its two refusals had fired was being discarded at that line. Each launch now writes to its own file in the scratch directory and the event log says where.

d1cd8bb **Archive two clips their own verifiers rejected, so no failed footage sits in the verification tree**

Both were recorded and both were REFUSED by the checks that ran immediately after them, and `abc_sweep_progress.json` records `ok=False` for each. Left in `recordings/verification/` they would read as the T1 and T1S2 evidence, which is precisely the confusion this project has been bitten by before. T1: `run exited 2; NO HOME RECORD; NO GRASP RECORDED` at all.

32751e3 **Wait for the fingers, not for a fixed number of ticks**

The first mode-06 T1 re-record picked two cubes and left two, and reported exit 0. `clip_scene`'s closest-approach numbers separate two different failures: `cube_0`'s pads reached 11.0 mm of it with the knuckle at 0.0007 – hand OPEN – and did not read closed until 48.8 mm away; `cube_2` never got closer than 42.3 mm at all, 58% of the way down a 100 mm standoff. The same run launched by hand, with nothing recording, picked all four.

962e264 **Pull frames out of a clip at the moments its own event log names**

The brief ends "the frame is the evidence", and every automatic check here has at some point passed a clip that was visibly wrong. This judges nothing: it extracts frames and prints where they are. Not evenly spaced – evenly spaced frames of a 30 s clip are mostly the arm traversing, which is the part nothing goes wrong in. The instants come from `scene_events.json`'s own wall clock, offset by the clip's `t0_wall`, so the frames land on the card, each grasp, each release and the final state.

84457b1 **Measure stage 2 at the approach it actually commands**

The scene decided where a graspable object is from `Scene._pad_offset()`, guarded by `if self.task == "t1"`. Stage 2 was rebuilt on stage 1's geometry on 2026-08-18 and commands stage 1's approach – `msc_clip_tasks` gives `t1` and `t1s2` the same `orient` – but an exact match on "t1" is not a match on "t1s2", so every stage-2 object was measured at the ANCHOR instead.

448b57f **Record T1 both stages under 06, from a typed sentence**

Two clips, mode 06 only, eight angles each, every one opening on a card that carries the operator's sentence verbatim and then the LOOK before the task: `06_full_autonomy/T1/S1_both_arms_centre` two blue cubes left, two green right, 4 grasps, 4 releases `06_full_autonomy/T1S2/S2_both_arms_random` seed 3, a 3/1 draw, 4 grasps, 4 releases into three distinct blue slots and one green Both ran from the instruction "put every cube on the pad of its own colour", both started AT home (0.0000 rad, both arms), and the `destinati` [...]

3abb809 **Write down what the two clips cost, and what still reads wrong**

CLAUDE.md gains the four rows this session earned: T1 recorded under 06 both stages, the stage-2 anchor defect, the frame extractor that never extracted a grasp, and the mislabel control's re-measurement. `NEXT_SESSION` gets a resume point whose point is that every one of the four failures was the INSTRUMENT, not the robot – including a "new" failing test that was two leftover stacks.

5bb604f **Record two things that are true today and would be assumed wrong tomorrow**

Neither is a defect. Both are assumptions a reader would make from the numbers as they stand, so TASK_SPEC 4B states them and the section-4 gap table points at them. U-4. A VOICE SCORE IS A PARSER SCORE. The 75-phrasing sweep runs voice_listener's TEXT source – no audio in that path at all.

438c91a **Re-record T1 both stages under 06, 2026-08-19**

A fresh recording of both cells on the current tree, not a re-use of yesterday's. Same instruction, same seed, and the same result, which is the point: the two clips reproduce. T1/S1_both_arms_centre 2 blue left, 2 green right; grasps at 0.42, 0.48 (left) and -0.48, -0.42 (right); 4 releases into the two pads' slots T1S2/S2_both_arms_random seed 3, the 3/1 draw; grasps at 0.42, 0.48, 0.54 (left) and -0.48 (right); three distinct blue slots at 0.2198 / 0.2798 / 0.3398 and one green at -0.2798 Both started AT home (0 [...])

800194a **VR over wifi works end to end with a real Quest, no adb**

The WebXR route was in the tree as a documented fallback and had never been run. Running it found that it could not start at all, and that its axis map was for a different frame. Both fixed; the route is now verified with a physical headset on the lab wifi.

20c8acc **Redesign for desk operation: motion capture, not VR**

The operator sits across the room FACING the wearer, holds the controllers as a 6-DOF input, and watches the real robot. Nobody wears the headset – it stands on a shelf as the tracking reference. Every head-worn assumption was wrong about who is where, and two of them were load-bearing. ROUTE DECISION, ON FACTS SteamVR/OpenVR over Quest Link was evaluated and rejected. Checked on this machine: Meta Quest Link PC app NOT installed, SteamVR NOT installed (Steam is), no OpenXR runtime registered, Quest not on USB.

2026-08-20 — 4 commits

3b8f051 **One window for the whole session, and the real arms are in it**

The GUI starts from one command and everything is reachable from it. Nothing was rewritten; the pieces that already worked are where they were. TWO VIEWS, SIDE BY SIDE. RViz is the COMMANDED view and is embedded where a window manager exists. That is a correction, not a preference: the "embedding does not work here" conclusion was measured on a host with no window manager and written down as if it were a property of the technique.

41cdf41 **The wearer is a person now, and the camera may only make them bigger**

The scene camera path is built end to end: a USB camera node, both calibration tools, a markerless body tracker, per-segment gating, the fused body in move_group's planning scene, and a GUI tab whose top line is which body the robot is planning against. NO CAMERA HAS EVER BEEN ATTACHED TO THIS MACHINE. Everything optical is still inferred and is listed as such in doc 19 section 6 rather than estimated into a table.

6e2bf68 **The observer can be somewhere, and the system can tell when they leave**

Finishing the VR phase for a lab day. The deliverable is docs/system/VR_REAL_ARM_RUN.md: a numbered procedure with a pass condition and a failure branch at every step, covering every connection problem this project has hit, and a section 14 listing what is still unverified ranked by how likely it is to bite. FOUR DEFECTS, ALL IN THE SAFETY PATH. The observer e-stop had NEVER BEEN SATISFIABLE.

534a033 **One button starts VR, and pressing it repeatedly is how the bugs fell out**
START VR TELEOP, top of the RUN tab. Twelve steps in an order that is load-bearing – the simulation before the mapper, because the mapper refuses to engage without robot TF and the symptom is a clutch that silently does nothing; the certificate and the port before the bridge, because a bridge that dies on a bound port leaves four healthy nodes and a URL nobody is serving. Each step fails in plain words with the fix as a button. It cannot reach a real arm, by construction and by test.

2026-08-21 — 1 commit

d1390f0 **The instrument was wrong six times out of eight, and the arms were fine**
Two real-arm sessions. Perception found the cube every time and nothing was picked up, because the failures were never detection: they were calibration, kinematics, and eight defects in the code that COMMANDS the arms. The one that was already in the tree: real_homing_node looked the arm link up as `real_<arm>_<link>` and the wearer part up as a bare `torso`. The real stack publishes the whole URDF under `real_`, so the wearer clearance guard resolved 0 of 9 parts in every run that has ever been made.

2026-08-22 — 15 commits

3c1eb46 **The pinned wrist cost a factor of seven, and freeing the roll was worth nothing**

"The arms barely move" has had four answers in this repository and none of them carried a size. Measured offline, wearer floor 0.15 m enforced under every policy, walking outward until the first refusal: from the arm's own HOME EE – far outboard, roomy pinned 0.460 m mean reach over 28 walks, 6 wearer-bound free 0.547 m 6 from the WORK POINT (0.400, 0.175, 1.120) – where the tasks run pinned 0.065 m mean reach over 12 walks, 11 of 12 wearer-bound cone15 0.319 m 7 cone45 0.440 m 5 free 0.450 m 4 The mechanism is n [...]

25a28e2 **Every joint parks a third of a degree short, and I called that speed-independent**

recordings/baselines/arm_directional_calibration.json has held 36 real runs since 2026-08-21 – both arms, six directions, 100 mm each, photographs in recordings/calibration_frames/ – and no code in this repository read it. The end effector lands 7.2 mm RMS from where it was sent. In joint space 211 of 252 per-joint terminal errors sit at $\pm 0.25..0.30$ deg with the sign a coin flip: a BOUND, not a spread. A joint parking just inside a band, on the side it arrived from.

6b10bf5 **It can say what is on the table, and the grasp pipeline's floor could never fire**

"It is unable to check what's on the table, it will just move here and there." That was accurate, and it was a MISSING piece rather than a broken one. There was a detector that finds a NAMED thing, a planner that grasps ONE given cloud, and a task layer commanding coordinates written down in advance. Nothing looked at a surface and enumerated what was standing on it, so a wrong name produced a move to somewhere plausible and empty.

bc99719 **The window said VR only, and RViz was embedding an invisible helper**

The button audit read 203/203 the whole time these were live, because it presses buttons and none of these are about buttons. RVIZ WAS NEVER ACTUALLY EMBEDDED. `_rviz_windows()` matched any X window with "rviz" in its line and the caller took the LOWEST id – which is "Qt Selection Owner for rviz2", invisible. The geometry dump measured the CONTAINER and read 1005..1554 exactly as designed, every check passed, and the real RViz sat in its own window on top of the GUI.

70ef8ae **LeRobot is worth its dataset format and not its policies, and the docs to run all this**

Installed lerobot 0.6.1, exported 23 episodes / 20 440 frames of the real teleop recording as a LeRobotDataset, and wrote down the honest reading. THE CHANNEL GATE IS THE INTERESTING PART. The newest channel baseline says the master arm was 5 ALIVE, 1 INTERMITTENT, 6 INCOHERENT, 2 DEAD. A dead channel is a column of one repeated value; exported without comment it becomes a zero-variance feature indistinguishable from a joint held still on purpose.

0c069e4 **The hand lands 33 mm from where it was told, and the wearer moves faster than the guard can hear**

Asked "how good is the system", this repository could produce a hundred numbers and no budget. A budget is what tells you which number to work on. `scripts/measure_control_budget.py` composes the recorded baselines into one; it measures no new hardware and prints UNMEASURED where nothing has been measured, because a budget with a guessed term hides the thing you should go and measure. WHERE THE HAND ENDS UP: 33.4 mm worst case against a 30 mm grasp gate.

1fceffc **There was no motion planner, and we were reading seven numbers off the arm**

Two findings, and they are most of "everything works in simulation and then real hardware messes up". THIS PROJECT HAD NO MOTION PLANNER. `ik_follower_node` solves ONE IK per commanded pose and clamps the joint step per cycle – pose streaming with a rate limiter. There is no OMPL in the command path, no `move_group.plan()`, no Cartesian path service; the only OMPL reference in the tree is a vendored Kinova launch file this project does not run. MoveIt is used for `/compute_ik` and the planning scene, and nothing else.

384db95 **Each joint was walked to the IK solution on its own, and the hand went 51 mm somewhere nobody asked for**

`ik_follower_node.clamp_towards()` was the whole of motion generation for teleoperation. It clamps EACH JOINT INDEPENDENTLY to `max_step_rad` per control cycle, so a joint needing 0.10 rad arrives in one cycle while one needing 0.50 takes two: the joints are at different fractions of their own travel at every instant, the path through joint space is not the straight line the IK solution implies, and the end effector traces a curve nobody asked for. On a rig whose grasp capture gate is 30 mm.

- dc726df **The diagnosis reproduced to the decimal place, and then four things wrong with the instrument**
 scripts/measure_teleop_motion.py. It reproduces the recorded diagnosis exactly before it is allowed to report anything else – max 135.3 mm, mean 45.1 mm on the left arm's shipped home plus [0.50 -0.30 0.10 0.25 -0.05 0.15 0.02] at 50 Hz – because a baseline you cannot reproduce is not a baseline.
- e3e4956 **The follower generates motion through the generator now, in BOTH branches – the fast path was the unguarded one**
 clamp_towards is replaced in ik_follower_node, and the replacement is in both branches on purpose. It used to be two: a solution within max_step was published RAW – a 0.35 rad jump in one 20 ms cycle, 17.5 rad/s against a 1.3963 rad/s joint limit – and only a solution outside it went through clamp_towards. So the path with "no guard needed" was the faster one. motion_generator:= is a parameter and a launch argument on both stacks. ruckig is the default in every mode.
- d22d832 **The audit's launch stub reimplemented the front of on_launch, so nothing added there could ever be checked**
 THE GUI RULE. How the arms move (next launch) on SET UP: which generator the next launch uses, what each costs in plain words and millimetres, and a button that runs the whole comparison offline with no stack up. It applies to the NEXT launch and says so – every parameter in ik_follower_node except motion_enabled is read once at startup, so a control that appeared to change a running follower would be a control that changed nothing.
- 9490a41 **The excursion settles at 68 mm rather than going away, so it was never a step size**
 The write-ups. docs/system/24_motion_planning.md moves Ruckig from "not yet wired; next" to what it measured, with the three decisions worth carrying forward and what is still unmeasured. docs/system/findings.md carries the full account, including the five things that were wrong in my own work on the way – four of which would have passed a test suite – and the six definitions of CONTINUOUS_IDX. CLAUDE.md gains one row.
- 6753293 **The pads stopped 13.45 mm short of every object T0, T2 and T3 declared, and the picture agreed with the task the whole time**
 The largest single term in scripts/measure_control_budget.py's error budget, against a 30 mm grasp capture gate, and it survived because it cancelled everywhere it was looked at. clip_tasks.PAD_OFFSET_BY_ARM was two hand-recorded world vectors of magnitude 0.1118 m. grasp_frames.PAD_MID_EE is the same quantity in the end effector frame, measured by /compute_fk on the two finger-tip links on 2026-08-18: 0.09833 m, both arms agreeing to 0.000 mm.
- 29ccfe0 **The pad midpoint is a curve, not a constant, and staging went to a sixth copy of the home pose**
 Asked to take the accuracy as far as it goes and then check that everything works. The error budget was the map. RSS of the measured terms 18.64 mm -> 12.91 mm WORST CASE (they add) 33.39 mm -> 19.99 mm against a 30 mm gate ## THE PAD MIDPOINT DEPENDS ON HOW OPEN THE HAND IS The Robotiq 85 is a FOUR-BAR LINKAGE: its fingers swing, they do not translate.

7f13d57 **The recording sweep could not record: how to run it, and what it was refusing about**

`scripts/record_everything.sh` drives the whole verification set one mode-task cell per stack, resumable, in `mode_upstreams.MODE_ORDER`. One cell per `sim_session` invocation and not one long sweep, for the reason `record_t1_both_stages.sh` is written the same way: churning processes against a live stack degrades the DDS graph until a fresh process receives nothing while every node is still up. `-fresh` only stops it SKIPPING.

2026-08-23 — 16 commits

83e467a **The scene drew T1's cubes 11.43 mm from where the hand was told to close, and filed the clip "1 OBJECT NEVER MOVED"**

MY OWN REGRESSION, CAUGHT BY THE RECORDING IT BROKE, and it is the third time this one function has produced that sentence. `t1_task.ee_for` moved onto the gripper opening a 40 mm cube needs earlier today – the Robotiq's fingers swing on a four-bar, so the pad sits 0.09833 m from the wrist wide open and 0.10976 m closed on a cube. `clip_scene . _pad_offset` stayed on the WIDE-OPEN value. So the task commanded a wrist that puts the pads on the cube and the scene drew the cube 11.43 mm from there.

12db4b7 **T2's grippers had never closed, and three instruments were reporting defects that were not there**

THE WHOLE VERIFICATION SET IS RE-RECORDED against the fixed geometry: 17 cells, eight camera angles each, every mode. The old set is archived, not deleted. grasp success, all five modes 100% positioning error at closure 0.000 mm T1 placement 319.7 -> 56.3 mm (var 264 -> 0.59) `verify_rviz_clips` 33 of 33 `verify_gripper_motion` 8 of 12 -> 12 of 12 `status_table` REAL GAPS NONE ## T2'S GRIPPERS HAVE NEVER CLOSED, AND THE TRAY WAS NOT WHY CLAUDE.md records "T2-1 'held by BOTH grippers' is false and has been for the life of [...]"

8e6b7ee **The one section whose job is "which file to open first" named four files that had not existed for weeks**

`make_clip_index.py` generates `recordings/verification/INDEX.md` and its own header says it is generated so that it "cannot claim a clip that is not there". That was true of the table of every clip and false of the shortlist above it: the five recommended paths were written out by hand against the A/B/C task set, and four of them – `06_full_autonomy/B/S1_bimanual`, `01_master_teleop/A/S1_single_arm`, `02_vr_teleop/B/S1_bimanual` and `06_full_autonomy/A/S1_single_arm` – were retired when the MSc tasks replaced them [...]

6d20f42 **The planner has never known the table exists, and everything runs on coordinates a file declares**

The operator watched the re-recorded clips: the arm "does not scan the area properly in a straight line, it moves in random directions", and asked for what a 3-D printer does before it prints – probe each point in order. Then the part that matters more: the real cell has a different table and different objects, teleoperation is unusable because there is no proper planning, and the robot should calibrate the environment from every source it has, build a 3-D map, and let the path and grasp layers adapt to THAT.

c1e39a3 **Objects come out of the picture now, not out of the point pile**

The calibration stage found things on a surface the way this repository always has: fit a plane to the fused cloud, cluster the points above it by Euclidean distance. Run against the T1 scene it reported object 0 at (-0.356, 0.471, 1.272), 140 mm across the jaws, NOT graspable where two 40 mm cubes are. They sit on a 60 mm pitch, so the gap between two faces is exactly 20 mm, which is the cluster distance. The map refused an object that does not exist, with a confident reason naming the gripper.

5621520 **The camera pose has to come off the frame, and the viewing angle is a measurement**

Two defects in the live half of the calibration stage, and the second one is the reason the first three rounds of this looked like a perception problem. THE POSE. `env_probe.camera_pose` looked the transform up with `Time()` – the LATEST – which is where the camera is now, not where it was when the frame was taken. The map came back with the cubes REPLICATED, one copy per view, each displaced by about one cell of the sweep: 13 objects for a 6-object scene, and two runs of the same command gave different maps.

4592bd1 **A pick and place planned from the map the robot measured**

Everything else in this repository that picks something up was told where it is. `run_abc` builds T1's path from `T1_CUBES`, four coordinates in a source file. That is verified N=10 over densified paths and it survives exactly one cell. `pick_from_map` reads what the arm SAW during the calibration sweep and picks an object out of it. Move the cube and the plan moves. Put a different object down and it appears in the list. Nothing in it knows what T1 is. And it is the first caller `joint_planner.VoxelWorld` has ever had.

74ce92c **The window says what the robot is doing, and the clip shows what it found**

THE GUI RULE: a capability that only exists in a terminal does not exist for the person running the session. MAP THE ENVIRONMENT is the top panel of the RUN tab – above the fold, checked by screenshot and not assumed, because this window has already had one defect where every mode button was below it. It carries a live phase banner off `/robot_say`, which is the robot's own sentence and the same one the log gets, so the window and the terminal cannot disagree.

306d3d8 **Write down what was measured, and what is still not true**

`docs/system/25_calibrate_then_plan.md` is the new stage end to end: the sweep, the segmenter, the map, the seam to the planner, the pick, the voice, the accuracy scored against the renderer's own geometry, and the viewing-angle table. `22_grasping.md` gets the finding itself, with sources – including that LeRobot has no perception stage at all, so there is nothing there to copy for "where is the cube in metres". Its loop is teleoperate, record, train, deploy, and the policies eat raw video.

2cb7d60 **The recordings of the calibration sweep and the pick it planned**

`recordings/calibration/20260823_125722/` holds the run as it happened: `calibrate.mp4` the arm sweeping the workspace in straight serpentine rows `pick.mp4` the pick and place, planned from the map that sweep made `narration.txt` all 24 sentences the robot published, timed `world_map.json` the map the pick was planned from `summary.txt` what it is and what it is not The map is drawn in both clips: the measured surface, the two cubes green at 40 and 41 mm, the two pads red and labelled with the reason the jaws cannot close on [...]

c0ef577 **Ignore the regenerated occupancy cloud**

1.1 MB of points written by every calibration run. `pick_from_map` names its absence rather than silently planning without the table.

- 0dce39b **The occupancy clouds really are out now, and the run logs are in**
 The previous commit's message said `world_map_occupancy.npy` was deliberately not committed. It was committed – 2.3 MB of it, twice. The ignore rule I wrote covered `recordings/baselines/` and the calibration run directories are somewhere else, so the message described an intention rather than the tree. Found by reading `git ls-files` instead of trusting what I had just written. Removed from tracking, ignored properly, and left on disk.
- d60ed9a **The wrist holds still now, the sweep is a volume, and both arms build one map**
 Four things the operator asked for, and the first is the one that mattered. THE GRIPPER NEVER HELD STILL. `look_at_quat` aimed the wrist AT each cell, so the orientation changed at every step of the grid: the arm reoriented between neighbouring cells 90 mm apart, swung continuously, and no two views were taken from the same attitude – which means a residual wrist error was a DIFFERENT error in every frame and could never cancel or be measured. A 3-D printer's probe does not re-aim.
- 2de054b **The two-arm volume sweep, recorded**
`recordings/calibration/20260823_202409/` is the run described in the previous commit, filmed: `calibrate.mp4` both arms sweeping the volume, three fixed-attitude passes each over two layers – 144 cells planned, 70 reachable, 67 photographed `pick.mp4` the pick and place, planned from the map that sweep made `narration.txt` 89 sentences, timed, including which pass and which attitude `run.log` every cell, every refusal, every region count `world_map.json` the map itself `summary.txt` coverage, stillness refusals and the measure [...]
- fe4dc5a **Write down the printer-probe sweep, and why the scene camera cannot help**
 The measured figures for the two-arm volume sweep, the coverage split into the two honest numbers, and the evidence rule. And one thing checked rather than assumed: THE SCENE CAMERA CANNOT CONTRIBUTE TO THE MAP AT ALL, and it is not a wiring gap I could close. `scene_camera_node` publishes `image_raw` and `camera_info` and NO DEPTH – it is a monocular colour camera. One colour image gives a direction to an object and never a distance, so it cannot place a point in metres.
- 1f66b98 **The swept volume is measured now, and the sweep clip shows the map arriving**
 Two things left over from the last pass, both of which I had written down as not done. THE VOLUME WAS A GUESS. `x` 0.30-0.58, `y` 0.30-0.52, and 74 of the 144 cells planned inside it came back "outside the arm's envelope". The obvious reading is that the arms are small. The real one is that a guessed box sat badly inside a much larger one. `measure_reachable_volume.py` solves IK at the shipped attitude over a deliberately over-wide grid, 924 poses per arm, commanding nothing: reachable at SOME facing+layer `x` 0.10 ..

2026-08-24 — 14 commits

- fc0a08e **The surface was never 5 mm low. I was asking about the wrong place.**
 The map reported the support surface at 1.2448 m against a table top of 1.2500 – 5.2 mm low – with 78% of all points below the true top. I wrote a plane-refinement to correct that bias before checking whether it was one. It was not. `Map.surface_z` returned `offset / n_z`: the plane's Z-INTERCEPT AT $x = y = 0$. A fitted support plane comes out slightly tilted, so its intercept is not its height anywhere a table exists – and on this rig the origin is INSIDE THE WEARER, half a metre from the nearest tabletop.
- 013d3d4 **A map is a fact about the scene, and the scene moves**
`pick_from_map` read the stored map and planned a grasp off it with NO FRESHNESS CHECK AT ALL. A map measured in a previous session looks exactly as authoritative as one measured a minute ago – same file, same fields, same confident object list – and the arm would have been driven at coordinates for a scene that no longer exists. The age is printed on every run now, and EXECUTING off a map older than 30 minutes is refused with the age, the command to re-measure, and the override.
- b9d0614 **It grasps from ABOVE now, and the angle is measured per object**
 The operator: "please try to pick the objects from above, not from inside the table or the side." They were right about what it looked like. The shipped anchor approaches at about -31 degrees – from BELOW and behind – so the wrist drove up out of the table toward the object. On the last recorded pick the wrist sat at $z = 1.227$ with the cube at 1.282: 55 mm UNDER it. CLAUDE.md says top-down is impossible: "top-down on a surface: 0 of 840 cells, any table height 0.70-1.10, either arm".
- a2a7360 **Re-measure only what changed: 18 minutes to 32 seconds, and the scene camera does the noticing**
 The operator: "if the table is the same then the scene should immediately understand and the robot should only look for the changes and grab that." THE FULL SWEEP IS THE RIGHT THING TO DO ONCE. It measures the support surface, learns which viewpoints the arm can reach, and learns which few of them see the whole table. None of that changes while the table stays put. What changes is the objects – and sweeping the volume again to find that out is why the calibration was unusable.
- 2a05963 **Pick accuracy against ground truth: 11.3 mm mean, 4 of 4 inside the gate**
 WHY THE EXISTING TABLE DOES NOT ANSWER THIS. `accuracy_table.py` reports 100% grasp and 0.000 mm positioning error for every mode. Both are sound and neither says whether the gripper is on the OBJECT, because the arm is scored against THE COORDINATE IT WAS GIVEN. Hand it a coordinate 50 mm from the cube and it will hit that coordinate to 0.000 mm and the table will say 100%.
- e40cf6f **Shared autonomy: the confidently-wrong rate goes from 47.5% to zero**
`IntentEstimator` is a Bayesian posterior over WHICH OBJECT the operator is reaching for, and everything in modes 03 and 04 hangs off it being right. Its inputs are the object positions. That is the whole connection to calibration: it can only be as right about "which cube" as it is about "where the cubes are". THIS CANNOT BE MEASURED IN THE SIMULATION AS IT STANDS.

- 4fec074 **The table is found now, not assumed to be where the old one was**
 The operator asked whether this adapts to a table kept anywhere or only to the one in front. It was only the one in front, and the split is worth stating. ADAPTED ALREADY: the table's HEIGHT (measured, 1.2502 against a true 1.2500), its SIZE, and every object's position, size and count. DID NOT ADAPT: its POSITION. BOUNDS was a constant – a box in front of the wearer, x 0.325..0.750, y 0.15..0.65 – the height guess was the constant 1.25, and the camera's base heading pointed forward.
- 82ebf84 **A table beside the wearer, 300 mm lower, found to 0.3 mm – and plainer words on the window**
 THE TABLE BESIDE THE WEARER was the untested case I had named. Tested now, by shifting the whole rendered scene (+0.35, -0.35, -0.30) while the task files stayed put – so the table is at the wearer's SIDE, close in, and at a different height, and every declared coordinate is wrong: table found: surface z = 0.9503 m, tilt 0.03 deg, spanning x -0.758..0.995 y 0.019..0.540 left arm will sweep x 0.325..0.850 y 0.019..0.540 right arm will sweep x -0.758..0.450 y 0.019..0.540 truth (after the shift) measured surface 0. [...]
- 9b8c632 **A table beside the wearer, and the last of the spurious objects**
 BESIDE, AND 300 mm LOWER. The untested case from the last commit, tested: the whole rendered scene shifted (+0.35, -0.35, -0.30) with the task files untouched, so the table is at the wearer's SIDE, close in, at a different height, and every declared coordinate is wrong.
- 5f7db07 **The whole session, filmed: scan, pick, quick check, and the window**
 record_session.py films the SEQUENCE, because the sequence is the claim. A scan alone says the perception works; a pick alone says the arm works; the quick check means nothing except after a scan it can compare against. Filmed separately each would be true and the thing that matters – a table measured once and then kept current in thirty seconds – would not be shown at all.
- 085f7c3 **The gripper cameras were subscribed to topics nothing publishes**
 The operator: "on the gui i never saw gripper camera connected". They were right, and it could never have worked. The panel subscribed to /<arm>_wrist_camera/image_raw and /wrist_mounted_camera/<arm>/image, and NOTHING IN THIS REPOSITORY PUBLISHES EITHER – checked, no create_publisher anywhere uses those names. Every camera producer and consumer here uses /<arm>_camera/color/image_raw: the mock, env_probe, the calibration sweep, verify_colour_vision, verify_scan_view.
- 8d5dc3c **Do not touch the splitter after RViz has been adopted**
 The operator reported the window not opening. It does open here – X reports the main window at 1920x1060, Map State IsViewable, with RViz embedded and the connection checks cycling normally – so what they hit was almost certainly my leavings: a GUI left running from my own testing, which start_gui.sh correctly refuses to duplicate and which looks exactly like "not opening", and the wedged shared memory their connection panel was reporting as shm=bad, daemon=bad. The machine is clean now.
- 5f315b8 **Any shape, any colour: tested rather than reasoned about**
 The operator asked whether the table can be any shape and colour. Both had an argument – the plane fit is geometry-only and the segmenter cuts on edges, not on particular colours – and an argument is not a measurement. Two hooks on the mock make them measurable: SRL_SCENE_TABLE_RGB recolours the table, SRL_SCENE_EXTRA_BOX adds solids so it need not be a rectangle. COLOUR: NO EFFECT. Rendered dark grey (0.30, 0.30, 0.34), close to the background the scene is drawn on – the hard case, not a token change.

daf7b91 **The window opens behind your terminal, and the refusal only gave you a pid**

The operator could not get the window to load. It loads: captured on the real WSLg display, the whole thing renders – the map panel with every button legible, all four modes, RViz embedded and running at 31 fps, the E-STOP bar. X reports it at 1920x1060, Map State IsViewable, with the RViz window correctly reparented inside it at 549x819. Nothing is broken. TWO REASONS SOMEBODY SEES NOTHING ANYWAY, and both are now handled.

2026-08-25 — 1 commit

3d33f8a **The window was six pixels too wide, and WSLg answered by not drawing it**

The operator saw a penguin icon in the taskbar that did nothing when clicked. Nothing was wrong with the window. Captured from the running process, every panel renders: the numbered RUN workflow, all four modes, the gripper camera panels, RViz embedded and reporting ready at 31 fps, the whole thing repainting at 3.0 ms/frame. `Gui.__init__` sized itself with `self.resize(1920, 1060)` – a constant, never measured against a display. This screen is 1920 wide, so the right edge landed at 1926.

CHAPTER A

Every program: the packages

Every first-party module in `src/`, with the description taken from the program's own docstring. Test modules are in Chapter C; launch files are listed at the end of this chapter.

A.1 srl_teleop

Teleoperation: master sensing, following, safety, the bridge, the window. 72 modules, 25,737 lines.

Program	What it is
<code>srl_teleop/analyse_sensing.py</code> 251 lines	PART 3 measurements, against the real recordings.
<code>srl_teleop/analyse_teleop.py</code> 457 lines	Turn a teleop_recorder CSV into numbers and plots . The central output is the 3x3 GAIN MATRIX: robot EE displacement regressed against commanded master displacement, per axis, over the Phase A sweeps.
<code>srl_teleop/arm_link_monitor.py</code> 100 lines	Is the network to each arm actually up?
<code>srl_teleop/arm_telemetry.py</code> 422 lines	Everything the Kinova arm already reports, and what it means.
<code>srl_teleop/block_monitor.py</code> 283 lines	Every mechanism that can stop motion reports through here.
<code>srl_teleop/blocking_aggregator.py</code> 168 lines	ONE PLACE that answers "why isn't the arm moving?".
<code>srl_teleop/boundary_feedback_node.py</code> 260 lines	WITHDRAWN 2026-08-09. NOT AN ENTRY POINT. DO NOT RE-ENABLE AS IT STANDS.
<code>srl_teleop/camera_relay.py</code> 158 lines	Wrist camera relay: what the gripper sees, shown to the operator.
<code>srl_teleop/camera_vr_publisher.py</code> 107 lines	Relay the wrist cameras into the VR overlay, with the same staleness rule.

Program	What it is
<code>srl_teleop/capability.py</code> 203 lines	THE CAPABILITY LADDER – what mapping to run on whatever channels survive.
<code>srl_teleop/capability_node.py</code> 210 lines	Live capability selection: run the best mapping the healthy channels allow.
<code>srl_teleop/capture_gyro_bias.py</code> 87 lines	Measure the gyro zero-rate offset only.
<code>srl_teleop/capture_zero.py</code> 176 lines	Capture the master arm's rest reference . Records, over the same N frames: * the pot ZEROS for j1..j7, and * a_hat, the arm's long axis in the wrist IMU sensor frame.
<code>srl_teleop/channel_manager.py</code> 185 lines	Every master-arm channel individually disableable at runtime, with the capability lost by each stated explicitly.
<code>srl_teleop/check_buttons.py</code> 134 lines	30-SECOND LAB CHECK: which physical button moves which index?
<code>srl_teleop/clearance.py</code> 207 lines	Distance from a candidate arm posture to the wearer.
<code>srl_teleop/compare_master_sim.py</code> 184 lines	Records MASTER pose vs SIMULATOR joint state for. BOTH arms simultaneously, side by side, for labeled test movements. .
<code>srl_teleop/console.py</code> 23 lines	'ros2 run srl_teleop console' -> scripts/srl_console.py (Dear PyGui).
<code>srl_teleop/dashboard.py</code> 256 lines	One-terminal live status for the teleop stack, 5 Hz.
<code>srl_teleop/degraded_mode.py</code> 385 lines	DEGRADED MODE – run on the channels that still work, freeze the rest.
<code>srl_teleop/dependency_check.py</code> 238 lines	Are the things this workspace depends on actually present and consistent?
<code>srl_teleop/divergence.py</code> 282 lines	Sim-vs-real divergence: the number, and the four ways it can be a lie.
<code>srl_teleop/estop_node.py</code> 496 lines	Emergency stop for both arms. Required before real motion.
<code>srl_teleop/fake_pot.py</code> 61 lines	Simulate the Teensy for testing without hardware . Publishes fake pot values to a virtual serial-like topic so you can test the full teleoperation pipeline (pot -> joint -> arm) in sim, before the real Teensy is connected.
<code>srl_teleop/fault_injector.py</code> 850 lines	Deliberately break things, and check the system says so.

Program	What it is
<code>srl_teleop/fsr_gripper_node.py</code> 489 lines	Squeeze the master FSR, close the Robotiq 2F-85.
<code>srl_teleop/full_state_recorder.py</code> 189 lines	Guided snapshot recorder capturing EVERYTHING: pots (raw joint degrees), FSR, buttons, computed master pose, and sim joint state – both arms. . Same workflow as <code>max_position_recorder.py</code> : the program tells you what to do, you do it, press ENTER once, it captures a single snapshot.
<code>srl_teleop/gripper_state.py</code> 233 lines	The gripper's three states, and the ONE piece of state that must survive a process death.
<code>srl_teleop/gui_launch_specs.py</code> 438 lines	Everything the GUI can launch, as ONE manifest that can be checked.
<code>srl_teleop/gui_shim.py</code> 27 lines	Entry point for the Qt operations GUI, which lives in <code>scripts/</code> .
<code>srl_teleop/ik_follower_node.py</code> 1494 lines	Continuously moves the sim <code>left_arm</code> to match <code>/master_arm_pose</code> , using MoveIt's existing <code>/compute_ik</code> service. . Simpler than MoveIt Servo: reuses the <code>/compute_ik</code> service that <code>move_group</code> already runs (no new package/config needed). For each incoming pose, asks.
<code>srl_teleop/imu_mount_calibration.py</code> 334 lines	Estimate the wrist IMU's mount rotation . The IMU is glued to the wrist at an unknown but FIXED orientation relative to the FK tip frame. Everything in the IMU-aided pot repair depends on that rotation, so it is measured here rather than assumed.
<code>srl_teleop/imu_orientation.py</code> 255 lines	Orientation from the master arm's IMUs.
<code>srl_teleop/kortex_convention.py</code> 76 lines	The ONE place Kortex angles become ROS angles.
<code>srl_teleop/kortex_highlevel_bridge.py</code> 1171 lines	Drive the REAL arm over the HIGH-LEVEL Kortex API.
<code>srl_teleop/launcher.py</code> 29 lines	' <code>ros2 run srl_teleop launcher</code> ' -> <code>scripts/srl_launcher.py</code> .
<code>srl_teleop/leader_follower_node.py</code> 124 lines	Mirrors a "leader" arm's joint state to a "follower" arm's controller (1:1, no scaling yet – that's the next feature to layer on top). . ARCHITECTURE: Right now the pot drives BOTH the sim and real arm directly, with no collision checking on the real arm's path at all.
<code>srl_teleop/live_monitor.py</code> 175 lines	Every raw Teensy channel, 5 Hz, in its own terminal.

Program	What it is
srl_teleop/master_calibration.py 628 lines	MEASURED from the 2026-07-31 button-gated recording (see CLAUDE.md). The old ("x", "y", "z") was derived for the FK path.
srl_teleop/master_imu_node.py 214 lines	IMU-PRIMARY orientation for the master arm.
srl_teleop/master_pose_node.py 1552 lines	Computes the master (left) arm's end-effector pose . POSITION: from Forward Kinematics using the 7 pot joint angles (k1j1-k1j7), same math/link-lengths as the verified kinematics.cpp, ported to Python.
srl_teleop/max_position_recorder.py 162 lines	Walks you through a fixed sequence of movements, capturing ONE snapshot (max position reached) per prompt. . No typing labels.
srl_teleop/mock_real_stack.py 145 lines	A stand-in for the real arm, so the whole gated flow can be exercised with no hardware attached.
srl_teleop/motion_generator.py 921 lines	How the joints get from where they are to where the IK solution says.
srl_teleop/mount_guard_node.py 333 lines	FAIL LOUDLY at startup if the home pose is in collision or any arm link is within the clearance floor of any wearer link.
srl_teleop/observer_bypass.py 246 lines	WORKING ALONE: a deliberate, logged, per-session bypass of the observer.
srl_teleop/operating_modes.py 186 lines	THE SIX OPERATING MODES. One definition, imported by every package.
srl_teleop/orientation_policy.py 369 lines	How much the wrist is allowed to differ from the commanded orientation.
srl_teleop/participant_safety_node.py 287 lines	The interlocks that only apply when a PERSON who is not the developer is in the loop.
srl_teleop/payload_manager.py 172 lines	Tell the Kinova arm what it is holding (Part 6a).
srl_teleop/pointing_direction_node.py 188 lines	POINTING DIRECTION as a first-class topic.
srl_teleop/pot_bridge.py 130 lines	DUAL ARM
srl_teleop/precision_speed.py 110 lines	ONE DIAL: precision at one end, speed at the other. Safety at neither.

Program	What it is
<code>srl_teleop/predictive_avoidance.py</code> 388 lines	Predictive avoidance: see it coming, move the ELBOW, keep the grasp.
<code>srl_teleop/preflight.py</code> 227 lines	Refuses to start a session if anything is wrong, and NAMES it.
<code>srl_teleop/procscan.py</code> 205 lines	Find processes without matching yourself. The ONLY sanctioned way here.
<code>srl_teleop/real_arm_doctor.py</code> 908 lines	The seven ways connecting the real arms goes wrong, each detected, each said in plain words, each with a fix that is a button.
<code>srl_teleop/real_arm_gate.py</code> 165 lines	The typed yes/no between sim teleoperation and real motion.
<code>srl_teleop/real_homing_node.py</code> 709 lines	Move a REAL arm to the home pose under a VELOCITY law.
<code>srl_teleop/real_robot_state_display.py</code> 114 lines	READ-ONLY visualization of the real robot's exact current joint values in RViz, highlighted in RED. . SAFETY: this node NEVER publishes to any controller or command topic. It only SUBSCRIBES to /joint_states (published by kortex_driver when running with use_fake_hardware:=false, i.e.
<code>srl_teleop/recovery_manager.py</code> 446 lines	Freeze, abort, recover. One node, four failure paths.
<code>srl_teleop/rviz_master.py</code> 524 lines	What RViz has to show, built as markers, with no ROS in the way.
<code>srl_teleop/rviz_master_node.py</code> 376 lines	The RViz master: three states, the floor as geometry, the whole path.
<code>srl_teleop/selftest_node.py</code> 201 lines	Prove the arm ACTUALLY MOVES, at startup, automatically.
<code>srl_teleop/serial_port.py</code> 186 lines	Find the Teensy, wherever usbpd put it this time . The board moves between /dev/ttyACM0 and /dev/ttyACM1 on every usbpd re-attach, which has caused several failed runs.
<code>srl_teleop/sim_to_real_bridge.py</code> 481 lines	Replay SIM joint angles onto the REAL arm, delayed.
<code>srl_teleop/sim_to_real_gap.py</code> 401 lines	The measured sim-to-real correction: one number, applied in JOINT space.
<code>srl_teleop/srl_teleop_node.py</code> 284 lines	Combined SRL teleoperation (ROS2) . Homing + countdown + pot teleop, publishing to BOTH arm controllers.

Program	What it is
<code>srl_teleop/teleop_gui.py</code> 919 lines	One terminal that answers "what is the rig doing?".
<code>srl_teleop/teleop_recorder.py</code> 468 lines	Button-gated 50 Hz recording of a scripted pass . The operator controls the pace, not a clock. For each segment:
<code>srl_teleop/vr_bringup.py</code> 1500 lines	ONE BUTTON: everything VR teleoperation needs, in order, or a plain reason.
<code>srl_teleop/wearer_posture.py</code> 375 lines	WHERE THE WEARER'S OWN ARMS ARE, and one place that says it.

A.2 srl_perception

Cameras, detection, scene understanding, the world model, wearer tracking, the planner. 26 modules, 9,431 lines.

Program	What it is
<code>srl_perception/apriltag_detector.py</code> 289 lines	PRIMARY object detector. Deliberately boring.
<code>srl_perception/calibration_sweep.py</code> 468 lines	WHERE THE ROBOT LOOKS FROM, AND IN WHAT ORDER.
<code>srl_perception/colour_shape_detector.py</code> 264 lines	CLEARLY-SECONDARY FALLBACK. Off by default.
<code>srl_perception/grasp_pipeline.py</code> 315 lines	PROMPT -> DETECTION -> GRASP -> REACHABILITY, as one callable.
<code>srl_perception/joint_planner.py</code> 595 lines	A motion planner, because this project did not have one.
<code>srl_perception/measure_detector.py</code> 219 lines	Detection rate, pose accuracy and latency, offline.
<code>srl_perception/mock_rgbd_camera.py</code> 579 lines	MOCK RGB-D on the wrist-camera topics, so the perception pipeline runs.
<code>srl_perception/object_pose_tracker.py</code> 146 lines	Fuse the two wrist cameras into one stable list of world-frame objects, and DROP STALE ONES.

Program	What it is
<code>srl_perception/prompt_detector.py</code> 385 lines	Open-vocabulary object detection: a SENTENCE in, objects out.
<code>srl_perception/rgbd_grasp.py</code> 245 lines	Object -> grasp, from an RGB-D frame. Pure geometry, no ROS, no model.
<code>srl_perception/scene_calibration.py</code> 155 lines	Calibrate the scene camera USING THE ARM AS THE REFERENCE OBJECT.
<code>srl_perception/scene_camera_node.py</code> 271 lines	The front-of-the-table USB camera, into ROS.
<code>srl_perception/scene_change.py</code> 270 lines	WHAT MOVED ON THE TABLE, FROM ONE FIXED CAMERA, WITHOUT MOVING THE ARM.
<code>srl_perception/scene_fingerprint.py</code> 410 lines	SCENE FINGERPRINTING – calibrate only when the bed has changed.
<code>srl_perception/scene_fingerprint_node.py</code> 420 lines	Sweep the wrist cameras, fingerprint the scene, and skip calibration if it has not changed.
<code>srl_perception/scene_geometry.py</code> 166 lines	Camera <-> world geometry for the scene camera. Pure arithmetic.
<code>srl_perception/scene_markers.py</code> 144 lines	Render a fingerprint sweep as RViz markers.
<code>srl_perception/segment_lift.py</code> 529 lines	SEGMENT THE PICTURE, THEN LIFT EACH MASK THROUGH THE DEPTH.
<code>srl_perception/srl_cameras.py</code> 292 lines	The two cameras this rig actually has, and how to get frames out of them.
<code>srl_perception/surface_from_depth.py</code> 174 lines	THE WORK SURFACE HEIGHT, MEASURED FROM A DEPTH FRAME. Arithmetic only.
<code>srl_perception/table_scene.py</code> 647 lines	What is on the table, and how to pick each of it up.
<code>srl_perception/vlm_object_locator.py</code> 280 lines	Vlm_locate_node.py — LocateAnything VLM object localization (ROS2) .

Program	What it is
<code>srl_perception/wearer_tracker_node.py</code> 461 lines	The wearer's actual body, into the planning scene.
<code>srl_perception/wearer_tracking.py</code> 446 lines	Turn observed body keypoints into a wearer model that is never LESS conservative than the mannequin.
<code>srl_perception/work_surface_node.py</code> 219 lines	MEASURE THE WORK SURFACE FROM DEPTH AND TELL EVERYONE. Closes U-3.
<code>srl_perception/world_model.py</code> 1042 lines	ONE MAP OF WHAT IS ACTUALLY THERE, BUILT FROM EVERY SOURCE THAT CAN SEE.

A.3 srl_autonomy

Grasp generation, intent, arbitration, language and voice. 12 modules, 3,886 lines.

Program	What it is
<code>srl_autonomy/autonomy_executive.py</code> 454 lines	Mode 5/6 executive: the ONE state machine, and the ONE place mode is set.
<code>srl_autonomy/grasp_generator.py</code> 251 lines	Offers ONLY grasps that /compute_ik has already accepted.
<code>srl_autonomy/grasp_library.py</code> 174 lines	The lookup table. No learned predictor, on purpose.
<code>srl_autonomy/handover_arbiter.py</code> 286 lines	DIRECT / ASSIST / GRASPED. Discrete, operator-confirmed.
<code>srl_autonomy/intent_inference.py</code> 252 lines	P(goal) over the detected objects.
<code>srl_autonomy/move_to_pose.py</code> 97 lines	Move_to_pose_node.py — drive the arm to a target pose via MoveIt (moveit_py) . Subscribes to /located_object_pose and plans+executes a motion to it using. MoveIt's Python API (moveit_py).
<code>srl_autonomy/named_places.py</code> 164 lines	NAMED PLACES -> A POSE THAT WAS MEASURED REACHABLE. Closes the MOVE_TO gap.
<code>srl_autonomy/referring.py</code> 375 lines	Free-form referring expressions, grounded against the scene.

Program	What it is
<code>srl_autonomy/scripted_pick_place.py</code> 230 lines	Pick_place_node.py — full pick-and-place using MoveIt (moveit_py) . Interactive: type a cube color, it picks that cube off the table and places it at a fixed drop zone. Uses the SAME cube positions spawn_cubes.py used (known ground truth, since there's no live camera yet).
<code>srl_autonomy/voice_intent.py</code> 1197 lines	Voice -> structured command. A DETERMINISTIC GRAMMAR, not an LLM.
<code>srl_autonomy/voice_listener.py</code> 169 lines	Microphone -> wake word -> transcript -> /voice_transcript.
<code>srl_autonomy/world_model.py</code> 237 lines	World model for an EYE-IN-HAND camera: scan, remember, re-verify, drop.

A.4 srl_experiments

Tasks, conditions, logging, sessions, analysis, the work surface. 34 modules, 12,278 lines.

Program	What it is
<code>experiments/abc/choreography.py</code> 540 lines	DANCE for the demonstration mode. Both arms, timed to a beat.
<code>experiments/abc/clip_tasks.py</code> 513 lines	The three CLIP tasks: A pick and place, B bimanual hold and place, C multimeter.
<code>experiments/abc/grasp_frames.py</code> 225 lines	WHERE THE FINGER PADS ARE, AND HOW TO POINT THE HAND SOMEWHERE ELSE.
<code>experiments/abc/msc_clip_tasks.py</code> 643 lines	Clip-recording paths for the FOUR MSc tasks: T0, T1, T2, T3.
<code>experiments/abc/msc_session.py</code> 188 lines	THE DATA COLLECTION SEQUENCE for the MSc set: what runs, in what order.
<code>experiments/abc/run_abc.py</code> 1603 lines	Run task A, B or C through a NAMED CONTROL MODE's own command path.
<code>experiments/abc/session_timeline.py</code> 178 lines	THE TWO-HOUR SESSION, as data plus the arithmetic that checks it.
<code>experiments/abc/t1_instruction.py</code> 606 lines	A TYPED INSTRUCTION PLUS WHAT THE CAMERA SAW, INTO A T1 PLAN.

Program	What it is
experiments/abc/t1_task.py 674 lines	T1 – COLOUR-MATCHED PICK AND PLACE, REBUILT 2026-08-17 FOR MODE 06 ALONE.
experiments/abc/task0.py 490 lines	TASK 0 – FOUR COLOURED SPHERES PER ARM.
experiments/abc/task3.py 216 lines	T3 – CIRCUIT BOX AND MULTIMETER. The realistic super-numerary scenario.
experiments/abc/tasks.py 398 lines	TASKS A, B AND C. The three-task set, consolidated from five.
experiments/abc/vision_grasp.py 694 lines	DETECTION THAT DRIVES THE GRASP. The one source, used by task and harness.
srl_experiments/by_design.py 67 lines	ABSENCE BY DESIGN vs A REAL GAP – one definition, not one per reporter.
srl_experiments/conditions.py 85 lines	Counterbalancing, and the ordering guarantees it must give.
srl_experiments/mode_adapters.py 237 lines	THE FIVE CONTROL MODES. This is the ONLY file allowed to name one.
srl_experiments/narration.py 266 lines	WHAT THE ROBOT IS DOING, IN WORDS, FROM ONE SOURCE.
srl_experiments/posthoc.py 124 lines	Post-hoc pairwise comparisons with the pre-registered Holm correction.
srl_experiments/readiness.py 164 lines	READY TO RUN — can a session start now, and if not, exactly what is wrong?
srl_experiments/runner.py 327 lines	The shared experiment runner every E<n> builds on.
srl_experiments/scene_spawner.py 122 lines	Spawn_cubes.py — add a table + 3 colored cubes to the MoveIt planning scene .
srl_experiments/scripted_operator.py 174 lines	A synthetic operator, so every experiment runs end to end with no human and no hardware.
srl_experiments/session.py 291 lines	Participant session state: the protocol, and surviving a failure mid-trial.
srl_experiments/session_manager.py 413 lines	The session manager: readiness from live topics, and failure handling.

Program	What it is
<code>srl_experiments/task_actions.py</code> 354 lines	THE TASK LAYER. Eight verbs, and it does not know which mode drives it.
<code>srl_experiments/task_scene.py</code> 280 lines	The experiment scene as MoveIt COLLISION OBJECTS, with grasp attachment.
<code>srl_experiments/task_trial.py</code> 224 lines	One reach-and-grasp trial, shared by E2, E3, E4 and E5.
<code>srl_experiments/trial_logger.py</code> 271 lines	The shared logging library. Every experiment uses this one.
<code>srl_experiments/validity.py</code> 83 lines	One definition of "analysable", shared by every analyser.
<code>srl_experiments/work_surface.py</code> 262 lines	THE WORK SURFACE HEIGHT. One owner, one value, every consumer reads it here.
<code>trajectory_capture/analyse_capture.py</code> 572 lines	Turn a capture into a CALIBRATION REPORT.
<code>trajectory_capture/capture_protocol.py</code> 190 lines	The segment list, and only the segment list.
<code>trajectory_capture/channel_report.py</code> 198 lines	The verdict table, from a capture CSV. Nothing else.
<code>trajectory_capture/record_trajectories.py</code> 606 lines	Button-gated trajectory capture, you set the pace.

A.5 srl_vr_teleop

The Quest bridge, the pose mapper, VR safety. 10 modules, 3,413 lines.

Program	What it is
<code>srl_vr_teleop/quest_bridge_node.py</code> 652 lines	Meta Quest -> ROS transport.
<code>srl_vr_teleop/quest_vendor_bridge.py</code> 478 lines	Quest bridge speaking the VENDOR wire protocol, over 'adb reverse'.

Program	What it is
<code>srl_vr_teleop/quest_vendor_mock.py</code> 133 lines	Desktop mock of the Quest Unity client, speaking the VENDOR schema.
<code>srl_vr_teleop/vr_feedback_node.py</code> 162 lines	What the operator sees and feels inside the headset.
<code>srl_vr_teleop/vr_gripper_node.py</code> 140 lines	Trigger -> Robotiq, with the SAME latch behaviour as the mannequin's FSR path.
<code>srl_vr_teleop/vr_mock_publisher.py</code> 113 lines	A desktop stand-in for the Quest, so the ENTIRE ROS side is testable without a headset.
<code>srl_vr_teleop/vr_pose_mapper.py</code> 731 lines	Quest controller pose -> robot end-effector target.
<code>srl_vr_teleop/vr_safety_node.py</code> 409 lines	The interlocks that only VR needs.
<code>srl_vr_teleop/vr_smoothing.py</code> 497 lines	Smoothing for a hand-held 6-DOF controller driving a robot arm.
<code>srl_vr_teleop/wearer_view.py</code> 98 lines	FIRST-PERSON VIEW: a camera frame at the WEARER'S EYES, not a free camera.

A.6 srl_vr_autonomy

VR-side intent and arbitration. 2 modules, 405 lines.

Program	What it is
<code>srl_vr_autonomy/vr_handover_arbiter.py</code> 277 lines	Assistance for an input device that is NOT. DOF-deficient.
<code>srl_vr_autonomy/vr_intent_source.py</code> 128 lines	Publishes the pointing vector srl_autonomy already consumes, sourced from the Quest instead of the mannequin's IMU.

A.7 srl_experiments/experiments — the task layer and the archive

The live task set is `experiments/abc/`; `experiments/_archive/` holds the superseded bimanual and E1–E6 sets, kept because recorded data still refers to them. 39 modules.

Program	What it is
<code>experiments/_archive/ bimanual/bimanual_metrics.py</code> 104 lines	Shared metrics for the bimanual tasks. One definition, five tasks.
<code>experiments/_archive/ bimanual/coupled_metrics.py</code> 273 lines	Metrics for the COUPLED-TRANSPORT pair (T3 rigid, T6 compliant) and for. T7 bimanual pursuit. One definition, shared, so the two coupling conditions are measured identically – that is the whole point of running them together.
<code>experiments/_archive/ bimanual/run_bimanual.py</code> 504 lines	THE runner for the current experiment set: T2, T3, T5, T6, T7.
<code>experiments/_archive/ bimanual/t1_bimanual_reach/ analyse_t1.py</code> 105 lines	T1 analysis — Fitts throughput, one arm vs two. The attention bottleneck.
<code>experiments/_archive/ bimanual/t2_hold_and_fill/ analyse_t2.py</code> 128 lines	T2 analysis — box disturbance first, then the discrete placement outcome.
<code>experiments/_archive/ bimanual/t2_metrics.py</code> 223 lines	T2 DISCRETE OUTCOME: blocks placed, blocks dropped, blocks missed.
<code>experiments/_archive/ bimanual/t3_coordinated_ carry/analyse_t3.py</code> 128 lines	T3 analysis — inter-arm coordination error, the headline metric.
<code>experiments/_archive/ bimanual/t4_inter_arm_ handover/analyse_t4.py</code> 106 lines	T4 analysis — and the guard that stops stepped and oriented being pooled.
<code>experiments/_archive/ bimanual/t5_handover_to_ wearer/analyse_t5.py</code> 109 lines	T5 analysis — handover latency, reported as a DISTRIBUTION.
<code>experiments/_archive/ bimanual/t7_pursuit/ targets.py</code> 83 lines	T7 moving targets: independent per arm, inside each arm’s own reachable set.
<code>experiments/_archive/ bimanual/t8_wearer_assisted_ reach/analyse_t8.py</code> 151 lines	T8 analysis — COORDINATION between two people, not reach by one.
<code>experiments/_archive/ bimanual/t9_wearer_motion/ analyse_t9.py</code> 184 lines	T9 analysis — does autonomy compensate a disturbance the operator cannot feel?

Program	What it is
experiments/_archive/e1_fitts_characterisation/analyse_fitts.py 144 lines	E1. Fits $MT = a + b \cdot ID$ and reports throughput in bits/s.
experiments/_archive/e1_fitts_characterisation/run_fitts.py 167 lines	E1, Fitts characterisation of the degraded master.
experiments/_archive/e2_autonomy_level/analyse_autonomy_level.py 156 lines	E2, the core comparison.
experiments/_archive/e2_autonomy_level/run_autonomy_level.py 110 lines	E2, the core comparison.
experiments/_archive/e3_divided_attention/analyse_divided_attention.py 185 lines	E3, divided attention.
experiments/_archive/e3_divided_attention/run_divided_attention.py 116 lines	E3, the SRL-specific experiment.
experiments/_archive/e4_dof_recovery/analyse_dof_recovery.py 185 lines	E4, DOF recovery.
experiments/_archive/e4_dof_recovery/run_dof_recovery.py 124 lines	E4, DOF recovery.
experiments/_archive/e5_intent_inference/analyse_intent_inference.py 184 lines	E5, intent characterisation.
experiments/_archive/e5_intent_inference/run_intent_inference.py 113 lines	E5, intent inference characterisation.
experiments/_archive/e6_vr_vs_mannequin/analyse_vr_vs_mannequin.py 156 lines	Analyse_autonomy_level.py — E2, the core comparison.

Program	What it is
experiments/_archive/e6_vr_vs_mannequin/run_vr_vs_mannequin.py 114 lines	E6, the experiment that isolates the central claim.
experiments/_archive/final5/tasks.py 208 lines	THE FIVE-TASK SET. Consolidated from nine.
experiments/_archive/task_b_rigid_and_compliant/task_b_merged.py 68 lines	SUPERSEDED 2026-08-11 – the merged Task B, rigid AND compliant.
experiments/abc/choreography.py 540 lines	DANCE for the demonstration mode. Both arms, timed to a beat.
experiments/abc/clip_tasks.py 513 lines	The three CLIP tasks: A pick and place, B bimanual hold and place, C multimeter.
experiments/abc/grasp_frames.py 225 lines	WHERE THE FINGER PADS ARE, AND HOW TO POINT THE HAND SOMEWHERE ELSE.
experiments/abc/msc_clip_tasks.py 643 lines	Clip-recording paths for the FOUR MSc tasks: T0, T1, T2, T3.
experiments/abc/msc_session.py 188 lines	THE DATA COLLECTION SEQUENCE for the MSc set: what runs, in what order.
experiments/abc/run_abc.py 1603 lines	Run task A, B or C through a NAMED CONTROL MODE's own command path.
experiments/abc/session_timeline.py 178 lines	THE TWO-HOUR SESSION, as data plus the arithmetic that checks it.
experiments/abc/t1_instruction.py 606 lines	A TYPED INSTRUCTION PLUS WHAT THE CAMERA SAW, INTO A T1 PLAN.
experiments/abc/t1_task.py 674 lines	T1 – COLOUR-MATCHED PICK AND PLACE, REBUILT 2026-08-17 FOR MODE 06 ALONE.
experiments/abc/task0.py 490 lines	TASK 0 – FOUR COLOURED SPHERES PER ARM.
experiments/abc/task3.py 216 lines	T3 – CIRCUIT BOX AND MULTIMETER. The realistic supernumerary scenario.
experiments/abc/tasks.py 398 lines	TASKS A, B AND C. The three-task set, consolidated from five.
experiments/abc/vision_grasp.py 694 lines	DETECTION THAT DRIVES THE GRASP. The one source, used by task and harness.

Program	What it is
---------	------------

A.8 Launch files

Program	What it is
<code>srl_autonomy/launch/move_to_pose.launch.py</code> 40 lines	Launches <code>move_to_pose_node</code> with proper MoveIt config . Adds <code>.moveit_cpp(file_path=...)</code> pointing at <code>moveit_cpp.yaml</code> – MoveItPy reads planning pipeline config from THIS file, in a different format than what <code>move_group</code> reads from its own auto-detected <code>*_planning.yaml</code> .
<code>srl_autonomy/launch/scripted_pick_place.launch.py</code> 38 lines	<code>Pick_place.launch.py</code> — launches <code>pick_place_node</code> with proper MoveIt config . Same pattern as <code>move_to_pose.launch.py</code> : injects MoveIt config (including <code>moveit_cpp.yaml</code> for the planning pipeline) via ROS parameters, which is what actually made MoveItPy load correctly.
<code>srl_autonomy/launch/shared_autonomy.launch.py</code> 65 lines	Teleop + perception + shared autonomy.
<code>srl_description/launch/display.launch.py</code> 54 lines	Visualize the SRL scene in RViz (no controllers). Shows the human + backpack + both Gen3 arms. Use this to tune the mount positions before bringing controllers online.
<code>srl_description/launch/srl_bringup.launch.py</code> 90 lines	Full dual-arm SRL with controllers . Loads human+backpack+2 Gen3 arms, starts <code>ros2_control</code> , spawns both arms' trajectory controllers + joint state broadcaster, opens RViz.
<code>srl_moveit_config/launch/demo.launch.py</code> 7 lines	(no description in source)
<code>srl_moveit_config/launch/move_group.launch.py</code> 7 lines	(no description in source)
<code>srl_moveit_config/launch/moveit_rviz.launch.py</code> 7 lines	(no description in source)
<code>srl_moveit_config/launch/rsp.launch.py</code> 7 lines	(no description in source)
<code>srl_moveit_config/launch/setup_assistant.launch.py</code> 7 lines	(no description in source)
<code>srl_moveit_config/launch/spawn_controllers.launch.py</code> 7 lines	(no description in source)

Program	What it is
srl_moveit_config/launch/ static_virtual_joint_ tfs.launch.py 7 lines	(no description in source)
srl_moveit_config/launch/ warehouse_db.launch.py 7 lines	(no description in source)
srl_perception/launch/ perception.launch.py 86 lines	Both wrist cameras through the AprilTag detector, fused into one world-frame object list.
srl_teleop/launch/mock_ real.launch.py 108 lines	The real-arm stack with the hardware replaced by a mock.
srl_teleop/launch/real_ arms.launch.py 160 lines	REAL arm drivers that CAN command, plus homing + bridge.
srl_teleop/launch/real_arms_ highlevel.launch.py 156 lines	REAL arm via the HIGH-LEVEL Kortex API.
srl_teleop/launch/real_ drivers_readonly.launch.py 102 lines	REAL Kortex drivers, READ ONLY.
srl_teleop/launch/ sensing.launch.py 42 lines	The IMU-primary sensing chain, on its own.
srl_teleop/launch/ teleop.launch.py 306 lines	Bring up the entire teleop stack with one command.
srl_vr_teleop/launch/vr_ teleop.launch.py 77 lines	The VR input path.

Total: 156 first-party modules, 55,150 lines, excluding tests and launch files.

CHAPTER B

Every program: the scripts

`scripts/` holds 237 Python programs and 28 shell entry points. Every one is listed below with its own description. The verb in the name is the project's taxonomy and it is used consistently.

B.1 Measurement — each produces a number and writes a baseline

Program	What it is
<code>measure_approach_geometry.py</code> 167 lines	What approach direction does the PINNED wrist actually give, and where can it go?
<code>measure_cad_step.py</code> 71 lines	Bounding dimensions from STEP files, with no CAD library.
<code>measure_capability_delta.py</code> 107 lines	JOB B: the capability change from the pot repair, before and after, in numbers.
<code>measure_capability_ladder.py</code> 143 lines	Measure what each rung of the capability ladder costs, on recorded data.
<code>measure_capture_rate.py</code> 225 lines	What frame rate can the RViz capture path ACTUALLY deliver?
<code>measure_centre_gap.py</code> 484 lines	WHAT IS IN THE GAP IN FRONT OF THE WEARER? PER ARM, PER COLUMN, BY LINK.
<code>measure_centre_reach.py</code> 202 lines	HOW FAR INBOARD CAN EACH ARM WORK, at every forward distance?
<code>measure_centre_vs_height.py</code> 405 lines	IS THE FRONT CENTRE REACHABLE AT ANY WORK HEIGHT? Asked properly.
<code>measure_centre_vs_wearer_posture.py</code> 524 lines	CAN THE ARMS WORK IN THE CENTRE IF THE WEARER MOVES THEIR ARMS?
<code>measure_clearance_region.py</code> 220 lines	WHERE CAN THE ARM WORK **AND** KEEP THE WEARER CLEARANCE FLOOR?
<code>measure_control_budget.py</code> 468 lines	How well can these arms actually be controlled, and what still stops them.

Program	What it is
measure_depth_pose_accuracy.py 220 lines	How accurately can an object's pose be recovered from RGB-D, without tags?
measure_detection.py 109 lines	Detection rate, pose error and latency on the task objects. THE MODE-6 GATE.
measure_detector_on_real_rgbd.py 202 lines	Open-vocabulary detection + known-dimension depth fit, on REAL RGB-D.
measure_dual_gui.py 367 lines	PART 4 – run the dual-view GUI for real, measure it, and PROVE IT DREW.
measure_forward_reach.py 462 lines	WHY IS EVERYTHING AT THE WEARER'S WAIST, and what moves it onto the table?
measure_freeform_language.py 153 lines	PART 5: free-form language, measured on phrasings it was not designed for.
measure_grasp_approach.py 628 lines	IS THE APPROACH RIGHT FOR EACH GRASP? Per task, per object, per arm.
measure_grasp_approach_angles.py 170 lines	CAN THESE ARMS GRASP FROM ABOVE? MEASURED, ON THE MOUNT THAT EXISTS.
measure_grasp_penetration.py 354 lines	JOB C: the gripper penetrates objects. Measure it, explain it, fix it.
measure_grasp_reality.py 415 lines	PART 1: is the grasp real, and how precise is it? Measure, do not assert.
measure_home_change.py 443 lines	IF THE PRESENTATION POSE BECOMES HOME, CAN THE TASKS STILL BE PERFORMED?
measure_home_clearance.py 380 lines	What a pose actually clears, PER WEARER LINK and PER ARM SEGMENT.
measure_home_render.py 421 lines	WHAT POSE DOES THE STACK ACTUALLY BOOT INTO? Measured from TF, shot from RViz.
measure_ik_stickiness.py 218 lines	How often does IK refuse a target it can solve? – the "arm sticks" bug.
measure_lift.py 176 lines	HOW MUCH LIFT is available at a grasp point, with the pinned wrist?
measure_objects_on_the_table.py 272 lines	CAN T1'S CUBES REST ON THE TABLE? Measured, because the spec requires it.

Program	What it is
measure_orientation_cost.py 575 lines	WHAT THE PINNED WRIST COSTS, in metres of reach, per direction, per arm.
measure_pad_columns.py 141 lines	WHAT LIMITS EACH ARM AT EACH PAD COLUMN, AND WHERE IT STOPS BEING THE PERSON.
measure_pad_mid_ee.py 264 lines	WHERE THE FINGER PADS ARE, RELATIVE TO THE WRIST, READ FROM FK.
measure_pick_accuracy.py 280 lines	WHERE DO THE GRIPPER PADS ACTUALLY END UP? MEASURED AGAINST GROUND TRUTH.
measure_randomisation_region.py 285 lines	MEASURE THE RANDOMISATION REGION, before anything is designed around it.
measure_reachable_volume.py 170 lines	WHAT VOLUME CAN EACH ARM ACTUALLY OBSERVE, HOLDING ITS WRIST STILL?
measure_ready_pose_envelope.py 147 lines	HOW FAR IN CAN THE HANDS COME, at chest height, with a level wrist?
measure_render_rate.py 231 lines	What frame rate can RViz RENDER on the capture display, and under which driver?
measure_scene_fingerprint.py 177 lines	Measure the three things the fingerprint has to be judged on.
measure_shared_autonomy.py 282 lines	WHAT CALIBRATION IS WORTH TO SHARED-AUTONOMY TELEOPERATION. MEASURED.
measure_sim_to_real_gap.py 834 lines	What the real arm does with a commanded move, against what the sim does.
measure_t1_cubes_t2_transfer.py 324 lines	T1 cube grasping, and T2 container transfer – measured before either is built.
measure_t2_band_top.py 126 lines	How far UP does T2's coordinated-carry band actually go?
measure_task_margin.py 157 lines	How much room does each task pose actually have, with the furniture in?
measure_teleop_motion.py 440 lines	What the teleoperation motion generator does to the end effector.
measure_view_geometry.py 75 lines	WHICH VIEWING GEOMETRY ACTUALLY SEES THE TABLE, measured on a live stack.

Program	What it is
<code>measure_vision_options.py</code> 207 lines	WHERE, IF ANYWHERE, CAN A WRIST CAMERA SEE THE WORK? Four options, measured.
<code>measure_wake_word.py</code> 189 lines	HOW TOLERANT MAY THE WAKE WORD BE? BOTH SIDES, MEASURED.
<code>measure_wearer_size_effect.py</code> 285 lines	What changes when the wearer is a person instead of the mannequin.
<code>measure_wearer_size_sweep.py</code> 248 lines	HOW MUCH OF THE CLEARANCE IS THE MANNEQUIN? Sweep the body, not one guess.
<code>measure_wearer_tracking_rate.py</code> 137 lines	How fast the body reaches the planning scene, and where the time goes.
<code>measure_what_binds.py</code> 558 lines	WHAT BINDS THE WORKSPACE, per direction, per arm, with the mechanism named.
<code>measure_workspace.py</code> 345 lines	Measure the CONTINUOUSLY reachable volume from home. No master needed.
<code>measure_wrist_capability.py</code> 264 lines	JOB B, question 1: does a working j7 restore commandable wrist orientation?

B.2 Verification — each produces a pass/fail verdict

Program	What it is
<code>verify_abc_scenarios.py</code> 410 lines	Verify every coordinate of TASKS A, B and C at N=10 over the FULL PATH.
<code>verify_autonomy.py</code> 263 lines	PART 6: run mode 6 and measure it. Refusals, latency, stop, mode switching.
<code>verify_autonomy_command_path.py</code> 179 lines	BLOCKER 1: does an autonomy pose actually move the arm? Live graph + drive.
<code>verify_capability_degradation.py</code> 129 lines	Disable channels one at a time and report what still works at each level.
<code>verify_centre_layout.py</code> 180 lines	SCORE ONE PROPOSED CENTRE LAYOUT, POINT BY POINT, ON THE FULL PICK PATH.
<code>verify_colour_vision.py</code> 521 lines	CAN THE WRIST CAMERA SEE WHAT COLOUR EACH CUBE IS? Measured, from pixels.
<code>verify_dance_paths.py</code> 176 lines	EVERY DANCE WAYPOINT THROUGH /compute_ik, BEFORE ANY OF IT IS FILMED.

Program	What it is
<code>verify_detection_path.py</code> 205 lines	U-1: RUN THE WHOLE DETECTION PATH, ONCE, END TO END.
<code>verify_follower_pause.py</code> 126 lines	IS THE FOLLOWER PAUSE REAL? A known-answer test with both controls.
<code>verify_frame_pose_gate.py</code> 184 lines	KNOWN-ANSWER TEST FOR THE 2026-08-17 DEPROJECTION FAULT.
<code>verify_grasp_quality.py</code> 227 lines	THE FOUR GRASP CRITERIA, MEASURED FROM PIXELS.
<code>verify_gripper_motion.py</code> 317 lines	DID THE FINGERS ACTUALLY OPEN AND CLOSE, AT THE RIGHT MOMENTS?
<code>verify_gripper_shutdown.py</code> 318 lines	PART 3.2 – DOES THE GRIPPER SURVIVE BEING KILLED MID-GRASP?
<code>verify_gui_buttons.py</code> 1005 lines	PART 4 – PRESS EVERY BUTTON IN THE GUI AND REQUIRE EVIDENCE IT DID SOMETHING.
<code>verify_indexing.py</code> 176 lines	PART 2 VERIFICATION: 10 clutch index cycles, jump per cycle, total travel.
<code>verify_look_then_grasp.py</code> 268 lines	LOOK, THEN GRASP: the whole sequence, driven by the camera. With timings.
<code>verify_mock_camera.py</code> 287 lines	Verify the MOCK camera’s PLUMBING against constructed ground truth.
<code>verify_mode_paths.py</code> 141 lines	JOB D: are the six modes real, or a bluff? Verified from the LIVE GRAPH.
<code>verify_msc_tasks.py</code> 379 lines	Verify the MSc tasks T0, T2 and T3 – N=10, full densified path, bench in the scene, BOTH arm assignments. T1 IS VERIFIED BY ‘scripts/verify_t1.py’.
<code>verify_object_attachment.py</code> 296 lines	DOES THE OBJECT ACTUALLY TRAVEL WITH THE GRIPPER?
<code>verify_objects_on_table.py</code> 625 lines	DOES EVERY OBJECT IN THE SCENE ACTUALLY REST ON A SOLID?
<code>verify_observe_pose.py</code> 320 lines	IS THE OBSERVE POSE ACTUALLY REACHABLE, AND IS THE MOVE THERE SAFE?
<code>verify_predictive_avoidance.py</code> 555 lines	DRIVE THE COMMANDED EE STRAIGHT AT THE WEARER AND SEE WHAT THE ARM DOES.
<code>verify_prompt_panel.py</code> 251 lines	THE GUI’S PROMPT BOX: DRIVE IT, AND REQUIRE THAT IT DRIVES THE ARM.

Program	What it is
<code>verify_rviz_clips.py</code> 881 lines	CHECK EVERY RVIZ CLIP SHOWS ITS TASK – automatically, then flag what to eyeball.
<code>verify_scan_view.py</code> 200 lines	From the SCAN POSE, does the camera actually see the work surface?
<code>verify_scene_camera.py</code> 270 lines	End to end: a picture in, a wearer model out, and every claim labelled.
<code>verify_t1.py</code> 427 lines	WALK THE T1 PATH THE TASK ACTUALLY BUILDS, WAYPOINT BY WAYPOINT.
<code>verify_t1_layout.py</code> 417 lines	Does a T1 layout FIT in the left arm’s region, at the CURRENT bench edge?
<code>verify_t1_paths.py</code> 225 lines	T1 AND T1 STAGE 2, EVERY WAYPOINT: reachable, and clear of the wearer.
<code>verify_t1_stage2.py</code> 56 lines	T1 Stage 2, N=10 over the FULL densified path, across seeds.
<code>verify_t8_t9_scenarios.py</code> 275 lines	Generate and VERIFY the two TWO-PERSON tasks, T8 and T9.
<code>verify_task0_scenarios.py</code> 290 lines	Verify TASK 0’s four spheres per arm, N=10, over the FULL densified path, WITH THE BENCH IN THE SCENE.
<code>verify_task_scenes.py</code> 354 lines	Is every task actually performable in sim? Per target, per arm, with reasons.
<code>verify_vision_drives_grasp.py</code> 225 lines	IS VISION DRIVING THE GRASP, OR DECORATING IT? One test settles it.
<code>verify_voice_instruction.py</code> 265 lines	T1 FROM A SPOKEN INSTRUCTION, THROUGH REAL AUDIO AND REAL SPEECH-TO-TEXT.
<code>verify_vr_one_button.py</code> 190 lines	Press the one button for real, end to end, and require READY TO OPERATE.
<code>verify_vr_wifi_route.py</code> 246 lines	Does the wifi/WebXR route actually work, end to end?
<code>verify_wearer_fallbacks.py</code> 230 lines	Break the scene camera six ways, through the LIVE NODE, and require the planning scene to fall back to the mannequin every time.
<code>verify_work_surface_from_depth.py</code> 207 lines	LIVE END-TO-END PROOF FOR U-3: depth -> a measured work-surface height.

B.3 Audit — checks the repository against its own declarations

Program	What it is
<code>audit_dual_arm_collisions.py</code> 184 lines	Find resources that COLLIDE when two real arms are instantiated.
<code>audit_recurring_patterns.py</code> 325 lines	The five bug classes this repo keeps reproducing. Find them by SHAPE.
<code>audit_scenario_reachability.py</code> 381 lines	RE-VERIFY every coordinate in every protocol at N repeats over the WHOLE. PATH, and report which figures change.
<code>audit_task_spec.py</code> 965 lines	ONE PASS OVER docs/TASK_SPEC.md, IN THE CODE, BEFORE ANY RECORDING.
<code>audit_task_specs_vs_scene.py</code> 257 lines	AUDIT: every explicit requirement in the four MSc task specs, against what 'clip_scene' actually publishes.
<code>audit_uncalled.py</code> 95 lines	WHAT EXISTS AND NOTHING CALLS. The recurring fault, enumerated once.
<code>audit_unreachable_clear.py</code> 187 lines	Audit for the LATCH pattern, not for its instances.
<code>audit_workspace.py</code> 368 lines	Audit the whole workspace for the failure classes this project actually has.

B.4 Sweeps — explore a parameter space and record the whole surface

Program	What it is
<code>sweep_gap_vs_mount.py</code> 309 lines	WOULD TILTING THE MOUNT OPEN THE GAP, AND WHAT WOULD IT COST?
<code>sweep_language_vision.py</code> 325 lines	PART 3.1 – THE LANGUAGE AND VISION SWEEP, INCLUDING PHRASINGS NOBODY DESIGNED FOR.
<code>sweep_mount_geometry.py</code> 299 lines	WHERE WOULD THE MOUNTS HAVE TO BE FOR THE CENTRE TO BE REACHABLE?
<code>sweep_mount_overlap.py</code> 234 lines	CAN A MOUNT CHANGE GIVE THE TWO ARMS A SHARED WORKSPACE? Quantified.
<code>sweep_mount_tradeoff.py</code> 129 lines	Does mount rotation trade BACK reach for FRONT reach? Report only.
<code>sweep_surface_vs_t1_path.py</code> 193 lines	THE SURFACE AGAINST T1'S WHOLE PATH, height and near edge together.
<code>sweep_t1_instructions.py</code> 585 lines	T1 FROM A TYPED SENTENCE: correct / asked / refused / MISUNDERSTOOD.

Program	What it is
<code>sweep_t1_supports.py</code> 223 lines	Which support geometry lets T1's objects REST on something and still be picked by the pinned wrist?
<code>sweep_table_edge.py</code> 153 lines	How far FORWARD can the table's near edge come before the grasp dies?
<code>sweep_task_a_bin.py</code> 153 lines	Where can task A's bin go so the transport is a CARRY, not a drop?
<code>sweep_wearer_postures.py</code> 159 lines	Run the centre sweep once per wearer arm posture, one stack each, and merge.

B.5 Searches over configuration space

Program	What it is
<code>search_centre_geometry.py</code> 406 lines	CAN THE CENTRE OF THE TABLE BE MADE REACHABLE BY BOTH ARMS? PER LEVER.
<code>search_centre_on_surface.py</code> 330 lines	IS THERE **ANY** TABLE THAT PUTS THE WORK IN THE CENTRE, ON THE SURFACE?
<code>search_t1_centre.py</code> 691 lines	HOW CLOSE TO THE CENTRELINE CAN A PAD GO, WITH THE OBJECTS ON THE SURFACE?
<code>search_t1_layout_on_surface.py</code> 532 lines	A WHOLE T1 LAYOUT THAT RESTS ON THE SURFACE. Arithmetic first, then IK.

B.6 Solvers — poses, layouts, transits

Program	What it is
<code>solve_home_pose.py</code> 1337 lines	SOLVE the home pose from geometric constraints. Do not guess joint angles.
<code>solve_observe_pose.py</code> 380 lines	WHERE MUST THE ARM STAND SO THE WRIST CAMERA CAN SEE THE WORK?
<code>solve_observe_transit.py</code> 185 lines	A CLEAN WAY TO GET TO THE OBSERVE POSE, since a straight line is not one.
<code>solve_t1_layout.py</code> 306 lines	BUILD T1'S LAYOUT FROM THE MEASUREMENT, AND VERIFY EVERY POINT OF IT.

B.7 Recording and capture

Program	What it is
<code>record_abc_sweep.py</code> 1753 lines	PART 5 – the 7-angle sweep: three tasks x four modes, launched THROUGH THE GUI.
<code>record_calibration.py</code> 263 lines	FILM THE CALIBRATION SWEEP AND THE PICK IT PLANS FROM THE MAP.
<code>record_gui_tutorials.py</code> 271 lines	Screen recordings of the GUI being used: click this, this happens.
<code>record_language_sweep.py</code> 190 lines	Full autonomy answering 40 phrasings, on video, with every outcome captioned.
<code>record_rviz.py</code> 1504 lines	REAL SCREEN RECORDINGS OF RVIZ, with the task objects visible and moving.
<code>record_session.py</code> 256 lines	FILM A WHOLE SESSION: the scan, the pick, the quick check, and the window.
<code>record_verification.py</code> 735 lines	Drive every task/scenario/condition in sim and RECORD it for review.

B.8 Rendering

Program	What it is
<code>render_cad_figures.py</code> 81 lines	Render the CAD to vector figures for the report.
<code>render_centre_plan.py</code> 101 lines	THE TOP-DOWN PLAN VIEW: both pads in front of the wearer, both arms on them.
<code>render_pose.py</code> 90 lines	SHOW ME THE POSE. One still per view, so a posture can be LOOKED at.
<code>render_t1_scene.py</code> 168 lines	SHOOT THE T1 SCENE, from the config the clips are actually filmed with.

B.9 Probes — small targeted experiments

Program	What it is
<code>probe_centre_limit.py</code> 352 lines	HOW CLOSE TO THE CENTRE CAN THE ARM GET AT ALL, AND WHAT IS THE LAST THING. IN THE WAY?
<code>probe_clutch_indexing.py</code> 158 lines	Is the reachable set UNBOUNDED across repeated clutch cycles?

Program	What it is
probe_collision_response.py 173 lines	Drive the commanded EE INTO the wearer and watch what the follower does.
probe_dial_safety.py 327 lines	JOB F: DOES SAFETY SCALE WITH THE DIAL? Measured at both extremes.
probe_gripper.py 138 lines	Ask the ARM directly whether it has a gripper, and try to move it.
probe_marker_shading.py 197 lines	WHAT COLOUR DOES A MARKER ACTUALLY RENDER AS? Measured, not assumed.
probe_task_band_clearance.py 210 lines	WHICH arm link is inside the 120 mm clearance floor, and where does the task band have to move to clear it?

B.10 Session checks

Program	What it is
check_layout_reachable.py 133 lines	Is every marked position in a task layout actually reachable? Walk to it.
check_placement.py 185 lines	Did each cube end on its OWN plane, the WRONG plane, or off both?
check_tests.py 131 lines	Run the unit suite and refuse on any NEW failure.
check_wearer_path.py 109 lines	Clearance of a planned arm path from the WEARER, not just the table.

B.11 Calibration

Program	What it is
calibrate_environment.py 960 lines	LOOK AT THE WORKSPACE FIRST, IN ORDER, AND BUILD ONE MAP OF WHAT IS THERE.
calibrate_operator_yaw.py 337 lines	Measure where the operator is standing, in one angle.
calibrate_scene_camera.py 246 lines	Calibrate the scene camera by driving the gripper to known points.

Program	What it is
<code>calibrate_scene_camera_extrinsics.py</code> 323 lines	Where the scene camera IS, in the robot's world frame.

B.12 Diagnosis

Program	What it is
<code>diagnose_front_reach.py</code> 278 lines	WHY does the front direction stop so early? Name the binding constraint.
<code>diagnose_gripper.py</code> 204 lines	Ask the arm, directly, why its gripper is not moving.
<code>diagnose_left_on_surface.py</code> 241 lines	WHY CAN THE RIGHT ARM WORK ON THE SURFACE AND THE LEFT ARM NOT AT ALL?
<code>diagnose_t1_waypoints.py</code> 143 lines	WHICH T1 WAYPOINTS FAIL, and where are they? Per waypoint, not per task.

B.13 Fault injection

Program	What it is
<code>inject_experiment_faults.py</code> 344 lines	Run E1-E6 in sim with a fault injected MID-TRIAL, and check the consequences.
<code>inject_lab_day_faults.py</code> 314 lines	LAB-DAY FAULT INJECTION: the faults <code>fault_injector.py</code> does not cover.

B.14 Pick execution

Program	What it is
<code>pick_cube_v2.py</code> 229 lines	Pick the cube: align laterally WHILE IT IS VISIBLE, then descend blind-safe.
<code>pick_from_map.py</code> 625 lines	PICK AND PLACE FROM THE MAP THE ROBOT MEASURED. No declared coordinates.
<code>pick_from_table.py</code> 505 lines	Look at the table, say what is on it, pick one up.

Program	What it is
<code>pick_from_top.py</code> 285 lines	Pick the cube from directly above, accounting for the camera->pad offset.

B.15 Operator-facing tools and windows

Program	What it is
<code>srl_arm_view.py</code> 433 lines	The ACTUAL panel: where the real arms are, drawn beside where the sim is.
<code>srl_console.py</code> 1685 lines	SRL CONSOLE — flight-software-grade operations GUI.
<code>srl_fk.py</code> 339 lines	Forward kinematics for <code>srl_dual</code> , straight off the URDF. No stack required.
<code>srl_gui.py</code> 7156 lines	SRL operations GUI – DUAL VIEW: commanded beside actual, and the gap named.
<code>srl_hud.py</code> 772 lines	The dark HUD: palette, type, and the schematic widgets.
<code>srl_launcher.py</code> 961 lines	SRL LAUNCHER – one window. Everything else is a button.
<code>srl_object_detector.py</code> 261 lines	Detect objects continuously, in every camera this rig has, at once.
<code>srl_pick_cube.py</code> 283 lines	Pick the green cube with the LEFT arm, start to finish, with no hand-solving.
<code>srl_pick_live.py</code> 493 lines	Find a cube with every camera on the rig and pick it up, tool pointing DOWN.
<code>srl_realsense_node.py</code> 169 lines	The RealSense D435i, across the room, as a ROS node.
<code>srl_scene.py</code> 248 lines	Table and object geometry from an EYE-IN-HAND depth camera.
<code>srl_scene_camera_node.py</code> 106 lines	The USB webcam across the room, as a ROS node.
<code>srl_sequence.py</code> 333 lines	HOME -> PICKUP POSE -> SCAN -> LOCATE -> PICK, for the LEFT arm.
<code>srl_wearer_panel.py</code> 303 lines	The WEARER tab: the scene camera, the tracked body over it, and which body the robot is actually planning against.

B.16 Scanning

Program	What it is
<code>scan_table.py</code> 233 lines	Scan the table with BOTH arms and build a 3D model of what is on it.

B.17 Staging

Program	What it is
<code>stage_observe_and_detect.py</code> 248 lines	STAGE THE LOOK: detect the cubes BEFORE the mode starts, and write them out.
<code>stage_presentation_pose.py</code> 419 lines	Drive both arms to the PRESENTATION POSE and wait until they arrive.

B.18 Analysis

Program	What it is
<code>analyse_cad.py</code> 183 lines	Decompose the STEP assemblies and check them against the URDF and the FK.
<code>analyse_mode_difference.py</code> 174 lines	IS THE NO-OPERATOR MODE DIFFERENCE REAL? Settled from the recorded set.

B.19 CAD

Program	What it is
<code>cad_export_meshes.py</code> 162 lines	STEP -> per-link STL, at two resolutions.
<code>cad_figures.py</code> 149 lines	Thesis figures for the CAD master arm, rendered straight from the STLs.
<code>cad_infer_joints.py</code> 186 lines	Infer the seven pot-joint frames from a model that has none.
<code>cad_inspect.py</code> 123 lines	Read the master-arm STEP files and report what is actually in them.

B.20 Finders

Program	What it is
<code>find_object.py</code> 195 lines	Say what you want; get its position, a grasp, and whether an arm can reach.
<code>find_presentation_pose.py</code> 899 lines	THE PRESENTATION POSE. Derived, checked, and saved. Closes R-7.
<code>find_scan_pose.py</code> 266 lines	Find a SCAN POSE per arm: one the wrist camera can actually see from.
<code>find_symmetric_home.py</code> 422 lines	A HOME POSE THAT LOOKS RIGHT: elbows down and in, hands in front, symmetric.

B.21 Generators

Program	What it is
<code>make_clip_index.py</code> 260 lines	Build recordings/verification/INDEX.md from what is ACTUALLY on disk.
<code>make_thesis_figures.py</code> 273 lines	Generate every data figure the thesis needs, FROM THE SOURCE DATA.

B.22 Runners

Program	What it is
<code>run_msc_session.py</code> 196 lines	Run the MSc data-collection sequence END TO END in sim, and report what was actually captured.

B.23 VR tooling

Program	What it is
<code>vr_desk_monitor.py</code> 175 lines	Everything the in-headset panel shows, on the PC screen.
<code>vr_guided_session.py</code> 597 lines	Run the headset measurements WITHOUT reading a terminal.
<code>vr_headset_check.py</code> 638 lines	The measurements that ONLY a physical Quest can give.
<code>vr_reachability.py</code> 349 lines	STEP 1 of the wifi VR route: can the headset reach this machine AT ALL?

B.24 Motion helpers

Program	What it is
<code>goto_pose_left.py</code> 73 lines	Drive the LEFT arm to a saved joint pose, and wait until it arrives.

B.25 Location

Program	What it is
<code>locate_cube_left.py</code> 89 lines	(no description in source)

B.26 Builders

Program	What it is
<code>build_safe_pick.py</code> 154 lines	Turn the grasp plan into a path that provably clears the table.

B.27 Gates

Program	What it is
<code>gate3_cup_stack.py</code> 203 lines	GATE 3, redone – and GATE 2’s right-arm question, which the first run asked in the wrong place.
<code>gate_check_experiment.py</code> 310 lines	GATES 1-3 for the MSc experiment brief, measured in one sim session.

B.28 Appliers

Program	What it is
<code>apply_mount_candidate.py</code> 182 lines	PUT A MOUNT CANDIDATE INTO THE URDF, OR TAKE IT BACK OUT.

B.29 Choosers

Program	What it is
<code>choose_t0_directions.py</code> 237 lines	Choose T0's THREE DISTINCT REACHING DIRECTIONS per arm, by measurement.

B.30 Drivers

Program	What it is
<code>drive_mode_paths.py</code> 127 lines	JOB D: inject at the head of each mode's chain, count at the arm.

B.31 Execution

Program	What it is
<code>execute_pick_left.py</code> 386 lines	Execute the planned top-down grasp of the green cube with the LEFT arm.

B.32 Readers

Program	What it is
<code>read_t1_frames.py</code> 182 lines	PULL FRAMES OUT OF A T1 CLIP SO THEY CAN BE LOOKED AT.

B.33 Scoring

Program	What it is
<code>score_pose.py</code> 118 lines	SCORE ONE POSTURE. Hand, elbow, limb apex, wearer clearance, wrist level.

B.34 Mapping

Program	What it is
map_table_topdown.py 184 lines	FIND THE REAL TABLE WITH THE REAL ARM, LOOKING STRAIGHT DOWN.

B.35 Merging

Program	What it is
merge_work_surface_region.py 113 lines	Build the ONE region file from the clearance surveys, with provenance.

B.36 Modes

Program	What it is
mode_upstreams.py 256 lines	WHICH UPSTREAM EACH MODE NEEDS, and the isolation that enforces it.

B.37 Automation

Program	What it is
auto_observe.py 217 lines	Find a pose from which the gripper camera can SEE the cube – automatically.

B.38 Observer

Program	What it is
observer_estop.py 183 lines	The observer's station: a heartbeat that says somebody is standing there.

B.39 Hand-eye

Program	What it is
handeye_from_table.py 177 lines	Recover the gripper camera's mount-rotation error from the table itself.

B.40 Guarded motion

Program	What it is
<code>guarded_descent.py</code> 184 lines	Descend onto an object while CONTINUOUSLY measuring the gap to the surface.

B.41 Completion

Program	What it is
<code>finish_grasp.py</code> 152 lines	Close the last few centimetres onto an object already between the fingers.

B.42 Follower control

Program	What it is
<code>follower_pause.py</code> 128 lines	PAUSE AND RESUME 'ik_follower_node'. The one implementation.

B.43 Description

Program	What it is
<code>describe_pose.py</code> 201 lines	WHICH POSE AM I LOOKING AT? Joint angles and the geometry, side by side.

B.44 Capture

Program	What it is
<code>shoot_pose.py</code> 132 lines	Command a joint pose to both arms, WAIT FOR ARRIVAL, and photograph it.

B.45 Sampling

Program	What it is
sample_place_branches.py 156 lines	HOW BADLY CAN ONE POSE GO? THE CLEARANCE DISTRIBUTION AT A SINGLE WAYPOINT.

B.46 Settling

Program	What it is
settle_blockers_ab.py 267 lines	Settle blocker A (T2's carry vs the bench) and blocker B (T3's box).

B.47 Surveys

Program	What it is
survey_task0_band.py 137 lines	Measure the band Task 0's spheres can live in, WITH THE BENCH IN SCENE.
survey_work_surface.py 214 lines	Where can each arm actually WORK on the table top?

B.48 Re-look

Program	What it is
relook.py 471 lines	RE-MEASURE ONLY WHAT CHANGED. Seconds, not half an hour.

B.49 Retraction

Program	What it is
retract_look_down.py 113 lines	Pull the arm back off the table and point the gripper straight DOWN.

B.50 Revalidation

Program	What it is
<code>revalidate_region.py</code> 172 lines	RE-VALIDATE THE SURVEYED REGION FROM THE CURRENT HOME, AT THE ANCHOR.

B.51 Recording layout

Program	What it is
<code>recording_layout.py</code> 131 lines	Where recordings live, and why the tree has the shape it has.

B.52 Safe motion

Program	What it is
<code>safe_goto_left.py</code> 176 lines	Move the LEFT arm to a saved joint pose WITHOUT sweeping through the table.

B.53 Servoing

Program	What it is
<code>servo_pick_left.py</code> 297 lines	Pick the cube by VISUAL SERVOING, not by trusting the camera extrinsic.

B.54 Simulated sessions

Program	What it is
<code>sim_session.py</code> 650 lines	Launch the sim, WAIT ON AN OBSERVABLE CONDITION, run a command, tear down.

B.55 Status

Program	What it is
<code>status_table.py</code> 338 lines	THE STATUS TABLE: every task x every mode, read FROM DISK.

B.56 Accuracy

Program	What it is
<code>accuracy_table.py</code> 295 lines	THE ACCURACY TABLE, derived from recorded data. Mode by metric.

B.57 Session faults

Program	What it is
<code>break_session.py</code> 159 lines	Break a running session on purpose, four ways, and check each is caught.

B.58 Clips

Program	What it is
<code>clip_contact_sheet.py</code> 93 lines	Build one contact sheet per clip so a human can LOOK at the whole set.
<code>clip_scene.py</code> 2374 lines	The task scene for the clip sweep: real objects, attached on a real grasp.

B.59 Environment

Program	What it is
<code>env_probe.py</code> 822 lines	THE LIVE HALF OF THE CALIBRATION STAGE: arm, camera, TF, and one voice.

B.60 Archived runs

Program	What it is
<code>final5_runs.py</code> 134 lines	Build recorder runs from the CURRENT five-task spec.

B.61 Generators

Program	What it is
<code>gen_wearer_posture_xacro.py</code> 124 lines	Generate the HUMAN ARMS block of <code>human_backpack.xacro</code> from <code>wearer_posture.py</code> .

B.62 Gripper

Program	What it is
<code>gripper.py</code> 275 lines	Open and close a real Robotiq gripper, and say what actually happened.

B.63 Instruction

Program	What it is
<code>instruct_t1.py</code> 147 lines	TYPE AN INSTRUCTION AND WATCH T1 HAPPEN. One command, the whole path.

B.64 LeRobot

Program	What it is
<code>lerobot_export.py</code> 580 lines	This repository's recordings, as a <code>LeRobotDataset</code> .

B.65 Pilot runs

Program	What it is
<code>pilot_bimanual.py</code> 264 lines	PILOT: every task end to end, scripted, with a fault injected mid-trial.

B.66 Planning

Program	What it is
<code>plan_pick_left.py</code> 161 lines	Plan (and optionally execute) a top-down grasp of the located cube.

B.67 Posing

Program	What it is
<code>pose_arms_at_pads.py</code> 107 lines	DRIVE BOTH ARMS TO THE TWO PADS AND HOLD, so a still shows the work.

B.68 Proofs

Program	What it is
<code>prove_degraded_mode.py</code> 243 lines	THE VIBRATION NUMBER: EE position std with degraded mode OFF vs ON.

B.69 Virtual hardware

Program	What it is
<code>virtual_tensy.py</code> 223 lines	A VIRTUAL TEENSY on a pty: replay real recorded frames, or hold still.

B.70 Shell entry points

Program	What it is
<code>attach_scene_camera.sh</code> 114 lines	<code>Scripts/attach_scene_camera.sh</code> - get the laptop camera into WSL, or say exactly why it is not there. <code>bash scripts/attach_scene_camera.sh # check</code> , and print the fix <code>bash scripts/attach_scene_camera.sh -attach #</code> also try the attach step
<code>bringup_arm.sh</code> 118 lines	Bring up ONE arm's high-level bridge and its gripper camera, idempotently. <code>bash scripts/bringup_arm.sh left</code> <code>bash scripts/bringup_arm.sh left -restart #</code> force a clean restart
<code>calibrate.sh</code> 17 lines	<zero gyro imu-mount max-position> Every calibration the rig needs, in one place, with the order that matters spelled out. Gyro bias MUST be captured from a settled stream: reopening the serial port resets the Teensy and the IMU emits nonsense while settling,
<code>check_arm_network.sh</code> 110 lines	Run this when the Kinova arms are connected. Verdict is about JITTER as much as reachability. The Kortex cyclic loop on UDP 10001 runs near 1 kHz; if a cyclic packet arrives late the arm's watchdog faults and the session drops mid-motion. WSL2 NAT adds a userspace

Program	What it is
check_channels.sh 76 lines	The wiring-repair acceptance test. ~3 minutes. bash scripts/check_channels.sh # sweep, then verdict table bash scripts/check_channels.sh -analyse-only <csv> bash scripts/check_channels.sh -baseline recordings/baselines/channels_20260806.json
diagnostics.sh 31 lines	The checks worth running BEFORE blaming the code. Every item here corresponds to a failure that has actually happened on this rig and cost real time to diagnose. See CLAUDE.md.
env.sh 211 lines	Scripts/env.sh – the single source of ROS 2 environment truth. source scripts/env.sh # SOURCE this, do not execute it Every script in this workspace sources this file so that terminal 1,
make_vr_cert.sh 127 lines	Self-signed TLS cert for the wifi/WebXR VR route. WHY A CERT AT ALL. WebXR refuses to start a session outside a SECURE CONTEXT. Over USB ('adb reverse') the origin is 127.0.0.1, which is secure by definition and needs no cert. Over wifi the origin is an IP address, so it
pick_cube_left.sh 73 lines	Put the LEFT arm on the operator's ideal pick pose, then pick the green cube. bash scripts/pick_cube_left.sh # move and pick bash scripts/pick_cube_left.sh -plan-only # everything except the motion
pick_the_cube.sh 109 lines	Recover from a latched e-stop and run the pick. bash scripts/pick_the_cube.sh [-arm left right] [-expect N] WHY THIS EXISTS. The observe move tripped the sim->real lag monitor's e-stop
record_everything.sh 107 lines	RE-RECORD THE VERIFICATION SET, one mode-task cell per stack. bash scripts/record_everything.sh # resume where it stopped bash scripts/record_everything.sh -fresh # ignore the ledger
record_t1_both_stages.sh 39 lines	RECORD T1 AND T1S2 UNDER 06_full_autonomy, FROM A TYPED INSTRUCTION. bash scripts/record_t1_both_stages.sh "put every cube on the pad of its own colour" ONE MODE PER INVOCATION, THROUGH sim_session.py, WHICH IS THE PROTOCOL THAT
recover.sh 41 lines	Back to a known-good state from ANY failure. PART 7f. Every step here corresponds to a state this rig has actually been stuck in. It is deliberately blunt: it kills everything srl_* and vendor-ROS related, clears the stale IPC that breaks discovery, and leaves you at a clean start.
run_autonomy.sh 14 lines	Teleoperation PLUS perception and shared autonomy. Starts the teleop stack first and layers perception + autonomy on top, so a failure in the autonomy layer leaves a working teleoperated robot rather than a dead one.
run_experiment.sh 91 lines	<task> [args] ONE dispatcher for both experiment sets. CURRENT t2 t3 t5 t6 t7 the bimanual set. run_bimanual.py.
run_teleop.sh 14 lines	TELEOPERATION ONLY. No perception, no autonomy. This is the baseline condition for every experiment, and it is deliberately runnable with srl_perception and srl_autonomy absent from the workspace. If this script ever needs one of them, the layering has been broken.

Program	What it is
run_teleop_capture.sh 116 lines	Calibration capture: runs the scripted teleop recording in THIS terminal, so the prompts are clean, unprefixed and readable. Intended flow (two terminals - see NOTE below for why): terminal 1: ros2 launch srl_teleop teleop.launch.py
run_wearer_tracker.sh 73 lines	Scripts/run_wearer_tracker.sh – start the body tracker in the ONE interpreter that can import both ROS and the pose model. bash scripts/run_wearer_tracker.sh
start_gui.sh 60 lines	Scripts/start_gui.sh – ONE COMMAND. Everything else is reachable from the window it opens. bash scripts/start_gui.sh
start_real.sh 513 lines	Terminal 3: connect the REAL arm(s), home, cascade. arm:=left right both (default: both) bash scripts/start_real.sh
start_vr.sh 58 lines	Bring up the VR path with nothing configured by hand. bash scripts/start_vr.sh # real headset over USB bash scripts/start_vr.sh -mock # desktop mock, no headset
start_vr_wifi.sh 176 lines	The VR path over WIFI and WebXR. NO adb, NO USB, NO Unity. bash scripts/start_vr_wifi.sh # everything bash scripts/start_vr_wifi.sh -page # bridge only, no mapper/gripper bash scripts/start_vr_wifi.sh -check # print the diagnosis and exit
sweep_gap_vs_posture.sh 32 lines	WOULD THE WEARER MOVING THEIR ARMS OPEN THE GAP? ONE STACK PER POSTURE. The wearer's arm posture reaches TWO places that share no file: the URDF, which is what MoveIt plans and collision-checks against, and 'mount_guard_node.WEARER', which is what the geometric clearance floor is
sweep_gap_vs_row.sh 34 lines	HOW FAR FORWARD DOES THE ROW HAVE TO BE BEFORE THE CENTRELINE OPENS? The wearer alone gives a hard bound that needs no kinematics: at the object height $z = 0.97$ a single tube point on the centreline is
sweep_rviz_resumable.sh 52 lines	Resumable RViz capture sweep. The first sweep had no progress file: a restart would have re-recorded all 87 clips from the beginning. This one records each completed run in recordings/verification/.rviz_progress and skips anything already listed, so
test_gripper.sh 82 lines	Open and close the gripper 3 times. Watch the hand and tell me what it does. bash scripts/test_gripper.sh # left arm bash scripts/test_gripper.sh right # right arm
vr_connect.sh 127 lines	Automatic Quest connection: find adb, find the headset, open the reverse tunnel, start the bridge, and keep the tunnel alive. WHY adb reverse AND NOT THE NETWORK. The headset connects to ws://127.0.0.1:8766, which is a secure origin by definition. Over a LAN it
vr_measure_session.sh 216 lines	The lab-day VR measurements, ONE AT A TIME. bash scripts/vr_measure_session.sh # what to run, in order bash scripts/vr_measure_session.sh yaw # ALWAYS FIRST bash scripts/vr_measure_session.sh rate

CHAPTER C

Every program: the tests

114 test modules, 1 322 passing tests. Each is listed with what it pins. A test name in this project is a sentence, and a great many of them name a defect that was once real.

C.1 srl_autonomy — 10 modules

Program	What it is
<code>test_grasp_offset.py</code> 54 lines	The grasp pose must not drive the fingers through the object.
<code>test_intent_inference.py</code> 157 lines	The three hard cases for intent inference, as executable tests.
<code>test_keepout_is_structural.py</code> 99 lines	A forbidden target must never become a disambiguation candidate.
<code>test_mode_switching.py</code> 62 lines	Switching between all six modes: no leaked state, no bypassed safety.
<code>test_modes_and_safety.py</code> 110 lines	The six modes, the shared safety stack, and mode 6's extra requirements.
<code>test_named_places.py</code> 194 lines	KNOWN ANSWERS FOR MOVE_TO's TARGET RESOLVER.
<code>test_referring.py</code> 110 lines	Known answers for free-form referring expressions.
<code>test_voice_intent.py</code> 241 lines	Voice grammar: what it accepts, and – more importantly – what it refuses.
<code>test_wake_word_is_measured.py</code> 115 lines	THE WAKE MATCHER, PINNED TO WHAT A REAL TRANSCRIBER ACTUALLY PRODUCED.
<code>test_world_model.py</code> 119 lines	World model: persistence, re-verification, and refusing to trust memory.

C.2 srl_experiments — 35 modules

Program	What it is
test_a_grip_with_no_object_is_not_left_pending.py 118 lines	A commanded grip change with nothing to arrive at must be APPLIED, not held.
test_abc_tasks_and_timeline.py 155 lines	Tasks A/B/C: the arithmetic, the geometry, and the two-hour claim.
test_bimanual_metrics.py 85 lines	Pin the bimanual metrics, especially the presence check.
test_capture_gate_is_opposite_button.py 39 lines	The capture gate must be the OPPOSITE arm's button.
test_carry_series_known_answers.py 141 lines	KNOWN ANSWERS FOR THE T2 CARRY SERIES.
test_claude_md_is_sufficient.py 83 lines	CLAUDE.md must stay short AND still carry every damaging constraint.
test_clip_frames_land_on_the_grasp.py 154 lines	THE FRAME EXTRACTOR MUST LAND A FRAME ON THE GRASP, OR SAY IT CANNOT.
test_clip_requirements_namespace.py 144 lines	KNOWN ANSWERS FOR WHICH OBJECTS A CLIP IS REQUIRED TO SHOW.
test_coupled_metrics_known_answers.py 176 lines	KNOWN-ANSWER TESTS. The standing rule, applied.
test_cube_must_be_on_work_plane.py 355 lines	A CUBE IS ON THE WORK PLANE; A FRAGMENT OF A PAD IS NOT.
test_grasp_attach_known_answers.py 77 lines	Known answers for the grasp attach test and the single-owner gripper.
test_grasp_orientation.py 43 lines	A grasp must match the object's ORIENTATION, not only its position.
test_grip_waits_for_the_fingers.py 76 lines	The run must wait for the FINGERS, by the same test its evidence uses.
test_mode_independence.py 235 lines	THE MODE-INDEPENDENCE PROOF. Build order item 1, before any task exists.
test_objects_on_table_can_fail.py 171 lines	THE OBJECTS-ON-TABLE CHECK MUST BE ABLE TO FAIL, AND TO PASS.

Program	What it is
test_one_work_surface_height.py 196 lines	THE SCENE HAS ONE OWNER FOR EACH SURFACE HEIGHT, AND THE GAP IS PINNED.
test_pad_offset_has_one_source.py 295 lines	THE WRIST-TO-PAD OFFSET IS ONE VECTOR, IN THE FRAME IT BELONGS TO.
test_readiness_never_green_on_unknown.py 104 lines	The one property that matters: NOT CHECKED can never read as HEALTHY.
test_run_starts_from_home.py 245 lines	A run starts from HOME, and until 2026-08-17 the second one did not.
test_scene_measures_both_t1_stages_at_t1s_approach.py 152 lines	THE SCENE MUST MEASURE BOTH T1 STAGES AT T1'S OWN APPROACH, NOT THE ANCHOR.
test_scene_travel_gate.py 167 lines	KNOWN ANSWERS FOR THE SWEEP'S SECOND TRAVEL GATE.
test_session_survives_failure.py 186 lines	Break the session on purpose and check it survives, names it, and resumes.
test_staging_move_duration.py 91 lines	KNOWN ANSWERS FOR HOW LONG THE STAGING MOVE IS GIVEN.
test_status_table_counts_its_own_task.py 128 lines	A CLIP COUNTS FOR THE TASK IT CONTAINS, NOT THE FOLDER IT SITS IN.
test_t0_sampling.py 173 lines	T0 randomisation: reproducible, safe, and it REFUSES rather than degrades.
test_t1_instruction.py 359 lines	T1 FROM A TYPED SENTENCE, and the colour comes from the CAMERA.
test_t1_layout_is_verified.py 213 lines	THE SHIPPED T1 CONSTANTS ARE THE ONES THAT WERE VERIFIED, OR THIS FAILS.
test_t1_stages_agree.py 267 lines	T1's two stages are ONE task, and as of 2026-08-18 they are one geometry.
test_t2_outcome_known_answers.py 227 lines	KNOWN-ANSWER tests for the T2 discrete outcome.
test_t2_tray_is_rigid.py 134 lines	T2's tray is a RIGID board, and it used to be drawn as elastic.
test_t3_spec.py 79 lines	T3's spec properties, asserted rather than trusted.

Program	What it is
<code>test_t7_targets_known_answers.py</code> 76 lines	T7 target generator: known-answer checks before it drives a participant.
<code>test_t9_interaction_known_answer.py</code> 130 lines	KNOWN-ANSWER test for T9's interaction fit — the headline of the task.
<code>test_the_scene_and_the_task_agree_where_an_object_is.py</code> 118 lines	The picture and the path must place an object in the SAME spot.
<code>test_work_surface.py</code> 77 lines	The work surface height has ONE owner and a 20 mm error must be NAMED.

C.3 srl_perception — 17 modules

Program	What it is
<code>test_fingerprint_yaw.py</code> 135 lines	KNOWN ANSWERS FOR THE YAW PATH. TASK_SPEC.md U-2.
<code>test_frame_carries_its_pose.py</code> 171 lines	A camera frame carries the pose it was RENDERED from, and the consumer uses it.
<code>test_grasp_pipeline.py</code> 174 lines	End-to-end pipeline tests on CONSTRUCTED RGB-D scenes.
<code>test_it_can_say_what_is_on_the_table.py</code> 314 lines	What is on the table, and can it be picked up: the chain, end to end.
<code>test_objects_come_from_the_picture.py</code> 528 lines	Object identity comes from the PICTURE, not from clustering the points.
<code>test_prompt_detector.py</code> 176 lines	Tests for prompt-driven detection.
<code>test_rgbd_grasp.py</code> 173 lines	Known-answer tests for the RGB-D grasp geometry.
<code>test_scene_calibration.py</code> 141 lines	Known-answer tests for scene-camera calibration.
<code>test_scene_fingerprint.py</code> 168 lines	Known answers for scene fingerprinting.
<code>test_scene_geometry.py</code> 130 lines	The camera <-> world transforms, with answers known in advance.
<code>test_scene_markers.py</code> 113 lines	The sweep picture must be the RIGHT picture.

Program	What it is
<code>test_surface_from_depth.py</code> 185 lines	KNOWN ANSWERS FOR THE WORK-SURFACE ESTIMATOR.
<code>test_the_grasp_pipeline_looks_before_it_says_no.py</code> 235 lines	Two defects in ‘grasp_pipeline.reachable’, and the controls that catch them.
<code>test_the_pick_path_corrects_the_shell_bias.py</code> 176 lines	A depth camera sees the FRONT of an object, and the pick path knew it – once.
<code>test_the_planner_goes_round_not_through.py</code> 191 lines	The planner must go ROUND, and must never mark its own homework.
<code>test_the_robot_maps_before_it_moves.py</code> 443 lines	Calibrate, map, then plan against the map – and say so while doing it.
<code>test_wearer_tracking.py</code> 272 lines	The tracked wearer must never be less conservative than the mannequin.

C.4 srl_teleop — 43 modules

Program	What it is
<code>test_arm_view_projection.py</code> 164 lines	The ACTUAL panel’s projection, with the answers known in advance.
<code>test_block_expiry_is_not_permission.py</code> 212 lines	An EXPIRED blocker must never read as an all-clear.
<code>test_both_button_estop_needs_hold.py</code> 70 lines	Gating taps must not fire the both-button e-stop; a held squeeze must.
<code>test_bridge_forces_high_level_servoing.py</code> 49 lines	The bridge must SET the servoing mode, not assume it.
<code>test_bridge_integral_closes_the_gap.py</code> 93 lines	The bridge’s integral term: it must exist, be bounded, and be reset.
<code>test_bridge_refuses_the_home_change.py</code> 92 lines	The sim home moved and the real arms did not. The bridge must REFUSE.
<code>test_capability_ladder.py</code> 145 lines	Known answers for the capability ladder.
<code>test_clearance_floor_follows_the_tracked_body.py</code> 124 lines	The clearance floor must enforce against the SAME body MoveIt plans against.

Program	What it is
test_clearance_guard_cannot_pass_on_no_data.py 51 lines	A clearance guard with no data must REFUSE, not report infinity.
test_closed_on_nothing_is_not_a_grasp.py 53 lines	A gripper closed on NOTHING must never read as holding something.
test_copyright.py 25 lines	Copyright 2015 Open Source Robotics Foundation, Inc. Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at
test_degraded_mode.py 188 lines	Degraded mode: freeze what is broken, and never invent a value.
test_divergence.py 142 lines	Divergence: the number, and the four ways it can be a lie.
test_flake8.py 25 lines	Copyright 2017 Open Source Robotics Foundation, Inc. Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at
test_grasp_constants_have_one_owner.py 97 lines	The grasp thresholds must have exactly ONE definition.
test_gripper_startup_reference.py 110 lines	The startup/shutdown open reference: what may be assumed, and what may not.
test_gui_launch_specs.py 169 lines	Launch buttons: every one must resolve, or be disabled WITH A REASON.
test_home_has_one_source.py 308 lines	The home pose has ONE source, and everything else must load it.
test_ik_guard.py 127 lines	The guard must converge, not deadlock.
test_imu_orientation.py 117 lines	Regression tests for the IMU sensing path (Part 3).
test_kortex_convention.py 125 lines	The seam is where this goes wrong, so test the seam.
test_motion_enabled_is_read_live.py 119 lines	‘motion_enabled‘ is honoured LIVE, and for three sessions it was not.
test_motion_generator.py 483 lines	The teleoperation motion generator: synchronised, continuous, and limited.
test_no_unresolved_module_names.py 178 lines	Every module-like name a node uses must actually be imported.

Program	What it is
test_observer_bypass_is_deliberate.py 156 lines	Working alone is allowed. Working alone BY ACCIDENT is not.
test_orientation_relaxation_is_a_last_resort.py 281 lines	The wrist may be unpinned, and only after the pinned answer has failed.
test_pep257.py 23 lines	Copyright 2015 Open Source Robotics Foundation, Inc. Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at
test_precision_speed.py 81 lines	The dial trades precision for speed and NOTHING else.
test_predictive_avoidance.py 308 lines	Predictive avoidance, on constructed inputs with known answers.
test_real_arm_doctor.py 405 lines	Every connection check must be able to FAIL, and must never be green on unknown.
test_real_path_safety.py 217 lines	The two silent real-robot failures must not come back.
test_right_arm_spherical_survives_dead_rolls.py 129 lines	The RIGHT arm must still give POSITION with j3/j5/j7 dead.
test_serial_exclusive.py 119 lines	Two readers on one master port must be IMPOSSIBLE, not merely detectable.
test_teleop_gui_launches.py 167 lines	Teleop_gui must LAUNCH. Exercises the real entry point, not an import.
test_the_arm_stops_short_and_the_bridge_knows.py 288 lines	The 0.305 deg terminal error: the model, the fix, and the trap in between.
test_the_arm_tells_us_more_than_seven_numbers.py 162 lines	Everything the Kinova arm already reports, and the gate that needs a baseline.
test_the_test_gate_cannot_invent_a_failure.py 74 lines	The gate must not report tests that do not exist.
test_the_window_fits_the_screen.py 81 lines	The operations window must fit the screen it is opened on.
test_vr_bringup_sequence.py 832 lines	Break the VR bring-up eight ways and require it to say which, in English.
test_wearer_check_covers_the_whole_body.py 137 lines	The homing clearance check must sweep the whole body and the whole arm.

Program	What it is
test_wearer_posture_has_one_source.py 194 lines	The wearer's arm posture has ONE source, and both consumers must read it.
test_wearer_presence_is_explicit.py 88 lines	Declaring nobody is in the rig must be explicit, loud, and never a default.
test_wearer_size_reaches_the_live_check.py 139 lines	The wearer's SIZE must reach the check that stops the arm, not just the offline one.

C.5 srl_vr_teleop — 10 modules

Program	What it is
test_bridge_rate_decays_to_zero.py 149 lines	A stalled stream must not keep reporting the rate it used to have.
test_observer_estop_silence.py 133 lines	Silence from the observer must be treated as absence, not as presence.
test_observer_gate_reaches_the_freeze.py 188 lines	'require_observer_estop' must gate the FREEZE, not only the enable service.
test_operator_alignment_is_a_rotation.py 136 lines	The operator-to-robot alignment must be a ROTATION, never a mirror.
test_passthrough_is_an_addition.py 145 lines	Passthrough changes the session mode and NOTHING on the arm's path.
test_quest_frames.py 102 lines	Quest -> world frame conversion, pinned against known rotations.
test_tracking_reference_stability.py 175 lines	The tracking reference is a physical object and it can be knocked.
test_vr_mapper_resets_between_runs.py 233 lines	Vr_pose_mapper must begin every run in the state it begins its life in.
test_vr_smoothing.py 227 lines	The VR command smoother: what it must do, and what it must not carry.
test_webxr_frames.py 172 lines	WebXR -> world frame conversion, pinned against known rotations.

CHAPTER D

Recorded baseline artefacts

recordings/baselines/ holds the stored output of every measurement script. A claim in this report that names a baseline file can be checked against it, and re-running the named script regenerates it.

abc_verification.json, approach_geometry.json, arm_directional_calibration.json, arm_reach_extents.json, band_vs_table_height.json, band_vs_work_height.json, blockers_ab.json, cad_inspect.json, cad_joints.json, capability_degradation.json, capability_delta.json, capability_ladder.json, capture_rate.json, centre_certified.json, centre_certified2.json, centre_certified3.json, centre_certified_sym.json, centre_gap_corner_e-20.json, centre_gap_corner_e-30.json, centre_gap_corner_e-40.json, centre_gap_corner_e-50.json, centre_gap_down.json, centre_gap_folded.json, centre_gap_h80.json, centre_gap_heads.json, centre_gap_none.json, centre_gap_row_0.300.json, centre_gap_row_0.320.json, centre_gap_row_0.340.json, centre_gap_row_0.380.json, centre_gap_topdown.json, centre_geom_height.json, centre_geom_stand.json, centre_geom_width.json, centre_layout.json, centre_limit.json, centre_limit_approach.json, centre_on_surface.json, centre_posture_behind.json, centre_posture_down.json, centre_posture_folded.json, centre_posture_none.json, centre_posture_out.json, centre_reach.json, centre_vs_height.json, centre_vs_height_topdown.json, centre_vs_wearer_posture.json, channels_20260806.json, choreography_verification.json, clearance_region.json, clearance_region_inboard.json, clearance_region_left_fine.json, clearance_region_outboard.json, colour_vision.json, colour_vision_frame.png, control_budget.json, cube_located_v2.json, dance_paths.json, dance_pulse_executed.csv, depth_pose_accuracy.json, detector_real_rgbd.json, dial_safety.json, dual_gui_measurement.json, experiment_gates.json, final5_verification.json, forward_reach.json, frame_pose_gate.json, freeform_language.json, gap_vs_mount.json, gate3_cup_stack.json, go_home.json, grasp_approach.json, grasp_approach_angles.json, grasp_penetration.json, grasp_reality.json, gui_button_verify.txt, home_change.json, home_change_applied.json, home_rederive.json, home_render.json, home_solved.json, home_wide.json, home_widest.json, home_widest_run2_x0780.json, lab_day_faults.json, language_vision_sweep.json, layout_onetable.json, layout_verified_zero.txt, left_arm_on_surface.json, lift.json, look_then_grasp.json, mode_difference.json, mode_paths.json, mount_as_built.json, mount_geometry.json, mount_geometry_best_table.json, mount_overlap_sweep.json, msc_verification.json, objects_on_table.json, observe_pose.json, observe_pose_left.json, observe_pose_right.json, observe_pose_verified.json, observe_t1_left.json, observe_t1s2_left.json, observe_t1s2_right.json, observe_transit_left.json, observe_transit_right.json, orientation_constraint_fk.txt, orientation_constraint_probe.txt, orientation_cost.json, orientation_cost_what_binds.json, orientation_cost_what_binds_as_follower.json, pad_clearance.json, pad_clearance.txt, pad_clearance_INVALID.txt, pad_columns.json, pad_mid_ee.json, pad_mid_ee_by_width.json, pick_accuracy.json, pick_finish.json, pick_plan_left_safe.json, pick_plan_left_v2.json, pick_pose_both_v3.json, pick_pose_manual.json, pick_pose_manual_v2.json, predictive_avoidance.json, presentation_pose.json, randomisation_region.json, reachable_cells.json, reachable_envelope_wide.json, reachable_rear.json, reachable_volume.json, ready_pose_envelope.json, render_rate.json, retreat_left.json, return_pickup.json, scan_pose.json, scenario_audit.yaml, scene_fingerprint.json, scene_reference.npz,

shared_autonomy_gain.json, sim_to_real_gap.json, surface_vs_t1_path.json,
symmetric_home.json, t0_directions.json, t1_centre_stage_a.json,
t1_centre_stage_b.json, t1_cubes_t2_transfer.json, t1_instruction_sweep.json,
t1_layout_on_surface.json, t1_layout_options.json, t1_layout_options.txt,
t1_pad_columns.json, t1_paths.json, t1_paths_BEFORE.json, t1_paths_independent.json,
t1_paths_stage1_objects.json, t1_paths_stage1_path.json, t1_paths_stage2_a.json,
t1_paths_stage2_b.json, t1_paths_stage2_seed3.json, t1_place_branches.json,
t1_stage2_paths.json, t1_stage2_verification.json, t1_support_sweep.json,
t1_waypoint_diagnosis.json, t2_band_top.json, t3_box_hold_sweep.json,
table_edge_sweep.json, table_model.json, table_topdown, task0_band_survey.json,
task0_verification.json, task_a_bin_sweep.json, task_margin.json, teleop_motion.json,
topdown_vs_pinned.json, topdown_vs_pinned.txt, vision_drives_grasp.json,
vision_options.json, voice_instruction.json, vr_one_button.json, wake_word.json,
wearer_size_effect.json, what_binds.json, work_surface_region.json,
work_surface_region_PRE20260815.json, workspace_20260806.json, workspace_audit.json,
workspace_n10_20260806.json, workspace_n10_retry_20260806.json,
workspace_waypoints.json, world_map.json, world_map_left.json,
world_map_occupancy.npy, wrist_capability.json, wrist_orientation.txt

CHAPTER E

The hard constraints, in full

These are the project's standing rules. Each one exists because violating it caused, or would cause, real damage. They are reproduced here because they are the most compressed statement of what the system is.

0. **Simulated and real home disagree on purpose.** Simulated home is the presentation pose; the physical arms are still at the legacy Kortex home, where the wrists point up by $+85^\circ$ and $+79^\circ$. The bridge refuses to enable on the roughly 1.9 rad difference rather than commanding it.
1. **Home joint angles are ground truth, and the real arm's are the ones that count.** Never change either to make a pose look right without re-measuring first. The workspace anchor is a *different* quantity: moving home cost nothing measurable, while re-deriving the anchor to match a level home costs task T2 its right arm.
2. **The arm permits exactly one Kortex session.** SIGINT the bridge; SIGKILL leaks the session and the next run cannot connect.
3. **Never run two stacks.** Two master pose nodes split the serial stream and invalidated a full day of measurements. A stray `robot_state_publisher` does the same damage and is not caught by a process count.
4. **The Kortex cyclic path is unusable over WSL.** Use the high-level API. Every cyclic write is a network round trip.
5. **FASTDDS_BUILTIN_TRANSPORTS=SHM.** UDP discovery is dead on this host. Clear `/dev/shm/fastrtps_*` with the stack stopped, then relaunch.
6. **Never wait for a service in the e-stop trip path.** It once blocked the e-stop for 4.0s. Park first, then talk to the driver.
7. **The dependency arrow runs one way.** `srl_teleop` imports nothing in-repository; it is the baseline condition of every experiment.
8. **Real-robot mode must be armed.** Motion is disabled until enabled by hand, and it refuses to start with a continuous joint wound beyond $\pm\pi$.
9. **Never pkill broad patterns while a stack you want is running.** Kill an explicit PID list, and kill the process group.
10. **Wrap ROS `setup.bash` in `set +u` / `set -u`** — it reads unbound variables and dies otherwise.
11. **The wearer is in the collision model and the clearance floor is the last thing between the arms and a person's chest.** Do not lower it globally. `avoid_collisions` is not the wearer check: the SRDF excludes the 44 proximal pairs a shoulder mount actually threatens, so a “valid” pose can have the tube inside the person. Measure clearance geometrically. The wearer's posture is part of the model and is a variable with

one source; do not instruct a wearer to clasp their arms behind their back or hold them out to the sides.

12. **Anonymity is enforced in code.** The manifest writer raises on name, email, date of birth, address and phone.
13. **A demonstration is not evidence.** Demonstration clips carry that caveat in their caption and produce no trial data, no condition and no metric.

CHAPTER F

Glossary

Term	Meaning in this project
Anchor / workspace anchor	The fixed wrist orientation, about 30.7° approach direction, written into every commanded waypoint by the task runner. Distinct from home, and never recomputed from it.
Capture gate	30 mm. The distance within which the gripper pads must arrive for a grasp to close on the object.
Clearance floor	150 mm, from any moving part of the arm chain to any part of the wearer's body. Measured geometrically, never through <code>avoid_collisions</code> .
Coherent / incoherent	A master channel verdict. Incoherent means present, fresh and wrong — more than a threshold fraction of updates jump over 60° .
Densified	A path resampled at 20 mm (or 0.05 rad in joint space) so that checks apply between waypoints as well as at them.
Desk operation	The shipped arrangement: the operator sits across the room facing the wearer, holding the controllers as motion capture, with the headset on a shelf as the tracking reference.
Dyad	The unit of analysis in the experimental design: one operator and one wearer, both participants.
<code>fuse()</code>	The function implementing the safety rule that the camera may only make the wearer bigger: per body part, keep whichever of tracked and mannequin is closer to the robot.
Mode 01/02/03/04/06	The five operating modes. There is no 05.
$N = 10$	The repeat count for a reachability claim, applied over the whole densified path, because TRAC-IK restarts randomly.
Pad midpoint	The point between the gripper's finger pads. Not a fixed distance from the wrist: 0.09833 m open, 0.10976 m on a 40 mm cube.
Pinned wrist	The pre-2026-08-22 policy of commanding an exact wrist orientation, which spends the arm's entire redundancy.
Presentation pose	The simulated home pose since 2026-08-15: wrists level, hands in front of the chest.
Real gap vs by-design absence	A missing data cell that is a defect, versus one that is missing because the task does not run under that mode. Rendered differently on purpose.
Shell bias	The systematic offset between the centre of the visible shell of an object and its true centre when measured from one view: 10.5 mm, and 0.0 mm once the support plane is used.

Term	Meaning in this project
Staging	Returning the arms to home and confirming it before a run starts. Refuses rather than proceeding from an unknown pose.
T0–T3	The four tasks, keyed <code>m0</code> <code>m1</code> <code>m1s2</code> <code>m2</code> <code>m3</code> internally.
VoxelWorld	The occupancy structure built per frame from the measured map, through which the joint-space planner knows the table exists.